

# 有界测向误差下无线电干扰源的集合定位、主动测向与多源滚动清除

——集合几何、主动测向与滚动调度

## 摘要

针对有界测向在有误差的情况下无线电干扰源的快速定位与清除问题，本文建立了一套完整的由**集合几何定位**、**鲁棒主动测向**和**多源在线滚动调度**组成的一体化模型。本文仅使用题目给出的  $\pm 1^\circ$  确定性误差界构造位置可行集合，而没有采用预设高斯噪声等概率分布的方法。本文以定位区域尺度与机器狗总任务时间作为核心的评价指标。

针对问题一，本文将单次有界示向信息表示为两个半平面的交，从而把多次交会定位转化为了凸集求交的问题；我们利用旋转卡壳计算定位区域直径，并基于 **Thales 点积判据**与最小包围圆给出“直径圆能否覆盖定位区域”的等价审计。根据分析，**采用定位区域直径为直径作圆并不一定可以完美覆盖整个定位区域**。在 1000 组随机凸区域交叉验证中，旋转卡壳与穷举直径的最大误差为 0，Thales 判据与最小包围圆判据无不一致案例，并且无 Jung 界违例。

针对问题二，本文将干扰源位置  $g$  与未知有效接收半径  $\rho$  进行联合建模，利用第一次已接收到的信号这一事实构造最大保证二次有信号区域  $C_{\text{sig}}^*$ ；在该区域内，采用第二次观测后定位区域的**最坏直径最小**为目标建立 min-max 主动测向模型，并利用近优集合  $C(\eta)$  给出具有容错性的第二检测点候选区域，避免仅凭“尽量接近  $90^\circ$  交会”出现经验选点。

针对问题三，本文建立了一套**有限视野主动测向-带状光学覆盖模型**。算法持续利用正、负观测更新各频道的位置可行域，并依据预计剩余时间在继续无线电测向与有限光学覆盖之间进行动态切换。120 组离线场景均完成全部 1560 个干扰源清除，按清除源数加权的平均定位清除时间为 **255.96 s/源**；三次官方正式测试共清除 42 个源，对应加权平均定位清除时间为 **237.423 s/源**。

针对问题四，考虑定向源使 no\_signal 同时具有“无未清除源、超出接收范围、处于定向盲区”三种解释，因而采用保守位置更新策略；进一步设计**局部共享补测-多源混合滚动调度模型**，将扫描任务与已发现干扰源的定位、清除任务纳入统一任务池，并通过多初值局部路径改进降低跨区域移动。100 组离线场景均完成全部干扰源清除，按清除源数加权的平均定位清除时间为 **472.26 s/源**；三次官方正式测试共清除 35 个源，对应加权平均定位清除时间为 **494.800 s/源**。

综上，本文在不牺牲集合一致性与完整清除约束的前提下，通过主动感知与滚动调度显著减少了无效清除、冗余检测与跨区域移动，为有界误差下的移动传感与主动定位问题提供了一套可解释、可验证、可在线执行的决策框架。

**关键词：**有界误差；集合估计；主动测向；滚动调度；干扰源定位

# 1 问题重述与总体分析

## 1.1 任务重述

任务约束与计时规则依据官方题面 [1]。目标区域为半径 1800 m 的圆域，干扰源频道位于 1–20 号频道，每个源的有效接收半径固定但未知，范围为 1000 m to 1500 m。测向机示向度误差在同一地点保持不变，在不同地点随环境变化，但全局严格有界于  $[-1^\circ, 1^\circ]$ 。机器狗移动速度为 5 m/s，一次检测固定耗时 5 s，检测频道发生变化时增加 1 s 切换时间；当距离目标不超过 20 m 时可进行光学精确定位与清除，成功清除固定服务时间为 5 s，未发现可清除目标时为 3 s。问题三仅含全向干扰源，问题四同时含全向源和定向源，后者的有效辐射角为半平面型  $180^\circ$  扇区。

四问分别对应同一决策过程的不同层次。问题一为确定观测后的可行位置集合；问题二为在该集合上选择第二测点；问题三则加入多频道搜索与清除；问题四进一步处理定向辐射造成的负观测歧义和跨源路径调度。



图 1 四个子问题构成从有界观测、可行集合、主动感知到滚动清除的统一方法链。硬可行几何负责正确性约束，主动测向与滚动路由仅负责提高执行效率。

## 1.2 总体建模思想

有界误差集合估计的基本思想是保留所有与观测相容的状态 [2]，本文据此构造测向一致集。把**正确性约束**与**效率启发式**进行了严格的分离：真实源位置硬可行域只由题设确定性约束和实际观测更新；概率式、有限视野式或路径式评分仅用于动作排序，不直接修改位置集合。由此形成了以下顺序：

有界观测 → 可行集合 → 主动感知 → 滚动调度 → 完整清除。

Q3/Q4 的在线控制建立在 Q1/Q2 的集合几何之上；即使效率评分失准，硬可行域仍可由真实观测进行约束。

## 2 模型假设与符号说明

### 2.1 模型假设

1. 严格采用题设二维平面模型，忽略测向机和干扰源的高度差。
2. 示向误差只假定满足  $[-1^\circ, 1^\circ]$  的确定性有界条件，不额外假定其服从高斯分布或独立同分布。
3. 同一地点重复测量的当地误差不变，因此不通过同点重复测量取平均来“降低噪声”。
4. 不同频道干扰源信号相互独立；每个源频道固定，单局内有效接收半径固定。
5. 移动、检测、切换频道和光学清除耗时完全按模拟器计时规则计算。
6. 在线决策只使用接口返回的当前位置、频道与测量结果，不读取隐藏源坐标、真实接收半径、真实源类型、随机种子或未来响应。
7. “已解决”仅在真实清除成功后成立；未命中清除尝试不能被视为成功清除，Q4 的普通 `no_signal` 也不能无条件作为位置排除证据。

## 2.2 主要符号

表 1 主要符号及含义

| 符号                                                    | 含义                | 单位/范围               |
|-------------------------------------------------------|-------------------|---------------------|
| $\Omega$                                              | 目标区域              | 半径 1800 m<br>圆域     |
| $g$                                                   | 干扰源真实位置           | m                   |
| $S_i$                                                 | 第 $i$ 个检测点        | m                   |
| $\theta_i$                                            | 第 $i$ 次示向度        | $[0, 360^\circ)$    |
| $\delta$                                              | 示向误差上界            | $1^\circ$           |
| $W_i$                                                 | 第 $i$ 次测向的楔形可行域   | —                   |
| $P$                                                   | 多次测向后的交会定位区域      | —                   |
| $D(P)$                                                | 定位区域直径            | m                   |
| $R_{\text{MEC}}(P)$                                   | 定位区域最小包围圆半径       | m                   |
| $\rho$                                                | 干扰源有效接收半径         | $[1000, 1500]$<br>m |
| $F_1, Z_1$                                            | 第一次正观测后的位置集与联合状态集 | —                   |
| $C_{\text{sig}}^*$                                    | 最大保证二次有信号区域       | —                   |
| $J_D(x)$                                              | 第二检测点 $x$ 的最坏定位直径 | m                   |
| $\mathcal{F}_c$                                       | 频道 $c$ 当前硬位置可行域   | —                   |
| $\mathcal{J}_{\text{scan}}, \mathcal{J}_{\text{src}}$ | 全局扫描任务集、源服务任务集    | —                   |
| $T$                                                   | 单局虚拟总任务时间         | s                   |

## 3 问题一：有界测向误差下的集合定位模型

### 3.1 单次示向观测的楔形集合

设第  $i$  个检测点为  $S_i = (x_i, y_i)$ ，测得示向度为  $\theta_i$ ，误差上界为  $\delta = 1^\circ$ 。定义方向单位向量

$$u(\alpha) = (\cos \alpha, \sin \alpha)^T,$$

以及二维叉积  $[a, b] = a_x b_y - a_y b_x$ 。真实源位置  $g$  的方位必位于  $[\theta_i - \delta, \theta_i + \delta]$ ，故单次测量对应楔形

$$W_i = \{g : [u(\theta_i - \delta), g - S_i] \geq 0, [u(\theta_i + \delta), g - S_i] \leq 0\}. \quad (1)$$

多次观测融合后，交会定位区域为

$$P = \Omega \cap \bigcap_{i=1}^n W_i. \quad (2)$$

此方法不估计单个“最可能点”，而保留全部误差相容位置。只要实际误差落在题设区间内，真实位置必属于  $P$ 。

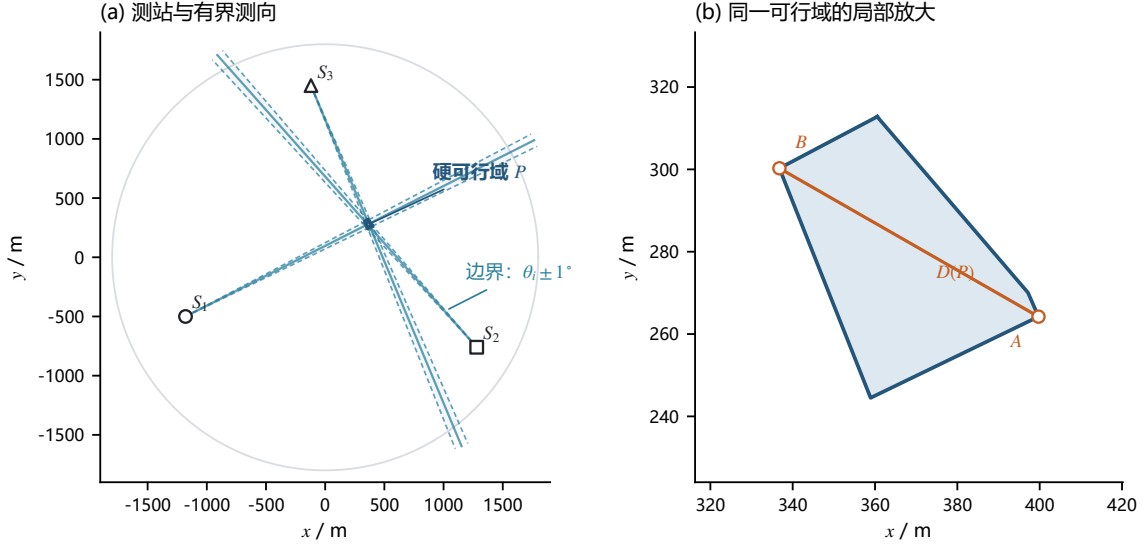


图 2 三次  $\pm 1^\circ$  有界示向观测在目标圆域内形成三个楔形约束，其交集为保留全部误差相容位置的可行区域  $P$ 。中心射线与边界射线分别用实线和虚线表示。

### 3.2 定位区域直径与旋转卡壳

采用凸多边形旋转卡壳方法 [5] 枚举对踵点。设凸多边形  $P$  的顶点集为  $V(P) = \{v_1, \dots, v_m\}$ 。定义

$$D(P) = \max_{p, q \in P} \|p - q\|_2. \quad (3)$$

对于凸多边形，最大距离必由某一对顶点取得：

$$D(P) = \max_{1 \leq i, j \leq m} \|v_i - v_j\|_2. \quad (4)$$

顶点按逆时针顺序排列后，用旋转卡壳扫描对踵点对，可在  $O(m)$  时间内得到直径。另保留  $O(m^2)$  顶点对穷举，用于独立核对数值结果。

### 3.3 直径圆覆盖性的精确判据

设  $A, B$  为一对直径端点， $\|A - B\| = D$ ，其中点为  $M = (A + B)/2$ 。对于任一点  $x$ ，有

$$(x - A) \cdot (x - B) = \|x - M\|^2 - \frac{D^2}{4}.$$

因此，以  $AB$  为直径的圆覆盖整个  $P$  当且仅当

$$(v_i - A) \cdot (v_i - B) \leq 0, \quad i = 1, \dots, m. \quad (5)$$

这给出直接的 Thales 点积判据。

该条件并非恒成立。以正三角形为例，其边长就是区域直径  $D$ ，但最小包围圆半径

为

$$R_{\text{MEC}} = \frac{D}{\sqrt{3}} > \frac{D}{2},$$

故任一半径为  $D/2$  的直径圆都不能覆盖整个三角形。因此

以定位区域直径为直径作圆，一般不能保证覆盖整个定位区域。

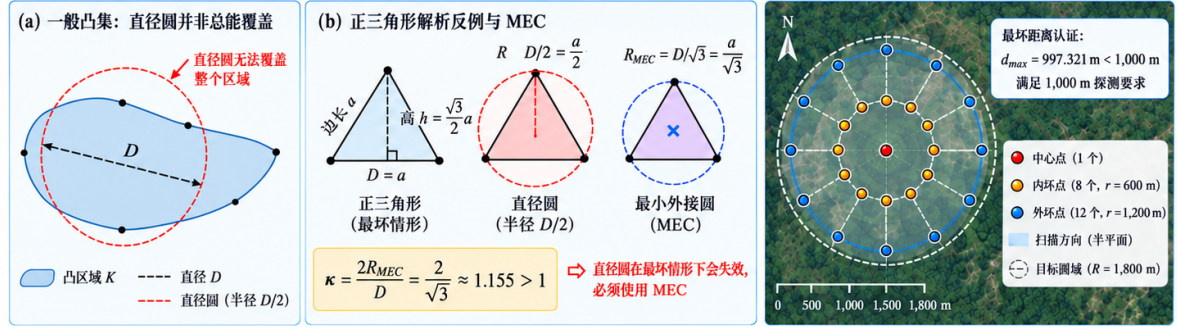
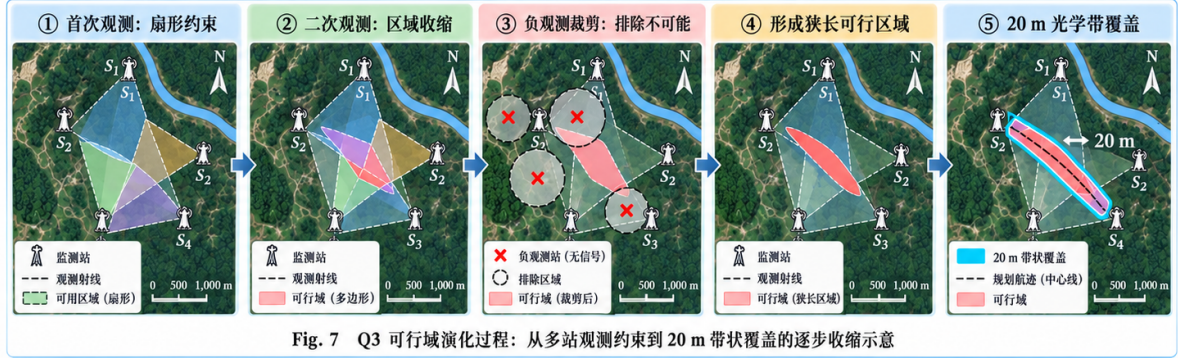


Fig. 12 Q4 21 点扫描布局与最坏距离认证

图 3 直径圆与最小外接圆 (MEC) 的对比及正三角形解析反例。左图表明一般凸可行域的顶点可落在直径圆外；右图中  $R_{\text{MEC}} = D/\sqrt{3} > D/2$ ，故直径圆不能保证覆盖整个可行域。

### 3.4 最小包围圆的独立审计

最小包围圆可用随机增量方法求解 [6]。任意覆盖  $P$  的圆都必须包含一对相距  $D$  的点，因此

$$R_{\text{MEC}}(P) \geq \frac{D(P)}{2}.$$

故存在半径  $D/2$  的圆覆盖  $P$  当且仅当

$$R_{\text{MEC}}(P) = \frac{D(P)}{2}. \quad (6)$$

定义无量纲指标

$$\kappa(P) = \frac{2R_{\text{MEC}}(P)}{D(P)}.$$

根据平面 Jung 定理,

$$1 \leq \kappa \leq \frac{2}{\sqrt{3}}.$$

$\kappa = 1$  对应存在直径圆覆盖,  $\kappa > 1$  则刻画直径圆与实际最小覆盖圆之间的尺度差异。

### 3.5 几何内核验证

表 2 Q1 几何算法交叉验证结果

| 验证项目              | 样本数  | 结果         |
|-------------------|------|------------|
| 旋转卡壳与顶点对穷举直径      | 1000 | 最大误差 = 0 m |
| Thales 判据与最小包围圆判据 | 1000 | 不一致案例 = 0  |
| Jung 界检查          | 1000 | 违例 = 0     |

1000 组样本中, 三项检查均未出现冲突或违例。

## 4 问题二: 保证二次可观测性的鲁棒主动测向模型

### 4.1 第一次正观测后的联合状态

设第一次检测点为  $S_1$ , 返回正常示向度  $\theta_1$ 。正常示向不仅提供角度楔形, 同时说明源距  $S_1$  大于 5 m 且不超过实际接收半径。因  $\rho \leq 1500$ , 得到位置可行集

$$F_1 = \Omega \cap W(S_1, \theta_1, \delta) \cap B(S_1, 1500) \setminus B(S_1, 5). \quad (7)$$

更完整的状态还应包含未知接收半径  $\rho$ 。第一次已经成功收到信号, 因此

$$Z_1 = \{(g, \rho) : g \in F_1, \max\{1000, \|g - S_1\|\} \leq \rho \leq 1500\}. \quad (8)$$

这一步把“第一次收到信号”本身转化为对未知参数  $\rho$  的信息, 而不是把  $\rho$  简单固定为最保守的 1000 m。

### 4.2 最大保证二次有信号区域

若只使用  $\rho \geq 1000$ , 保证第二点仍在接收半径内的保守区域为

$$C_{\text{sig}} = \bigcap_{g \in F_1} B(g, 1000).$$

对给定候选位置  $g$ ，与第一次正观测相容的最小接收半径为

$$\rho_{\min}(g) = \max\{1000, \|g - S_1\|\}.$$

于是定义

$$C_{\text{sig}}^* = \bigcap_{g \in F_1} B(g, \rho_{\min}(g)). \quad (9)$$

显然  $C_{\text{sig}} \subseteq C_{\text{sig}}^*$ 。

**命题：**在只使用题设先验和第一次正观测的条件下， $C_{\text{sig}}^*$  是能够保证第二次检测不因距离不足而失去信号的最大区域。

**证明：**若  $x \in C_{\text{sig}}^*$ ，则对任意  $g \in F_1$  有  $\|x - g\| \leq \rho_{\min}(g)$ ，而任意与第一次观测相容的实际  $\rho$  均满足  $\rho \geq \rho_{\min}(g)$ ，故  $\|x - g\| \leq \rho$ 。反之，若  $x \notin C_{\text{sig}}^*$ ，存在某个  $g_0$  使  $\|x - g_0\| > \rho_{\min}(g_0)$ ；取  $\rho = \rho_{\min}(g_0)$  仍与第一次观测相容，但第二点  $x$  将因距离不足失去信号。因此任何更大的区域都无法保持该最坏情形保证。  $\square$

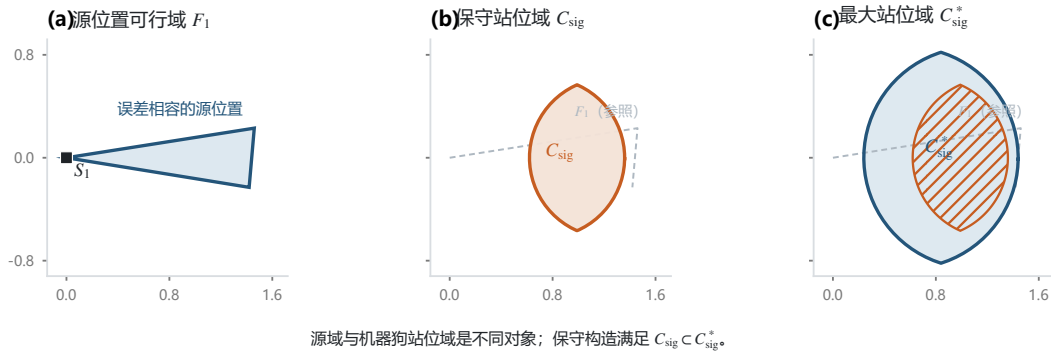


图4 第一次正观测后的可行状态提高了接收半径的可行下界，使最大保证二次有信号区域  $C_{\text{sig}}^*$  严格包含保守区域  $C_{\text{sig}}$ 。纹理与边界线型使两个集合在灰度打印中仍可区分。

### 4.3 第二检测点的 min-max 设计

有界误差测向布局与三角定位的研究提供了几何选点背景 [3, 4]；以下目标与保证接收约束按本题条件建立。对候选第二点  $x \in C_{\text{sig}}^*$ ，若返回正常示向度  $\theta$ ，严格观测一致集写为

$$F_2(x, \theta) = F_1 \cap W(x, \theta, \delta) \setminus B(x, 5). \quad (10)$$

若返回 **near**，则源距离不超过 5 m，同时必在 20 m 光学清除半径内，可直接转入精确定位与清除。记候选点  $x$  的所有可能正常第二示向度为  $\Theta_2(x)$ ，定义最坏定位直径

$$J_D(x) = \max_{\theta \in \Theta_2(x)} D(F_2(x, \theta)). \quad (11)$$

从而选择

$$x^* = \arg \min_{x \in C_{\text{sig}}^* \cap \mathcal{X}} J_D(x). \quad (12)$$

与“尽量形成 90° 交会角”的经验规则相比，式(12)直接最小化题目关心的定位区域尺度，同时自动权衡交会角、源到测点的距离以及保证接收约束。

#### 4.4 由最优点扩展到近优候选区域

考虑到后续路径调度不一定能恰好经过  $x^*$ ，再定义

$$C(\eta) = \{x \in C_{\text{sig}}^* : J_D(x) \leq (1 + \eta)J_D(x^*)\}. \quad (13)$$

结果同时给出  $x^*$  和近优集合  $C(\eta)$ ； $C(\eta)$  中各点的最坏直径不超过最优值的  $(1 + \eta)$  倍。若多个候选点的  $J_D$  数值相同，再按

$$(J_D(x), J_R(x), \|x - S_1\|)$$

进行字典序比较。

#### 4.5 数值求解与代表性算例

$C_{\text{sig}}^*$  是由连续圆盘约束定义的二维半无限约束集合。数值求解采用支撑函数与约束生成：先用有限活动约束求给定方向的边界候选点，再检查其对全部  $F_1$  的最大违反量；若仍有违反，就加入最严重约束继续迭代。 $J_D(x)$  采用候选搜索，并用更密集的离散计算独立核对。

代表性算例的网格最优点约为 (800, 600)，对应  $J_D \approx 112.13 \text{ m}$ 。该数值只检验求解程序和几何趋势；式 (7)–(13) 本身是参数化模型， $F_1, C_{\text{sig}}^*, x^*, C(\eta)$  都随首次观测位置和示向度重新计算。

图 5 只对应上述代表性算例；其数值不作为题目给定的固定实例结果。

### 5 问题三：全向多源主动搜索定位与清除模型

#### 5.1 总时间目标

问题三的目标并不是单纯最小化定位误差，而是在全部清除的前提下尽可能缩短任务时间。总虚拟时间被分解为

$$T = T_{\text{move}} + T_{\text{switch}} + T_{\text{measure}} + T_{\text{clear}}. \quad (14)$$

其中移动时间由相邻动作位置间欧氏距离除以 5 m/s 得到，检测固定 5 s，切换频道额外 1 s；光学清除成功与未命中的固定服务时间分别为 5 s 与 3 s。这意味着“少测一次”“少

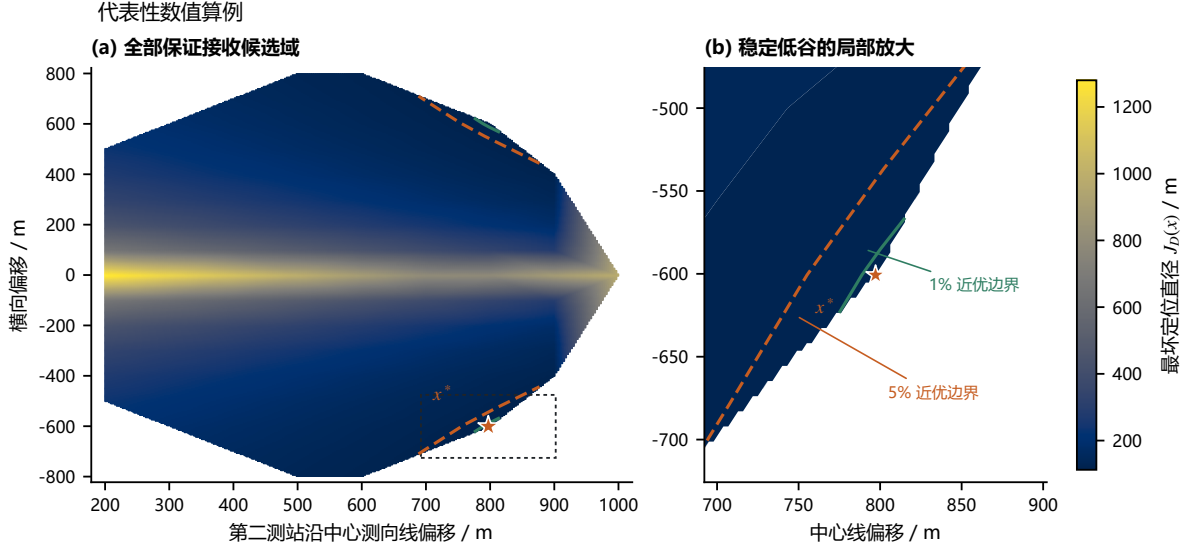


图 5 Q2 min-max 目标的代表性数值算例。颜色表示最坏定位直径  $J_D(x)$ ，星号为网格最优值，等值线给出相对最小值的 1% 与 5% 近优候选区域。

一次未命中清除尝试”与“少走几百米”几种情况必须放到同一时间尺度上进行比较。

## 5.2 七点全域发现布局

全向源有效接收半径至少为 1000 m。采用原点加半径 1130 m 的正六边形六个顶点，共 7 个扫描站。对目标圆半径 1800 m，由六重对称性只需检查一个  $60^\circ$  扇区；最不利边界点与相邻环点的距离上界为

$$\sqrt{1800^2 + 1130^2 - 2 \cdot 1800 \cdot 1130 \cos 30^\circ} \approx 996.95 \text{ m} < 1000 \text{ m}.$$

因此任一全向干扰源至少在一个扫描站处满足距离接收条件，从而完成确定性的全域发现。

## 5.3 单频道硬可行域与负观测几何

对每个已发现频道  $c$  维护硬位置可行域  $\mathcal{F}_c$ 。正常示向产生  $\pm 1^\circ$  楔形并与旧区域求交；由于源是全向的，Q3 的 no\_signal 还可安全利用。若点  $P$  曾正常收到该源，而点  $Q$  无信号，则

$$\|P - g\| \leq \rho < \|Q - g\|,$$

故

$$2(Q - P) \cdot g < \|Q\|^2 - \|P\|^2. \quad (15)$$

式(15)是关于  $g$  的线性半平面约束，可直接裁剪  $\mathcal{F}_c$ 。此外，Q3 的普通 no\_signal 还意味着  $\|Q - g\| > \rho \geq 1000$ ，因而相应测点附近不会被误当作可清除位置。

### Q3

### 全向源的有限视野主动测向与光学清除

状态机 / 决策流程

- 确定性更新（保持可行性 / 正确性）
- 启发式评估（影响效率的决策模块）

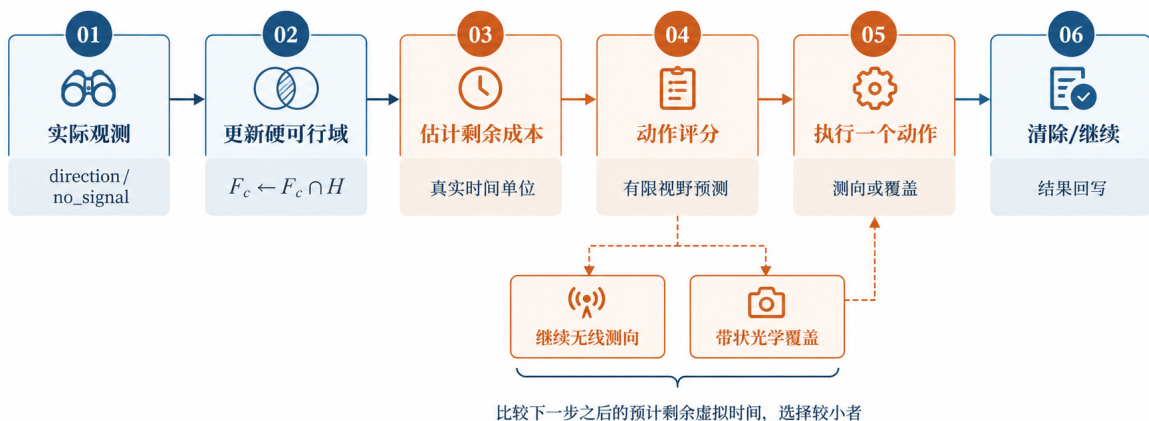


图 6 全向源的有限视野主动测向与光学清除决策流程（机制示意）。蓝色模块表示改变硬可行域的不确定性状态更新，橙色模块仅用于动作评分；效率启发式不会改变位置集合的一致性保证。

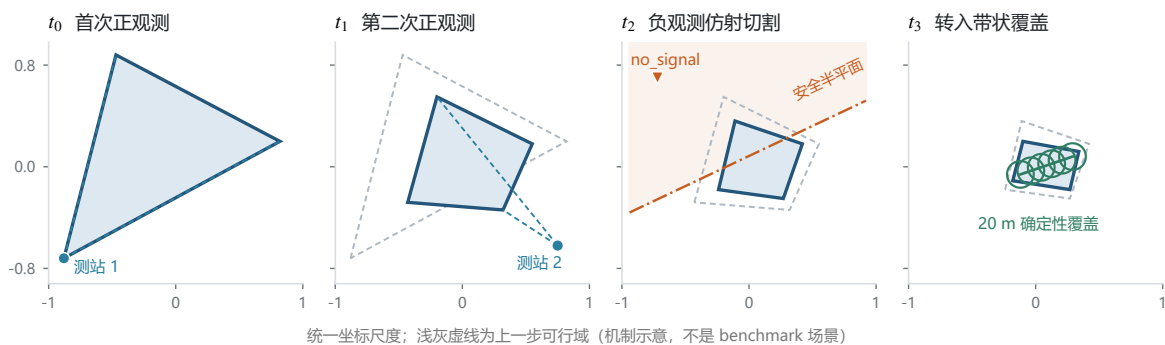


图 7 同一合成源例下 Q3 单频道硬可行域的逐步演化。浅灰轮廓保留上一状态作为参照，深蓝区域为当前硬可行域，橙色菱形表示实际探测点，绿色圆表示最终带状光学覆盖点。

## 5.4 有限视野主动测向

当当前可行域仍较大时，算法从基准候选点、区域质心以及沿最新示向主轴与横向方向构造的候选点中选择下一测点。对可行域沿主直径方向做少量区域求积，形成代表节点，并估计候选动作的正观测/负观测后继状态。动作评分抽象为

$$q^* = \arg \min_q \left[ T_{\text{move}}(q) + t_m + \hat{V}(\mathcal{F}_c | q) \right], \quad (16)$$

其中  $\hat{V}$  只用于估计后续定位与清除成本，并不会修改硬可行域。动作执行后，位置集合仍只按真实接口响应更新。

## 5.5 带状光学覆盖与无线电–光学切换

当  $\mathcal{F}_c$  已收缩为狭长区域时，继续测向未必比直接做光学覆盖更省时。沿多边形主直径建立中心线；若横向半宽小于光学覆盖半径，相邻清除点间距取

$$\Delta = 2\sqrt{r_{\text{cov}}^2 - h^2},$$

其中  $h$  为带状区域半宽， $r_{\text{cov}} < 20 \text{ m}$  用于保留数值裕度。按该间距构造有限的带状光学覆盖序列，并比较

$$\hat{T}_{\text{radio}} \quad \text{与} \quad \hat{T}_{\text{optical}},$$

若  $\hat{T}_{\text{optical}} < \hat{T}_{\text{radio}}$ ，则转入光学覆盖，这样避免为已经足够窄的区域继续付出远距离无线电测量成本。

## 5.6 离线消融与性能归因

在统一的 120 组离线场景中，基线策略与本文最终模型均完成全部源清除。最终模型相较基线策略，按清除源数加权的平均定位清除时间由 299.68 s/源降至 255.96 s/源，下降约 14.6%；平均移动距离由 13764.51 m/局降至 12536.53 m/局，检测数由 131.36 次/局降至 122.06 次/局，未命中清除尝试由 101.78 次/局降至 10.36 次/局。

表 3 Q3 离线模型消融与关键指标

| 方案     | 完整清除    | 加权时间/(s·源 <sup>-1</sup> ) | 单局时间/s  | 移动距离/(m·局 <sup>-1</sup> ) | 检测次数/局 | 未命中清除尝试/局 |
|--------|---------|---------------------------|---------|---------------------------|--------|-----------|
| 基线策略   | 120/120 | 299.68                    | 3895.89 | 13764.51                  | 131.36 | 101.78    |
| 本文最终模型 | 120/120 | 255.96                    | 3327.43 | 12536.53                  | 122.06 | 10.36     |

最终模型平均每局触发有限视野预测 20.95 次、带状光学覆盖 3.675 次。表 3 和图 8 显示，时间下降同时伴随未命中清除、移动距离和测量次数减少，改进并非由单一动作类型造成。

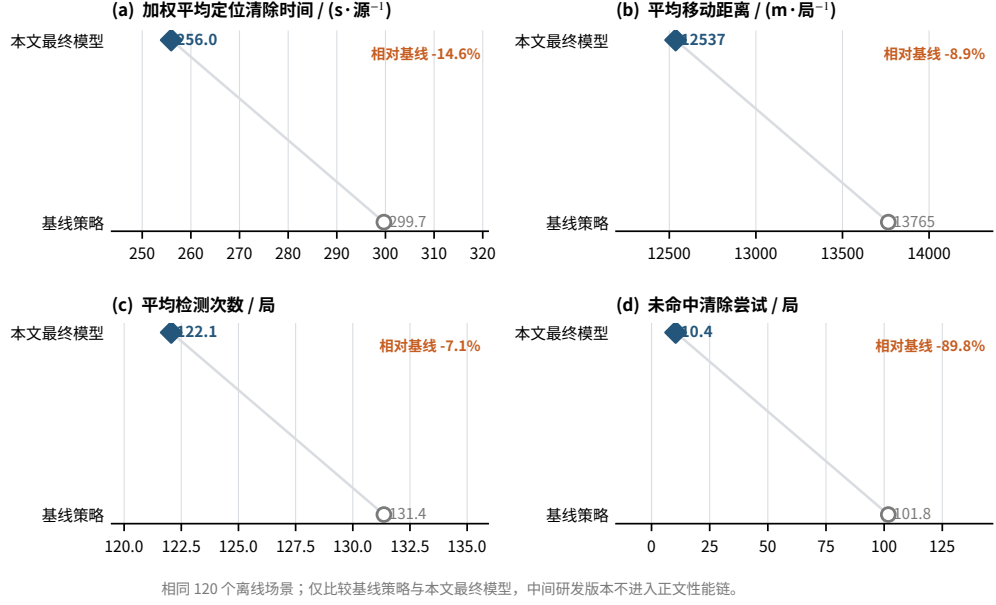


图 8 Q3 基线策略与本文最终模型在相同 120 组离线场景上的对齐比较。四个面板分别报告加权平均定位清除时间、平均移动距离、测量次数和未命中清除尝试数，并标注相对基线的变化幅度。

## 6 问题四：定向干扰下的保守多源滚动调度模型

### 6.1 定向源引入的负观测歧义

Q4 中的定向源只在其发射方向两侧各  $90^\circ$  的半平面扇区内辐射。于是同一个 `no_signal` 可能由三种原因导致：该频道无未清除源、当前点距离超过接收半径、或者当前点位于定向盲区。因而

$$\text{no\_signal} \nRightarrow \|Q - g\| > \rho,$$

Q3 的普通负观测半平面裁剪不能无条件迁移到 Q4。

因此本文在 Q4 对普通 `no_signal` 采用记录而不收缩源位置域的语义；正常 `direction` 与 `near` 仍按题设更新，未命中的 `clear` 不记为已解决。

### 6.2 21 点认证搜索布局

初始扫描采用 21 个站点，以覆盖混合全向/定向源：原点 1 个，外环 12 点，其半径

$$r_o = \frac{1802}{\cos(\pi/12)} \approx 1865.57 \text{ m};$$

内环 8 点，半径  $r_i = 997.32 \text{ m}$ 。独立几何认证给出：外环凸包的內切圆半径为  $1802 \text{ m}$ ，覆盖目标圆；对全部定向边界组合进行 408 次有向检查后，最坏接收距离上界为  $997.321 \text{ m} < 1000 \text{ m}$ 。该布局据此作为在线调度的固定扫描骨架。

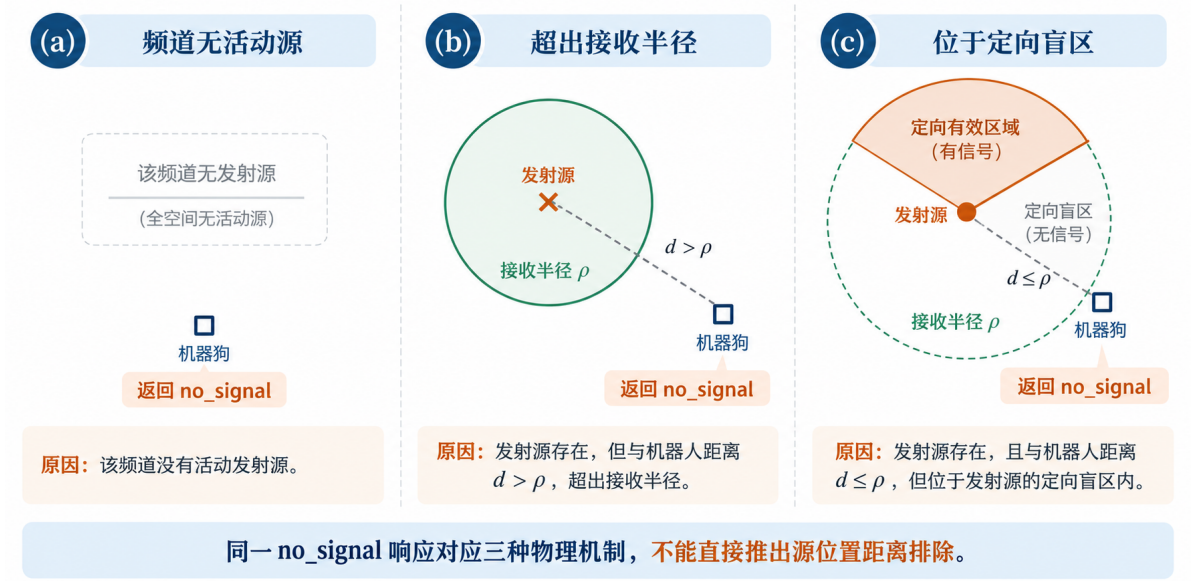


图9 Q4 中同一 no\_signal 响应对应三种不同物理机制：频道无活动源、机器人超出接收半径，或机器人虽在接收半径内但位于定向盲区。因此普通负响应不能直接转化为源位置的距离排除。

### 6.3 局部共享补测

若一个扫描站同时发现多个高不确定度频道，逐频道远距离寻找第二测点会产生重复绕行。算法在当前站点附近约 50m 的圆周上生成有限候选补测点，按估计的几何收益和额外路程选择一个公共位置，再依次补测这些频道。该评分只决定动作位置，普通 Q4 no\_signal 仍不会用于位置裁剪。

### 6.4 SCAN/SOURCE 混合任务池

若采用“扫描全部站点，再逐源定位清除”的串行结构会让已经发现的源等待很久，且往往导致回头路。本文将尚待访问的扫描点与已发现源的下一步服务任务同时放入

$$\mathcal{J}_t = \mathcal{J}_{\text{scan}}(t) \cup \mathcal{J}_{\text{src}}(t). \quad (17)$$

每次只执行规划序列中的**第一个原子动作**；得到实际响应后立即更新源状态、重建任务池，再计算下一条路线：

execute one  $\rightarrow$  observe  $\rightarrow$  update  $\rightarrow$  rebuild  $\rightarrow$  replan.

### 6.5 多初值局部路径改进

对当前任务池的位置节点生成 nearest-neighbor、farthest-insertion、angular order 及反向 angular order 等初始路线，并把上一轮规划作为 warm start。各路线先用 2-opt 去除交叉，再用长度 1–3 的 Or-opt 链做局部搬移。比较后只执行最优候选路线的第一个任务，

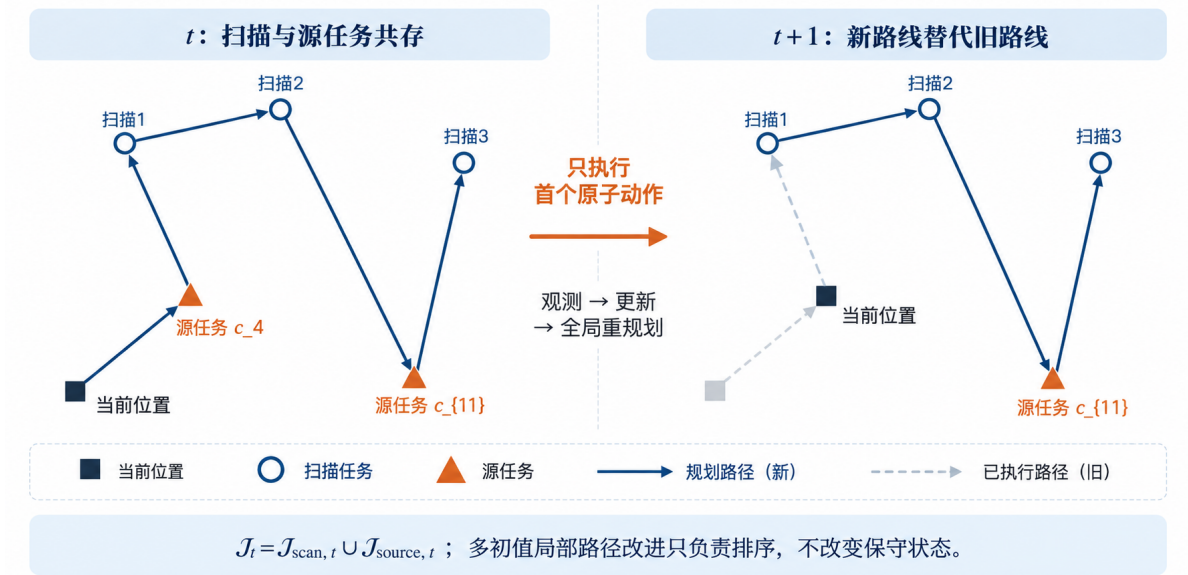


图 10 Q4 的混合滚动调度机制示意：时刻  $t$  同时保留扫描任务与源服务任务，但只执行首个原子动作；新观测到达后立即更新并重规划，使时刻  $t+1$  的新路线替代旧路线。

下一响应到达后重新规划，这样既保留了实时滚动性，又降低了每轮的预计路程。

**证据完整性保护。** 为避免成对探测上下文被无关同频道 `no_signal` 污染，程序将 `generation`、`channel`、`action` 与 `coordinate` 绑定到同一证据上下文，仅允许预定探测位置产生的响应进入对应证据链。三次官方正式测试中该机制未触发成对裁剪，因此这一机制只作为安全保护，不计入性能增益。

## 6.6 离线消融结果

在 100 组统一离线场景中，基线策略、混合滚动调度与本文最终模型均达到 100/100 全清，且离线真值审计中未发现可行域包含违例。加权平均定位清除时间依次为 514.57、477.73 和 472.26 s/源。第一阶段降幅明显大于第二阶段，说明主要收益来自扫描任务与源服务任务的混合调度；多初值局部路径改进在此基础上再减少约 5.47 s/源。

表 4 Q4 100 组离线消融结果

| 方案     | 完整清除    | 加权时间/(s·源 <sup>-1</sup> ) | 单局时间/s  | 移动距离/(m·局 <sup>-1</sup> ) | 检测次数/局 | 未命中清除尝试/局 | 可行域违例 |
|--------|---------|---------------------------|---------|---------------------------|--------|-----------|-------|
| 基线策略   | 100/100 | 514.57                    | 6689.44 | 25801.13                  | 230.50 | 27.91     | 0     |
| 混合滚动调度 | 100/100 | 477.73                    | 6210.47 | 22460.04                  | 263.98 | 26.34     | 0     |
| 本文最终模型 | 100/100 | 472.26                    | 6139.33 | 21994.87                  | 265.46 | 30.35     | 0     |

最终模型的未命中清除尝试数并非最低，但其总时间仍最优，说明 Q4 的主瓶颈是跨区域移动，而不是单次仅增加 3 s 服务时间的未命中清除尝试；这一判断与最终移动距离显著下降相一致。

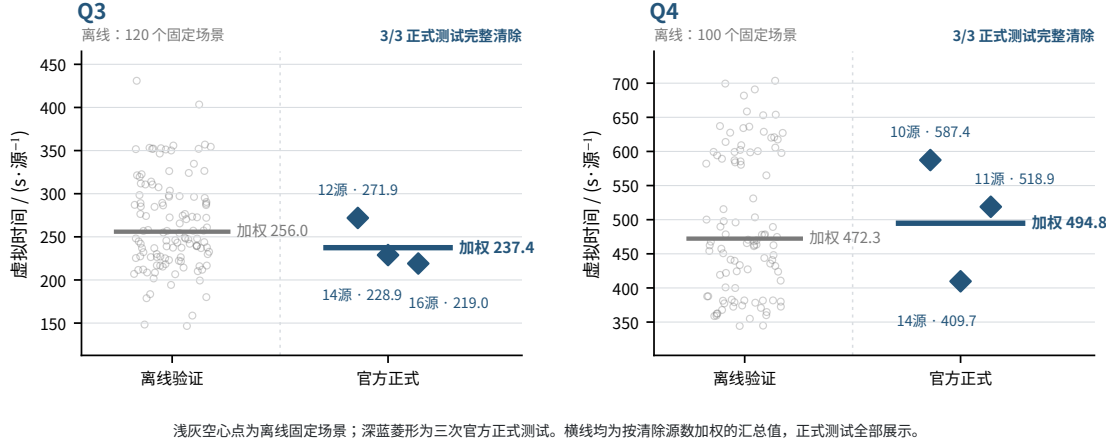


图 11 Q3 与 Q4 的离线验证和官方正式测试两层证据。浅灰空心点展示离线场景分布，深色菱形展示三次正式测试，横线为按清除源数加权的汇总值；正式测试点全部展示。

## 7 模型检验

本题没有用于拟合的连续真值序列，示向误差也由题设直接给出确定性上界，因此不采用 RMSE、MAPE 这类回归误差指标。检验集中在三个可以直接核对的方面：独立实现的一致性、关键几何阈值的扰动裕量，以及跨随机场景和正式测试的执行稳定性。

### 7.1 误差分析与一致性检验

Q1 的直径分别由旋转卡壳和  $O(m^2)$  顶点对穷举计算，直径圆覆盖性则分别由 Thales 判据和最小包围圆检查。1000 组随机凸多边形中，两种直径算法的最大绝对差为 0；两个覆盖判据没有不一致案例，Jung 界也未出现违例。这里报告的是两种独立实现之间的数值差异，而不是另行假定随机噪声。

Q2 的半无限约束求解保留独立复核：主计算用约束生成得到边界，再用更密的离散点检查全部约束的最大违反量。这样可以把边界生成算法与结果检查分开，避免 Q1/Q2 的几何结论只依赖单一数值实现。

### 7.2 灵敏度分析

题设中的 1000 m 最小接收半径、20 m 光学清除半径等常数属于竞赛规则，不适合人为改写后再讨论“参数敏感性”。这里考察模型自身可调量以及确定性布局相对规则阈值的裕量。

Q2 以近优集合  $C(\eta)$  描述第二检测点对站位偏差的容忍度。由定义，若  $x \in C(\eta)$ ，则

$$J_D(x) \leq (1 + \eta)J_D(x^*).$$

因此图 5 中的 1% 与 5% 近优边界分别把最坏定位直径的相对损失限制在 1% 与 5% 以

内；报告一个近优区域比只给单个网格最优点更能反映站位扰动下的稳定性。

Q3、Q4 的全域发现布局则直接检查几何裕量。Q3 七点布局的最坏覆盖距离为 996.95 m，相对最小接收半径 1000 m 留有 3.05 m 裕量；Q4 二十一点布局的认证最坏距离为 997.321 m，对应裕量 2.679 m。两者均满足确定性接收条件，但裕量较小，因此还需要用离线样本和正式测试检验执行层面的稳定性。

表 5 关键参数的灵敏度与几何裕量检查

| 检验对象      | 扰动或阈值             | 检验结果                                        | 含义                 |
|-----------|-------------------|---------------------------------------------|--------------------|
| Q2 第二检测点  | $\eta = 1\%, 5\%$ | $J_D(x) \leq 1.01J_D(x^*)$ 或 $1.05J_D(x^*)$ | 站位偏离最优点时，性能损失有显式上界 |
| Q3 七点发现布局 | 最小接收半径 1000 m     | 最坏距离 996.95 m，裕量 3.05 m                     | 保持确定性全域发现          |
| Q4 二十一点布局 | 最小接收半径 1000 m     | 最坏距离 997.321 m，裕量 2.679 m                   | 满足定向边界组合下的接收条件     |

### 7.3 稳健性检验与证据口径

跨场景结果用于检查算法是否依赖少数有利实例。Q3 的最终模型在 120 组离线场景中全部完成清除，Q4 在 100 组离线场景中同样达到 100/100 全清；Q4 的离线真值审计中可行域违例为 0。正式测试又覆盖了不同源数的随机案例：Q3 为 12、14、16 个源，Q4 为 10、14、11 个源，六局均完整结束，且 HTTP error 与 retry 均为 0。上述结果说明算法在当前测试范围内没有出现由场景变化引起的失效，但正式测试每题只有 3 局，因此不据此做总体分布或显著性推断。

官方演练只用于提交前核验 HTTP 接口、动作串行性和计时规则，记录保留在支撑材料中，不与离线结果或正式测试混合。对正式测试，记第  $i$  局总虚拟时间为  $T_i$ 、清除源数为  $N_i$ ，采用

$$\bar{t}_{\text{weighted}} = \frac{\sum_i T_i}{\sum_i N_i}, \quad \bar{t}_{\text{case}} = \frac{1}{n} \sum_i \frac{T_i}{N_i}.$$

图 11 的横线和正式结果表汇总行均使用  $\bar{t}_{\text{weighted}}$ 。逐局单位源时间再平均时，Q3、Q4 分别为 239.939 s/源和 505.339 s/源；正文不把这一口径与加权汇总值混用。

7.4 官方正式测试结果

表 6 问题三官方正式测试结果

| 测试 | 案例编码                | 清除数 | 虚拟时间/s   | 平均定位清除时间/(s·源 <sup>-1</sup> ) | 程序时间/s |
|----|---------------------|-----|----------|-------------------------------|--------|
| 1  | 5C33-U6RS-2WAE-G8R5 | 12  | 3262.741 | 271.895                       | 1.687  |
| 2  | CV44-VASJ-3VVJ-8PRU | 14  | 3204.233 | 228.874                       | 1.594  |
| 3  | 2XMV-BG6G-ZMS8-NH5T | 16  | 3504.795 | 219.050                       | 2.078  |
| 汇总 | —                   | 42  | 9971.768 | 237.423                       | —      |

三局均完整清除，合计清除 42 个源，总虚拟时间为 9971.768 s，按清除源数加权得到 237.423 s/源；HTTP error 与 retry 均为 0。

表 7 问题四官方正式测试结果

| 测试 | 案例编码                | 清除数 | 虚拟时间/s    | 平均定位清除时间/(s·源 <sup>-1</sup> ) | 程序时间/s |
|----|---------------------|-----|-----------|-------------------------------|--------|
| 1  | 5PW2-F66D-VFKB-Q55H | 10  | 5873.712  | 587.371                       | 4.110  |
| 2  | ACVC-HHUD-XCFM-F2H7 | 14  | 5736.187  | 409.728                       | 5.984  |
| 3  | ZK97-PA48-VUHD-E5UV | 11  | 5708.100  | 518.918                       | 5.250  |
| 汇总 | —                   | 35  | 17317.999 | 494.800                       | —      |

三局均完整清除，合计清除 35 个源，总虚拟时间为 17 317.999 s，按清除源数加权得到 494.800 s/源；HTTP error 与 retry 均为 0。

7.5 时间瓶颈分析

在 Q3 最终离线模型中，平均每局移动距离约 12 536.53 m，对应移动时间约 2507 s，约占总 3327.43 s 的 75%；Q4 最终离线模型平均每局移动距离约 21 994.87 m，对应移动时间约 4399 s，约占总 6139.33 s 的 72%。因此 Q3/Q4 的优化重点分别落在“避免不必要的局部清除/测量”和“降低多源跨区域往返”上，与最终算法结构相吻合。

表 8 Q3 与 Q4 最终离线策略的平均单局虚拟时间构成

| 模型      | 移动/s    | 检测/s    | 切换/s   | 成功清除/s | 未命中清除/s | 单局总时间/s |
|---------|---------|---------|--------|--------|---------|---------|
| Q3 最终模型 | 2507.31 | 610.29  | 113.76 | 65.00  | 31.07   | 3327.43 |
| Q4 最终模型 | 4398.97 | 1327.30 | 257.01 | 65.00  | 91.05   | 6139.33 |

## 8 模型评价与推广

### 8.1 模型优势

1. **与题设误差模型一致。**全文只使用有界误差集合，不额外引入题目噪声分布；硬位置状态具有清晰的集合语义。
2. **正确性与效率分层。**位置可行域由真实观测和确定性几何更新，而有限视野评分、共享补测和路由搜索仅承担效率优化，避免启发式直接破坏完备性。
3. **从定位误差转向任务级目标。**Q2 直接优化最坏定位直径，Q3/Q4 直接围绕总虚拟时间设计动作，目标与竞赛评分函数一致。
4. **具有多层验证。**Q1 有双算法交叉审计，Q3/Q4 均有大样本离线验证与三次官方正式测试；六次正式运行均无 HTTP error 或 retry。

### 8.2 模型不足

1. Q2 的连续 min-max 求解依赖数值约束生成与离散审计；不同首次观测实例需要重新生成可行域和近优集合。
2. Q3 的有限视野价值函数为低维近似，并非全局 Bellman 最优；其优势主要通过大样本消融验证。

### 8.3 推广价值

若用于其他有确定性测量误差界的移动感知任务，可把本文的楔形一致集替换为相应传感器的观测一致集；硬状态仍由真实观测更新，主动选点和滚动路径继续作为效率层。搜救定位、无人巡检和移动传感器部署都可按这一方式改写状态约束，而不必沿用本题的具体测向几何。

## 9 结论

本文围绕“如何在有界测向误差和未知接收半径下快速、完整地定位并清除多个干扰源”建立统一模型。Q1 通过楔形半平面交得到完整定位区域，并以旋转卡壳、Thales 判据与最小包围圆准确评价区域尺度与直径圆覆盖性；Q2 将位置与接收半径联合建模，构造最大保证二次有信号域  $C_{\text{sig}}^*$ ，再以最坏定位直径最小化设计主动第二测点；Q3 在确定性全域发现基础上，引入全向负观测半平面、有限视野主动测向与带状光学覆盖，在 120 组离线场景中全部完成清除；Q4 针对定向源的 no\_signal 歧义采用保守更新，并通过共享补测、SCAN/SOURCE 混合任务池和多初值局部路径改进完成 100 组离线全清。三次官方正式测试中，Q3 共清除 42 个源，加权平均定位清除时间为 237.423 s/源；Q4

共清除 35 个源，对应值为 494.800 s/源。六次正式测试均完整结束，且无 HTTP error 或 retry。

## AI 工具使用声明

本参赛队在竞赛过程中使用了 AI 工具，主要用于语言润色和代码调试，详细使用情况见支撑材料。

## 参考文献

- [1] 全国大学生数学建模竞赛组委会. 2026 年高教社杯全国大学生数学建模竞赛 B 题: 无线电干扰源的快速自动定位与清除, 2026.
- [2] Jaulin L, Walter E. Set inversion via interval analysis for nonlinear bounded-error estimation. *Automatica*, 1993, 29(4): 1053–1064.
- [3] Tokekar P, Isler V. Sensor placement and selection for bearing sensors with bounded uncertainty. *2013 IEEE International Conference on Robotics and Automation*, 2013: 2515–2520.
- [4] Tekdas O, Isler V. Sensor placement for triangulation based localization. *IEEE Transactions on Automation Science and Engineering*, 2010, 7(3): 681–685.
- [5] Toussaint G T. Solving geometric problems with the rotating calipers. *Proc. IEEE MELECON*, 1983.
- [6] Welzl E. Smallest enclosing disks (balls and ellipsoids). In: *New Results and New Trends in Computer Science*, LNCS 555. Springer, 1991: 359–370.
- [7] 姜启源, 谢金星, 叶俊. 数学模型 (第 5 版) [M]. 北京: 高等教育出版社, 2018.
- [8] 周培德. 计算几何——算法设计与分析 (第 2 版) [M]. 北京: 清华大学出版社, 2005.

## A 支撑材料清单与复现入口

本附录逐文件列出随文代码副本并打印全部 Python 源代码。Q3/Q4 文件取自现有脱敏代码归档，逐字节复制；Q1/Q2 仅将输出根目录改为脚本所在目录。此处未重新运行算法或官方测试。Q3 对应 Q3\_BREAKTHROUGH\_R2，Q4 对应 Q4\_SANDBOX\_EXACT\_FULL；ROUTINGPLUS 对应多初值局部路径改进，PAIR\_BOUND 对应成对负观测证据绑定。运行标识通过 `--robot-id` 传入，不提供真实默认值。演练入口分别为 `scripts/run_q3_rehearsal_r2.py` 与 `scripts/run_q4_rehearsal_full.py`。环境依赖见 `requirements.txt`。本附录的入口用于演练，不自动启动任何测试。图表脚本、输入数据与逐文件哈希清单随源文件包提供；正式加密行为日志保持原文件，单独留存，不在此改写。最终支撑材料压缩包另含 AI .pdf、AI .docx、 /README.md 和 .txt；其中 AI 工具使用详情以参赛队最终提供的独立说明文件为准。

| 文件（相对于代码目录）                                                       | 字节   |
|-------------------------------------------------------------------|------|
| README.md                                                         | 687  |
| requirements.txt                                                  | 275  |
| scripts/audit_submission_privacy.py                               | 1218 |
| scripts/requirements.txt                                          | 275  |
| scripts/run_q3_rehearsal_r2.py                                    | 2533 |
| scripts/run_q4_rehearsal_full.py                                  | 1920 |
| src/b_problem/__init__.py                                         | 52   |
| src/b_problem/batch_geometry/__init__.py                          | 79   |
| src/b_problem/batch_geometry/shared_second_bearing.py             | 8796 |
| src/b_problem/benchmark/__init__.py                               | 57   |
| src/b_problem/benchmark/coverage.py                               | 2622 |
| src/b_problem/benchmark/extreme_speed_probe.py                    | 1582 |
| src/b_problem/benchmark/generator.py                              | 2959 |
| src/b_problem/benchmark/metrics.py                                | 3646 |
| src/b_problem/benchmark/offline_arena.py                          | 5686 |
| src/b_problem/benchmark/runner.py                                 | 2189 |
| src/b_problem/benchmark/suite.py                                  | 6495 |
| src/b_problem/benchmark/types.py                                  | 9573 |
| src/b_problem/certificate_experiments/__init__.py                 | 84   |
| src/b_problem/certificate_experiments/q4_21_certificate.py        | 4260 |
| src/b_problem/certificate_experiments/q4_certificate_recycling.py | 2536 |
| src/b_problem/certificate_experiments/q4_mincert.py               | 7036 |
| src/b_problem/certificate_experiments/q4_movement_forensics.py    | 5721 |

| 文件（相对于代码目录）                                                   | 字节    |
|---------------------------------------------------------------|-------|
| src/b_problem/certificate_experiments/q4_radc_replay.py       | 1509  |
| src/b_problem/certificate_experiments/q4_station_selection.py | 10367 |
| src/b_problem/experimental/__init__.py                        | 115   |
| src/b_problem/experimental/q3_breakthrough_port.py            | 7360  |
| src/b_problem/experimental/sandbox_exact_port.py              | 9720  |
| src/b_problem/geometry/__init__.py                            | 930   |
| src/b_problem/geometry/core.py                                | 20982 |
| src/b_problem/localization/__init__.py                        | 624   |
| src/b_problem/localization/q4_all_bearing.py                  | 3466  |
| src/b_problem/localization/q4_pair_certificate.py             | 3097  |
| src/b_problem/localization/q4_pair_orientation.py             | 1765  |
| src/b_problem/localization/robust.py                          | 10596 |
| src/b_problem/simulator/__init__.py                           | 55    |
| src/b_problem/simulator/client.py                             | 13797 |
| src/b_problem/simulator/config.py                             | 2518  |
| src/b_problem/simulator/http_transport.py                     | 4586  |
| src/b_problem/simulator/logger.py                             | 3181  |
| src/b_problem/simulator/mock_server.py                        | 9589  |
| src/b_problem/simulator/protocol.py                           | 2674  |
| src/b_problem/simulator/session.py                            | 6543  |
| src/b_problem/strategy/__init__.py                            | 51    |
| src/b_problem/strategy/q3.py                                  | 29809 |
| src/b_problem/strategy/q34_receding_horizon.py                | 5722  |
| src/b_problem/strategy/q3_extreme.py                          | 15815 |
| src/b_problem/strategy/q3_speed.py                            | 24948 |
| src/b_problem/strategy/q4.py                                  | 28511 |
| src/b_problem/strategy/q4_batcr.py                            | 6474  |
| src/b_problem/strategy/q4_batcr_a.py                          | 11313 |
| src/b_problem/strategy/q4_cert_d.py                           | 16554 |
| src/b_problem/strategy/q4_extreme.py                          | 17966 |
| src/b_problem/strategy/q4_mbfc.py                             | 2377  |
| src/b_problem/strategy/q4_mincert.py                          | 4630  |
| src/b_problem/strategy/q4_mixed_atomic.py                     | 14898 |
| src/b_problem/strategy/q4_region_speed.py                     | 5005  |
| src/b_problem/strategy/q4_routingplus.py                      | 6148  |
| src/b_problem/strategy/q4_smar.py                             | 633   |
| src/b_problem/strategy/q4_static_routing.py                   | 6298  |
| src/b_problem/strategy/q4_telemetry.py                        | 6829  |

| 文件（相对于代码目录）                                           | 字节    |
|-------------------------------------------------------|-------|
| vendor/portable_sandbox/ablation_preview.py           | 4369  |
| vendor/portable_sandbox/ablation_structural.py        | 5334  |
| vendor/portable_sandbox/certify_q4_21.py              | 3008  |
| vendor/portable_sandbox/lib/geom.py                   | 13482 |
| vendor/portable_sandbox/lib/sim.py                    | 5617  |
| vendor/portable_sandbox/lib/strategy.py               | 12889 |
| vendor/portable_sandbox/lib/strategy2.py              | 12228 |
| vendor/portable_sandbox/run_bench.py                  | 2148  |
| vendor/portable_sandbox/run_best.py                   | 2169  |
| vendor/portable_sandbox/run_mincert25_external.py     | 1879  |
| vendor/portable_sandbox/run_q3.py                     | 2059  |
| vendor/portable_sandbox/run_q3_7pt.py                 | 1750  |
| vendor/portable_sandbox/run_q3_failures.py            | 916   |
| vendor/portable_sandbox/run_q3_nextgen.py             | 8273  |
| vendor/portable_sandbox/run_q3_shared.py              | 4894  |
| vendor/portable_sandbox/run_q4_21pt.py                | 1957  |
| vendor/portable_sandbox/run_q4_23pt.py                | 2172  |
| vendor/portable_sandbox/run_q4_nextgen.py             | 11318 |
| vendor/portable_sandbox/run_q4_pairlocal.py           | 6175  |
| vendor/portable_sandbox/run_q4_routing_ablate.py      | 4448  |
| vendor/portable_sandbox/run_q4_routingplus.py         | 6763  |
| vendor/portable_sandbox/run_q4_routingplus_bound.py   | 6779  |
| vendor/portable_sandbox/run_q4_routingplus_enroute.py | 4548  |
| vendor/portable_sandbox/run_q4_routingplus_fast.py    | 7426  |
| vendor/portable_sandbox/run_q4_serviceanchor.py       | 2879  |
| vendor/portable_sandbox/run_q4_shared.py              | 5738  |
| vendor/portable_sandbox/run_q4_shared_pair.py         | 4316  |
| vendor/portable_sandbox/run_q4_shared_pair_bound.py   | 6313  |
| vendor/portable_sandbox/run_q4_shared_richroute.py    | 4490  |
| vendor/portable_sandbox/run_q4_staged.py              | 8238  |
| vendor/portable_sandbox/run_q4_staged_routingplus.py  | 2396  |
| vendor/portable_sandbox/run_rho30.py                  | 1339  |
| vendor/portable_sandbox/run_safe7_filtered.py         | 1615  |
| vendor/portable_sandbox/run_safe7_minr.py             | 1625  |
| vendor/portable_sandbox/run_safe7_r1130.py            | 1615  |
| vendor/portable_sandbox/strict_filtered_test.py       | 4131  |
| vendor/portable_sandbox/strict_q3_mec18.py            | 1308  |
| vendor/portable_sandbox/strict_q3_test.py             | 4349  |
| vendor/portable_sandbox/strict_q4_mec18.py            | 1367  |

| 文件（相对于代码目录）                                                 | 字节    |
|-------------------------------------------------------------|-------|
| vendor/portable_sandbox/strict_q4_mincert25.py              | 3540  |
| vendor/portable_sandbox/strict_q4_sched_ablate.py           | 1992  |
| vendor/q3_breakthrough_sandbox/lib/geom.py                  | 13482 |
| vendor/q3_breakthrough_sandbox/lib/sim.py                   | 5617  |
| vendor/q3_breakthrough_sandbox/lib/strategy.py              | 12889 |
| vendor/q3_breakthrough_sandbox/lib/strategy2.py             | 12228 |
| vendor/q3_breakthrough_sandbox/q3_breakthrough.py           | 11936 |
| vendor/q3_breakthrough_sandbox/reference/run_q3_7pt.py      | 1750  |
| vendor/q3_breakthrough_sandbox/reference/run_q3_nextgen.py  | 8280  |
| py                                                          |       |
| vendor/q3_breakthrough_sandbox/reference/run_q3_shared.py   | 4901  |
| vendor/q3_breakthrough_sandbox/reference/run_safe7_r1130.py | 1622  |
| py                                                          |       |
| vendor/q3_breakthrough_sandbox/reference/strict_q3_mec18.py | 1315  |
| py                                                          |       |
| vendor/q3_breakthrough_sandbox/run_q3_nextgen.py            | 8273  |
| vendor/q3_breakthrough_sandbox/run_safe7_r1130.py           | 1615  |
| vendor/q3_breakthrough_sandbox/strict_filtered_test.py      | 4131  |
| vendor/q3_breakthrough_sandbox/strict_q3_test.py            | 4349  |
| q1q2/generate_fast.py                                       | 9375  |
| q1q2/generate_q1q2_artifacts.py                             | 14555 |

## B 完整源程序

文件: scripts/audit\_submission\_privacy.py

```

"""Read-only privacy gate for the code submission directory."""
from __future__ import annotations
import argparse, re
from pathlib import Path

TEXT={" .py", ".json", ".yaml", ".yml", ".toml", ".ini", ".cfg", ".md", ".txt", ".bat", ".cmd", ".ps1", ".sh", ".ipynb"}
PATTERNS={"quoted 12-digit identifier":re.compile(r"['\"]d{12}['\"]"), "robot identifier
↪ assignment":re.compile(r"robot[_-]?id\s*[:=].{0,8}\d", re.I), "Windows absolute
↪ path":re.compile(r"[A-Za-z]:\\(?:Users|WSN)\\\\"), "plaintext rehearsal
↪ log":re.compile(r"(?:rehearsal|practice).*\.(?:jsonl|log)$", re.I)}
def main():
    p=argparse.ArgumentParser();p.add_argument("directory",nargs="?",default=".");a=p.parse_args();root=Path|
↪ (a.directory);issues=[]
    for f in root.rglob("*"):
        if not f.is_file():continue
        if f.suffix.lower()==" .jlog":continue
        if f.name==" .env" or f.suffix.lower()==" .jsonl":issues.append((f,"forbidden runtime/log
↪ artifact"));continue

```

```

    if f.suffix.lower() not in TEXT:continue
    text=f.read_text(encoding="utf-8",errors="replace")
    for label,rx in PATTERNS.items():
        if rx.search(text):issues.append((f,label))
if issues:
    for f,reason in issues:print(f"FAIL {f}: {reason}")
    raise SystemExit(1)
print("PASS")
if __name__=="__main__":main()

```

## 文件: scripts/run\_q3\_rehearsal\_r2.py

```

"""Run Q3_BREAKTHROUGH_R2 only after an explicit human rehearsal confirmation."""
from __future__ import annotations

import argparse
from dataclasses import asdict
from datetime import datetime
import json
from pathlib import Path
import subprocess
import sys
from urllib.parse import urlparse

ROOT = Path(__file__).resolve().parents[1]
sys.path.insert(0, str(ROOT / "src"))
from b_problem.experimental.q3_breakthrough_port import execute_q3_breakthrough_official
from b_problem.simulator.config import PRACTICE_READY_TOKEN, SimulatorConfig, SimulatorMode
from b_problem.simulator.session import run_validated_session

ALGORITHM = "Q3_BREAKTHROUGH_R2"

def _port(value: str) -> int:
    parsed = urlparse(value)
    if parsed.scheme != "http" or parsed.hostname != "127.0.0.1" or parsed.path not in {"", "/" } or
    ↪ parsed.query or parsed.fragment or parsed.port is None:
        raise argparse.ArgumentTypeError("base URL must be exactly http://127.0.0.1:<port>")
    return parsed.port

def main() -> None:
    parser = argparse.ArgumentParser(description=__doc__)
    parser.add_argument("--robot-id", required=True)
    parser.add_argument("--base-url", type=_port, default=2026, help="loopback simulator URL")
    parser.add_argument("--confirm-q3-rehearsal", action="store_true")
    parser.add_argument("--run-index", type=int, choices=(1, 2, 3), default=1)
    args = parser.parse_args()
    sha = subprocess.check_output(["powershell", "-NoProfile", "-File", str(ROOT / "scripts" /
    ↪ "cumcm-git.ps1"), "rev-parse", "HEAD"], text=True).strip()
    print("THIS RUNNER IS FOR Q3 REHEARSAL ONLY")
    print(f"STRATEGY = {ALGORITHM}")
    print(f"git_sha = {sha}")
    print("NO REQUEST HAS BEEN SENT")
    if not args.confirm_q3_rehearsal:
        parser.error("--confirm-q3-rehearsal is required before /enter")

```

```

config = SimulatorConfig(mode=SimulatorMode.PRACTICE_Q3, robot_id=args.robot_id, port=args.base_url,
    ↪ ready_signal=PRACTICE_READY_TOKEN)
stamp = datetime.now().strftime("%Y%m%dT%H%M%S")
out = ROOT / "outputs" / "q3_rehearsal_20260912"
log = out / f"q3_rehearsal_{args.run_index:02d}_{stamp}.jsonl"
summary = out / f"q3_rehearsal_{args.run_index:02d}_{stamp}.json"
result = run_validated_session(config, log_path=log, summary_path=summary,
    q3_strategy=lambda robot, _: execute_q3_breakthrough_official(robot))
print(json.dumps(asdict(result), ensure_ascii=False, default=str))
print(f"log_path = {log}")
print(f"summary_path = {summary}")

if __name__ == "__main__": main()

```

文件: scripts/run\_q4\_rehearsal\_full.py

```

"""Q4 SandboxExact FULL / PAIR_BOUND rehearsal runner only."""
from __future__ import annotations
import argparse, json, sys
from dataclasses import asdict
from datetime import datetime
from pathlib import Path
from urllib.parse import urlparse
ROOT=Path(__file__).resolve().parents[1];sys.path.insert(0,str(ROOT/'src'))
from b_problem.experimental.sandbox_exact_port import execute_q4_sandbox_exact_official
from b_problem.simulator.config import PRACTICE_READY_TOKEN, SimulatorConfig, SimulatorMode
from b_problem.simulator.session import run_validated_session
def port(v:str)->int:
    p=urlparse(v)
    if p.scheme!='http' or p.hostname!='127.0.0.1' or p.path not in {'','/'} or p.port is None: raise
    ↪ argparse.ArgumentTypeError('base URL must be http://127.0.0.1:<port>')
    return p.port
def main():
    a=argparse.ArgumentParser();a.add_argument('--robot-id',required=True);a.add_argument('--base-url',type=
    ↪ port,default=2026);a.add_argument('--confirm-q4-rehearsal',action='store_true');a.add_argument('--ru
    ↪ n-index',type=int,choices=(1,2,3),default=1);x=a.parse_args()
    print('THIS RUNNER IS FOR Q4 REHEARSAL ONLY');print('STRATEGY = Q4_SANDBOX_EXACT_FULL');print('PAIR =
    ↪ BOUND');print('ROUTING = ROUTINGPLUS');print('NO REQUEST HAS BEEN SENT')
    if not x.confirm_q4_rehearsal:a.error('--confirm-q4-rehearsal is required before /enter')
    c=SimulatorConfig(mode=SimulatorMode.PRACTICE_Q4,robot_id=x.robot_id,port=x.base_url,ready_signal=PRACTI
    ↪ CE_READY_TOKEN);d=ROOT/'outputs'/'q4_rehearsal_20260912';stamp=datetime.now().strftime('%Y%m%dT%H%M%
    ↪ S');log=d/f'q4_rehearsal_{x.run_index:02d}_{stamp}.jsonl';summary=d/f'q4_rehearsal_{x.run_index:02d}
    ↪ _{stamp}.json'
    r=run_validated_session(c,log_path=log,summary_path=summary,q4_strategy=lambda robot, _:execute_q4_sandbo
    ↪ x_exact_official(robot));print(json.dumps(asdict(r),ensure_ascii=False));print(f'log_path =
    ↪ {log}');print(f'summary_path = {summary}')
if __name__=='__main__':main()

```

文件: src/b\_problem/\_\_init\_\_.py

```

"""Models and offline tools for CUMCM problem B."""

```

文件: src/b\_problem/batch\_geometry/\_\_init\_\_.py

```
"""Standalone geometry helpers for batched observations; not strategy code."""
```

文件: src/b\_problem/batch\_geometry/shared\_second\_bearing.py

```
"""Deterministic geometry ranking for one shared second-bearing location ``B``.
```

```
This is intentionally a small 2-D proxy, not a Fisher-information model. A
candidate ``B`` is useful for anchor ``A`` when the displacement ``B-A`` has a
large component perpendicular to A's first bearing. The score uses
``|sin(delta)|`` and its metre lever ``|B-A| |sin(delta)|``; both are zero for
a parallel second observation and improve before any target-dependent second
bearing is available.
```

```
"""
```

```
from __future__ import annotations
```

```
from dataclasses import dataclass
```

```
from math import atan2, cos, degrees, hypot, radians, sin
```

```
from typing import Iterable
```

```
_OFFSETS_DEG = (-90.0, -60.0, -45.0, 45.0, 60.0, 90.0)
```

```
@dataclass(frozen=True, order=True)
```

```
class BatchAnchor:
```

```
    channel: int
```

```
    bearing_deg: float
```

```
    x: float
```

```
    y: float
```

```
@dataclass(frozen=True)
```

```
class BearingCluster:
```

```
    anchors: tuple[BatchAnchor, ...]
```

```
    mean_bearing_deg: float
```

```
    mean_x: float
```

```
    mean_y: float
```

```
@dataclass(frozen=True)
```

```
class CandidateBScore:
```

```
    x: float
```

```
    y: float
```

```
    heading_deg: float
```

```
    nominal_distance_m: float
```

```
    cluster_channels: tuple[int, ...]
```

```
    served_channels: tuple[int, ...]
```

```
    nonparallel_pair_count: int
```

```
    minimum_cross_range_sine: float
```

```
    mean_cross_range_sine: float
```

```
    minimum_cross_range_lever_m: float
```

```
    mean_cross_range_lever_m: float
```

```
def _normalize(angle_deg: float) -> float:
```

```
    return angle_deg % 360.0
```

```

def _circular_gap(first_deg: float, second_deg: float) -> float:
    return abs((second_deg - first_deg + 180.0) % 360.0 - 180.0)

def _circular_mean(anchors: Iterable[BatchAnchor]) -> float:
    values = tuple(anchors)
    if not values:
        raise ValueError("a circular mean needs at least one anchor")
    sine = sum(sin(radians(anchor.bearing_deg)) for anchor in values)
    cosine = sum(cos(radians(anchor.bearing_deg)) for anchor in values)
    return _normalize(degrees(atan2(sine, cosine)))

def cluster_circular_bearings(anchors: Iterable[BatchAnchor], *, max_gap_deg: float = 20.0) ->
↳ tuple[BearingCluster, ...]:
    """Cluster circular bearings, explicitly merging the 360°/0° boundary."""
    if not 0.0 < max_gap_deg < 180.0:
        raise ValueError("max_gap_deg must be in (0, 180)")
    ordered = sorted(tuple(anchors), key=lambda anchor: (_normalize(anchor.bearing_deg), anchor.channel,
↳ anchor.x, anchor.y))
    if not ordered:
        return ()
    groups: list[list[BatchAnchor]] = [[ordered[0]]]
    for previous, anchor in zip(ordered, ordered[1:]):
        if _circular_gap(previous.bearing_deg, anchor.bearing_deg) <= max_gap_deg:
            groups[-1].append(anchor)
        else:
            groups.append([anchor])
    if len(groups) > 1 and _circular_gap(ordered[-1].bearing_deg, ordered[0].bearing_deg) <= max_gap_deg:
        groups[0] = groups[-1] + groups[0]
        groups.pop()
    clusters = []
    for group in groups:
        members = tuple(sorted(group, key=lambda anchor: (anchor.channel, anchor.x, anchor.y,
↳ anchor.bearing_deg)))
        clusters.append(BearingCluster(
            members, _circular_mean(members),
            sum(anchor.x for anchor in members) / len(members),
            sum(anchor.y for anchor in members) / len(members),
        ))
    return tuple(sorted(clusters, key=lambda cluster: tuple(anchor.channel for anchor in cluster.anchors)))

def second_bearing_condition_proxy(anchor: BatchAnchor, *, x: float, y: float) -> tuple[float, float]:
    """Return (angular separation sine, cross-range lever metres) for ``A→B``."""
    dx, dy = x - anchor.x, y - anchor.y
    baseline = hypot(dx, dy)
    if baseline <= 1e-9:
        return 0.0, 0.0
    heading = degrees(atan2(dy, dx))
    separation_sine = abs(sin(radians(_circular_gap(anchor.bearing_deg, heading))))
    return separation_sine, baseline * separation_sine

def forward_ray_intersection(
    anchor: BatchAnchor, *, second_x: float, second_y: float, second_bearing_deg: float
) -> tuple[float, float] | None:

```

```

"""Intersect two forward bearing rays, rejecting parallel or backward crossings.

This is only a light, target-independent 2-D geometry helper for a future
caller that has obtained the second bearing; it performs no optimisation.
"""
first_x, first_y = cos(radians(anchor.bearing_deg)), sin(radians(anchor.bearing_deg))
second_x_dir, second_y_dir = cos(radians(second_bearing_deg)), sin(radians(second_bearing_deg))
determinant = first_x * second_y_dir - first_y * second_x_dir
if abs(determinant) <= 1e-9:
    return None
delta_x, delta_y = second_x - anchor.x, second_y - anchor.y
first_distance = (delta_x * second_y_dir - delta_y * second_x_dir) / determinant
second_distance = (delta_x * first_y - delta_y * first_x) / determinant
if first_distance < -1e-9 or second_distance < -1e-9:
    return None
return anchor.x + first_distance * first_x, anchor.y + first_distance * first_y

def _nonparallel_pair_count(anchors: tuple[BatchAnchor, ...], threshold_deg: float) -> int:
    return sum(
        _circular_gap(first.bearing_deg, second.bearing_deg) >= threshold_deg
        for index, first in enumerate(anchors)
        for second in anchors[index + 1:]
    )

def _score_candidate(
    cluster: BearingCluster,
    anchors: tuple[BatchAnchor, ...],
    *,
    heading_deg: float,
    distance_m: float,
    minimum_cross_range_sine: float,
    nonparallel_deg: float,
) -> CandidateBScore:
    x = cluster.mean_x + distance_m * cos(radians(heading_deg))
    y = cluster.mean_y + distance_m * sin(radians(heading_deg))
    served: list[BatchAnchor] = []
    sines: list[float] = []
    levers: list[float] = []
    for anchor in anchors:
        separation_sine, lever = second_bearing_condition_proxy(anchor, x=x, y=y)
        if separation_sine >= minimum_cross_range_sine:
            served.append(anchor)
            sines.append(separation_sine)
            levers.append(lever)
    return CandidateBScore(
        x, y, _normalize(heading_deg), distance_m,
        tuple(anchor.channel for anchor in cluster.anchors), tuple(anchor.channel for anchor in served),
        _nonparallel_pair_count(tuple(served), nonparallel_deg),
        min(sines, default=0.0), sum(sines) / len(sines) if sines else 0.0,
        min(levers, default=0.0), sum(levers) / len(levers) if levers else 0.0,
    )

def score_batch_candidates(
    anchors: Iterable[BatchAnchor],
    *,
    route_directions_deg: Iterable[float] = (),

```

```

distances_m: Iterable[float] = (100.0, 200.0, 300.0, 400.0, 500.0),
max_gap_deg: float = 20.0,
minimum_cross_range_sine: float = 0.25,
nonparallel_deg: float = 5.0,
) -> tuple[CandidateBScore, ...]:
    """Score only the requested cross-range and route-aligned B candidates."""
    anchor_tuple = tuple(sorted(anchors, key=lambda anchor: (anchor.channel, anchor.x, anchor.y,
↪ anchor.bearing_deg)))
    if not anchor_tuple:
        return ()
    distances = tuple(float(distance) for distance in distances_m)
    if not distances or any(distance < 100.0 or distance > 500.0 for distance in distances):
        raise ValueError("B distances must stay in the requested [100, 500] m range")
    if not 0.0 < minimum_cross_range_sine <= 1.0 or not 0.0 < nonparallel_deg < 180.0:
        raise ValueError("invalid geometry thresholds")
    route_headings = tuple(_normalize(float(heading)) for heading in route_directions_deg)
    candidates: list[CandidateBScore] = []
    for cluster in cluster_circular_bearings(anchor_tuple, max_gap_deg=max_gap_deg):
        headings = {_normalize(cluster.mean_bearing_deg + offset) for offset in _OFFSETS_DEG}
        headings.update(route_headings)
        for heading in sorted(headings):
            for distance in sorted(set(distances)):
                candidates.append(_score_candidate(
                    cluster, anchor_tuple, heading_deg=heading, distance_m=distance,
                    minimum_cross_range_sine=minimum_cross_range_sine, nonparallel_deg=nonparallel_deg,
                ))
    return tuple(candidates)

def best_shared_second_bearing(anchors: Iterable[BatchAnchor], **kwargs: object) -> CandidateBScore:
    """Select the deterministic best B, prioritising shared nonparallel service."""
    candidates = score_batch_candidates(anchors, **kwargs)
    if not candidates:
        raise ValueError("at least one anchor is required")
    return max(
        candidates,
        key=lambda candidate: (
            len(candidate.served_channels), candidate.nonparallel_pair_count,
            candidate.minimum_cross_range_lever_m, candidate.mean_cross_range_lever_m,
            candidate.minimum_cross_range_sine, candidate.mean_cross_range_sine,
            -candidate.nominal_distance_m, -candidate.heading_deg, -candidate.x, -candidate.y,
        ),
    )

```

文件: src/b\_problem/benchmark/\_\_init\_\_.py

```

"""Pure offline hidden-truth benchmark for problem B."""

```

文件: src/b\_problem/benchmark/coverage.py

```

from __future__ import annotations

from .types import Case

```

```

def coverage_certificate(cases: tuple[Case, ...], *, holdout_seeds: set[int], tuning_seeds: set[int]) ->
↳ dict:
    observed = {
        "counts": sorted({case.spec.target_count for case in cases}),
        "questions": sorted({case.spec.question for case in cases}),
        "spatial": sorted({case.spec.spatial for case in cases}),
        "radius": sorted({case.spec.radius_mode for case in cases}),
        "errors": sorted({case.spec.error_mode for case in cases}),
        "orientation": sorted({case.spec.orientation_mode for case in cases}),
    }
    required = {
        "questions": {"Q3", "Q4"}, "counts": {10, 13, 16},
        "spatial": {"uniform", "boundary", "cluster", "separated", "grid_boundary"},
        "radius": {"r1000", "r1500", "mixed"},
        "errors": {"zero", "plus1", "minus1", "fixed_hash"},
        "orientation": {"none", "random", "adversarial", "boundary"},
    }
    missing = {key: sorted(values - set(observed[key])) for key, values in required.items()}
    complete = all(not values for values in missing.values())
    disjoint = holdout_seeds.isdisjoint(tuning_seeds)
    expected_combinations = {
        (question, count, spatial, radius, error, orientation)
        for question, orientations in (("Q3", ("none",)), ("Q4", ("random", "adversarial", "boundary")))
        for count in required["counts"]
        for spatial in required["spatial"]
        for radius in required["radius"]
        for error in required["errors"]
        for orientation in orientations
    }
    observed_combinations = {
        (case.spec.question, case.spec.target_count, case.spec.spatial, case.spec.radius_mode,
         case.spec.error_mode, case.spec.orientation_mode)
        for case in cases
    }
    missing_combinations = sorted(expected_combinations - observed_combinations)
    split_bound = all(
        (case.spec.split == "holdout" and case.spec.seed in holdout_seeds)
        or (case.spec.split == "tuning" and case.spec.seed in tuning_seeds)
        for case in cases
    )
    unique_cases = len({case.truth_digest for case in cases}) == len(cases)
    return {"case_ids": [case.truth_digest for case in cases], "observed": observed,
            "missing": missing, "holdout_disjoint": disjoint,
            "observed_combinations": sorted(observed_combinations),
            "missing_combinations": missing_combinations,
            "split_bound": split_bound, "unique_cases": unique_cases,
            "pass": bool(cases) and disjoint and split_bound and unique_cases and complete and not
↳ missing_combinations}

```

文件: src/b\_problem/benchmark/extreme\_speed\_probe.py

```

"""Offline-only parameter probes; they never modify or invoke production strategies."""
from __future__ import annotations

from ..strategy import q3

```

```

def evenly_spaced_indices(population_size: int, sample_size: int) -> tuple[int, ...]:
    """Choose deterministic coverage across an immutable offline-suite ordering."""
    if not 1 <= sample_size <= population_size:
        raise ValueError("sample_size must be in 1..population_size")
    if sample_size == 1:
        return (0,)
    return tuple(round(index * (population_size - 1) / (sample_size - 1)) for index in range(sample_size))

def rank_q3_ring_variants(ring_radii_m: tuple[float, ...]) -> list[dict[str, float | bool]]:
    """Return certified Q3 survey variants ordered by their analytic survey cost."""
    ranked: list[dict[str, float | bool]] = []
    for ring_radius_m in ring_radii_m:
        plan = q3.build_center_ring_plan(ring_radius=ring_radius_m)
        coverage_proved = plan.coverage_certificate.worst_nearest_distance <=
        ↪ plan.coverage_certificate.receive_lower_bound
        if not coverage_proved:
            continue
        actions = q3.snake_scan_plan(plan)
        cost = q3.survey_cost(plan, actions)
        ranked.append({
            "ring_radius_m": ring_radius_m,
            "coverage_proved": coverage_proved,
            "survey_virtual_seconds": cost.virtual_seconds,
            "worst_nearest_distance_m": plan.coverage_certificate.worst_nearest_distance,
        })
    return sorted(ranked, key=lambda item: (float(item["survey_virtual_seconds"]),
    ↪ float(item["ring_radius_m"])))

```

文件: src/b\_problem/benchmark/generator.py

```

from __future__ import annotations

import hashlib
import math
import random

from .types import Case, Point, ScenarioSpec, Target

def build_case(spec: ScenarioSpec, targets: tuple[Target, ...] | None = None) -> Case:
    if not isinstance(spec, ScenarioSpec):
        raise ValueError("spec must be a ScenarioSpec")
    generated = tuple(targets) if targets is not None else _generate_targets(spec)
    digest_source = _digest_repr(spec, generated).encode("utf-8")
    return Case(spec, generated, hashlib.sha256(digest_source).hexdigest())

def _digest_repr(spec: ScenarioSpec, targets: tuple[Target, ...]) -> str:
    """Keep pre-extension frozen case IDs stable while binding explicit channel IDs."""
    if spec.channel_ids is None:
        spec_repr = (
            f"ScenarioSpec(question={spec.question!r}, target_count={spec.target_count!r}, "
            f"spatial={spec.spatial!r}, radius_mode={spec.radius_mode!r}, "
            f"error_mode={spec.error_mode!r}, orientation_mode={spec.orientation_mode!r}, "
            f"seed={spec.seed!r}, split={spec.split!r})"
        )

```

```

        return f"({spec_repr}, {targets!r})"
    return repr((spec, targets))

def _generate_targets(spec: ScenarioSpec) -> tuple[Target, ...]:
    rng = random.Random(spec.seed)
    result = []
    channels = spec.channel_ids or tuple(range(1, spec.target_count + 1))
    for index, channel in enumerate(channels):
        point = _point(spec.spatial, index, spec.target_count, rng)
        radius = {"r1000": 1000.0, "r1500": 1500.0, "mixed": 1000.0 if index % 2 == 0 else
        ↪ 1500.0}[spec.radius_mode]
        directional = spec.question == "Q4" and index % 2 == 0
        kind = "directional" if directional else "omni"
        bearing = None if kind == "omni" else _bearing(spec.orientation_mode, index, rng)
        result.append(Target(channel, point, radius, kind, bearing))
    return tuple(result)

def _point(family: str, index: int, count: int, rng: random.Random) -> Point:
    angle = 2 * math.pi * index / count
    if family == "boundary":
        # Keep floating point radial reconstruction safely inside the closed disk.
        radius = 1800.0 - 1e-9
    elif family == "cluster":
        radius = 80.0 + 20.0 * (index % 3)
    elif family == "separated":
        radius = 1500.0
    elif family == "grid_boundary":
        x = min(1800.0, max(-1800.0, -1800.0 + 3600.0 * index / max(count - 1, 1)))
        return Point(x, 0.0)
    else: # uniform disk
        angle = rng.random() * 2 * math.pi
        radius = 1800.0 * math.sqrt(rng.random())
    return Point(radius * math.cos(angle), radius * math.sin(angle))

def _bearing(mode: str, index: int, rng: random.Random) -> float:
    if mode == "boundary":
        return 90.0 if index % 2 == 0 else 270.0
    if mode == "adversarial":
        return 180.0
    if mode == "random":
        return rng.random() * 360.0
    raise ValueError("directional targets need an explicit orientation mode")

```

文件: src/b\_problem/benchmark/metrics.py

```

from __future__ import annotations

import math

from .types import Case, MeasurementObservation, Metrics, Point, Trace

def _distance(first: Point, second: Point) -> float:
    return math.hypot(first.x - second.x, first.y - second.y)

```

```

def _validate_trace_identity(case: Case, trace: tuple[Trace, ...]) -> None:
    position = Point(0.0, 0.0)
    measure_channel = 1
    cleared: set[int] = set()
    targets = {target.channel: target for target in case.targets}
    for event in trace:
        movement = _distance(position, event.action.position)
        if not math.isclose(event.movement_m, movement, rel_tol=0.0, abs_tol=1e-9):
            raise ValueError("trace movement does not match action positions")
        if event.action.kind == "measure":
            if not isinstance(event.result, MeasurementObservation):
                raise ValueError("measure trace has no observation")
            switched = event.action.channel != measure_channel
            expected = movement / 5.0 + (1.0 if switched else 0.0) + 5.0
            if event.switched is not switched:
                raise ValueError("trace switch flag does not match channel state")
            measure_channel = event.action.channel
        else:
            if event.switched:
                raise ValueError("clear action cannot switch measurement channel")
            if event.result == "success":
                target = targets.get(event.action.channel)
                if target is None or event.action.channel in cleared or _distance(event.action.position,
↪ target.position) > 20.0:
                    raise ValueError("trace claims an impossible clear success")
                cleared.add(event.action.channel)
                expected = movement / 5.0 + 5.0
            else:
                expected = movement / 5.0 + 3.0
        if not math.isclose(event.delta_s, expected, rel_tol=0.0, abs_tol=1e-9):
            raise ValueError("trace duration does not match action mechanics")
    position = event.action.position

def score_trace(case: Case, trace: tuple[Trace, ...]) -> Metrics:
    _validate_trace_identity(case, trace)
    measures = [event for event in trace if event.action.kind == "measure"]
    clears = [event for event in trace if event.action.kind == "clear"]
    successes = [event for event in clears if event.result == "success"]
    failed = [event for event in clears if event.result != "success"]
    movement = sum(event.movement_m for event in trace)
    switches = sum(event.switched for event in measures)
    total = sum(event.delta_s for event in trace)
    return Metrics(len(successes) / len(case.targets), total,
                    total / len(successes) if successes else None, movement, len(measures),
                    switches, len(clears), len(failed), len(successes))

def aggregate(items: tuple[Metrics, ...]) -> dict:
    def summary(values: list[float], lower_is_worse: bool = False) -> dict:
        if not values:
            return {"p50": None, "p90": None, "worst": None}
        ordered = sorted(values)
        percentile = lambda p: ordered[math.ceil(p * len(ordered)) - 1]
        return {"p50": percentile(.5), "p90": percentile(.9),
                "worst": min(values) if lower_is_worse else max(values)}

```

```

return {
    "clear_rate": summary([item.clear_rate for item in items], True),
    "total_virtual_time_s": summary([item.total_virtual_time_s for item in items]),
    "mean_clear_time_s": summary([item.mean_clear_time_s for item in items if item.mean_clear_time_s
↪ is not None]),
    "movement_distance_m": summary([item.movement_distance_m for item in items]),
}

```

文件: src/b\_problem/benchmark/offline\_arena.py

```

from __future__ import annotations

import math
from dataclasses import replace
import time
from typing import Callable, Protocol

from .types import Action, Case, MeasurementObservation, Point, PublicStateView, Trace

class Strategy(Protocol):
    def run(self, arena: "OfflineArena") -> None: ...

class OfflineArena:
    """Deterministic arena for cooperative strategies, not a security boundary."""

    def __init__(self, case: Case, *, max_virtual_duration_s: float = 360_000.0,
                 max_real_duration_s: float = 1_200.0, max_actions: int = 100_000,
                 clock: Callable[[], float] = time.monotonic) -> None:
        if not isinstance(case, Case):
            raise ValueError("case must be a Case")
        if not all(isinstance(value, (int, float)) and not isinstance(value, bool) and
↪ math.isfinite(value) and value >= 0.0
                    for value in (max_virtual_duration_s, max_real_duration_s)):
            raise ValueError("time budgets must be finite and non-negative")
        if isinstance(max_actions, bool) or not isinstance(max_actions, int) or max_actions < 1:
            raise ValueError("max_actions must be a positive integer")
        self._case = case
        self._cleared: set[int] = set()
        self._state = PublicStateView(Point(0.0, 0.0), 1, 0.0)
        self._trace: list[Trace] = []
        self._max_virtual_duration_s = float(max_virtual_duration_s)
        self._max_real_duration_s = float(max_real_duration_s)
        self._max_actions = max_actions
        self._clock = clock
        self._started_at = clock()

    @property
    def public_state(self) -> PublicStateView:
        return self._state

    @property
    def virtual_time_s(self) -> float:
        return self._state.virtual_time_s

    @property

```

```

def trace(self) -> tuple[Trace, ...]:
    return tuple(self._trace)

def measure(self, position: Point, channel: int) -> MeasurementObservation:
    self._validate(channel)
    position = self._coerce_point(position)
    movement = self._distance(self._state.position, position)
    switched = channel != self._state.measure_channel
    delta = movement / 5 + (1 if switched else 0) + 5
    self._reserve(delta)
    target = next((target for target in self._case.targets if target.channel == channel), None)
    if target is None or target.channel in self._cleared or not self._covered(target, position):
        observation = MeasurementObservation("no_signal", None)
    elif self._distance(position, target.position) <= 5:
        observation = MeasurementObservation("near", None)
    else:
        bearing = math.degrees(math.atan2(target.position.y - position.y, target.position.x -
        ↪ position.x)) % 360.0
        observation = MeasurementObservation("direction", (bearing + self._case.error_deg(channel,
        ↪ position)) % 360.0)
    self._advance(position, channel, delta)
    self._trace.append(Trace(Action("measure", position, channel), observation, movement, switched,
    ↪ delta))
    return observation

def clear(self, position: Point, channel: int) -> str:
    self._validate(channel)
    position = self._coerce_point(position)
    movement = self._distance(self._state.position, position)
    target = next((target for target in self._case.targets if target.channel == channel), None)
    success = target is not None and channel not in self._cleared and self._distance(position,
    ↪ target.position) <= 20
    delta = movement / 5 + (5 if success else 3)
    self._reserve(delta)
    if success:
        self._cleared.add(channel)
    self._state = replace(self._state, position=position, virtual_time_s=self.virtual_time_s + delta)
    result = "success" if success else "no_target_in_range"
    self._trace.append(Trace(Action("clear", position, channel), result, movement, False, delta))
    return result

def _advance(self, position: Point, channel: int, delta: float) -> None:
    self._state = PublicStateView(position, channel, self.virtual_time_s + delta)

def _reserve(self, delta: float) -> None:
    if len(self._trace) >= self._max_actions:
        raise RuntimeError("action budget exhausted")
    if self._clock() - self._started_at > self._max_real_duration_s:
        raise RuntimeError("real-time budget exhausted")
    if self.virtual_time_s + delta > self._max_virtual_duration_s:
        raise RuntimeError("virtual-time budget exhausted")

def _covered(self, target, point: Point) -> bool:
    if self._distance(point, target.position) > target.receive_radius:
        return False
    if target.kind == "omni":
        return True
    angle = math.degrees(math.atan2(point.y - target.position.y, point.x - target.position.x)) % 360

```

```

        difference = abs((angle - target.bearing_deg + 180) % 360 - 180)
        return difference <= 90

    @staticmethod
    def _distance(a: Point, b: Point) -> float:
        return math.hypot(a.x - b.x, a.y - b.y)

    @staticmethod
    def _validate(channel: int) -> None:
        if isinstance(channel, bool) or not isinstance(channel, int) or not 1 <= channel <= 20:
            raise ValueError("channel must be in 1..20")

    @staticmethod
    def _coerce_point(point: Point) -> Point:
        """Accept a typed public point or a strategy point with finite x/y."""
        if isinstance(point, Point):
            return point
        try:
            return Point(point.x, point.y)
        except (AttributeError, TypeError, ValueError) as exc:
            raise ValueError("position must expose valid finite x/y coordinates") from exc

```

文件: src/b\_problem/benchmark/runner.py

```

from __future__ import annotations

from dataclasses import dataclass
import inspect
from typing import Callable

from .offline_arena import OfflineArena
from .types import Case, Point, PublicStateView, Trace

class PublicArena:
    """Narrow cooperative-strategy facade; Python reflection remains unresolved."""

    def __init__(self, arena: OfflineArena) -> None:
        self.__arena = arena

    @property
    def public_state(self) -> PublicStateView:
        return self.__arena.public_state

    def measure(self, position: Point, channel: int) -> str:
        return self.__arena.measure(position, channel)

    def clear(self, position: Point, channel: int) -> str:
        return self.__arena.clear(position, channel)

@dataclass(frozen=True)
class IsolationAssessment:
    status: str
    reason: str

```

```

def cooperative_strategy_assessment(strategy: Callable[[PublicArena], None]) -> IsolationAssessment:
    """Reject common direct truth imports; this is not process isolation."""
    if not callable(strategy):
        raise ValueError("strategy must be callable")
    prohibited = {"_PublicArena__arena", "_case", "OfflineArena", "Case", "Target"}
    pending = [strategy.__code__] if hasattr(strategy, "__code__") else []
    names: set[str] = set()
    while pending:
        code = pending.pop()
        names.update(code.co_names)
        pending.extend(item for item in code.co_consts if inspect.iscode(item))
    if names & prohibited:
        raise RuntimeError("cooperative strategy may not access benchmark truth")
    closure = inspect.getclosurevars(strategy)
    if any(isinstance(value, (Case, OfflineArena)) for value in (*closure.globals.values(),
        ↪ *closure.nonlocals.values())):
        raise RuntimeError("cooperative strategy may not capture benchmark truth")
    return IsolationAssessment(
        "UNRESOLVED",
        "Static checks reject common direct truth access; same-process reflection is not strong
        ↪ isolation.",
    )

def run_strategy(strategy: Callable[[PublicArena], None], case: Case) -> tuple[Trace, ...]:
    cooperative_strategy_assessment(strategy)
    arena = OfflineArena(case)
    strategy(PublicArena(arena))
    return arena.trace

```

文件: src/b\_problem/benchmark/suite.py

```

"""Frozen full-factor offline suite and auditable single-case runner."""
from __future__ import annotations

from dataclasses import asdict
from itertools import product

from b_problem.strategy import q3, q4

from .generator import build_case
from .metrics import score_trace
from .runner import run_strategy
from .types import Case, ScenarioSpec

COUNTS = (10, 13, 16)
SPATIAL_FAMILIES = ("uniform", "boundary", "cluster", "separated", "grid_boundary")
RADIUS_MODES = ("r1000", "r1500", "mixed")
ERROR_MODES = ("zero", "plus1", "minus1", "fixed_hash")
EXTENDED_COUNTS = tuple(range(10, 17))
_EXTENDED_VARIANTS = (
    ("boundary", "r1000", "plus1"),
    ("cluster", "r1500", "minus1"),
    ("separated", "mixed", "fixed_hash"),
)

```

```

def build_full_suite(seed_base: int = 202_609_100) -> tuple[Case, ...]:
    """Return one immutable holdout case for every declared factor tuple."""
    cases: list[Case] = []
    seed = seed_base
    for question, orientations in (("Q3", ("none",)), ("Q4", ("random", "adversarial", "boundary"))):
        for count, spatial, radius, error, orientation in product(
            COUNTS, SPATIAL_FAMILIES, RADIUS_MODES, ERROR_MODES, orientations
        ):
            cases.append(build_case(ScenarioSpec(
                question, count, spatial, radius, error, orientation, seed, "holdout"
            )))
            seed += 1
    return tuple(cases)

def build_extended_holdout_suite(seed_base: int = 202_610_100) -> tuple[Case, ...]:
    """Return a bounded 42-case holdout spanning untested source counts and channel IDs."""
    cases: list[Case] = []
    seed = seed_base
    for count in EXTENDED_COUNTS:
        for question, orientations in (("Q3", ("none",) * 3), ("Q4", ("adversarial",) * 3)):
            for variant, (spatial, radius, error) in enumerate(_EXTENDED_VARIANTS):
                cases.append(build_case(ScenarioSpec(
                    question, count, spatial, radius, error, orientations[variant], seed,
                    "holdout", _extended_channels(count, variant),
                )))
                seed += 1
    return tuple(cases)

def _extended_channels(count: int, variant: int) -> tuple[int, ...]:
    if variant == 0:
        return tuple(range(1, count + 1))
    if variant == 1:
        return tuple(range(21 - count, 21))
    if count == 10:
        return tuple(range(1, 21, 2))
    pool = (1, 20, 3, 18, 5, 16, 7, 14, 9, 12, 2, 19, 4, 17, 6, 15, 8, 13, 10, 11)
    return pool[:count]

def extended_coverage_certificate(cases: tuple[Case, ...]) -> dict:
    """Audit the compact extended matrix without changing frozen-suite coverage rules."""
    counts = {case.spec.target_count for case in cases}
    questions = {case.spec.question for case in cases}
    channel_shapes = {_channel_shape(case.spec.channel_ids or ()) for case in cases}
    q4_orientations = {case.spec.orientation_mode for case in cases if case.spec.question == "Q4"}
    spatial = {case.spec.spatial for case in cases}
    radius_modes = {case.spec.radius_mode for case in cases}
    error_modes = {case.spec.error_mode for case in cases}
    required_shapes = {"low", "high", "sparse", "mixed_non_contiguous"}
    unique_cases = len({case.truth_digest for case in cases}) == len(cases)
    ids_match = all(tuple(target.channel for target in case.targets) == case.spec.channel_ids for case in
        ↪ cases)
    missing_counts = sorted(set(EXTENDED_COUNTS) - counts)
    missing_channel_shapes = sorted(required_shapes - channel_shapes)
    missing_questions = sorted({"Q3", "Q4"} - questions)

```

```

required_q4_orientations = {"adversarial"}
passed = (
    len(cases) == 42 and not missing_counts and not missing_questions and not missing_channel_shapes
    and required_q4_orientations <= q4_orientations
    and {"boundary", "cluster", "separated"} <= spatial
    and {"r1000", "r1500", "mixed"} <= radius_modes
    and {"plus1", "minus1"} <= error_modes
    and all(case.spec.split == "holdout" for case in cases) and unique_cases and ids_match
)
return {
    "case_count": len(cases), "counts": sorted(counts), "questions": sorted(questions),
    "channel_shapes": sorted(channel_shapes), "missing_counts": missing_counts,
    "missing_questions": missing_questions, "missing_channel_shapes": missing_channel_shapes,
    "spatial": sorted(spatial), "radius_modes": sorted(radius_modes),
    "error_modes": sorted(error_modes), "q4_orientations": sorted(q4_orientations),
    "threshold_contract_m": [5.0, 20.0], "all_holdout": all(case.spec.split == "holdout" for case in
↪ cases),
    "unique_cases": unique_cases, "channel_ids_match_targets": ids_match, "pass": passed,
}

def _channel_shape(channels: tuple[int, ...]) -> str:
    count = len(channels)
    if channels == tuple(range(1, count + 1)):
        return "low"
    if channels == tuple(range(21 - count, 21)):
        return "high"
    if channels == tuple(range(1, 21, 2)):
        return "sparse"
    return "mixed_non_contiguous"

def run_case(case: Case) -> dict:
    """Execute the matching strategy and return metrics plus time components."""
    result_box = []
    execute = q3.execute_q3_strategy if case.spec.question == "Q3" else q4.execute_q4_strategy

    def strategy(arena) -> None:
        result_box.append(execute(arena, arena.public_state))

    trace = run_strategy(strategy, case)
    metrics = score_trace(case, trace)
    components = {
        "movement": metrics.movement_distance_m / 5.0,
        "switching": float(metrics.switch_count),
        "measurement": 5.0 * metrics.measure_count,
        "successful_clear": 5.0 * metrics.successful_clear_count,
        "failed_clear": 3.0 * metrics.failed_clear_count,
    }
    return {
        "case_id": case.truth_digest,
        "spec": asdict(case.spec),
        "complete": bool(result_box and result_box[0].complete),
        "metrics": asdict(metrics),
        "action_count": len(trace),
        "virtual_time_components_s": components,
    }

```

```

def run_extended_case(case: Case) -> dict:
    """Run one extended holdout case and retain a non-sensitive failure record if needed."""
    try:
        result = run_case(case)
    except Exception as exc:
        return {
            "case_id": case.truth_digest,
            "spec": asdict(case.spec),
            "strategy_execution_status": "failed",
            "execution_error_type": type(exc).__name__,
        }
    return {**result, "strategy_execution_status": "completed"}

```

文件: src/b\_problem/benchmark/types.py

```

from __future__ import annotations

from dataclasses import dataclass
import hashlib
import math
from typing import Literal

MAX_POSITION_M = 2_000_000.0
_QUESTION = {"Q3", "Q4"}
_COUNTS = set(range(10, 17))
_SPATIAL = {"uniform", "boundary", "cluster", "separated", "grid_boundary"}
_RADIUS_MODES = {"r1000", "r1500", "mixed"}
_ERROR_MODES = {"zero", "plus1", "minus1", "fixed_hash"}
_Q4_ORIENTATIONS = {"random", "adversarial", "boundary"}

@dataclass(frozen=True)
class Point:
    x: float
    y: float

    def __post_init__(self) -> None:
        if not all(isinstance(value, (int, float)) and not isinstance(value, bool)
                    and math.isfinite(value) and abs(value) <= MAX_POSITION_M for value in (self.x,
                    ↪ self.y)):
            raise ValueError("point coordinates must be finite and within 2,000,000 m")

@dataclass(frozen=True)
class Target:
    channel: int
    position: Point
    receive_radius: float
    kind: Literal["omni", "directional"]
    bearing_deg: float | None

    def __post_init__(self) -> None:
        if isinstance(self.channel, bool) or not isinstance(self.channel, int) or not 1 <= self.channel <=
        ↪ 20:
            raise ValueError("target channel must be an integer in 1..20")

```

```

        if not isinstance(self.position, Point):
            raise ValueError("target position must be a Point")
        if self.position.x * self.position.x + self.position.y * self.position.y > 1800.0 ** 2 + 1e-8:
            raise ValueError("target position must be inside the 1800 m target disk")
        if isinstance(self.receive_radius, bool) or not isinstance(self.receive_radius, (int, float)) or
        ↪ not math.isfinite(self.receive_radius) or not 1000.0 <= self.receive_radius <= 1500.0:
            raise ValueError("target receive radius must be finite and in [1000, 1500]")
        if self.kind not in {"omni", "directional"}:
            raise ValueError("target kind must be omni or directional")
        if self.kind == "omni" and self.bearing_deg is not None:
            raise ValueError("omnidirectional targets must not have a bearing")
        if self.kind == "directional":
            if isinstance(self.bearing_deg, bool) or not isinstance(self.bearing_deg, (int, float)) or not
            ↪ math.isfinite(self.bearing_deg) or not 0.0 <= self.bearing_deg < 360.0:
                raise ValueError("directional targets need a normalized finite bearing")

@dataclass(frozen=True)
class ScenarioSpec:
    question: Literal["Q3", "Q4"]
    target_count: int
    spatial: str
    radius_mode: str
    error_mode: str
    orientation_mode: str
    seed: int
    split: Literal["tuning", "holdout"] = "holdout"
    channel_ids: tuple[int, ...] | None = None

    def __post_init__(self) -> None:
        if self.question not in _QUESTIONS or self.target_count not in _COUNTS:
            raise ValueError("supported suites use Q3/Q4 and 10 through 16 targets")
        if self.spatial not in _SPATIAL or self.radius_mode not in _RADIUS_MODES or self.error_mode not in
        ↪ _ERROR_MODES:
            raise ValueError("unsupported scenario mode")
        if not isinstance(self.seed, int) or isinstance(self.seed, bool):
            raise ValueError("seed must be an integer")
        if self.split not in {"tuning", "holdout"}:
            raise ValueError("split must be tuning or holdout")
        if self.channel_ids is not None:
            if (not isinstance(self.channel_ids, tuple)
                or len(self.channel_ids) != self.target_count
                or any(isinstance(channel, bool) or not isinstance(channel, int)
                    or not 1 <= channel <= 20 for channel in self.channel_ids)
                or len(set(self.channel_ids)) != len(self.channel_ids)):
                raise ValueError("channel_ids must be unique integer channels in 1..20 matching
                ↪ target_count")
        if self.question == "Q3" and self.orientation_mode != "none":
            raise ValueError("Q3 uses orientation_mode none")
        if self.question == "Q4" and self.orientation_mode not in _Q4_ORIENTATIONS:
            raise ValueError("Q4 needs an explicit directional orientation mode")

@dataclass(frozen=True)
class Action:
    kind: Literal["measure", "clear"]
    position: Point
    channel: int

```

```

def __post_init__(self) -> None:
    if self.kind not in {"measure", "clear"} or not isinstance(self.position, Point):
        raise ValueError("invalid action")
    if isinstance(self.channel, bool) or not isinstance(self.channel, int) or not 1 <= self.channel <=
↪ 20:
        raise ValueError("action channel must be an integer in 1..20")

@dataclass(frozen=True)
class MeasurementObservation:
    result: Literal["no_signal", "near", "direction"]
    svd_deg: float | None

    def __post_init__(self) -> None:
        if self.result not in {"no_signal", "near", "direction"}:
            raise ValueError("invalid measurement result")
        if self.result == "direction":
            if isinstance(self.svd_deg, bool) or not isinstance(self.svd_deg, (int, float)) or not
↪ math.isfinite(self.svd_deg) or not 0.0 <= self.svd_deg < 360.0:
                raise ValueError("direction observation needs normalized svd_deg")
        elif self.svd_deg is not None:
            raise ValueError("non-direction observation must not have svd_deg")

@dataclass(frozen=True)
class Trace:
    action: Action
    result: MeasurementObservation | Literal["success", "no_target_in_range"]
    movement_m: float
    switched: bool
    delta_s: float

    def __post_init__(self) -> None:
        if not isinstance(self.action, Action) or isinstance(self.switched, bool) is False:
            raise ValueError("invalid trace")
        if self.action.kind == "measure" and not isinstance(self.result, MeasurementObservation):
            raise ValueError("measure trace needs a measurement observation")
        if self.action.kind == "clear" and self.result not in {"success", "no_target_in_range"}:
            raise ValueError("clear trace needs a clear result")
        if not all(isinstance(value, (int, float)) and not isinstance(value, bool) and
↪ math.isfinite(value) and value >= 0.0
            for value in (self.movement_m, self.delta_s)):
            raise ValueError("trace movement and duration must be finite and non-negative")

@dataclass(frozen=True)
class PublicStateView:
    position: Point
    measure_channel: int
    virtual_time_s: float

    def __post_init__(self) -> None:
        if not isinstance(self.position, Point):
            raise ValueError("state position must be a Point")
        if isinstance(self.measure_channel, bool) or not isinstance(self.measure_channel, int) or not 1 <=
↪ self.measure_channel <= 20:
            raise ValueError("state channel must be an integer in 1..20")

```

```

        if isinstance(self.virtual_time_s, bool) or not isinstance(self.virtual_time_s, (int, float)) or
        ↪ not math.isfinite(self.virtual_time_s) or self.virtual_time_s < 0.0:
            raise ValueError("state virtual time must be finite and non-negative")

@dataclass(frozen=True)
class Metrics:
    clear_rate: float
    total_virtual_time_s: float
    mean_clear_time_s: float | None
    movement_distance_m: float
    measure_count: int
    switch_count: int
    clear_count: int
    failed_clear_count: int
    successful_clear_count: int

@dataclass(frozen=True)
class Case:
    spec: ScenarioSpec
    targets: tuple[Target, ...]
    truth_digest: str

    def __post_init__(self) -> None:
        if not isinstance(self.spec, ScenarioSpec) or len(self.targets) != self.spec.target_count:
            raise ValueError("case targets must match the declared target count")
        if not all(isinstance(target, Target) for target in self.targets):
            raise ValueError("case targets must be Target values")
        if len({target.channel for target in self.targets}) != len(self.targets):
            raise ValueError("case channels must be unique")
        if self.spec.channel_ids is not None and tuple(target.channel for target in self.targets) !=
        ↪ self.spec.channel_ids:
            raise ValueError("case channels must preserve the declared channel_ids order")
        if self.spec.question == "Q3" and any(target.kind != "omni" for target in self.targets):
            raise ValueError("Q3 cases contain only omnidirectional targets")
        if self.spec.question == "Q4" and not any(target.kind == "directional" for target in self.targets):
            raise ValueError("Q4 cases need at least one directional target")
        radii = {target.receive_radius for target in self.targets}
        if self.spec.radius_mode == "r1000" and radii != {1000.0}:
            raise ValueError("r1000 cases require only 1000 m targets")
        if self.spec.radius_mode == "r1500" and radii != {1500.0}:
            raise ValueError("r1500 cases require only 1500 m targets")
        if self.spec.radius_mode == "mixed" and not radii <= {1000.0, 1500.0}:
            raise ValueError("mixed cases require 1000 m or 1500 m targets")

    def error_deg(self, channel: int, point: Point) -> float:
        mode = self.spec.error_mode
        if mode == "zero":
            return 0.0
        if mode == "plus1":
            return 1.0
        if mode == "minus1":
            return -1.0
        key = f"{self.spec.seed}:{channel}:{point.x:.9f}:{point.y:.9f}".encode()
        return (int.from_bytes(hashlib.sha256(key).digest()[8:], "big") / 2**64) * 2 - 1

@property

```

```
def manifest(self) -> dict:
    return {"spec": self.spec.__dict__, "truth_digest": self.truth_digest}
```

文件: src/b\_problem/certificate\_experiments/\_\_init\_\_.py

```
"""Standalone mathematical certificate experiments; never strategy entry points."""
```

文件: src/b\_problem/certificate\_experiments/q4\_21\_certificate.py

```
"""Finite, fail-closed certificate for the 21-point Q4 discovery candidate.
```

```
This is diagnostic geometry only; no strategy imports this module.
"""
```

```
from __future__ import annotations
```

```
from dataclasses import dataclass
from itertools import permutations
from math import cos, hypot, pi, sin
```

```
from b_problem.strategy.q4 import Point
```

```
_EPS = 1e-8
```

```
@dataclass(frozen=True)
```

```
class Certificate21:
```

```
    certified: bool
    hull_inradius_m: float
    oriented_pair_checks: int
    worst_receive_radius_m: float
```

```
def candidate_21() -> tuple[Point, ...]:
```

```
    """Origin, 12 outer sites and 8 inner sites; no expected result is embedded."""
```

```
    outer = 1802.0 / cos(pi / 12.0)
```

```
    return (Point(0.0, 0.0),
            *(Point(outer * cos(2 * pi * k / 12), outer * sin(2 * pi * k / 12)) for k in range(12)),
            *(Point(997.32 * cos(2 * pi * k / 8), 997.32 * sin(2 * pi * k / 8)) for k in range(8)))
```

```
def _cross(a: Point, b: Point, c: Point) -> float:
```

```
    return (b.x-a.x)*(c.y-a.y)-(b.y-a.y)*(c.x-a.x)
```

```
def _hull(points: tuple[Point, ...]) -> tuple[Point, ...]:
```

```
    ordered = sorted(set(points))
```

```
    lower: list[Point] = []
```

```
    for point in ordered:
```

```
        while len(lower) >= 2 and _cross(lower[-2], lower[-1], point) <= 0: lower.pop()
        lower.append(point)
```

```
    upper: list[Point] = []
```

```
    for point in reversed(ordered):
```

```
        while len(upper) >= 2 and _cross(upper[-2], upper[-1], point) <= 0: upper.pop()
        upper.append(point)
```

```
    return tuple(lower[:-1] + upper[:-1])
```

```

def _clip(poly: tuple[Point, ...], nx: float, ny: float, bound: float) -> tuple[Point, ...]:
    out: list[Point] = []
    if not poly: return ()
    previous = poly[-1]; fp = nx*previous.x + ny*previous.y - bound
    for point in poly:
        fc = nx*point.x + ny*point.y-bound
        if (fp <= 0) != (fc <= 0):
            t = fp/(fp-fc); out.append(Point(previous.x+t*(point.x-previous.x),
            ↪ previous.y+t*(point.y-previous.y)))
        if fc <= _EPS: out.append(point)
        previous, fp = point, fc
    return tuple(out)

def _max_cell_radius(sites: tuple[Point, ...], nx: float, ny: float, offset: float, disk_r: float) ->
    ↪ float:
    # The pair support half-plane clipped to the source disk is partitioned by
    # Voronoi cells. Candidate extrema are cell vertices and edge/circle hits.
    base = _clip((Point(-disk_r,-disk_r), Point(disk_r,-disk_r), Point(disk_r,disk_r),
    ↪ Point(-disk_r,disk_r)), -nx, -ny, -offset)
    worst = 0.0
    for site in sites:
        cell = base
        for other in sites:
            if other == site: continue
            dx, dy = other.x-site.x, other.y-site.y
            cell = _clip(cell, dx, dy, (dx*(other.x+site.x)+dy*(other.y+site.y))/2.0)
            if not cell: break
        candidates = [p for p in cell if p.radius() <= disk_r+_EPS]
        for a, b in zip(cell, cell[1:]+cell[:1]):
            dx, dy = b.x-a.x, b.y-a.y; length2 = dx*dx+dy*dy
            if length2 <= _EPS: continue
            t0 = -(a.x*dx+a.y*dy)/length2; discriminant = disk_r*disk_r-(a.x+t0*dx)**2-(a.y+t0*dy)**2
            if discriminant >= -_EPS:
                dt = (max(discriminant, 0.0)/length2) ** 0.5
                for t in (t0-dt, t0+dt):
                    if -_EPS <= t <= 1+_EPS: candidates.append(Point(a.x+t*dx, a.y+t*dy))
            if candidates: worst = max(worst, max(site.distance_to(p) for p in candidates))
    return worst

def certify_21(points: tuple[Point, ...] | None = None, *, target_radius_m: float = 1800.0,
    ↪ receive_radius_m: float = 1000.0) -> Certificate21:
    points = candidate_21() if points is None else tuple(points)
    hull = _hull(points)
    inradius = min(abs(a.x*b.y-a.y*b.x)/a.distance_to(b) for a, b in zip(hull, hull[1:]+hull[:1]))
    worst, checks = 0.0, 0
    for a, b in permutations(points, 2):
        dx, dy = b.x-a.x, b.y-a.y; length = hypot(dx, dy)
        if length <= _EPS: continue
        nx, ny = -dy/length, dx/length; offset = nx*a.x+ny*a.y
        if offset >= target_radius_m: continue
        sites = tuple(p for p in points if nx*p.x+ny*p.y > offset+_EPS)
        worst = max(worst, _max_cell_radius(sites, nx, ny, offset, target_radius_m)); checks += 1
    return Certificate21(inradius > target_radius_m+_EPS and worst < receive_radius_m-_EPS, inradius,
    ↪ checks, worst)

```

文件: src/b\_problem/certificate\_experiments/q4\_certificate\_recycling.py

```
"""Conservative offline replay of per-channel CERT-D measurement witnesses.

The proved CERT-C construction requires its complete 31-point radial-support
set. This replay therefore credits a witness only when an actual ``no_signal``
measurement occurred exactly at one of those proved points. Arbitrary visited
or measured points are deliberately not substituted: the current validator has
no analytic certificate for such a replacement.
"""

from __future__ import annotations

from dataclasses import dataclass
from typing import Iterable

from b_problem.strategy.q4 import Point

@dataclass(frozen=True)
class ReplayMeasurement:
    channel: int
    position: Point
    result: str
    virtual_time_s: float

def cert_d_reference_stations() -> frozenset[Point]:
    """Return only the independently proved 31-point CERT-D basis."""
    from b_problem.certificate_experiments.q4_station_selection import radial_support_candidate

    candidate = radial_support_candidate(925.0)
    if len(candidate.stations) != 31:
        raise RuntimeError("CERT-D reference certificate invariant failed")
    return candidate.stations

def no_signal_witnesses(
    events: Iterable[ReplayMeasurement], *, channel: int, through_time_s: float | None = None
) -> frozenset[Point]:
    """Extract only valid no-signal measurements for one channel and time cut."""
    return frozenset(
        event.position
        for event in events
        if event.channel == channel
        and event.result == "no_signal"
        and (through_time_s is None or event.virtual_time_s <= through_time_s)
    )

def remaining_certificate_stations(
    channel: int, witness_positions: Iterable[Point], reference_31_station_set: Iterable[Point]
) -> tuple[Point, ...]:
    """Return the proved reference witnesses still missing for one channel.

    ``channel`` is intentionally explicit to make callers retain per-channel
    evidence rather than pooling no-signal observations across channels.
    """
    if not 1 <= channel <= 20:
        raise ValueError("channel must be in 1..20")
```

```

reference = frozenset(reference_31_station_set)
witnesses = frozenset(witness_positions)
return tuple(sorted(reference - witnesses))

def is_channel_certificate_complete(
    channel: int, witness_positions: Iterable[Point], reference_31_station_set: Iterable[Point]
) -> bool:
    """Conservative deterministic completion check for the current validator."""
    return not remaining_certificate_stations(channel, witness_positions, reference_31_station_set)

```

文件: src/b\_problem/certificate\_experiments/q4\_mincert.py

```

"""Conservative arbitrary-station oracle for offline MINCERT search.

PASS is deliberately stronger than sampled success: the station Delaunay mesh
must cover the entire source disk and every triangle intersecting that disk
must have edges at most the 1000 m guaranteed receive radius. Any source in
such a triangle is in the convex hull of three in-range stations, proving the
closed-half-plane directional condition.
"""

from __future__ import annotations

from dataclasses import dataclass
from math import atan2, cos, hypot, pi, sin
from typing import Iterable

import numpy as np
from scipy.spatial import Delaunay, QhullError

from b_problem.strategy import q4

@dataclass(frozen=True)
class DirectionalCertificateValidation:
    status: str
    method: str
    failure_kind: str | None
    counterexample: q4.Point | None
    counterexample_orientation_deg: float | None
    max_active_triangle_edge_m: float | None
    margin_m: float | None
    detail: str

def two_ring_candidate(*, outer_radius_m: float, inner_radius_m: float, count: int = 12) ->
    ↪ frozenset[q4.Point]:
    """A boundary-adaptive polar mesh; the inner ring is half a sector offset."""
    if count < 3 or outer_radius_m <= 0.0 or inner_radius_m <= 0.0:
        raise ValueError("ring counts and radii must be positive")
    return frozenset({
        q4.Point(0.0, 0.0),
        *(q4.Point(outer_radius_m * cos(2.0 * pi * index / count), outer_radius_m * sin(2.0 * pi * index /
        ↪ count)) for index in range(count)),
        *(q4.Point(inner_radius_m * cos(2.0 * pi * (index + 0.5) / count), inner_radius_m * sin(2.0 * pi *
        ↪ (index + 0.5) / count)) for index in range(count)),
    })

```

```

def _cross(first: q4.Point, second: q4.Point, point: q4.Point) -> float:
    return (second.x - first.x) * (point.y - first.y) - (second.y - first.y) * (point.x - first.x)

def _convex_hull(points: Iterable[q4.Point]) -> tuple[q4.Point, ...]:
    ordered = sorted(set(points))
    if len(ordered) <= 1:
        return tuple(ordered)
    lower: list[q4.Point] = []
    for point in ordered:
        while len(lower) >= 2 and _cross(lower[-2], lower[-1], point) <= q4.TOLERANCE:
            lower.pop()
        lower.append(point)
    upper: list[q4.Point] = []
    for point in reversed(ordered):
        while len(upper) >= 2 and _cross(upper[-2], upper[-1], point) <= q4.TOLERANCE:
            upper.pop()
        upper.append(point)
    return tuple(lower[:-1] + upper[:-1])

def _disk_hull_witness(hull: tuple[q4.Point, ...], radius_m: float) -> tuple[q4.Point, float] | None:
    """Return a real uncovered source/orientation when the disk exits the hull."""
    if len(hull) < 3:
        return q4.Point(radius_m, 0.0), 0.0
    worst: tuple[float, q4.Point, q4.Point, float, float] | None = None
    for first, second in zip(hull, hull[1:] + hull[:1]):
        dx, dy = second.x - first.x, second.y - first.y
        length = hypot(dx, dy)
        inward_x, inward_y = -dy / length, dx / length
        # The minimum signed distance over the source disk occurs opposite the
        # inward normal. A negative result lies strictly outside this hull edge.
        minimum = -radius_m - (inward_x * first.x + inward_y * first.y)
        source = q4.Point(-radius_m * inward_x, -radius_m * inward_y)
        if worst is None or minimum < worst[0]:
            worst = (minimum, first, second, inward_x, inward_y)
    assert worst is not None
    minimum, _first, _second, inward_x, inward_y = worst
    if minimum >= -q4.TOLERANCE:
        return None
    source = q4.Point(-radius_m * inward_x, -radius_m * inward_y)
    # Outward normal is an emitting direction that strictly excludes every
    # station of the convex hull from its closed half-plane through ``source``.
    orientation = atan2(-inward_y, -inward_x) * 180.0 / 3.141592653589793 % 360.0
    return source, orientation

def _point_in_triangle(point: q4.Point, triangle: tuple[q4.Point, q4.Point, q4.Point]) -> bool:
    signs = [_cross(first, second, point) for first, second in zip(triangle, triangle[1:] + triangle[:1])]
    return all(value >= -q4.TOLERANCE for value in signs) or all(value <= q4.TOLERANCE for value in signs)

def _segment_origin_distance(first: q4.Point, second: q4.Point) -> float:
    dx, dy = second.x - first.x, second.y - first.y
    denominator = dx * dx + dy * dy
    if denominator <= q4.TOLERANCE:
        return first.radius()

```

```

ratio = max(0.0, min(1.0, -(first.x * dx + first.y * dy) / denominator))
return hypot(first.x + ratio * dx, first.y + ratio * dy)

def _triangle_intersects_disk(triangle: tuple[q4.Point, q4.Point, q4.Point], radius_m: float) -> bool:
    if _point_in_triangle(q4.Point(0.0, 0.0), triangle):
        return True
    return min(_segment_origin_distance(first, second) for first, second in zip(triangle, triangle[1:] +
↪ triangle[:1])) <= radius_m + q4.TOLERANCE

def validate_directional_certificate(
    stations: Iterable[q4.Point], *, source_radius_m: float = q4.TARGET_RADIUS_M, receive_radius_m: float =
↪ q4.MIN_RECEIVE_RADIUS_M
) -> DirectionalCertificateValidation:
    """Conservatively prove or reject an arbitrary precomputed station set."""
    points = tuple(sorted(set(stations)))
    if len(points) < 3:
        return DirectionalCertificateValidation("FAIL", "delaunay_triangle_cover", "counterexample",
↪ q4.Point(source_radius_m, 0.0), 0.0, None, None, "fewer than three stations")
    hull_witness = _disk_hull_witness(_convex_hull(points), source_radius_m)
    if hull_witness is not None:
        source, orientation = hull_witness
        return DirectionalCertificateValidation("FAIL", "delaunay_triangle_cover", "counterexample",
↪ source, orientation, None, None, "source disk is not inside station convex hull")
    try:
        simplices = Delaunay(np.array([(point.x, point.y) for point in points], dtype=float)).simplices
    except QhullError:
        return DirectionalCertificateValidation("FAIL", "delaunay_triangle_cover",
↪ "unproven_degenerate_mesh", None, None, None, None, "Delaunay triangulation is degenerate")
    active_edges = []
    for simplex in simplices:
        triangle = tuple(points[index] for index in simplex)
        if not _triangle_intersects_disk(triangle, source_radius_m):
            continue
        edge = max(first.distance_to(second) for first, second in zip(triangle, triangle[1:] +
↪ triangle[:1]))
        active_edges.append(edge)
    if not active_edges:
        return DirectionalCertificateValidation("FAIL", "delaunay_triangle_cover", "unproven_mesh", None,
↪ None, None, None, "no Delaunay triangle intersects source disk")
    maximum = max(active_edges)
    margin = receive_radius_m - maximum
    if margin < -q4.TOLERANCE:
        return DirectionalCertificateValidation("FAIL", "delaunay_triangle_cover", "unproven_long_edge",
↪ None, None, maximum, margin, "active Delaunay triangle exceeds guaranteed receive radius")
    return DirectionalCertificateValidation("PASS", "delaunay_triangle_cover", None, None, None, maximum,
↪ margin, "conservative triangulation proof")

```

文件: src/b\_problem/certificate\_experiments/q4\_movement\_forensics.py

```

"""Read-only, reconciled movement taxonomy for frozen CERT-D traces."""
from __future__ import annotations

from dataclasses import dataclass
from math import hypot
from typing import Iterable

```

```

from b_problem.benchmark.types import Trace
from b_problem.strategy.q4 import Point

@dataclass(frozen=True)
class MovementSegment:
    case_id: str
    event_index: int
    timestamp_before_s: float
    timestamp_after_s: float
    from_point: Point
    to_point: Point
    distance_m: float
    virtual_time_cost_s: float
    current_channel: int | None
    target_channel: int
    strategy_state_before: str
    strategy_state_after: str
    primary_category: str
    secondary_purposes: tuple[str, ...]
    source_channel: int | None
    current_target_type: str
    current_target_id: str | None
    certificate_target_id: int | None
    task_reason: str
    route_plan_id: str
    route_plan_index: int | None

@dataclass(frozen=True)
class MovementLedger:
    segments: tuple[MovementSegment, ...]
    reported_movement_m: float
    trace_movement_m: float
    reconciliation_pass: bool

def _same(point, station: Point) -> bool:
    return abs(point.x - station.x) <= 1e-8 and abs(point.y - station.y) <= 1e-8

def _station_index(point, stations: tuple[Point, ...]) -> int | None:
    for index, station in enumerate(stations):
        if _same(point, station):
            return index
    return None

def build_movement_ledger(
    case_id: str, trace: Iterable[Trace], stations: Iterable[Point], *, last_source_clear_event_index: int
    ↪ | None = None
) -> MovementLedger:
    """Assign one deterministic primary label to every positive-movement trace leg."""
    items = tuple(trace)
    route = tuple(stations)
    position = Point(0.0, 0.0)
    virtual_time = 0.0

```

```

current_channel: int | None = 1
in_source_handling = False
survey_complete = False
route_index: int | None = None
last_clear_failed: set[int] = set()
segments: list[MovementSegment] = []
for event_index, event in enumerate(items):
    target = Point(event.action.position.x, event.action.position.y)
    station_index = _station_index(event.action.position, route)
    before_state = "FALLBACK" if survey_complete else ("SOURCE_HANDLING" if in_source_handling else
↪ "SURVEY")
    if event.movement_m > 1e-12:
        if survey_complete:
            category, secondary, source = "FALLBACK", (), event.action.channel
        elif station_index is not None:
            category = "ROUTE_RECONNECTION" if in_source_handling else (
                "FINAL_CERTIFICATE" if last_source_clear_event_index is not None and event_index >
↪ last_source_clear_event_index else "CERT_ROUTE"
            )
            secondary, source = (("certificate_gain",), current_channel) if in_source_handling else
↪ ((), None)
        elif event.action.kind == "measure":
            if not in_source_handling:
                category, secondary = "SECOND_BEARING", ("source_geometry",)
            else:
                category, secondary = "LOCALIZATION_RECOVERY", ("source_geometry",)
                source = event.action.channel
        elif event.action.channel in last_clear_failed:
            category, secondary, source = "CLEAR_RETRY", (), event.action.channel
        else:
            category, secondary, source = "CLEAR_APPROACH", (), event.action.channel
    after_state = "FALLBACK" if survey_complete else ("SURVEY" if station_index is not None else
↪ "SOURCE_HANDLING")
    target_type = "certificate_station" if station_index is not None else ("fallback_clear" if
↪ survey_complete else "source_local")
    target_id = str(station_index) if station_index is not None else str(event.action.channel)
    task_reason = {"CERT_ROUTE": "cert_d_station_scan", "ROUTE_RECONNECTION":
↪ "resume_cert_d_station_scan",
                    "SECOND_BEARING": "fast_c_second_bearing", "LOCALIZATION_RECOVERY":
↪ "fast_c_alternate_bearing",
                    "CLEAR_APPROACH": "fast_c_clear_attempt", "CLEAR_RETRY":
↪ "post_failed_clear_retry",
                    "FALLBACK": "anchor_wedge_certified_clear", "FINAL_CERTIFICATE":
↪ "post_last_clear_certificate_completion"}[category]
    segments.append(MovementSegment(
        case_id, event_index, virtual_time, virtual_time + event.delta_s, position, target,
        event.movement_m, event.delta_s, current_channel, event.action.channel, before_state,
        after_state, category, secondary, source, target_type, target_id, station_index,
↪ task_reason,
        "cert_d_rolling_open", route_index,
    ))
if station_index is not None and event.action.kind == "measure":
    route_index = station_index
    if station_index == len(route) - 1:
        survey_complete = True
    if in_source_handling:
        in_source_handling = False
    elif not survey_complete and station_index is None and event.movement_m > 1e-12:

```

```

        in_source_handling = True
    if event.action.kind == "clear":
        if event.result == "success":
            last_clear_failed.discard(event.action.channel)
        else:
            last_clear_failed.add(event.action.channel)
    position = target
    virtual_time += event.delta_s
    if event.action.kind == "measure":
        current_channel = event.action.channel
    total = sum(event.movement_m for event in items)
    traced = sum(segment.distance_m for segment in segments)
    return MovementLedger(tuple(segments), total, traced, abs(total - traced) <= max(1e-6, 1e-6 * total))

```

文件: src/b\_problem/certificate\_experiments/q4\_radc\_replay.py

```

"""Conservative arithmetic for Route-Aware Deferred Clear replays only."""
from __future__ import annotations

from dataclasses import dataclass

from b_problem.strategy.q4 import Point

@dataclass(frozen=True)
class DetourCounterfactual:
    immediate_detour_m: float
    best_defer_detour_m: float
    avoidable_detour_m: float

def insertion_detour(start: Point, end: Point, clear_point: Point) -> float:
    """Exact additional distance for inserting a clear candidate in one base edge."""
    return start.distance_to(clear_point) + clear_point.distance_to(end) - start.distance_to(end)

def counterfactual_avoidable_detour(
    *, immediate_segment_m: float, base_direct_m: float, future_insertion_costs_m: tuple[float, ...]
) -> DetourCounterfactual:
    """Compare observed interruption surplus against the best future insertion.

    This uses only action-trace geometry and a successful historic clear point;
    it is not an online decision rule and does not expose benchmark truth.
    """
    if immediate_segment_m < 0.0 or base_direct_m < 0.0 or not future_insertion_costs_m:
        raise ValueError("counterfactual distances must be non-negative and non-empty")
    if any(value < 0.0 for value in future_insertion_costs_m):
        raise ValueError("future insertion costs must be non-negative")
    immediate = max(0.0, immediate_segment_m - base_direct_m)
    deferred = min(future_insertion_costs_m)
    return DetourCounterfactual(immediate, deferred, max(0.0, immediate - deferred))

```

文件: src/b\_problem/certificate\_experiments/q4\_station\_selection.py

```
"""Static Q4 station-set certificates for the 180-degree directional guarantee.
```

```
For a source ``x``, let ``N(x)`` be the scheduled stations within 1000 m.  
If ``x`` lies in ``conv(N(x)``), then every closed half-plane through ``x``  
contains a receiver: otherwise a separating normal is negative at every  
member of ``N(x)`` and therefore at their convex hull, contradicting ``x``.
```

```
The triangular-lattice construction below is a sufficient certificate for  
that premise. It is deliberately independent from ``q4.execute_q4_strategy``  
and is intended only for static design experiments.
```

```
"""
```

```
from __future__ import annotations
```

```
from dataclasses import dataclass  
from math import cos, radians, sin  
from random import Random
```

```
from b_problem.strategy import q4
```

```
SPACINGS_M = (650.0, 700.0, 750.0, 800.0, 850.0, 900.0, 925.0, 950.0, 975.0, 990.0)
```

```
@dataclass(frozen=True)
```

```
class StationCandidate:
```

```
    name: str  
    edge_m: float  
    stations: frozenset[q4.Point]  
    route: tuple[q4.Point, ...]  
    construction: str
```

```
@dataclass(frozen=True)
```

```
class StationCertificateAudit:
```

```
    name: str  
    edge_m: float  
    station_count: int  
    route_length_m: float  
    required_support_count: int  
    redundant_station_count: int  
    analytic_status: str  
    deterministic_stress_status: str  
    stress_sample_count: int  
    critical_sample_count: int  
    minimum_nearby_station_count: int  
    proof: str
```

```
def _full_cells(edge_m: float) -> tuple[q4.Triangle, ...]:
```

```
    cells, _ = q4._full_lattice_cells( # noqa: SLF001 - certificate reuses exact lattice geometry.  
        q4.TARGET_RADIUS_M, edge_m, q4.Point(0.0, 0.0), outer_layers=2  
    )  
    return cells
```

```
def required_radial_support(edge_m: float) -> frozenset[q4.Point]:
```

```
    """Return the finite lattice collar required by the triangle proof.
```

```
    If a target point belongs to an edge-``h`` triangular cell, every vertex
```

```

is within ``h`` of that point and hence within ``R+h`` of the origin.
Retaining this radial collar therefore retains all three cell vertices.
"""
if edge_m <= 0.0:
    raise ValueError("edge_m must be positive")
_, points = q4._full_lattice_cells( # noqa: SLF001 - same exact lattice as Q4.
    q4.TARGET_RADIUS_M, edge_m, q4.Point(0.0, 0.0), outer_layers=2
)
return frozenset(
    point for point in points
    if point.radius() <= q4.TARGET_RADIUS_M + edge_m + q4.TOLERANCE
)

def _route(stations: frozenset[q4.Point]) -> tuple[q4.Point, ...]:
    return q4._q4_snake_order(tuple(stations)) # noqa: SLF001 - static route metric only.

def current_exact_candidate() -> StationCandidate:
    """CERT-B: the exact Q4 static station set, without modifying Q4."""
    plan = q4.build_survey_plan()
    return StationCandidate(
        "CERT-B exact current Q4 set", plan.edge_m, plan.stations, plan.route,
        "Exact build_survey_plan() stations and its deterministic snake route.",
    )

def radial_support_candidate(edge_m: float) -> StationCandidate:
    """CERT-C: only the analytically necessary radial lattice support."""
    stations = required_radial_support(edge_m)
    return StationCandidate(
        f"CERT-C radial support h={edge_m:g}", edge_m, stations, _route(stations),
        "Keep every triangular-lattice vertex with radius at most R+h.",
    )

def _candidate_from_stations(name: str, base: StationCandidate, stations: set[q4.Point], construction:
↳ str) -> StationCandidate:
    frozen = frozenset(stations)
    return StationCandidate(name, base.edge_m, frozen, _route(frozen), construction)

def greedy_redundant_deletion(candidate: StationCandidate) -> StationCandidate:
    """Delete only stations whose removal leaves the radial-support proof intact."""
    required = required_radial_support(candidate.edge_m)
    working = set(candidate.stations)
    extras = sorted(working - required, key=lambda point: (-point.radius(), point.x, point.y))
    for point in extras:
        proposal = working - {point}
        if required <= proposal:
            working = proposal
    return _candidate_from_stations(
        f"{candidate.name} greedy-pruned", candidate, working,
        "Greedy deletion accepts a removal only when every R+h support vertex remains.",
    )

def randomized_redundant_deletion(candidate: StationCandidate, *, seed: int) -> StationCandidate:

```

```

"""Seeded deletion-order check; it has the same analytic acceptance rule."""
required = required_radial_support(candidate.edge_m)
working = set(candidate.stations)
extras = list(working - required)
Random(seed).shuffle(extras)
for point in extras:
    proposal = working - {point}
    if required <= proposal:
        working = proposal
return _candidate_from_stations(
    f"{candidate.name} randomized-pruned seed={seed}", candidate, working,
    "Randomized deletion order, but each accepted deletion retains all R+h support vertices.",
)

def _cross(origin: q4.Point, first: q4.Point, second: q4.Point) -> float:
    return (first.x - origin.x) * (second.y - origin.y) - (first.y - origin.y) * (second.x - origin.x)

def _convex_hull(points: tuple[q4.Point, ...]) -> tuple[q4.Point, ...]:
    ordered = sorted(set(points))
    if len(ordered) <= 1:
        return tuple(ordered)
    lower: list[q4.Point] = []
    for point in ordered:
        while len(lower) >= 2 and _cross(lower[-2], lower[-1], point) <= q4.TOLERANCE:
            lower.pop()
        lower.append(point)
    upper: list[q4.Point] = []
    for point in reversed(ordered):
        while len(upper) >= 2 and _cross(upper[-2], upper[-1], point) <= q4.TOLERANCE:
            upper.pop()
        upper.append(point)
    return tuple(lower[:-1] + upper[:-1])

def source_in_nearby_convex_hull(source: q4.Point, stations: frozenset[q4.Point]) -> tuple[bool, int]:
    """Directly validate the directional sufficient premise at one source."""
    nearby = tuple(point for point in stations if point.distance_to(source) <= q4.MIN_RECEIVE_RADIUS_M +
        ↪ q4.TOLERANCE)
    hull = _convex_hull(nearby)
    if len(hull) == 0:
        return False, 0
    if len(hull) == 1:
        return hull[0].distance_to(source) <= q4.TOLERANCE, len(nearby)
    if len(hull) == 2:
        on_line = abs(_cross(hull[0], hull[1], source)) <= q4.TOLERANCE
        between = (source.x - hull[0].x) * (source.x - hull[1].x) + (source.y - hull[0].y) * (source.y -
            ↪ hull[1].y) <= q4.TOLERANCE
        return on_line and between, len(nearby)
    return all(_cross(first, second, source) >= -q4.TOLERANCE for first, second in zip(hull, hull[1:] +
        ↪ hull[:1])), len(nearby)

def _critical_samples(candidate: StationCandidate) -> tuple[q4.Point, ...]:
    samples: set[q4.Point] = set()
    for triangle in _full_cells(candidate.edge_m):
        if not all(vertex in candidate.stations for vertex in triangle.vertices):

```

```

        continue
    a, b, c = triangle.vertices
    for point in (a, b, c, q4.Point((a.x + b.x) / 2.0, (a.y + b.y) / 2.0),
                  q4.Point((b.x + c.x) / 2.0, (b.y + c.y) / 2.0),
                  q4.Point((c.x + a.x) / 2.0, (c.y + a.y) / 2.0),
                  q4.Point((a.x + b.x + c.x) / 3.0, (a.y + b.y + c.y) / 3.0)):
        if point.radius() <= q4.TARGET_RADIUS_M + q4.TOLERANCE:
            samples.add(point)
    return tuple(sorted(samples))

def _dense_disk_samples(*, radial_step_m: float, angle_step_deg: int) -> tuple[q4.Point, ...]:
    if radial_step_m <= 0.0 or angle_step_deg <= 0 or 360 % angle_step_deg:
        raise ValueError("use a positive radial step and an integer divisor of 360 degrees")
    samples = {q4.Point(0.0, 0.0)}
    radius = radial_step_m
    while radius < q4.TARGET_RADIUS_M - q4.TOLERANCE:
        for angle in range(0, 360, angle_step_deg):
            samples.add(q4.Point(radius * cos(radians(angle)), radius * sin(radians(angle))))
        radius += radial_step_m
    for angle in range(0, 360, angle_step_deg):
        samples.add(q4.Point(q4.TARGET_RADIUS_M * cos(radians(angle)), q4.TARGET_RADIUS_M *
                               ↪ sin(radians(angle))))
    return tuple(sorted(samples))

def audit_candidate(
    candidate: StationCandidate, *, radial_step_m: float = 25.0, angle_step_deg: int = 2
) -> StationCertificateAudit:
    """Check the analytic lattice premise plus deterministic dense/critical witnesses."""
    required = required_radial_support(candidate.edge_m)
    analytic = candidate.edge_m <= q4.MIN_RECEIVE_RADIUS_M + q4.TOLERANCE and required <=
    ↪ candidate.stations
    dense = _dense_disk_samples(radial_step_m=radial_step_m, angle_step_deg=angle_step_deg)
    critical = _critical_samples(candidate)
    minimum_nearby = len(candidate.stations)
    stress_ok = True
    for source in (*dense, *critical):
        contained, nearby_count = source_in_nearby_convex_hull(source, candidate.stations)
        minimum_nearby = min(minimum_nearby, nearby_count)
        if not contained:
            stress_ok = False
            break
    return StationCertificateAudit(
        candidate.name, candidate.edge_m, len(candidate.stations),
        q4._q4_route_length(q4.Point(0.0, 0.0), candidate.route), # noqa: SLF001 - metric only.
        len(required), max(0, len(candidate.stations - required)),
        "PROVED" if analytic else "UNRESOLVED",
        "NUMERICALLY_VERIFIED" if stress_ok else "FAILED",
        len(dense) + len(critical), len(critical), minimum_nearby,
        (
            "For every target point, its containing lattice triangle has all three vertices inside R+h; "
            "these vertices are scheduled, within h<=1000 m, and their convex hull contains the source. "
            "The convex-hull half-plane lemma then supplies a receiver for every closed 180-degree
            ↪ direction."
            if analytic else
            "This candidate does not satisfy the triangular-cell/radial-support sufficient condition."
        ),
    ),

```

)

```
def run_static_experiments() -> tuple[StationCertificateAudit, ...]:
    """CERT-B/C comparison, including greedy/random pruning and requested spacings."""
    current = current_exact_candidate()
    candidates = [current, greedy_redundant_deletion(current)]
    candidates.extend(randomized_redundant_deletion(current, seed=seed) for seed in (17, 53, 97))
    candidates.extend(radial_support_candidate(edge_m) for edge_m in SPACINGS_M)
    return tuple(audit_candidate(candidate) for candidate in candidates)
```

文件: src/b\_problem/experimental/\_\_init\_\_.py

```
"""Isolated, non-production experiments.
```

```
Nothing in this package is imported by the competition entry points.
"""
```

文件: src/b\_problem/experimental/q3\_breakthrough\_port.py

```
"""Direct adapter for the 2026-09-12 Q3 breakthrough sandbox.
```

```
Experimental only: this imports the supplied controller as-is and routes its
public simulator calls to the repository's local OfflineArena.
"""
```

```
from __future__ import annotations
```

```
from dataclasses import dataclass, field
from pathlib import Path
import sys
from typing import Any
```

```
import numpy as np
```

```
from b_problem.benchmark.offline_arena import OfflineArena
from b_problem.benchmark.types import Point
from b_problem.simulator.protocol import Point as OfficialPoint
```

```
EXPERIMENTAL_UNPROVEN = True
```

```
SANDBOX_ROOT = Path(__file__).resolve().parents[3] / "vendor" / "q3_breakthrough_sandbox"
```

```
def _require_offline(arena: Any) -> None:
    if not isinstance(arena, OfflineArena) or any(hasattr(arena, name) for name in ("base_url",
    ↪ "robot_id", "request_id")):
        raise RuntimeError("Q3_BREAKTHROUGH is local OfflineArena only")
```

```
class ArenaAdapter:
```

```
    def __init__(self, arena: OfflineArena) -> None:
        _require_offline(arena); self.arena = arena
```

```
    @property
```

```
    def pos(self) -> np.ndarray:
```

```
        p = self.arena.public_state.position; return np.array([p.x, p.y], dtype=float)
```

```
    @property
```

```

def channel(self) -> int: return self.arena.public_state.measure_channel
def measure(self, position: Any, channel: int):
    obs = self.arena.measure(Point(float(position[0]), float(position[1])), int(channel))
    return obs.result, obs.svd_deg
def clear(self, position: Any, channel: int):
    return self.arena.clear(Point(float(position[0]), float(position[1])), int(channel)),
        ↪ self.arena.virtual_time_s
def stats(self) -> dict[str, float | int | None]:
    trace = self.arena.trace
    cleared = sum(item.action.kind == "clear" and item.result == "success" for item in trace)
    return {"n_sources": 0, "n_cleared": cleared, "ratio": 0.0, "total_time_s":
        ↪ self.arena.virtual_time_s,
            "avg_time_s": None, "n_requests": len(trace), "move_m": sum(item.movement_m for item in
        ↪ trace),
            "t_move": sum(item.movement_m / 5 for item in trace),
            "t_switch": sum(float(item.switched) for item in trace if item.action.kind == "measure"),
            "t_measure": 5 * sum(item.action.kind == "measure" for item in trace),
            "t_clear": sum(5 if item.result == "success" else 3 for item in trace if item.action.kind
        ↪ == "clear")}]

class OfficialArenaAdapter:
    """Tuple-shaped R2 adapter over the validated serial official client."""
    def __init__(self, robot: Any) -> None:
        self._robot = robot
        self.measure_count = self.switch_count = self.successful_clears = self.failed_clears = 0
        self.negative_observations = 0
    @property
    def pos(self) -> np.ndarray:
        p = self._robot.state.position; return np.array([p.x, p.y], dtype=float)
    @property
    def channel(self) -> int: return self._robot.state.measure_channel
    def measure(self, position: Any, channel: int):
        before = self.channel
        response = self._robot.measure(OfficialPoint(float(position[0]), float(position[1])), int(channel))
        self.measure_count += 1; self.switch_count += int(before != int(channel))
        self.negative_observations += int(response.measure_result == "no_signal")
        return response.measure_result, response.svd_deg
    def clear(self, position: Any, channel: int):
        response = self._robot.clear(OfficialPoint(float(position[0]), float(position[1])), int(channel))
        if response.clear_result == "success": self.successful_clears += 1
        else: self.failed_clears += 1
        return response.clear_result, response.virtual_time_s
    def stats(self) -> dict[str, float | int | None]:
        return {"n_sources": 0, "n_cleared": self.successful_clears, "ratio": 0.0,
            "total_time_s": self._robot.state.virtual_time_s, "avg_time_s": None,
            "n_requests": self.measure_count + self.successful_clears + self.failed_clears,
            "move_m": 0., "t_move": 0., "t_switch": float(self.switch_count),
            "t_measure": 5. * self.measure_count,
            "t_clear": 5. * self.successful_clears + 3. * self.failed_clears}

def _load() -> Any:
    if not SANDBOX_ROOT.is_dir(): raise RuntimeError(f"Q3 breakthrough sandbox missing: {SANDBOX_ROOT}")
    for path in (str(SANDBOX_ROOT), str(SANDBOX_ROOT / "lib")):
        if path not in sys.path: sys.path.insert(0, path)
    # Avoid reusing an identically named module from the earlier sandbox.
    for name in ("strategy", "strategy2", "geom", "sim", "run_q3_nextgen", "q3_breakthrough"):

```

```

        old = sys.modules.get(name)
        if old is not None and not str(getattr(old, "__file__", "")).startswith(str(SANDBOX_ROOT)):
            del sys.modules[name]
import q3_breakthrough # type: ignore
return q3_breakthrough

@dataclass
class BreakthroughTelemetry:
    variant: str
    lookahead_choices: int = 0
    band_sweeps: int = 0
    band_attempts: int = 0
    early16: bool = False
    rotation_phase: float = 0.0
    controller_result: dict[str, Any] = field(default_factory=dict)

def execute_q3_breakthrough(arena: OfflineArena, *, variant: str = "r2", telemetry: BreakthroughTelemetry |
↳ None = None) -> BreakthroughTelemetry:
    _require_offline(arena)
    if variant not in {"r1", "r2", "r3"}: raise ValueError("variant must be r1, r2, or r3")
    module = _load(); telemetry = telemetry or BreakthroughTelemetry(variant)
    controller = module.CLASSES[variant](ArenaAdapter(arena), module.PLAN,
        module.Params2(name=f"real_{variant}", theta_miss=0, use_belief=False), 3,
        neg=True, conditional=True, threshold=0, outer_n=9, outer_r=1700, rotate=True)
    telemetry.controller_result = controller.run()
    telemetry.lookahead_choices = int(getattr(controller, "lookahead_choices", 0))
    telemetry.band_sweeps = int(getattr(controller, "band_sweeps", 0))
    telemetry.band_attempts = int(getattr(controller, "band_attempts", 0))
    telemetry.early16 = bool(getattr(controller, "early16", False))
    telemetry.rotation_phase = float(getattr(controller, "base_phase", 0.0))
    return telemetry

def execute_q3_breakthrough_official(robot: Any) -> dict[str, Any]:
    """Run exactly R2 through an official-client adapter; no live configuration here."""
    module = _load(); telemetry = BreakthroughTelemetry("r2")
    adapter = OfficialArenaAdapter(robot)
    controller = module.CLASSES["r2"](adapter, module.PLAN,
        module.Params2(name="official_r2", theta_miss=0, use_belief=False), 3,
        neg=True, conditional=True, threshold=0, outer_n=9, outer_r=1700, rotate=True)
    result = controller.run()
    return {**result, "complete": not result.get("unresolved") and not result.get("gave_up"),
        "algorithm_version": "Q3_BREAKTHROUGH_R2", "strategy_class": type(controller).__name__,
        "lookahead_choices": int(getattr(controller, "lookahead_choices", 0)),
        "band_sweeps": int(getattr(controller, "band_sweeps", 0)),
        "band_attempts": int(getattr(controller, "band_attempts", 0)),
        "negative_observations": adapter.negative_observations, "measure_count": adapter.measure_count,
        "switch_count": adapter.switch_count, "successful_clears": adapter.successful_clears,
        "failed_clears": adapter.failed_clears}

```

文件: src/b\_problem/experimental/sandbox\_exact\_port.py

"""Direct portable-sandbox controller adapter for the local offline harness.

This is intentionally an *\*unproven\** behavior port: it runs the supplied

```

sandbox classes unchanged wherever their simulator API can be represented by
``OfflineArena``. It is isolated from production strategy modules and rejects
anything other than the in-process local benchmark arena.
"""
from __future__ import annotations

from dataclasses import dataclass, field
from pathlib import Path
import sys
from typing import Any

import numpy as np

from b_problem.benchmark.offline_arena import OfflineArena
from b_problem.benchmark.types import Point
from b_problem.simulator.protocol import Point as OfficialPoint

EXPERIMENTAL_UNPROVEN = True
PORTABLE_ROOT = Path(__file__).resolve().parents[3] / "vendor" / "portable_sandbox"

def _offline_only(arena: Any) -> None:
    """Make accidental use with official/online clients fail before an action."""
    if not isinstance(arena, OfflineArena) or any(hasattr(arena, name) for name in ("base_url",
    ↪ "robot_id", "request_id")):
        raise RuntimeError("SANDBOX_EXACT is offline-only; online/official execution is refused")

def _load_portable() -> dict[str, Any]:
    if not PORTABLE_ROOT.is_dir():
        raise RuntimeError(f"portable sandbox missing: {PORTABLE_ROOT}")
    for path in (str(PORTABLE_ROOT), str(PORTABLE_ROOT / "lib")):
        if path not in sys.path:
            sys.path.insert(0, path)
    # Imports intentionally use the portable sandbox's own implementation.
    import strategy # type: ignore
    import strategy2 # type: ignore
    import run_q3_nextgen # type: ignore
    import run_safe7_r1130 # type: ignore
    import run_q4_shared_pair # type: ignore
    import run_q4_shared_pair_bound # type: ignore
    import run_q4_routingplus # type: ignore
    import run_q4_routingplus_bound # type: ignore
    strategy.MARGIN = 2.0
    return {"strategy2": strategy2, "q3": run_q3_nextgen, "safe7": run_safe7_r1130,
        "pair": run_q4_shared_pair, "pair_bound": run_q4_shared_pair_bound,
        "routing": run_q4_routingplus, "routing_bound": run_q4_routingplus_bound}

class ArenaSimulatorAdapter:
    """Sandbox ``Simulator`` surface mapped to real local-harness billing."""
    def __init__(self, arena: OfflineArena) -> None:
        _offline_only(arena)
        self.arena = arena

    @property
    def pos(self) -> np.ndarray:
        point = self.arena.public_state.position

```

```

        return np.array([point.x, point.y], dtype=float)

@property
def channel(self) -> int:
    return self.arena.public_state.measure_channel

def measure(self, position: Any, channel: int) -> tuple[str, float | None]:
    point = Point(float(position[0]), float(position[1]))
    response = self.arena.measure(point, int(channel))
    return response.result, response.svd_deg

def clear(self, position: Any, channel: int) -> tuple[str, float]:
    point = Point(float(position[0]), float(position[1]))
    result = self.arena.clear(point, int(channel))
    return result, self.arena.virtual_time_s

def stats(self) -> dict[str, float | int | None]:
    trace = self.arena.trace
    cleared = sum(item.action.kind == "clear" and item.result == "success" for item in trace)
    return {"n_sources": 0, "n_cleared": cleared, "ratio": 0.0,
            "total_time_s": self.arena.virtual_time_s, "avg_time_s": None,
            "n_requests": len(trace), "move_m": sum(item.movement_m for item in trace),
            "t_move": sum(item.movement_m / 5.0 for item in trace),
            "t_switch": sum(float(item.switched) for item in trace if item.action.kind == "measure"),
            "t_measure": 5.0 * sum(item.action.kind == "measure" for item in trace),
            "t_clear": sum(5.0 if item.result == "success" else 3.0 for item in trace if
                           ↪ item.action.kind == "clear")}]

class OfficialSimulatorAdapter:
    """Portable sandbox simulator API over the already-validated HTTP client."""
    def __init__(self, robot: Any) -> None:
        self.robot=robot; self.measures=self.switches=self.successes=self.failures=0
    @property
    def pos(self) -> np.ndarray:
        p=self.robot.state.position; return np.array([p.x,p.y],float)
    @property
    def channel(self) -> int: return self.robot.state.measure_channel
    def measure(self, position: Any, channel: int):
        old=self.channel;
        ↪ r=self.robot.measure(OfficialPoint(float(position[0]),float(position[1])),int(channel))
        self.measures+=1; self.switches+=int(old!=int(channel)); return r.measure_result,r.svd_deg
    def clear(self, position: Any, channel: int):
        r=self.robot.clear(OfficialPoint(float(position[0]),float(position[1])),int(channel))
        self.successes+=int(r.clear_result=="success"); self.failures+=int(r.clear_result!="success");
        ↪ return r.clear_result,r.virtual_time_s
    def stats(self) -> dict[str,float|int|None]:
        return {"n_sources":0,"n_cleared":self.successes,"ratio":0.,"total_time_s":self.robot.state.virtu
        ↪ al_time_s,"avg_time_s":None,"n_requests":self.measures+self.successes+self.failures,"move_m":
        ↪ 0.,"t_move":0.,"t_switch":float(self.switches),"t_measure":5.*self.measures,"t_clear":5.*self
        ↪ .successes+3.*self.failures}

@dataclass
class ExactTelemetry:
    family: str
    pair_events: list[dict[str, Any]] = field(default_factory=list)
    controller_result: dict[str, Any] = field(default_factory=dict)

```

```

def _audit_pair_class(base: type, telemetry: ExactTelemetry) -> type:
    """Record polygon transitions only; target truth is audited after execution."""
    class Audited(base): # type: ignore[misc, valid-type]
        def _record_nosig(self, channel: int, point: Any) -> None:
            before = [np.asarray(vertex, float).tolist() for vertex in self.tracked.get(channel, [])]
            clips = getattr(self, "pair_clips", 0)
            super()._record_nosig(channel, point)
            if getattr(self, "pair_clips", 0) > clips:
                after = [np.asarray(vertex, float).tolist() for vertex in self.tracked.get(channel, [])]
                telemetry.pair_events.append({"channel": int(channel), "before": before, "after": after,
                                             "probe": np.asarray(point, float).tolist()})
    return Audited

def execute_q4_sandbox_exact(arena: OfflineArena, *, variant: str, telemetry: ExactTelemetry | None =
↳ None) -> ExactTelemetry:
    _offline_only(arena)
    if variant not in {"PAIR_ORIGINAL", "PAIR_BOUND", "FULL", "FULL_NN"}:
        raise ValueError("invalid Q4 sandbox-exact variant")
    modules = _load_portable(); telemetry = telemetry or ExactTelemetry(variant)
    sim = ArenaSimulatorAdapter(arena); params = modules["strategy2"].Params2(name=f"exact_{variant}",
↳ theta_miss=0, use_belief=False)
    plan = {"level1": [list(point) for point in modules["pair"].PTS21], "escalation": []}
    if variant == "PAIR_ORIGINAL":
        cls = _audit_pair_class(modules["pair"].SharedPairStrict, telemetry)
        controller = cls(sim, plan, params, 4, shared_radius=50, fresh_only=True)
    elif variant == "PAIR_BOUND":
        cls = _audit_pair_class(modules["pair_bound"].SharedPairBoundStrict, telemetry)
        controller = cls(sim, plan, params, 4, shared_radius=50, fresh_only=True)
    else:
        cls = _audit_pair_class(modules["routing"].RoutingPlusPair, telemetry)
        controller = cls(sim, plan, params, 4, shared_radius=50, fresh_only=True,
↳ oropt=variant == "FULL", warm=variant == "FULL", starts=4 if variant == "FULL"
↳ else 1)
    telemetry.controller_result = controller.run()
    return telemetry

def execute_q3_sandbox_exact(arena: OfflineArena, *, variant: str, telemetry: ExactTelemetry | None =
↳ None) -> ExactTelemetry:
    _offline_only(arena)
    if variant not in {"NEXTGEN", "SAFE7", "FULL"}:
        raise ValueError("invalid Q3 sandbox-exact variant")
    modules = _load_portable(); telemetry = telemetry or ExactTelemetry(variant)
    sim = ArenaSimulatorAdapter(arena); params = modules["strategy2"].Params2(name=f"exact_{variant}",
↳ theta_miss=0, use_belief=False)
    if variant == "SAFE7":
        controller = modules["safe7"].StrictFiltered(sim, modules["safe7"].PLAN, params, 3)
    else:
        cfg = {"base": dict(neg=False, conditional=False, rotate=False),
↳ "full": dict(neg=True, conditional=True, rotate=True)}["base" if variant == "NEXTGEN" else
↳ "full"]
        controller = modules["q3"].Q3NegDynamic(sim, modules["q3"].PLAN, params, 3, **cfg)
    telemetry.controller_result = controller.run()
    return telemetry

```

```

def execute_q4_sandbox_exact_official(robot: Any) -> dict[str, Any]:
    """Q4 rehearsal-only FULL routing controller, explicitly PAIR_BOUND."""
    modules=_load_portable(); telemetry=ExactTelemetry("FULL_PAIR_BOUND_ROUTINGPLUS")
    sim=OfficialSimulatorAdapter(robot);
    ↪ params=modules["strategy2"].Params2(name="official_full_pair_bound",theta_miss=0,use_belief=False)
    plan={"level1":[list(point) for point in modules["pair"].PTS21],"escalation":[]}
    cls=_audit_pair_class(modules["routing_bound"].RoutingPlusPair,telemetry)
    controller=cls(sim,plan,params,4,shared_radius=50,fresh_only=True,oropt=True,warm=True,starts=4)
    result=controller.run(); telemetry.controller_result=result
    return {**result,"complete":not result.get("unresolved") and not result.get("gave_up"),"algorithm_ver
    ↪ sion":"Q4_SANDBOX_EXACT_FULL","strategy_class":type(controller).__name__,"pair_mode":"BOUND","rou
    ↪ ting":"ROUTINGPLUS","pair_events":telemetry.pair_events,"measure_count":sim.measures,"switch_coun
    ↪ t":sim.switches,"successful_clears":sim.successes,"failed_clears":sim.failures}

```

文件: src/b\_problem/geometry/\_\_init\_\_.py

```

"""Robust planar geometry primitives for bearing localization."""

from .core import (
    Circle,
    CurvilinearMetricCertification,
    CurvilinearRegion,
    DiameterResult,
    HalfPlane,
    IntersectionKind,
    MetricCertificationStatus,
    MetricInterval,
    Point,
    bearing_wedge,
    certify_curvilinear_metrics,
    clip_to_target_disk,
    intersect_halfplanes,
    minimum_enclosing_circle,
    polygon_diameter_bruteforce,
    polygon_diameter_calipers,
    regular_polygon_inner_error,
)

__all__ = [
    "Circle", "CurvilinearMetricCertification", "CurvilinearRegion", "DiameterResult", "HalfPlane",
    "IntersectionKind", "MetricCertificationStatus", "MetricInterval", "Point", "bearing_wedge",
    "certify_curvilinear_metrics", "clip_to_target_disk", "intersect_halfplanes",
    ↪ "minimum_enclosing_circle",
    "polygon_diameter_bruteforce", "polygon_diameter_calipers",
    "regular_polygon_inner_error",
]

```

文件: src/b\_problem/geometry/core.py

```

"""Small, dependency-free primitives for closed planar half-planes."""
from __future__ import annotations

from dataclasses import dataclass
from enum import Enum

```

```

import math
from itertools import combinations

from scipy.optimize import linprog

@dataclass(frozen=True)
class Point:
    x: float
    y: float

    def __sub__(self, other: "Point") -> "Point":
        return Point(self.x - other.x, self.y - other.y)

def _cross(a: Point, b: Point) -> float:
    return a.x * b.y - a.y * b.x

@dataclass(frozen=True)
class HalfPlane:
    """The closed left side of a directed line through ``origin``."""

    origin: Point
    direction: Point
    keep_left: bool = True

    def __post_init__(self) -> None:
        if not math.isfinite(self.direction.x) or not math.isfinite(self.direction.y) or
        ↪ math.hypot(self.direction.x, self.direction.y) == 0.0:
            raise ValueError("half-plane direction must be finite and non-zero")

    def contains(self, point: Point, tolerance: float = 1e-10) -> bool:
        signed = _cross(self.direction, point - self.origin)
        scaled_tolerance = tolerance * math.hypot(self.direction.x, self.direction.y)
        return signed >= -scaled_tolerance if self.keep_left else signed <= scaled_tolerance

    def normal_and_offset(self) -> tuple[Point, float]:
        """Return ``n, b`` for the equivalent constraint ``n dot x >= b``."""
        normal = Point(-self.direction.y, self.direction.x)
        if not self.keep_left:
            normal = Point(-normal.x, -normal.y)
        return normal, normal.x * self.origin.x + normal.y * self.origin.y

@dataclass(frozen=True)
class BearingWedge:
    left_boundary: HalfPlane
    right_boundary: HalfPlane

    def contains(self, point: Point, tolerance: float = 1e-10) -> bool:
        return self.left_boundary.contains(point, tolerance) and self.right_boundary.contains(point,
        ↪ tolerance)

def bearing_wedge(station: Point, bearing_rad: float, error_rad: float = math.radians(1.0)) ->
↪ BearingWedge:
    """Return the closed angular wedge centered at a bearing.

```

```

Unit vectors avoid any special branch at 0/360 degrees. The construction
is convex exactly for the supported small-error case ``2*error_rad <= pi``.
"""
if not 0.0 <= error_rad <= math.pi / 2:
    raise ValueError("bearing wedge requires an error angle in [0, pi/2]")
left_angle = bearing_rad - error_rad
right_angle = bearing_rad + error_rad
left = Point(math.cos(left_angle), math.sin(left_angle))
right = Point(math.cos(right_angle), math.sin(right_angle))
return BearingWedge(HalfPlane(station, left, True), HalfPlane(station, right, False))

class IntersectionKind(str, Enum):
    EMPTY = "empty"
    UNBOUNDED = "unbounded"
    POINT = "point"
    SEGMENT = "segment"
    POLYGON = "polygon"

@dataclass(frozen=True)
class HalfPlaneIntersection:
    kind: IntersectionKind
    vertices: tuple[Point, ...] = ()

def _line_intersection(first: HalfPlane, second: HalfPlane, tolerance: float) -> Point | None:
    a, b = first.normal_and_offset()
    c, d = second.normal_and_offset()
    determinant = a.x * c.y - a.y * c.x
    scale = math.hypot(a.x, a.y) * math.hypot(c.x, c.y)
    if abs(determinant) <= tolerance * scale:
        return None
    return Point((b * c.y - a.y * d) / determinant, (a.x * d - b * c.x) / determinant)

def _deduplicate(points: list[Point], tolerance: float) -> list[Point]:
    unique: list[Point] = []
    for point in points:
        if not any(math.hypot(point.x - known.x, point.y - known.y) <= tolerance for known in unique):
            unique.append(point)
    return unique

def _ordered_hull(points: list[Point], tolerance: float) -> tuple[IntersectionKind, tuple[Point, ...]]:
    if len(points) == 1:
        return IntersectionKind.POINT, (points[0],)
    if len(points) == 2:
        return IntersectionKind.SEGMENT, tuple(points)

    center = Point(sum(p.x for p in points) / len(points), sum(p.y for p in points) / len(points))
    ordered = sorted(points, key=lambda p: math.atan2(p.y - center.y, p.x - center.x))
    twice_area = abs(sum(
        ordered[i].x * ordered[(i + 1) % len(ordered)].y
        - ordered[i].y * ordered[(i + 1) % len(ordered)].x
        for i in range(len(ordered))
    ))
    if twice_area <= tolerance:

```

```

        first, second = max(combinations(points, 2), key=lambda pair: math.hypot(pair[0].x - pair[1].x,
        ↪ pair[0].y - pair[1].y))
        return IntersectionKind.SEGMENT, (first, second)
return IntersectionKind.POLYGON, tuple(ordered)

def intersect_halfplanes(halfplanes: list[HalfPlane], tolerance: float = 1e-10) -> HalfPlaneIntersection:
    """Classify the intersection of closed half-planes in the full plane.

    Feasibility and unboundedness are delegated to HiGHS through frozen SciPy;
    finite vertices are reconstructed only from non-parallel boundary pairs.
    """
    if not halfplanes:
        return HalfPlaneIntersection(IntersectionKind.UNBOUNDED)
    if tolerance <= 0.0:
        raise ValueError("tolerance must be positive")
    # HiGHS applies feasibility tolerances to the submitted coefficients.
    # Normalize each line first, otherwise an equivalent rescaling changes
    # whether a narrow contradiction is reported feasible.
    constraints = []
    for plane in halfplanes:
        normal, offset = plane.normal_and_offset()
        scale = math.hypot(normal.x, normal.y)
        constraints.append((Point(normal.x / scale, normal.y / scale), offset / scale))
    a_ub = [[-normal.x, -normal.y] for normal, _ in constraints]
    b_ub = [-offset for _, offset in constraints]
    solver_options = {
        "primal_feasibility_tolerance": max(tolerance, 1e-10),
        "dual_feasibility_tolerance": max(tolerance, 1e-10),
    }
    feasibility = linprog(
        [0.0, 0.0], A_ub=a_ub, b_ub=b_ub, bounds=[(None, None), (None, None)],
        method="highs", options=solver_options,
    )
    if feasibility.status == 2:
        return HalfPlaneIntersection(IntersectionKind.EMPTY)
    if not feasibility.success:
        raise RuntimeError(f"half-plane feasibility failed: {feasibility.message}")

    for objective in ([1.0, 0.0], [-1.0, 0.0], [0.0, 1.0], [0.0, -1.0]):
        bound = linprog(
            objective, A_ub=a_ub, b_ub=b_ub, bounds=[(None, None), (None, None)],
            method="highs", options=solver_options,
        )
        if bound.status == 3:
            return HalfPlaneIntersection(IntersectionKind.UNBOUNDED)
        if not bound.success:
            raise RuntimeError(f"half-plane bound failed: {bound.message}")

    candidates = []
    for first, second in combinations(halfplanes, 2):
        point = _line_intersection(first, second, tolerance)
        if point is not None and all(plane.contains(point, tolerance) for plane in halfplanes):
            candidates.append(point)
    unique = _deduplicate(candidates, tolerance)
    if not unique:
        point = Point(float(feasibility.x[0]), float(feasibility.x[1]))
        return HalfPlaneIntersection(IntersectionKind.POINT, (point,))

```

```

kind, vertices = _ordered_hull(unique, tolerance)
return HalfPlaneIntersection(kind, vertices)

@dataclass(frozen=True)
class DiameterResult:
    distance: float
    first: Point
    second: Point

def _distance(first: Point, second: Point) -> float:
    return math.hypot(first.x - second.x, first.y - second.y)

def polygon_diameter_bruteforce(vertices: tuple[Point, ...]) -> DiameterResult:
    """Return a convex polygon's diameter by exhaustive vertex-pair search."""
    if not vertices:
        raise ValueError("diameter is undefined for an empty polygon")
    if len(vertices) == 1:
        return DiameterResult(0.0, vertices[0], vertices[0])
    first, second = max(combinations(vertices, 2), key=lambda pair: _distance(*pair))
    return DiameterResult(_distance(first, second), first, second)

def _triangle_area2(first: Point, second: Point, third: Point) -> float:
    return abs(_cross(second - first, third - first))

def _validate_ccw_convex_polygon(vertices: tuple[Point, ...], tolerance: float) -> None:
    """Reject sequences for which antipodal-pair traversal has no proof."""
    if len(vertices) <= 2:
        return
    if len({(point.x, point.y) for point in vertices}) != len(vertices):
        raise ValueError("calipers requires distinct polygon vertices")
    turns = [_cross(vertices[(index + 1) % len(vertices)] - vertices[index], vertices[(index + 2) %
    ↪ len(vertices)] - vertices[(index + 1) % len(vertices)]) for index in range(len(vertices))]
    if any(turn < -tolerance for turn in turns) or not any(turn > tolerance for turn in turns):
        raise ValueError("calipers requires a non-degenerate CCW convex polygon")

    points = sorted(vertices, key=lambda point: (point.x, point.y))

    def build_half(sequence: list[Point]) -> list[Point]:
        hull: list[Point] = []
        for point in sequence:
            while len(hull) >= 2 and _cross(hull[-1] - hull[-2], point - hull[-1]) <= tolerance:
                hull.pop()
            hull.append(point)
        return hull

    hull = tuple(build_half(points)[: -1] + build_half(list(reversed(points)))[: -1])
    if len(hull) < 3 or abs(_polygon_area2(vertices) - _polygon_area2(hull)) > tolerance * len(vertices):
        raise ValueError("calipers requires vertices ordered once around their CCW convex hull")

def polygon_diameter_calipers(vertices: tuple[Point, ...], tolerance: float = 1e-10) -> DiameterResult:
    """Return the diameter of a CCW convex polygon via antipodal pairs."""
    count = len(vertices)

```

```

if count <= 2:
    return polygon_diameter_bruteforce(vertices)
_validate_ccw_convex_polygon(vertices, tolerance)
best = polygon_diameter_bruteforce(vertices[:2])
opposite = 1
for index in range(count):
    following = (index + 1) % count
    while _triangle_area2(vertices[index], vertices[following], vertices[(opposite + 1) % count]) > (
        _triangle_area2(vertices[index], vertices[following], vertices[opposite]) + tolerance
    ):
        opposite = (opposite + 1) % count
    for pair in ((vertices[index], vertices[opposite]), (vertices[following], vertices[opposite]]):
        candidate = DiameterResult(_distance(*pair), *pair)
        if candidate.distance > best.distance:
            best = candidate
    next_area = _triangle_area2(vertices[index], vertices[following], vertices[(opposite + 1) % count])
    current_area = _triangle_area2(vertices[index], vertices[following], vertices[opposite])
    if abs(next_area - current_area) <= tolerance:
        pair = (vertices[index], vertices[(opposite + 1) % count])
        candidate = DiameterResult(_distance(*pair), *pair)
        if candidate.distance > best.distance:
            best = candidate
return best

@dataclass(frozen=True)
class Circle:
    center: Point
    radius: float

    def contains(self, point: Point, tolerance: float = 1e-10) -> bool:
        return _distance(self.center, point) <= self.radius + tolerance

@dataclass(frozen=True)
class CurvilinearRegion:
    """An exact disk-and-half-plane region; its circular constraint is retained."""

    disk: Circle
    halfplanes: tuple[HalfPlane, ...]

    @property
    def has_circular_boundary(self) -> bool:
        return True

    @property
    def is_polygon(self) -> bool:
        return False

    def contains(self, point: Point, tolerance: float = 1e-10) -> bool:
        return self.disk.contains(point, tolerance) and all(plane.contains(point, tolerance) for plane in
        ↪ self.halfplanes)

def clip_to_target_disk(halfplanes: list[HalfPlane], radius: float = 1800.0, center: Point = Point(0.0,
↪ 0.0)) -> CurvilinearRegion:
    """Keep the target circle exact while intersecting it with closed half-planes."""
    if radius <= 0.0:

```

```

        raise ValueError("target disk radius must be positive")
    return CurvilinearRegion(Circle(center, radius), tuple(halfplanes))

def regular_polygon_inner_error(radius: float, sides: int) -> float:
    """Return the maximum radial gap to an inscribed regular polygon.

    The bound is the sagitta ``R * (1 - cos(pi / sides))``. It certifies
    only the disk approximation; any clipped-region use must propagate it.
    """
    if radius <= 0.0:
        raise ValueError("radius must be positive")
    if sides < 3:
        raise ValueError("an inscribed polygon needs at least three sides")
    return radius * (1.0 - math.cos(math.pi / sides))

def _circle_from_pair(first: Point, second: Point) -> Circle:
    center = Point((first.x + second.x) / 2.0, (first.y + second.y) / 2.0)
    return Circle(center, _distance(first, second) / 2.0)

def _circle_from_triple(first: Point, second: Point, third: Point, tolerance: float) -> Circle | None:
    determinant = 2.0 * _cross(second - first, third - first)
    if abs(determinant) <= tolerance:
        return None
    first_norm = first.x * first.x + first.y * first.y
    second_norm = second.x * second.x + second.y * second.y
    third_norm = third.x * third.x + third.y * third.y
    center = Point(
        (first_norm * (second.y - third.y) + second_norm * (third.y - first.y) + third_norm * (first.y -
        ↪ second.y)) / determinant,
        (first_norm * (third.x - second.x) + second_norm * (first.x - third.x) + third_norm * (second.x -
        ↪ first.x)) / determinant,
    )
    return Circle(center, _distance(center, first))

def minimum_enclosing_circle(points: tuple[Point, ...], tolerance: float = 1e-10) -> Circle:
    """Return the exact MEC by incremental one-, two-, and three-point supports."""
    if not points:
        raise ValueError("minimum enclosing circle is undefined for no points")
    circle = Circle(points[0], 0.0)
    for first_index, first in enumerate(points):
        if circle.contains(first, tolerance):
            continue
        circle = Circle(first, 0.0)
        for second_index in range(first_index):
            second = points[second_index]
            if circle.contains(second, tolerance):
                continue
            circle = _circle_from_pair(first, second)
            for third_index in range(second_index):
                third = points[third_index]
                if circle.contains(third, tolerance):
                    continue
            triple = _circle_from_triple(first, second, third, tolerance)
            if triple is None:

```

```

        pairs = (
            _circle_from_pair(first, second), _circle_from_pair(first, third),
            ⇨ _circle_from_pair(second, third),
        )
        circle = min(
            (candidate for candidate in pairs if all(candidate.contains(point, tolerance) for
            ⇨ point in (first, second, third))),
            key=lambda candidate: candidate.radius,
        )
    else:
        circle = triple
    return circle

class MetricCertificationStatus(str, Enum):
    """Whether polygonal bounds prove metrics for a full-dimensional region."""

    PROVED = "PROVED"
    UNRESOLVED = "UNRESOLVED"

@dataclass(frozen=True)
class MetricInterval:
    """Monotone lower/upper bound for one compact-set metric."""

    lower: float
    upper: float

    @property
    def error(self) -> float:
        return self.upper - self.lower

@dataclass(frozen=True)
class CurvilinearMetricCertification:
    """Certified metric intervals from inner/outer disk polygon approximations."""

    status: MetricCertificationStatus
    diameter: MetricInterval | None
    mec_radius: MetricInterval | None
    sides: int
    reason: str

def _polygon_area2(vertices: tuple[Point, ...]) -> float:
    return sum(
        vertices[index].x * vertices[(index + 1) % len(vertices)].y
        - vertices[index].y * vertices[(index + 1) % len(vertices)].x
        for index in range(len(vertices))
    )

def _deduplicate_cycle(points: list[Point], tolerance: float) -> tuple[Point, ...]:
    unique: list[Point] = []
    for point in points:
        if not unique or _distance(point, unique[-1]) > tolerance:
            unique.append(point)
    if len(unique) > 1 and _distance(unique[0], unique[-1]) <= tolerance:

```

```

        unique.pop()
    return tuple(unique)

def _clip_convex_polygon(
    vertices: tuple[Point, ...], plane: HalfPlane, tolerance: float, *, inward: bool
) -> tuple[Point, ...]:
    """Clip a CCW convex polygon to one closed half-plane."""
    if not vertices:
        return ()

    def margin(point: Point) -> float:
        signed = _cross(plane.direction, point - plane.origin)
        return signed if plane.keep_left else -signed

    # Inner bounds use a slightly shrunken half-plane; outer bounds use the
    # matching expansion. This keeps finite arithmetic from reversing the
    # required subset/superset relation.
    threshold = (1.0 if inward else -1.0) * tolerance * math.hypot(plane.direction.x, plane.direction.y)
    clipped: list[Point] = []
    previous = vertices[-1]
    previous_margin = margin(previous)
    previous_inside = previous_margin >= threshold
    for current in vertices:
        current_margin = margin(current)
        current_inside = current_margin >= threshold
        if current_inside != previous_inside:
            fraction = (previous_margin - threshold) / (previous_margin - current_margin)
            clipped.append(Point(
                previous.x + fraction * (current.x - previous.x),
                previous.y + fraction * (current.y - previous.y),
            ))
        if current_inside:
            clipped.append(current)
        previous, previous_margin, previous_inside = current, current_margin, current_inside
    return _deduplicate_cycle(clipped, tolerance)

def _clip_disk_polygon(
    vertices: tuple[Point, ...], halfplanes: tuple[HalfPlane, ...], tolerance: float, *, inward: bool
) -> tuple[Point, ...]:
    clipped = vertices
    for plane in halfplanes:
        clipped = _clip_convex_polygon(clipped, plane, tolerance, inward=inward)
        if not clipped:
            break
    return clipped

def _regular_disk_polygon(circle: Circle, sides: int, *, outer: bool) -> tuple[Point, ...]:
    """Return an inscribed polygon or a circumscribed polygon containing a disk."""
    radial = circle.radius / math.cos(math.pi / sides) if outer else circle.radius
    phase = math.pi / sides if outer else 0.0
    return tuple(
        Point(circle.center.x + radial * math.cos(phase + 2.0 * math.pi * index / sides),
              circle.center.y + radial * math.sin(phase + 2.0 * math.pi * index / sides))
        for index in range(sides)
    )

```

```

def certify_curvilinear_metrics(
    region: CurvilinearRegion, *, sides: int = 256, tolerance: float = 1e-10
) -> CurvilinearMetricCertification:
    """Bound diameter and MEC radius of a disk-and-half-plane region.

    The inscribed disk polygon clipped by the half-planes is a subset of the
    true region; clipping a circumscribed disk polygon yields a superset.
    Diameter and MEC radius are monotone under set inclusion, so their two
    polygon metrics are rigorous lower/upper bounds. A non-positive-area
    inner polygon cannot certify a full-dimensional region and is returned as
    ``UNRESOLVED`` rather than assigned metrics for a point or segment.
    """
    if sides < 8:
        raise ValueError("at least eight polygon sides are required")
    if tolerance <= 0.0:
        raise ValueError("tolerance must be positive")

    inner = _clip_disk_polygon(
        _regular_disk_polygon(region.disk, sides, outer=False), region.halfplanes, tolerance, inward=True,
    )
    outer = _clip_disk_polygon(
        _regular_disk_polygon(region.disk, sides, outer=True), region.halfplanes, tolerance, inward=False,
    )
    if len(inner) < 3 or abs(_polygon_area2(inner)) <= tolerance:
        return CurvilinearMetricCertification(
            MetricCertificationStatus.UNRESOLVED, None, None, sides,
            "The inner approximation has no certified positive area; the exact region may be empty or
            ↪ lower-dimensional.",
        )
    if len(outer) < 3 or abs(_polygon_area2(outer)) <= tolerance:
        return CurvilinearMetricCertification(
            MetricCertificationStatus.UNRESOLVED, None, None, sides,
            "The outer approximation is lower-dimensional; no finite full-dimensional metric bracket is
            ↪ certified.",
        )

    inner_diameter = polygon_diameter_calipers(inner, tolerance).distance
    outer_diameter = polygon_diameter_calipers(outer, tolerance).distance
    inner_mec = minimum_enclosing_circle(inner, tolerance).radius
    outer_mec = minimum_enclosing_circle(outer, tolerance).radius
    return CurvilinearMetricCertification(
        MetricCertificationStatus.PROVED,
        MetricInterval(inner_diameter, outer_diameter),
        MetricInterval(inner_mec, outer_mec),
        sides,
        "Inner and outer convex polygon bounds certify the displayed metric intervals.",
    )

```

文件: src/b\_problem/localization/\_\_init\_\_.py

```

"""Localization components for CUMCM 2026 B problem."""

```

```

from .robust import (
    CandidateEvaluation,
    DirectionResult,
    FirstFeasibleRegion,

```

```

    Point,
    PosteriorScenario,
    ReceptionCertificate,
    evaluate_candidate,
    evaluate_certified_candidate,
    safe_seed_certificates,
    universal_reception_certificate,
)

__all__ = [
    "CandidateEvaluation",
    "DirectionResult",
    "FirstFeasibleRegion",
    "Point",
    "PosteriorScenario",
    "ReceptionCertificate",
    "evaluate_candidate",
    "evaluate_certified_candidate",
    "safe_seed_certificates",
    "universal_reception_certificate",
]

```

文件: src/b\_problem/localization/q4\_all\_bearing.py

```

"""Diagnostic all-positive-bearing geometry for Q4 SMAR experiments.

This module deliberately separates a numeric fit from the conservative
measurement set.  ``all_positive_bearing_fit`` is a heuristic diagnostic only;
it never certifies a clear candidate.  ``point_satisfies_bearing_set`` checks
the public +/- angular-error wedges and receive-range bounds independently.
"""

from __future__ import annotations

from dataclasses import dataclass
from math import atan2, cos, degrees, hypot, radians, sin
from typing import Iterable

from b_problem.strategy.q4 import Point

ANGLE_TOLERANCE_DEG = 1.0
RANK_EPSILON = 1e-9

@dataclass(frozen=True)
class BearingObservation:
    position: Point
    bearing_deg: float

@dataclass(frozen=True)
class AllBearingFit:
    estimate: Point | None
    observation_count: int
    condition_number: float | None
    max_perpendicular_residual_m: float | None
    degenerate: bool

```

```

reason: str

def _unit(bearing_deg: float) -> tuple[float, float]:
    angle = radians(bearing_deg)
    return cos(angle), sin(angle)

def _wrapped_delta_deg(first: float, second: float) -> float:
    return (first - second + 180.0) % 360.0 - 180.0

def all_positive_bearing_fit(observations: Iterable[BearingObservation]) -> AllBearingFit:
    """Fit every positive bearing ray with 2-D perpendicular least squares.

    The normal matrix is also the geometry diagnostic: a nearly zero small
    eigenvalue represents collinear/parallel rays and produces no estimate.
    """
    rows = tuple(observations)
    if len(rows) < 2:
        return AllBearingFit(None, len(rows), None, None, True, "insufficient_observations")
    a00 = a01 = a11 = b0 = b1 = 0.0
    normals: list[tuple[float, float, float]] = []
    for observation in rows:
        dx, dy = _unit(observation.bearing_deg)
        nx, ny = -dy, dx
        rhs = nx * observation.position.x + ny * observation.position.y
        normals.append((nx, ny, rhs))
        a00 += nx * nx
        a01 += nx * ny
        a11 += ny * ny
        b0 += nx * rhs
        b1 += ny * rhs
    trace = a00 + a11
    discriminant = max(0.0, (a00 - a11) ** 2 + 4.0 * a01 * a01)
    root = discriminant ** 0.5
    largest, smallest = (trace + root) / 2.0, (trace - root) / 2.0
    if smallest <= RANK_EPSILON * max(1.0, largest):
        return AllBearingFit(None, len(rows), None, None, True, "rank_deficient")
    determinant = a00 * a11 - a01 * a01
    estimate = Point((a11 * b0 - a01 * b1) / determinant, (a00 * b1 - a01 * b0) / determinant)
    residuals = [abs(nx * estimate.x + ny * estimate.y - rhs) for nx, ny, rhs in normals]
    return AllBearingFit(
        estimate, len(rows), largest / smallest, max(residuals, default=0.0), False, "well_conditioned"
    )

def point_satisfies_bearing_set(
    point: Point,
    observations: Iterable[BearingObservation],
    *,
    max_range_m: float,
    angle_tolerance_deg: float = ANGLE_TOLERANCE_DEG,
) -> bool:
    """Conservative public feasibility check; intentionally unrelated to LS fit."""
    for observation in observations:
        dx, dy = point.x - observation.position.x, point.y - observation.position.y
        if hypot(dx, dy) > max_range_m + 1e-9:
            return False

```

```

    if hypot(dx, dy) <= 1e-12:
        continue
    bearing = degrees(atan2(dy, dx)) % 360.0
    if abs(_wrapped_delta_deg(bearing, observation.bearing_deg)) > angle_tolerance_deg + 1e-12:
        return False
    return True

```

文件: src/b\_problem/localization/q4\_pair\_certificate.py

```

"""Truth-independent in-range predicates for Q4 paired-probe research."""
from __future__ import annotations
from dataclasses import dataclass, field
from .robust import Point

@dataclass(frozen=True)
class InRangeCertificate:
    passed:bool; method:str; margin:float

@dataclass
class PairBinding:
    """Identity-bound pair probes for an interruptible Q4 controller.

    This object is deliberately stateful: a response can be consumed once only
    by the exact generated pair, channel and designed coordinate. It records
    evidence; it does not itself authorise a directional-shadow cut.
    """
    generation:int
    channel:int
    snapshot_version:int
    probes:tuple[Point,Point]
    vertices:tuple[Point,...]
    consumed_actions:set[str]=field(default_factory=set)
    consumed_probe_indices:set[int]=field(default_factory=set)

@dataclass(frozen=True)
class BoundPairObservation:
    accepted:bool
    reason:str
    generation:int
    probe_index:int|None=None

@dataclass(frozen=True)
class PairEvidence:
    first:BoundPairObservation
    second:BoundPairObservation

    @property
    def complete(self)->bool:
        return (self.first.accepted and self.second.accepted and
                self.first.generation == self.second.generation and
                self.first.probe_index != self.second.probe_index)

def bind_pair_observation(binding:PairBinding, channel:int, point:Point, result:str, *, action_id:str,

```

```

        tolerance_m:float=1e-6)->BoundPairObservation:
    """Accept only one exact, designed ``no_signal`` response for a pair probe."""
    if channel != binding.channel:
        return BoundPairObservation(False,"channel_mismatch",binding.generation)
    if result != "no_signal":
        return BoundPairObservation(False,"result_not_no_signal",binding.generation)
    if action_id in binding.consumed_actions:
        return BoundPairObservation(False,"duplicate_action",binding.generation)
    matches=[index for index,probe in enumerate(binding.probes) if point.distance_to(probe)<=tolerance_m]
    if len(matches) != 1:
        return BoundPairObservation(False,"coordinate_mismatch",binding.generation)
    index=matches[0]
    if index in binding.consumed_probe_indices:
        return BoundPairObservation(False,"duplicate_probe",binding.generation,index)
    binding.consumed_actions.add(action_id)
    binding.consumed_probe_indices.add(index)
    return BoundPairObservation(True,"bound",binding.generation,index)

def certify_min_radius(q:Point,vertices:tuple[Point,...],*,radius:float=1000.,margin:float=.1)->InRangeCertificate:
    maximum=max(q.distance_to(v) for v in vertices)
    return InRangeCertificate(maximum<radius-margin,"minimum_receive_radius",radius-margin-maximum)

def certify_positive_dominance(q:Point,p:Point,vertices:tuple[Point,...],*,margin_sq:float=.001)->InRangeCertificate:
    values=((q.x*q.x+q.y*q.y-p.x*p.x-p.y*p.y)-2*(q.x-p.x)*g.x-2*(q.y-p.y)*g.y for g in vertices)
    maximum=max(values)
    return InRangeCertificate(maximum < -margin_sq,"positive_distance_dominance",-margin_sq-maximum)

```

## 文件: src/b\_problem/localization/q4\_pair\_orientation.py

```

"""Exact 2-D orientation feasibility for bound Q4 paired negatives.

Only the inclusive directional half-plane is modeled here. Distance is an
independent prerequisite supplied by the paired-probe in-range certificate.
"""

from __future__ import annotations
from math import atan2, pi
from .robust import Point

_EPS=1e-10

def _dot(a:Point,b:Point)->float:
    return a.x*b.x+a.y*b.y
def _vector(g:Point,p:Point)->Point:
    return Point(p.x-g.x,p.y-g.y)

def orientation_feasible(g:Point,positive:Point,negative_b:Point,negative_c:Point)->bool:
    """Whether an axis exists with P in its closed 180° lobe and B/C out.

    Boundaries are checked explicitly: P may sit on a lobe boundary, while a
    no-signal point cannot because the official receiver half-plane is closed.
    """
    vectors=(_vector(g,positive),_vector(g,negative_b),_vector(g,negative_c))
    if any(abs(v.x)+abs(v.y)<=_EPS for v in vectors):
        return False
    boundaries=[]
    for v in vectors:
        theta=atan2(v.y,v.x)
        boundaries.extend((theta-pi/2,theta+pi/2))

```

```

values=[]
ordered=sorted((value%(2*pi) for value in boundaries))
values.extend(ordered)
for i in range(len(ordered)):
    midpoint=(ordered[i]+((ordered[(i+1)%len(ordered)]-ordered[i])%(2*pi))/2)%(2*pi))
    values.append(midpoint)
for angle in values:
    axis=Point(__import__('math').cos(angle),__import__('math').sin(angle))
    if _dot(axis,vectors[0])>=_EPS and _dot(axis,vectors[1])<=_EPS and
        ↪ _dot(axis,vectors[2])<=_EPS: return True
return False

def proposed_cut_excludes(g:Point,positive:Point,negative_b:Point,negative_c:Point)->bool:
    """Fail-closed production proposal: no positional exclusion until proof."""
    _=(g,positive,negative_b,negative_c)
    return False

```

## 文件: src/b\_problem/localization/robust.py

```

"""Conservative Q2 geometry that does not depend on the shared Q1 module.

The reported bearing has a bounded angular error. This module keeps that
error as a closed angular wedge and treats the ``direction`` distance bound
as open. Its reception certificate is deliberately conservative: it proves
membership in a universal safe subset that covers the whole 1500 m wedge;
it does not claim to solve the exact instance-specific robust domain.
"""

from __future__ import annotations

from dataclasses import dataclass
from enum import Enum
from math import atan2, cos, degrees, hypot, pi, radians, sin
from typing import Iterable

DEFAULT_ERROR_DEG = 1.0
MIN_RECEIVE_RADIUS = 1000.0
MAX_RECEIVE_RADIUS = 1500.0
NEAR_DISTANCE = 5.0
ROBOT_SPEED_MPS = 5.0
NUMERIC_TOLERANCE = 1e-9

@dataclass(frozen=True)
class Point:
    """A two-dimensional Cartesian point in metres."""

    x: float
    y: float

    def distance_to(self, other: "Point") -> float:
        return hypot(self.x - other.x, self.y - other.y)

    def bearing_to(self, other: "Point") -> float:
        """Return the east-zero, counter-clockwise bearing in degrees."""
        return degrees(atan2(other.y - self.y, other.x - self.x)) % 360.0

```

```

def _signed_angle_deg(angle_deg: float) -> float:
    return (angle_deg + 180.0) % 360.0 - 180.0

@dataclass(frozen=True)
class FirstFeasibleRegion:
    """Closed geometric envelope and open ``direction`` subset after scan one."""

    station: Point
    bearing_deg: float
    target_radius: float = 1800.0
    error_deg: float = DEFAULT_ERROR_DEG
    max_receive_radius: float = MAX_RECEIVE_RADIUS
    direction_open_distance: float = NEAR_DISTANCE

    def __post_init__(self) -> None:
        if self.target_radius <= 0.0:
            raise ValueError("target_radius must be positive")
        if not 0.0 < self.error_deg < 90.0:
            raise ValueError("error_deg must be in (0, 90)")
        if self.max_receive_radius <= 0.0:
            raise ValueError("max_receive_radius must be positive")
        if self.direction_open_distance < 0.0:
            raise ValueError("direction_open_distance must be non-negative")

    def contains_closure(self, source: Point) -> bool:
        """Test the closed wedge/disk envelope used for stable numerics.

        The apex is retained as a closure point although a physical
        ``direction`` response cannot occur at distance zero.
        """
        distance = self.station.distance_to(source)
        if hypot(source.x, source.y) > self.target_radius + NUMERIC_TOLERANCE:
            return False
        if distance > self.max_receive_radius + NUMERIC_TOLERANCE:
            return False
        if distance <= NUMERIC_TOLERANCE:
            return True
        bearing_error = abs(_signed_angle_deg(self.station.bearing_to(source) - self.bearing_deg))
        return bearing_error <= self.error_deg + NUMERIC_TOLERANCE

    def contains_direction(self, source: Point) -> bool:
        """Test the physical result branch, with the 5 m threshold open.

        Values at the threshold are treated as ``near``. A tolerance is kept
        on the conservative side, so a numerically ambiguous point is not
        promoted to a ``direction`` observation.
        """
        return self.contains_closure(source) and (
            self.station.distance_to(source) > self.direction_open_distance + NUMERIC_TOLERANCE
        )

    def minimum_consistent_radius(self, source: Point) -> float:
        """Smallest source radius consistent with the first successful scan."""
        if not self.contains_closure(source):
            raise ValueError("source is outside the first feasible closure")
        return max(MIN_RECEIVE_RADIUS, self.station.distance_to(source))

```

```

@dataclass(frozen=True)
class ReceptionCertificate:
    """A proof status for a second-station reception candidate."""

    candidate: Point
    side: int | None
    is_member: bool
    status: str
    worst_projection_m: float
    reason: str

def _minimum_wedge_projection(region: FirstFeasibleRegion, candidate: Point) -> float:
    """Compute  $\min q \cdot u(\theta)$  over the entire first-bearing error wedge."""
    qx = candidate.x - region.station.x
    qy = candidate.y - region.station.y
    length = hypot(qx, qy)
    if length <= NUMERIC_TOLERANCE:
        return 0.0

    direction = atan2(qy, qx)
    centre = radians(region.bearing_deg)
    half_width = radians(region.error_deg)
    lo, hi = centre - half_width, centre + half_width
    angles = [lo, hi]
    # Dot products have extrema where theta equals q's direction or its
    # opposite. Include all wrapped copies that can lie in this interval.
    for base in (direction, direction + pi):
        for turns in range(-2, 3):
            theta = base + 2.0 * pi * turns
            if lo <= theta <= hi:
                angles.append(theta)
    return min(qx * cos(theta) + qy * sin(theta) for theta in angles)

def universal_reception_certificate(
    region: FirstFeasibleRegion, candidate: Point, *, side: int | None = None
) -> ReceptionCertificate:
    """Certify a conservative, source-independent second-station safe subset.

    Let  $q$  be the candidate displacement from scan one. Requiring

    
$$\|q\| \leq 1000,$$

    
$$\|q\|^2 \leq 2000 \min_{\{|\theta - \text{bearing}| \leq \delta\}} q \cdot u(\theta)$$


    is equivalent to reception for every source in the full 0--1500 m first
    wedge, using its smallest radius consistent with scan one. Since the
    actual first feasible set is a subset of that wedge, a successful result
    is a proof for the actual instance. A failed result is intentionally
    ``UNRESOLVED`` rather than a proof of unsafety for the smaller exact set.
    """
    displacement = region.station.distance_to(candidate)
    minimum_projection = _minimum_wedge_projection(region, candidate)
    member = (
        displacement <= MIN_RECEIVE_RADIUS + NUMERIC_TOLERANCE
        and displacement * displacement
        <= 2.0 * MIN_RECEIVE_RADIUS * minimum_projection + NUMERIC_TOLERANCE
    )

```

```

    if member:
        reason = "Universal wedge certificate proves second-station reception."
        status = "PROVED"
    else:
        reason = "Not certified by the universal wedge subset; exact-domain checking is still possible."
        status = "UNRESOLVED"
    return ReceptionCertificate(
        candidate=candidate,
        side=side,
        is_member=member,
        status=status,
        worst_projection_m=minimum_projection,
        reason=reason,
    )

def safe_seed_certificates(region: FirstFeasibleRegion) -> tuple[ReceptionCertificate,
↳ ReceptionCertificate]:
    """Return the two proved 1000 m,  $\pm(60^\circ - \delta)$  robust reception seeds."""
    beta_deg = 60.0 - region.error_deg
    if beta_deg <= 0.0:
        raise ValueError("error_deg must remain below 60 degrees for this seed construction")
    certificates: list[ReceptionCertificate] = []
    for side in (-1, 1):
        angle = radians(region.bearing_deg + side * beta_deg)
        candidate = Point(
            region.station.x + MIN_RECEIVE_RADIUS * cos(angle),
            region.station.y + MIN_RECEIVE_RADIUS * sin(angle),
        )
        certificates.append(universal_reception_certificate(region, candidate, side=side))
    return (certificates[0], certificates[1])

class DirectionResult(str, Enum):
    DIRECTION = "direction"
    NEAR = "near"

@dataclass(frozen=True)
class PosteriorScenario:
    """Externally computed posterior bounds for one possible second outcome."""

    result: DirectionResult
    mec_radius: float = 0.0
    diameter: float = 0.0

    @classmethod
    def direction(cls, *, mec_radius: float, diameter: float) -> "PosteriorScenario":
        if mec_radius < 0.0 or diameter < 0.0:
            raise ValueError("posterior metrics must be non-negative")
        return cls(DirectionResult.DIRECTION, mec_radius, diameter)

    @classmethod
    def near(cls) -> "PosteriorScenario":
        return cls(DirectionResult.NEAR, 0.0, 0.0)

@dataclass(frozen=True)

```

```

class CandidateEvaluation:
    """A lexicographic comparison result, not a global-optimality claim."""

    candidate: Point
    worst_mec_radius: float
    worst_diameter: float
    travel_time_s: float
    branch: DirectionResult
    optimization_status: str = "HEURISTIC"

    @property
    def ranking_key(self) -> tuple[float, float, float]:
        return (self.worst_mec_radius, self.worst_diameter, self.travel_time_s)

def evaluate_candidate(
    first_station: Point, candidate: Point, scenarios: Iterable[PosteriorScenario]
) -> CandidateEvaluation:
    """Rank supplied posterior scenarios by MEC radius, diameter, then travel.

    The geometric posterior/MEC computation belongs to the Q1 geometry module
    once that shared API is stable. This function deliberately accepts its
    bounds as data so this module does not duplicate or modify Q1 geometry.
    """
    values = tuple(scenarios)
    if not values:
        raise ValueError("at least one possible second-station scenario is required")
    worst_mec = max(scenario.mec_radius for scenario in values)
    worst_diameter = max(scenario.diameter for scenario in values)
    branch = DirectionResult.NEAR if all(s.result is DirectionResult.NEAR for s in values) else
    ↪ DirectionResult.DIRECTION
    return CandidateEvaluation(
        candidate=candidate,
        worst_mec_radius=worst_mec,
        worst_diameter=worst_diameter,
        travel_time_s=first_station.distance_to(candidate) / ROBOT_SPEED_MPS,
        branch=branch,
    )

def evaluate_certified_candidate(
    first_station: Point,
    candidate: Point,
    posterior_regions: Iterable[object],
    *,
    sides: int = 256,
):
    """Evaluate direction posteriors using certified geometry upper bounds.

    Lower-dimensional or numerically unresolved regions are rejected instead
    of being assigned optimistic localization metrics.
    """
    from b_problem.geometry import MetricCertificationStatus, certify_curvilinear_metrics

    certificates = tuple(
        certify_curvilinear_metrics(region, sides=sides) for region in posterior_regions
    )
    if any(certificate.status is not MetricCertificationStatus.PROVED for certificate in certificates):

```

```

        raise ValueError("every posterior region must have a full-dimensional metric certification")
    scenarios = tuple(
        PosteriorScenario.direction(
            mec_radius=certificate.mec_radius.upper,
            diameter=certificate.diameter.upper,
        )
        for certificate in certificates
    )
    return evaluate_candidate(first_station, candidate, scenarios), certificates

```

文件: src/b\_problem/simulator/\_\_init\_\_.py

```

"""Offline-safe simulator protocol client and mock."""

```

文件: src/b\_problem/simulator/client.py

```

from __future__ import annotations

from dataclasses import replace
from contextlib import contextmanager
import hashlib
import json
import math
import threading
import time
from typing import Callable, Protocol

from .config import SimulatorConfig, SimulatorMode
from .logger import JsonlLogger
from .protocol import (
    ClearResponse, EnterResponse, ExitResponse, MeasureResponse, Point, ProtocolError,
    SessionState, validate_channel, validate_response, validate_utf8_identifier,
)

class TransportTimeout(RuntimeError):
    pass

class ConnectionClosed(RuntimeError):
    pass

class ConnectionRefused(RuntimeError):
    pass

class HttpStatusError(RuntimeError):
    pass

class ActionRejected(RuntimeError):
    pass

```

```

class Transport(Protocol):
    def post(self, path: str, body: bytes) -> tuple[int, object]: ...

class SimulatorClient:
    """Serial protocol client. Its transport is injected; it never opens a socket."""

    def __init__(self, transport: Transport, robot_id: str, *, arena_id: str = "default",
                  request_id_factory: Callable[[], str] | None = None,
                  logger: JsonlLogger | None = None,
                  mode: SimulatorMode = SimulatorMode.OFFLINE,
                  _validated_live_config: bool = False,
                  clock: Callable[[], float] = time.monotonic,
                  sleeper: Callable[[float], None] = time.sleep) -> None:
        validate_utf8_identifier(robot_id, field="robot_id", maximum_bytes=64)
        if arena_id != "default":
            raise ValueError("arena_id must be exactly ASCII default")
        if getattr(transport, "is_official_live_transport", False) and not _validated_live_config:
            raise ValueError("official HTTP transport requires validated live configuration")
        if mode is not SimulatorMode.OFFLINE and not _validated_live_config:
            raise ValueError("practice and formal modes require SimulatorClient.from_config")
        self._transport = transport
        self._robot_id = robot_id
        self._arena_id = arena_id
        self._ids = request_id_factory or self._default_id_factory()
        self._logger = logger
        self.mode = mode
        self._clock = clock
        self._sleeper = sleeper
        self._deadline: float | None = None
        self._reserve_s = 0.0
        self._lock = threading.Lock()
        self.state = SessionState()

    @classmethod
    def from_config(cls, transport: Transport, config: SimulatorConfig, *, arena_id: str = "default",
                   request_id_factory: Callable[[], str] | None = None,
                   logger: JsonlLogger | None = None,
                   clock: Callable[[], float] = time.monotonic,
                   sleeper: Callable[[float], None] = time.sleep) -> "SimulatorClient":
        """Build an injected-transport client only after mode authorization validates."""
        if not isinstance(config, SimulatorConfig):
            raise ValueError("config must be a SimulatorConfig")
        config.validate()
        robot_id = config.robot_id if config.mode is not SimulatorMode.OFFLINE else (config.robot_id or
        ↪ "offline")
        return cls(
            transport,
            robot_id,
            arena_id=arena_id,
            request_id_factory=request_id_factory,
            logger=logger,
            mode=config.mode,
            _validated_live_config=True,
            clock=clock,
            sleeper=sleeper,
        )

```

```

@staticmethod
def _default_id_factory() -> Callable[[], str]:
    counter = 0
    def next_id() -> str:
        nonlocal counter
        counter += 1
        return f"action-{counter}"
    return next_id

def set_real_time_deadline(self, *, deadline: float, reserve_s: float) -> None:
    if isinstance(deadline, bool) or not isinstance(deadline, (int, float)):
        raise ValueError("deadline must be a number")
    if isinstance(reserve_s, bool) or not isinstance(reserve_s, (int, float)) or reserve_s < 0:
        raise ValueError("reserve_s must be a non-negative number")
    self._deadline = float(deadline)
    self._reserve_s = float(reserve_s)

def enter(self) -> EnterResponse:
    with self._serialized():
        if self.state.phase != "new":
            raise RuntimeError("enter is only valid for a new client")
        _, response = self._execute_locked("/enter", {})
    try:
        remaining = response.get("remaining_real_duration_s")
        if not isinstance(remaining, int) or isinstance(remaining, bool) or not 0 <= remaining <=
            ↪ 1200:
            raise ProtocolError("invalid enter remaining_real_duration_s")
        virtual = self._validated_virtual_time(response)
    except ProtocolError:
        self._desynchronize(response)
        raise
    self.state = SessionState("active", Point(0.0, 0.0), 1, virtual)
    return EnterResponse(virtual, remaining)

def measure(self, position: Point, channel: int) -> MeasureResponse:
    validate_channel(channel)
    with self._serialized():
        self._require_active()
        _, response = self._execute_locked("/measure", {"position": self._point(position), "channel":
            ↪ channel})
    try:
        result = response.get("measure_result")
        if result not in {"no_signal", "near", "direction"}:
            raise ProtocolError("invalid measure_result")
        svd = response.get("svd_deg")
        if result == "direction":
            if isinstance(svd, bool) or not isinstance(svd, (int, float)) or not
                ↪ math.isfinite(svd) or not 0 <= svd < 360:
                raise ProtocolError("direction result needs normalized svd_deg")
            svd_value: float | None = float(svd)
        else:
            if svd is not None:
                raise ProtocolError("non-direction result must not include svd_deg")
            svd_value = None
        virtual = self._validated_virtual_time(response)
    except ProtocolError:
        self._desynchronize(response)
        raise

```

```

        self.state = replace(self.state, position=position, measure_channel=channel,
        ↪ virtual_time_s=virtual)
        return MeasureResponse(virtual, result, svd_value)

def clear(self, position: Point, channel: int) -> ClearResponse:
    validate_channel(channel)
    with self._serialized():
        self._require_active()
        _, response = self._execute_locked("/clear", {"position": self._point(position), "channel":
        ↪ channel})
        try:
            result = response.get("clear_result")
            if result not in {"success", "no_target_in_range"}:
                raise ProtocolError("invalid clear_result")
            virtual = self._validated_virtual_time(response)
        except ProtocolError:
            self._desynchronize(response)
            raise
        self.state = replace(self.state, position=position, virtual_time_s=virtual)
        return ClearResponse(virtual, result)

def exit(self) -> ExitResponse:
    with self._serialized():
        self._require_active()
        _, response = self._execute_locked("/exit", {})
        try:
            if response.get("exit_reason") != "user_exit":
                raise ProtocolError("invalid exit_reason")
            virtual = self._validated_virtual_time(response)
        except ProtocolError:
            self._desynchronize(response)
            raise
        self.state = replace(self.state, phase="closed", virtual_time_s=virtual)
        return ExitResponse(virtual, "user_exit")

@staticmethod
def _point(point: Point) -> dict[str, float]:
    return {"x": point.x, "y": point.y}

def _require_active(self) -> None:
    if self.state.phase == "desynchronized":
        raise RuntimeError("client is desynchronized")
    if self.state.phase != "active":
        raise RuntimeError("action requires an active session")

@contextmanager
def _serialized(self):
    if not self._lock.acquire(blocking=False):
        raise RuntimeError("concurrent new simulator action rejected")
    try:
        yield
    finally:
        self._lock.release()

def _validated_virtual_time(self, response: dict) -> float:
    virtual = float(response["virtual_time_s"])
    if virtual < 0.0 or virtual < self.state.virtual_time_s:
        raise ProtocolError("virtual_time_s must be non-negative and non-decreasing")

```

```

    return virtual

def _desynchronize(self, response: dict) -> None:
    if response.get("accepted") is True:
        self.state = replace(self.state, phase="desynchronized")

def _execute_locked(self, path: str, fields: dict) -> tuple[bytes, dict]:
    request_id = self._ids()
    validate_utf8_identifier(request_id, field="request_id", maximum_bytes=128)
    payload = {"arena_id": self._arena_id, "robot_id": self._robot_id,
               "request_id": request_id, **fields}
    body = json.dumps(payload, ensure_ascii=False, sort_keys=True, separators=(",",
↵  ":")).encode("utf-8")
    audit_request = {
        "arena_id": self._arena_id,
        "request_id": request_id,
        "position": fields.get("position"),
        "channel": fields.get("channel"),
        "body_sha256": hashlib.sha256(body).hexdigest(),
    }
    for attempt in range(4):
        try:
            status, raw_response = self._post_with_remaining_timeout(path, body)
        except (TransportTimeout, ConnectionClosed, ConnectionRefused) as exc:
            self._log_attempt(path, request_id, None, None, None, self._transport_outcome(exc),
↵  audit_request)
            if self._retry_after_backoff(path, attempt):
                continue
            raise TransportTimeout("retry budget exhausted") from exc
        try:
            if status != 200:
                self._log_attempt(path, request_id, status, self._accepted_value(raw_response),
↵  raw_response,
                                   "http_retryable" if self._retryable_http_status(status) else
                                   ↵  "http_error", audit_request)
                if self._retryable_http_status(status) and self._retry_after_backoff(path, attempt):
                    continue
                raise HttpStatusError(f"simulator returned HTTP {status}")
            try:
                response = validate_response(raw_response)
            except ProtocolError:
                self._log_attempt(path, request_id, status, self._accepted_value(raw_response),
↵  raw_response,
                                   "protocol_error", audit_request)
                if isinstance(raw_response, dict):
                    self._desynchronize(raw_response)
                raise
            if response["accepted"] is not True:
                self._log_attempt(path, request_id, status, False, response, "rejected", audit_request)
                raise ActionRejected("simulator rejected action")
            self._log_attempt(path, request_id, status, True, response, "accepted", audit_request)
            return body, response
        except (HttpStatusError, ProtocolError, ActionRejected):
            raise
    raise TransportTimeout("retry budget exhausted")

def _post_with_remaining_timeout(self, path: str, body: bytes) -> tuple[int, object]:
    timeout_s = self._remaining_attempt_timeout(path)

```

```

        if getattr(self._transport, "is_official_live_transport", False) and timeout_s is not None:
            return self._transport.post(path, body, timeout_s=timeout_s)
        return self._transport.post(path, body)

def _remaining_attempt_timeout(self, path: str) -> float | None:
    if self._deadline is None:
        return None
    reserve_s = 0.0 if path == "/exit" else self._reserve_s
    timeout_s = self._deadline - self._clock() - reserve_s
    if timeout_s <= 0.0:
        raise TransportTimeout("real-time action budget exhausted")
    return timeout_s

def _retry_after_backoff(self, path: str, attempt: int) -> bool:
    if attempt >= 3:
        return False
    delay_s = 0.05
    if self._deadline is not None:
        reserve_s = 0.0 if path == "/exit" else self._reserve_s
        if self._clock() + delay_s + reserve_s >= self._deadline:
            return False
    self._sleeper(delay_s)
    return True

@staticmethod
def _retryable_http_status(status: int) -> bool:
    return status in {409, 429} or 500 <= status <= 599

def _log_attempt(self, path: str, request_id: str, status: int | None, accepted: bool | None,
                 response: object | None, outcome: str, request: dict) -> None:
    if self._logger:
        self._logger.record(path=path, request_id=request_id, robot_id=self._robot_id,
                           status=status, accepted=accepted, response=response, outcome=outcome,
                           request=request)

@staticmethod
def _accepted_value(response: object) -> bool | None:
    return response.get("accepted") if isinstance(response, dict) and
        ↳ isinstance(response.get("accepted"), bool) else None

@staticmethod
def _transport_outcome(exc: BaseException) -> str:
    if isinstance(exc, TransportTimeout):
        return "transport_timeout"
    if isinstance(exc, ConnectionRefused):
        return "connection_refused"
    return "connection_closed"

```

文件: src/b\_problem/simulator/config.py

```

from __future__ import annotations

from dataclasses import dataclass
from enum import Enum

from .protocol import validate_utf8_identifier

```

```

PRACTICE_READY_TOKEN = " 模拟器演练已就绪"
FORMAL_Q3_CONFIRMATION_TOKEN = " 允许开始问题 3 正式测试"
FORMAL_Q4_CONFIRMATION_TOKEN = " 允许开始问题 4 正式测试"

```

```

class SimulatorMode(str, Enum):
    OFFLINE = "OFFLINE"
    PRACTICE_Q3 = "PRACTICE_Q3"
    PRACTICE_Q4 = "PRACTICE_Q4"
    FORMAL_Q3 = "FORMAL_Q3"
    FORMAL_Q4 = "FORMAL_Q4"

@dataclass(frozen=True)
class SimulatorConfig:
    mode: SimulatorMode = SimulatorMode.OFFLINE
    robot_id: str | None = None
    port: int | None = None
    ready_signal: str | None = None
    hard_confirmation: str | None = None

    def validate(self) -> None:
        if not isinstance(self.mode, SimulatorMode):
            raise ValueError("mode must be a SimulatorMode")
        if self.robot_id is not None:
            validate_utf8_identifier(self.robot_id, field="robot_id", maximum_bytes=64)
        if self.mode is SimulatorMode.OFFLINE:
            if self.port is not None or self.ready_signal is not None or self.hard_confirmation is not None:
                raise ValueError("offline mode cannot carry live authorization fields")
            return
        if self.robot_id is None:
            raise ValueError("practice and formal modes require robot_id")
        if isinstance(self.port, bool) or not isinstance(self.port, int) or not 1 <= self.port <= 65535:
            raise ValueError("practice and formal modes require a port in 1..65535")
        if self.ready_signal != PRACTICE_READY_TOKEN:
            raise ValueError("practice and formal modes require the exact ready token")
        if self.mode in {SimulatorMode.FORMAL_Q3, SimulatorMode.FORMAL_Q4}:
            required = (FORMAL_Q3_CONFIRMATION_TOKEN if self.mode is SimulatorMode.FORMAL_Q3
                        else FORMAL_Q4_CONFIRMATION_TOKEN)
            if self.hard_confirmation != required:
                raise ValueError("formal mode requires its exact hard confirmation token")

    def build_http_transport(self, *, timeout_s: float = 10.0, opener=None):
        """Return a loopback-only transport bound to this validated live-mode port."""
        self.validate()
        if self.mode is SimulatorMode.OFFLINE:
            raise ValueError("offline mode has no HTTP transport")
        from .http_transport import HttpJsonTransport
        return HttpJsonTransport(self.port, timeout_s=timeout_s, opener=opener)

```

文件: src/b\_problem/simulator/http\_transport.py

```

from __future__ import annotations

import json

```

```

import socket
from typing import Any, Callable
from urllib import error, request

from .client import ConnectionClosed, ConnectionRefused, TransportTimeout
from .protocol import ProtocolError

_PATHS = {"/enter", "/measure", "/clear", "/exit"}

class HttpJsonTransport:
    """Loopback-only JSON transport; callers may inject an opener for tests."""

    is_official_live_transport = True

    def __init__(self, port: int, *, timeout_s: float = 10.0,
                 opener: Callable[..., Any] | None = None) -> None:
        if isinstance(port, bool) or not isinstance(port, int) or not 1 <= port <= 65535:
            raise ValueError("port must be an integer in 1..65535")
        if isinstance(timeout_s, bool) or not isinstance(timeout_s, (int, float)) or timeout_s <= 0:
            raise ValueError("timeout_s must be positive")
        self.port = port
        self.timeout_s = float(timeout_s)
        self._opener = opener or request.urlopen

    def post(self, path: str, body: bytes, *, timeout_s: float | None = None) -> tuple[int, object]:
        if path not in _PATHS:
            raise ValueError("unsupported simulator path")
        if not isinstance(body, bytes):
            raise ValueError("HTTP transport requires an exact bytes body")
        if timeout_s is not None and (isinstance(timeout_s, bool) or not isinstance(timeout_s, (int,
        ↪ float)) or timeout_s <= 0):
            raise ValueError("per-attempt timeout_s must be positive")
        try:
            decoded_request = json.loads(body.decode("utf-8"))
        except (UnicodeDecodeError, json.JSONDecodeError) as exc:
            raise ValueError("HTTP transport requires a UTF-8 JSON body") from exc
        if not isinstance(decoded_request, dict):
            raise ValueError("HTTP transport requires a JSON object body")
        outgoing = request.Request(
            f"http://127.0.0.1:{self.port}{path}", data=body, method="POST",
            headers={"Content-Type": "application/json", "Accept": "application/json"},
        )
        try:
            response = self._opener(outgoing, timeout=min(self.timeout_s, float(timeout_s)) if timeout_s
            ↪ is not None else self.timeout_s)
            return self._read_response(response)
        except error.HTTPError as exc:
            return int(exc.code), self._read_error_body(exc)
        except (socket.timeout, TimeoutError) as exc:
            raise TransportTimeout("HTTP transport timed out") from exc
        except ConnectionRefusedError as exc:
            raise ConnectionRefused("HTTP connection refused") from exc
        except (ConnectionResetError, BrokenPipeError) as exc:
            raise ConnectionClosed("HTTP connection closed") from exc
        except error.URLError as exc:
            self._raise_url_error(exc)

```

```

@staticmethod
def _read_response(response: Any) -> tuple[int, object]:
    try:
        status_value = getattr(response, "status", None)
        status = int(response.getcode() if status_value is None else status_value)
        raw = response.read()
    finally:
        close = getattr(response, "close", None)
        if close is not None:
            close()
    return status, HttpJsonTransport._decode_response(raw)

@staticmethod
def _read_error_body(response: Any) -> object:
    try:
        return HttpJsonTransport._decode_response(response.read())
    except ProtocolError:
        return {}
    finally:
        close = getattr(response, "close", None)
        if close is not None:
            close()

@staticmethod
def _decode_response(raw: object) -> object:
    if not isinstance(raw, bytes):
        raise ProtocolError("HTTP response body must be bytes")
    try:
        return json.loads(raw.decode("utf-8"))
    except (UnicodeDecodeError, json.JSONDecodeError) as exc:
        raise ProtocolError("HTTP response body must be UTF-8 JSON") from exc

@staticmethod
def _raise_url_error(exc: error.URLError) -> None:
    reason = exc.reason
    if isinstance(reason, (socket.timeout, TimeoutError)):
        raise TransportTimeout("HTTP transport timed out") from exc
    if isinstance(reason, ConnectionRefusedError):
        raise ConnectionRefused("HTTP connection refused") from exc
    if isinstance(reason, (ConnectionResetError, BrokenPipeError)):
        raise ConnectionClosed("HTTP connection closed") from exc
    raise ConnectionClosed("HTTP connection failed") from exc

```

文件: src/b\_problem/simulator/logger.py

```

from __future__ import annotations

import hashlib
import json
from pathlib import Path
from typing import Any

DEFAULT_LOCAL_LOG_BYTES = 8 * 1024 * 1024
OFFICIAL_ENCRYPTED_LOG_LIMIT_BYTES = 2 * 1024 * 1024
WORST_OFFLINE_ACTIONS = 4_159

```

ESTIMATED\_ATTEMPT\_BYTES = 512

```
class JsonlLogger:
    """Small local audit logger; it never writes a robot id verbatim."""

    def __init__(self, path: Path, max_bytes: int = DEFAULT_LOCAL_LOG_BYTES) -> None:
        self._path = path
        self._max_bytes = max_bytes
        self._written = 0
        self._file = path.open("w", encoding="utf-8")

    def preflight_offline_capacity(self, action_count: int = WORST_OFFLINE_ACTIONS) -> dict[str, int |
    ↪ bool]:
        if isinstance(action_count, bool) or not isinstance(action_count, int) or action_count < 0:
            raise ValueError("action_count must be a non-negative integer")
        estimated = action_count * ESTIMATED_ATTEMPT_BYTES
        return {
            "action_count": action_count,
            "estimated_local_bytes": estimated,
            "local_capacity_bytes": self._max_bytes,
            "local_capacity_ok": estimated <= self._max_bytes,
            "official_encrypted_limit_bytes": OFFICIAL_ENCRYPTED_LOG_LIMIT_BYTES,
            "official_export_preflight_required": True,
        }

    def record(self, *, path: str, request_id: str, robot_id: str, status: int | None,
        accepted: bool | None, response: object | None, outcome: str,
        request: dict[str, Any]) -> None:
        event = {
            "path": path,
            "request_id": request_id,
            "robot_id_hash": hashlib.sha256(robot_id.encode("utf-8")).hexdigest()[:16],
            "status": status,
            "accepted": accepted,
            "outcome": outcome,
            "request": self._redact_request(request, robot_id),
            "response": self._redact(response, robot_id),
        }
        encoded = (json.dumps(event, ensure_ascii=False, separators=(",", ":")) + "\n").encode("utf-8")
        if self._written + len(encoded) > self._max_bytes:
            raise RuntimeError("local audit log size limit reached")
        self._file.write(encoded.decode("utf-8"))
        self._file.flush()
        self._written += len(encoded)

    @staticmethod
    def _redact(value: object, robot_id: str) -> object:
        if value == robot_id:
            return "[redacted]"
        if isinstance(value, dict):
            return {key: JsonlLogger._redact(item, robot_id) for key, item in value.items()}
        if isinstance(value, list):
            return [JsonlLogger._redact(item, robot_id) for item in value]
        return value

    @staticmethod
    def _redact_request(value: dict[str, Any], robot_id: str) -> dict[str, Any]:
```

```

    return {
        "arena_id": value.get("arena_id"),
        "robot_id_hash": hashlib.sha256(robot_id.encode("utf-8")).hexdigest()[:16],
        "request_id": value.get("request_id"),
        "position": value.get("position"),
        "channel": value.get("channel"),
        "body_sha256": value.get("body_sha256"),
    }

def close(self) -> None:
    self._file.close()

```

文件: src/b\_problem/simulator/mock\_server.py

```

from __future__ import annotations

from dataclasses import dataclass
import json
import math
import threading
import time
from typing import Any, Callable

from .client import ConnectionClosed, TransportTimeout
from .protocol import Point

@dataclass
class MockTarget:
    channel: int
    position: Point
    receive_radius: float = 1000.0
    cleared: bool = False

class MockServer:
    """In-process protocol model for tests; it owns no socket."""

    def __init__(self, robot_id: str, targets: tuple[MockTarget, ...] = (), *,
                 max_virtual_duration_s: float = 360_000.0, max_real_duration_s: float = 1_200.0,
                 max_actions: int = 100_000, clock: Callable[[], float] = time.monotonic) -> None:
        if max_virtual_duration_s < 0.0 or max_real_duration_s < 0.0 or max_actions < 1:
            raise ValueError("mock budgets must be non-negative with at least one action")
        self.robot_id = robot_id
        self.targets = {target.channel: target for target in targets}
        self.entered = False
        self.closed = False
        self.position = Point(0.0, 0.0)
        self.measure_channel = 1
        self.virtual_time_s = 0.0
        self.ledger: dict[str, tuple[str, bytes, tuple[int, dict[str, Any]]]] = {}
        self.request_ids: list[str] = []
        self.request_bodies: list[bytes] = []
        self.accepted_virtual_times: list[float] = []
        self.accepted_action_count = 0
        self.max_virtual_duration_s = float(max_virtual_duration_s)
        self.max_real_duration_s = float(max_real_duration_s)

```

```

self.max_actions = max_actions
self._clock = clock
self._started_at = clock()
self.reject_next = False
self.http_error_next: int | None = None
self.timeout_after_accept_next = False
self.closed_before_response_next = False
self.block_next: threading.Event | None = None
self.started_event: threading.Event | None = None

def make_body(self, request_id: str, x: float | None = None, y: float | None = None,
              channel: int | None = None) -> bytes:
    data: dict[str, Any] = {"arena_id": "default", "robot_id": self.robot_id, "request_id": request_id}
    if x is not None:
        data["position"] = {"x": x, "y": y}
    data["channel"] = channel
    return json.dumps(data, sort_keys=True, separators=(",", ":")).encode("utf-8")

def post(self, path: str, body: bytes) -> tuple[int, object]:
    try:
        data = json.loads(body.decode("utf-8"))
    except (UnicodeDecodeError, json.JSONDecodeError):
        return 400, self._base(False)
    if not self._valid_request(path, data):
        return 400, self._base(False)
    request_id = data.get("request_id", "")
    self.request_ids.append(request_id)
    self.request_bodies.append(body)
    if self.block_next is not None:
        event, self.block_next = self.block_next, None
        if self.started_event:
            self.started_event.set()
        event.wait(2)
    if self.http_error_next is not None:
        code, self.http_error_next = self.http_error_next, None
        return code, self._base(False)
    previous = self.ledger.get(request_id)
    if previous is not None:
        if previous[0] != path or previous[1] != body:
            return 409, self._base(False)
        return previous[2]
    if self.reject_next:
        self.reject_next = False
        return 200, self._base(False)
    if path not in {"/enter", "/measure", "/clear", "/exit"}:
        return 404, self._base(False)
    if data.get("arena_id") != "default" or data.get("robot_id") != self.robot_id:
        return 200, self._base(False)
    if path == "/enter":
        if self.entered or self.closed:
            return 200, self._base(False)
        self.entered = True
        result = (200, {**self._base(True), "max_virtual_duration_s": self.max_virtual_duration_s,
                        "max_real_duration_s": self.max_real_duration_s,
                        "remaining_real_duration_s": self._remaining_real_duration_s()})
    elif not self.entered or self.closed:
        return 200, self._base(False)
    elif not self._can_accept(0.0):
        return 200, self._base(False)

```

```

elif path == "/measure":
    result = self._measure(data)
elif path == "/clear":
    result = self._clear(data)
else:
    self.closed = True
    result = (200, {**self._base(True), "exit_reason": "user_exit"})
self.ledger[request_id] = (path, body, result)
if result[1]["accepted"]:
    self.accepted_action_count += 1
    self.accepted_virtual_times.append(float(result[1]["virtual_time_s"]))
if self.timeout_after_accept_next:
    self.timeout_after_accept_next = False
    raise TransportTimeout("injected timeout after action accepted")
if self.closed_before_response_next:
    self.closed_before_response_next = False
    raise ConnectionClosed("injected closed connection after action accepted")
return result

@staticmethod
def _valid_request(path: str, data: object) -> bool:
    if path not in {"/enter", "/measure", "/clear", "/exit"} or not isinstance(data, dict):
        return False
    common = {"arena_id", "robot_id", "request_id"}
    allowed = common if path in {"/enter", "/exit"} else common | {"position", "channel"}
    if set(data) - allowed or not common <= set(data):
        return False
    if not isinstance(data["arena_id"], str) or not isinstance(data["robot_id"], str):
        return False
    if not isinstance(data["request_id"], str) or not data["request_id"]:
        return False
    if path in {"/enter", "/exit"}:
        return True
    position, channel = data.get("position"), data.get("channel")
    if not isinstance(position, dict) or set(position) != {"x", "y"}:
        return False
    if isinstance(channel, bool) or not isinstance(channel, int) or not 1 <= channel <= 20:
        return False
    return all(isinstance(position[key], (int, float)) and not isinstance(position[key], bool)
                and math.isfinite(position[key]) and abs(position[key]) <= 2_000_000
                for key in ("x", "y"))

def _measure(self, data: dict[str, Any]) -> tuple[int, dict[str, Any]]:
    point, channel = self._action_args(data)
    movement = self._distance(self.position, point)
    delta = movement / 5.0 + (1 if channel != self.measure_channel else 0) + 5
    if not self._can_accept(delta):
        return 200, self._base(False)
    self._move_to(point)
    self.virtual_time_s += (1 if channel != self.measure_channel else 0) + 5
    self.measure_channel = channel
    target = self.targets.get(channel)
    if target is None or target.cleared or self._distance(point, target.position) >
    ↪ target.receive_radius:
        result: dict[str, Any] = {**self._base(True), "measure_result": "no_signal"}
    elif self._distance(point, target.position) <= 5:
        result = {**self._base(True), "measure_result": "near"}
    else:

```

```

        angle = math.degrees(math.atan2(target.position.y - point.y, target.position.x - point.x)) %
        ↪ 360
        result = {**self._base(True), "measure_result": "direction", "svd_deg": round(angle, 2)}
    return 200, result

def _clear(self, data: dict[str, Any]) -> tuple[int, dict[str, Any]]:
    point, channel = self._action_args(data)
    target = self.targets.get(channel)
    success = target is not None and not target.cleared and self._distance(point, target.position) <=
    ↪ 20
    delta = self._distance(self.position, point) / 5.0 + (5 if success else 3)
    if not self._can_accept(delta):
        return 200, self._base(False)
    self._move_to(point)
    self.virtual_time_s += 5 if success else 3
    if success:
        target.cleared = True
    return 200, {**self._base(True), "clear_result": "success" if success else "no_target_in_range"}

@staticmethod
def _action_args(data: dict[str, Any]) -> tuple[Point, int]:
    position, channel = data.get("position"), data.get("channel")
    if not isinstance(position, dict) or not isinstance(channel, int) or not 1 <= channel <= 20:
        raise ValueError("mock action must be valid")
    return Point(float(position["x"]), float(position["y"])), channel

def _move_to(self, point: Point) -> None:
    self.virtual_time_s += self._distance(self.position, point) / 5
    self.position = point

def _can_accept(self, delta_s: float) -> bool:
    return (
        self.accepted_action_count < self.max_actions
        and self._clock() - self._started_at <= self.max_real_duration_s
        and self.virtual_time_s + delta_s <= self.max_virtual_duration_s
    )

def _remaining_real_duration_s(self) -> int:
    return max(0, min(int(self.max_real_duration_s - (self._clock() - self._started_at)),
    ↪ int(self.max_real_duration_s)))

@staticmethod
def _distance(left: Point, right: Point) -> float:
    return math.hypot(left.x - right.x, left.y - right.y)

def _base(self, accepted: bool) -> dict[str, Any]:
    return {"accepted": accepted, "real_timestamp_ms": 0, "virtual_time_s": self.virtual_time_s if
    ↪ accepted else 0}

```

文件: src/b\_problem/simulator/protocol.py

```

from __future__ import annotations

from dataclasses import dataclass
import math
import unicodedata

```

```

class ProtocolError(RuntimeError):
    pass

def validate_utf8_identifier(value: object, *, field: str, maximum_bytes: int) -> str:
    if not isinstance(value, str):
        raise ValueError(f"{field} must be a string")
    try:
        encoded = value.encode("utf-8")
    except UnicodeEncodeError as exc:
        raise ValueError(f"{field} must be UTF-8 encodable") from exc
    if not 1 <= len(encoded) <= maximum_bytes:
        raise ValueError(f"{field} must use 1..{maximum_bytes} UTF-8 bytes")
    if any(unicodedata.category(character) in {"Cc", "Cf"} for character in value):
        raise ValueError(f"{field} must not contain Unicode control or format characters")
    return value

@dataclass(frozen=True)
class Point:
    x: float
    y: float

    def __post_init__(self) -> None:
        if not all(
            isinstance(value, (int, float))
            and not isinstance(value, bool)
            and math.isfinite(value)
            and abs(value) <= 2_000_000
            for value in (self.x, self.y)
        ):
            raise ValueError("position must be finite and within 2,000,000 m")

@dataclass(frozen=True)
class SessionState:
    phase: str = "new"
    position: Point = Point(0.0, 0.0)
    measure_channel: int = 1
    virtual_time_s: float = 0.0

@dataclass(frozen=True)
class EnterResponse:
    virtual_time_s: float
    remaining_real_duration_s: int
    accepted: bool = True

@dataclass(frozen=True)
class MeasureResponse:
    virtual_time_s: float
    measure_result: str
    svd_deg: float | None
    accepted: bool = True

@dataclass(frozen=True)

```

```

class ClearResponse:
    virtual_time_s: float
    clear_result: str
    accepted: bool = True

@dataclass(frozen=True)
class ExitResponse:
    virtual_time_s: float
    exit_reason: str
    accepted: bool = True

def validate_channel(channel: int) -> None:
    if isinstance(channel, bool) or not isinstance(channel, int) or not 1 <= channel <= 20:
        raise ValueError("channel must be an integer from 1 through 20")

def validate_response(payload: object) -> dict:
    if not isinstance(payload, dict):
        raise ProtocolError("response must be a JSON object")
    if not isinstance(payload.get("accepted"), bool):
        raise ProtocolError("response missing boolean accepted")
    for field in ("real_timestamp_ms", "virtual_time_s"):
        value = payload.get(field)
        if isinstance(value, bool) or not isinstance(value, (int, float)) or not math.isfinite(value):
            raise ProtocolError(f"response missing finite {field}")
    return payload

```

文件: src/b\_problem/simulator/session.py

```

from __future__ import annotations

from dataclasses import asdict, dataclass, is_dataclass
from enum import Enum
import hashlib
import json
from pathlib import Path
import time
from typing import Any, Callable

from .client import SimulatorClient, Transport
from .config import SimulatorConfig, SimulatorMode
from .logger import JsonlLogger

class RealTimeBudgetExceeded(RuntimeError):
    """Raised before a new action when the official real-time reserve is reached."""

class DeadlineRobot:
    """Strategy-facing client wrapper that checks the /enter wall-time budget."""

    def __init__(self, client: SimulatorClient, *, deadline: float,
                 clock: Callable[[], float], reserve_s: float) -> None:
        self._client = client
        self._deadline = deadline

```

```

        self._clock = clock
        self._reserve_s = reserve_s

    @property
    def state(self):
        return self._client.state

    def _check(self) -> None:
        if self._clock() + self._reserve_s >= self._deadline:
            raise RealTimeBudgetExceeded("official real-time safety reserve reached")

    def measure(self, position, channel):
        self._check()
        return self._client.measure(position, channel)

    def clear(self, position, channel):
        self._check()
        return self._client.clear(position, channel)

@dataclass(frozen=True)
class SessionRunResult:
    mode: SimulatorMode
    complete: bool
    exit_reason: str
    virtual_time_s: float
    wall_time_s: float
    remaining_real_duration_s: int
    log_size_bytes: int
    attempt_count: int

def _json_safe(value: Any) -> Any:
    if isinstance(value, Enum):
        return _json_safe(asdict(value))
    if isinstance(value, dict):
        return {str(key): _json_safe(item) for key, item in value.items()}
    if isinstance(value, (tuple, list, set, frozenset)):
        return [_json_safe(item) for item in value]
    if isinstance(value, (str, int, float, bool)) or value is None:
        return value
    return repr(value)

def _complete(result: Any) -> bool:
    if isinstance(result, dict):
        return result.get("complete") is True
    return getattr(result, "complete", False) is True

def run_validated_session(
    config: SimulatorConfig,
    *,
    log_path: Path,
    summary_path: Path,
    transport: Transport | None = None,
    q3_strategy: Callable[[Any, Any], Any] | None = None,

```

```

q4_strategy: Callable[[Any, Any], Any] | None = None,
clock: Callable[[], float] = time.monotonic,
reserve_s: float = 5.0,
) -> SessionRunResult:
    """Run one validated live session; tests inject transport and perform no I/O."""
    config.validate()
    if config.mode is SimulatorMode.OFFLINE:
        raise ValueError("OFFLINE mode cannot start a live session")
    if isinstance(reserve_s, bool) or not isinstance(reserve_s, (int, float)) or reserve_s < 0.0:
        raise ValueError("reserve_s must be a non-negative number")
    if log_path == summary_path:
        raise ValueError("log_path and summary_path must be different")
    existing_outputs = [path for path in (log_path, summary_path) if path.exists()]
    if existing_outputs:
        names = ", ".join(str(path) for path in existing_outputs)
        raise FileExistsError(f"refusing to overwrite practice output: {names}")

    if q3_strategy is None or q4_strategy is None:
        from b_problem.strategy.q3 import execute_q3_strategy
        from b_problem.strategy.q4 import execute_q4_strategy
        q3_strategy = q3_strategy or execute_q3_strategy
        q4_strategy = q4_strategy or execute_q4_strategy

    log_path.parent.mkdir(parents=True, exist_ok=True)
    summary_path.parent.mkdir(parents=True, exist_ok=True)
    active_transport = transport if transport is not None else config.build_http_transport()
    logger = JsonlLogger(log_path)
    started = clock()
    client = SimulatorClient.from_config(active_transport, config, logger=logger, clock=clock)
    strategy_result: Any = None
    remaining = 0
    exit_reason = "not_entered"
    deadline: float | None = None
    primary_exception: BaseException | None = None
    cleanup_exception: BaseException | None = None
    try:
        entered = client.enter()
        remaining = entered.remaining_real_duration_s
        deadline = clock() + remaining
        client.set_real_time_deadline(deadline=deadline, reserve_s=float(reserve_s))
        robot = DeadlineRobot(client, deadline=deadline, clock=clock, reserve_s=float(reserve_s))
        strategy = q3_strategy if config.mode in {SimulatorMode.PRACTICE_Q3, SimulatorMode.FORMAL_Q3} else
        ↪ q4_strategy
        try:
            strategy_result = strategy(robot, robot.state)
        except RealTimeBudgetExceeded:
            exit_reason = "real_time_guard"
    except BaseException as exc:
        primary_exception = exc
        raise
    finally:
        if client.state.phase == "active" and (deadline is None or clock() < deadline):
            try:
                exit_reason = client.exit().exit_reason
            except BaseException as exc:
                if primary_exception is None and exit_reason != "real_time_guard":
                    cleanup_exception = exc
        wall_time = max(0.0, clock() - started)

```

```

complete = _complete(strategy_result)
logger.close()
log_size_bytes = log_path.stat().st_size
attempt_count = sum(1 for line in log_path.read_text(encoding="utf-8").splitlines() if line)
summary = {
    "mode": config.mode.value,
    "robot_id_hash": hashlib.sha256(config.robot_id.encode("utf-8")).hexdigest()[:16],
    "complete": complete,
    "exit_reason": exit_reason,
    "virtual_time_s": client.state.virtual_time_s,
    "wall_time_s": wall_time,
    "remaining_real_duration_s": remaining,
    "log_size_bytes": log_size_bytes,
    "attempt_count": attempt_count,
    "strategy_state": _json_safe(strategy_result),
}
summary_path.write_text(json.dumps(summary, ensure_ascii=False, indent=2), encoding="utf-8")
if cleanup_exception is not None:
    raise cleanup_exception

return SessionRunResult(
    mode=config.mode,
    complete=complete,
    exit_reason=exit_reason,
    virtual_time_s=client.state.virtual_time_s,
    wall_time_s=wall_time,
    remaining_real_duration_s=remaining,
    log_size_bytes=log_size_bytes,
    attempt_count=attempt_count,
)

```

文件: src/b\_problem/strategy/\_\_init\_\_.py

"""Planning strategies for the CUMCM B problem."""

文件: src/b\_problem/strategy/q3.py

"""Certified baseline planning primitives for Q3 omnidirectional sources."""

from \_\_future\_\_ import annotations

from dataclasses import dataclass, field

from enum import Enum

import math

from typing import Any

from b\_problem.geometry import Circle, Point

TARGET\_RADIUS = 1800.0

RECEIVE\_RADIUS\_LOWER\_BOUND = 1000.0

RING\_RADIUS = 1200.0

CHANNELS = tuple(range(1, 21))

def \_distance(first: Point, second: Point) -> float:

```

    return math.hypot(first.x - second.x, first.y - second.y)

@dataclass(frozen=True)
class CoverageCertificate:
    stations: tuple[Point, ...]
    target_radius: float
    receive_lower_bound: float
    worst_nearest_distance: float

    @property
    def margin(self) -> float:
        return self.receive_lower_bound - self.worst_nearest_distance

    def nearest_distance(self, point: Point) -> float:
        return min(_distance(point, station) for station in self.stations)

@dataclass(frozen=True)
class SurveyPlan:
    stations: tuple[Point, ...]
    route: tuple[Point, ...]
    coverage_certificate: CoverageCertificate

@dataclass(frozen=True)
class ScanAction:
    station_id: int
    position: Point
    channel: int

@dataclass(frozen=True)
class ClearAction:
    channel: int
    position: Point

@dataclass(frozen=True)
class SurveyCost:
    route_length: float
    movement_seconds: float
    measure_seconds: float
    switch_seconds: float

    @property
    def virtual_seconds(self) -> float:
        return self.movement_seconds + self.measure_seconds + self.switch_seconds

def _six_ring_worst_distance(target_radius: float, ring_radius: float) -> float:
    """Exact farthest nearest-station distance for center plus six equal ring sites."""
    half_sector = math.pi / 6.0
    cosine = math.cos(half_sector)
    crossover = ring_radius / (2.0 * cosine)
    outer = math.sqrt(target_radius**2 + ring_radius**2 - 2.0 * target_radius * ring_radius * cosine)
    if target_radius <= crossover:
        return target_radius
    return max(crossover, min(target_radius, outer))

```

```

def build_center_ring_plan(
    target_radius: float = TARGET_RADIUS,
    receive_lower_bound: float = RECEIVE_RADIUS_LOWER_BOUND,
    ring_radius: float = RING_RADIUS,
) -> SurveyPlan:
    """Build the analytic center-plus-six-ring Q3 coverage baseline."""
    if min(target_radius, receive_lower_bound, ring_radius) <= 0.0:
        raise ValueError("radii must be positive")
    center = Point(0.0, 0.0)
    ring = tuple(
        Point(ring_radius * math.cos(index * math.pi / 3.0), ring_radius * math.sin(index * math.pi / 3.0))
        for index in range(6)
    )
    stations = (center, *ring)
    worst = _six_ring_worst_distance(target_radius, ring_radius)
    certificate = CoverageCertificate(stations, target_radius, receive_lower_bound, worst)
    return SurveyPlan(stations, stations, certificate)

def snake_scan_plan(plan: SurveyPlan, channels: tuple[int, ...] = CHANNELS, initial_channel: int = 1) ->
    ↳ tuple[ScanAction, ...]:
    """Scan all unresolved channels in alternating orders without avoidable switches."""
    if set(channels) != set(CHANNELS) or initial_channel not in channels:
        raise ValueError("the fixed survey scans each channel 1..20 exactly once per station")
    current = initial_channel
    actions: list[ScanAction] = []
    for station_id, position in enumerate(plan.route):
        order = tuple(channel for channel in channels if channel >= current) + tuple(channel for channel
            ↳ in reversed(channels) if channel < current)
        actions.extend(ScanAction(station_id, position, channel) for channel in order)
        current = order[-1]
    return tuple(actions)

def survey_cost(plan: SurveyPlan, actions: tuple[ScanAction, ...], speed_mps: float = 5.0) -> SurveyCost:
    """Compute virtual cost using only accepted measure-action mechanics."""
    if speed_mps <= 0.0:
        raise ValueError("speed must be positive")
    route_length = sum(_distance(first, second) for first, second in zip(plan.route, plan.route[1:]))
    switches = sum(first.channel != second.channel for first, second in zip(actions, actions[1:]))
    return SurveyCost(route_length, route_length / speed_mps, 5.0 * len(actions), float(switches))

class ChannelState(str, Enum):
    UNKNOWN = "unknown"
    PRESENT_LOCALIZING = "present_localizing"
    CLEARED = "cleared"
    ABSENT_CERTIFIED = "absent_certified"

@dataclass
class ChannelTracker:
    """State machine retaining every unfulfilled absence-proof obligation."""

    plan: SurveyPlan
    states: dict[int, ChannelState] = field(default_factory=lambda: {channel: ChannelState.UNKNOWN for
        ↳ channel in CHANNELS})

```

```

_no_signal_stations: dict[int, set[int]] = field(default_factory=lambda: {channel: set() for channel
↪ in CHANNELS})
_failed_clears: dict[int, list[Point]] = field(default_factory=lambda: {channel: [] for channel in
↪ CHANNELS})

def state_of(self, channel: int) -> ChannelState:
    return self.states[channel]

def record_measurement(self, channel: int, station_id: int, *, accepted: bool, result: str) ->
↪ ClearAction | None:
    """Apply a simulator response; rejected/unknown responses create no evidence."""
    if channel not in CHANNELS or station_id not in range(len(self.plan.stations)) or not accepted:
        return None
    state = self.states[channel]
    if state in (ChannelState.CLEARED, ChannelState.ABSENT_CERTIFIED):
        return None
    if result == "direction":
        self.states[channel] = ChannelState.PRESENT_LOCALIZING
        return None
    if result == "near":
        self.states[channel] = ChannelState.PRESENT_LOCALIZING
        return ClearAction(channel, self.plan.stations[station_id])
    if result != "no_signal":
        return None
    if state is ChannelState.UNKNOWN:
        self._no_signal_stations[channel].add(station_id)
        if len(self._no_signal_stations[channel]) == len(self.plan.stations):
            self.states[channel] = ChannelState.ABSENT_CERTIFIED
    return None

def record_clear(self, channel: int, position: Point, *, accepted: bool, success: bool) -> None:
    """Only a successful accepted clear resolves a channel globally."""
    if channel not in CHANNELS or not accepted:
        return
    if success:
        self.states[channel] = ChannelState.CLEARED
    else:
        self._failed_clears[channel].append(position)

def failed_clear_points(self, channel: int) -> tuple[Point, ...]:
    return tuple(self._failed_clears[channel])

def remaining_obligations(self) -> tuple[tuple[int, int], ...]:
    """Return every (station, channel) scan still needed for absence certification."""
    obligations = []
    for channel in CHANNELS:
        if self.states[channel] is not ChannelState.UNKNOWN:
            continue
        obligations.extend((station_id, channel) for station_id in range(len(self.plan.stations)) if
↪ station_id not in self._no_signal_stations[channel])
    return tuple(obligations)

def is_complete(self) -> bool:
    if sum(state is ChannelState.CLEARED for state in self.states.values()) >= 16:
        return True
    return all(state in (ChannelState.CLEARED, ChannelState.ABSENT_CERTIFIED) for state in
↪ self.states.values())

```

```

def hybrid_scan_plan(tracker: ChannelTracker) -> tuple[ScanAction, ...]:
    """Resume only outstanding UNKNOWN-channel survey obligations after a detour."""
    return tuple(
        ScanAction(station_id, tracker.plan.stations[station_id], channel)
        for station_id, channel in tracker.remaining_obligations()
    )

@dataclass(frozen=True)
class ClearancePlan:
    kind: str
    actions: tuple[ClearAction, ...]
    coverage_radius: float
    certified: bool
    eta_c_m: float = 0.0
    resource_status: str = "LEGACY_GEOMETRY_ONLY"

def _triangular_box_cover(box: Circle, coverage_radius: float) -> tuple[Point, ...]:
    """Cover a circle's bounding box using a padded triangular lattice.

    A triangular lattice with side  $\sqrt{3} \cdot \text{coverage\_radius}$  has covering
    radius  $\text{coverage\_radius}$ . Padding each coordinate by that radius retains
    every lattice point that could be nearest to a point in the bounding box.
    """
    spacing = math.sqrt(3.0) * coverage_radius
    row_step = 1.5 * coverage_radius
    x_min = box.center.x - box.radius - coverage_radius
    x_max = box.center.x + box.radius + coverage_radius
    y_min = box.center.y - box.radius - coverage_radius
    y_max = box.center.y + box.radius + coverage_radius
    first_row = math.floor(y_min / row_step)
    last_row = math.ceil(y_max / row_step)
    points: list[Point] = []
    for row in range(first_row, last_row + 1):
        offset = (row & 1) * spacing / 2.0
        first_col = math.floor((x_min - offset) / spacing)
        last_col = math.ceil((x_max - offset) / spacing)
        points.extend(Point(column * spacing + offset, row * row_step) for column in range(first_col,
            ↪ last_col + 1))
    return tuple(points)

def clearance_plan(channel: int, mec: Circle, clear_radius: float = 20.0) -> ClearancePlan:
    """Use one certified MEC-center clear or a certified triangular fallback."""
    if channel not in CHANNELS or clear_radius <= 0.0:
        raise ValueError("invalid channel or clearance radius")
    if mec.radius <= clear_radius:
        return ClearancePlan("single_clear", (ClearAction(channel, mec.center),), clear_radius, True, 0.0,
            ↪ "CERTIFIED_SINGLE_CLEAR")
    eta_c_m = min(0.01, clear_radius / 2.0)
    grid_radius = clear_radius - eta_c_m
    points = _triangular_box_cover(mec, grid_radius)
    return ClearancePlan(
        "triangular_fallback",
        tuple(ClearAction(channel, point) for point in points),
        grid_radius,
    )

```

```

        True,
        eta_c_m,
        "LEGACY_GEOMETRY_ONLY",
    )

# The legacy MEC helper above remains for compatibility with the first-batch
# Q3 tests. The executable fallback below starts from the actual first
# direction wedge instead of treating an arbitrary 1500 m enclosing circle as
# a runnable primary plan.

DEFAULT_CLEAR_ETA_M = 0.01
MAX_VIRTUAL_SECONDS = 360000.0
MAX_LOG_BYTES = 2 * 1024 * 1024
ESTIMATED_LOG_BYTES_PER_ACTION = 256
SQRT3 = math.sqrt(3.0)
SAFE_SECOND_OFFSET_DEG = 5.0

@dataclass(frozen=True)
class SampleCoverageCertificate:
    sample_count: int
    all_samples_covered: bool
    maximum_nearest_distance_m: float
    coverage_radius_m: float
    continuous_sector_coverage: bool
    status: str
    proof: str = ""

@dataclass(frozen=True)
class ClearanceResourceCertificate:
    per_channel_actions: int
    projected_actions_for_16_channels: int
    baseline_virtual_seconds: float
    route_start: Point
    route_length_m: float
    projected_minimum_virtual_seconds: float
    projected_route_virtual_seconds: float
    estimated_log_bytes: int
    status: str
    reason: str

@dataclass(frozen=True)
class AnchorClearancePlan:
    kind: str
    channel: int
    anchor: Point
    bearing_deg: float
    delta_deg: float
    eta_c_m: float
    spacing_m: float
    coverage_radius_m: float
    actions: tuple[ClearAction, ...]
    sample_coverage_certificate: SampleCoverageCertificate
    bounding_box_area_m2: float
    route_length_m: float

```

```

def _angle_delta_deg(angle_deg: float, bearing_deg: float) -> float:
    return (angle_deg - bearing_deg + 180.0) % 360.0 - 180.0

def _in_anchor_wedge(point: Point, anchor: Point, bearing_deg: float, delta_deg: float, target_radius:
↪ float) -> bool:
    distance = _distance(point, anchor)
    if distance > 1500.0 + 1e-9 or _distance(point, Point(0.0, 0.0)) > target_radius + 1e-9:
        return False
    if distance <= 1e-9:
        return True
    return abs(_angle_delta_deg(math.degrees(math.atan2(point.y - anchor.y, point.x - anchor.x)),
↪ bearing_deg)) <= delta_deg + 1e-9

def _anchor_wedge_samples(anchor: Point, bearing_deg: float, delta_deg: float, target_radius: float) ->
↪ tuple[Point, ...]:
    samples: list[Point] = []
    radii = {0.0, 5.0, 20.0, 1000.0, 1500.0}
    radii.update(1500.0 * index / 75.0 for index in range(76))
    for radius in sorted(radii):
        for offset in (-delta_deg, -delta_deg / 2.0, 0.0, delta_deg / 2.0, delta_deg):
            angle = math.radians(bearing_deg + offset)
            candidate = Point(anchor.x + radius * math.cos(angle), anchor.y + radius * math.sin(angle))
            if _in_anchor_wedge(candidate, anchor, bearing_deg, delta_deg, target_radius):
                samples.append(candidate)
    return tuple(dict.fromkeys(samples))

def _triangular_box_points(x_min: float, x_max: float, y_min: float, y_max: float, spacing_m: float) ->
↪ tuple[Point, ...]:
    """Return a padded triangular-lattice patch whose Voronoi radius is h/sqrt(3)."""
    pad = spacing_m
    row_step = SQRT3 * spacing_m / 2.0
    first_row = math.floor((y_min - pad) / row_step)
    last_row = math.ceil((y_max + pad) / row_step)
    points: list[Point] = []
    for row in range(first_row, last_row + 1):
        offset = (row & 1) * spacing_m / 2.0
        first_col = math.floor((x_min - pad - offset) / spacing_m)
        last_col = math.ceil((x_max + pad - offset) / spacing_m)
        points.extend(Point(column * spacing_m + offset, row * row_step) for column in range(first_col,
↪ last_col + 1))
    return tuple(dict.fromkeys(points))

def _snake_order(points: tuple[Point, ...]) -> tuple[Point, ...]:
    rows: dict[float, list[Point]] = {}
    for point in points:
        rows.setdefault(point.y, []).append(point)
    ordered: list[Point] = []
    for index, y in enumerate(sorted(rows)):
        ordered.extend(sorted(rows[y], key=lambda point: point.x, reverse=bool(index % 2)))
    return tuple(ordered)

def _route_length(start: Point, points: tuple[Point, ...]) -> float:

```

```

return sum(_distance(first, second) for first, second in zip((start, *points), points))

def _distance_to_closed_sector(point: Point, *, radius_m: float, delta_deg: float) -> float:
    """Return exact Euclidean distance from local ``point`` to a closed sector."""
    angle = math.atan2(point.y, point.x)
    half_width = math.radians(delta_deg)
    radial_distance = math.hypot(point.x, point.y)
    if abs(angle) <= half_width + 1e-9:
        return max(0.0, radial_distance - radius_m)

    def distance_to_segment(end: Point) -> float:
        length_squared = end.x * end.x + end.y * end.y
        projection = max(0.0, min(1.0, (point.x * end.x + point.y * end.y) / length_squared))
        return math.hypot(point.x - projection * end.x, point.y - projection * end.y)

    upper = Point(radius_m * math.cos(half_width), radius_m * math.sin(half_width))
    lower = Point(radius_m * math.cos(-half_width), radius_m * math.sin(-half_width))
    return min(distance_to_segment(upper), distance_to_segment(lower))

def build_anchor_clearance_plan(
    *,
    channel: int,
    anchor: Point,
    bearing_deg: float,
    delta_deg: float = 1.0,
    eta_c_m: float = DEFAULT_CLEAR_ETA_M,
    target_radius: float = TARGET_RADIUS,
) -> AnchorClearancePlan:
    """Build a narrow first-bearing wedge clearance grid with explicit margin."""
    if channel not in CHANNELS or not 0.0 < delta_deg < 90.0 or eta_c_m <= 0.0:
        raise ValueError("invalid anchor clearance parameters")
    spacing_m = 20.0 * SQRT3 - eta_c_m
    coverage_radius_m = spacing_m / SQRT3
    # Build the lattice in bearing-aligned coordinates. The complete first-
    # bearing cone is contained in 0<=longitudinal<=1500 and
    # |transverse|<=1500*sin(delta), independent of the world bearing.
    x_min, x_max = 0.0, 1500.0
    transverse = 1500.0 * math.sin(math.radians(delta_deg))
    y_min, y_max = -transverse, transverse
    candidates = _triangular_box_points(x_min, x_max, y_min, y_max, spacing_m)
    # The full lattice covers the closed sector with radius h/sqrt(3). A
    # nearest lattice point to any sector point is itself no farther than that
    # radius from the sector, so this exact-distance filter cannot discard it.
    local_points = _snake_order(tuple(
        point for point in candidates
        if _distance_to_closed_sector(point, radius_m=1500.0, delta_deg=delta_deg) <= coverage_radius_m +
        ↪ 1e-9
    ))
    heading = math.radians(bearing_deg)
    cosine, sine = math.cos(heading), math.sin(heading)
    points = tuple(
        Point(anchor.x + cosine * point.x - sine * point.y, anchor.y + sine * point.x + cosine * point.y)
        for point in local_points
    )
    samples = _anchor_wedge_samples(anchor, bearing_deg, delta_deg, target_radius)
    maximum_nearest = max(min(_distance(sample, point) for point in points) for sample in samples)

```

```

sample_certificate = SampleCoverageCertificate(
    sample_count=len(samples),
    all_samples_covered=maximum_nearest <= coverage_radius_m + 1e-9,
    maximum_nearest_distance_m=maximum_nearest,
    coverage_radius_m=coverage_radius_m,
    continuous_sector_coverage=bool(local_points),
    status="PROVED" if local_points else "UNRESOLVED",
    proof=(
        "The full triangular lattice covers the closed sector with radius spacing/sqrt(3). "
        "Every nearest lattice point of a sector point is within that radius of the sector and
        ↪ survives the exact-distance filter."
    ),
)
)
return AnchorClearancePlan(
    kind="anchor_wedge_clearance",
    channel=channel,
    anchor=anchor,
    bearing_deg=bearing_deg,
    delta_deg=delta_deg,
    eta_c_m=eta_c_m,
    spacing_m=spacing_m,
    coverage_radius_m=coverage_radius_m,
    actions=tuple(ClearAction(channel, point) for point in points),
    sample_coverage_certificate=sample_certificate,
    bounding_box_area_m2=(x_max - x_min) * (y_max - y_min),
    # Rigid rotation/translation preserves the bearing-aligned snake route.
    route_length_m=_route_length(anchor, points),
)

def certify_clearance_resources(
    plan: AnchorClearancePlan,
    *,
    baseline_virtual_seconds: float,
    max_channels: int = 16,
    max_virtual_seconds: float = MAX_VIRTUAL_SECONDS,
    max_log_bytes: int = MAX_LOG_BYTES,
    baseline_actions: int = 140,
    fallback_start: Point | None = None,
) -> ClearanceResourceCertificate:
    """Reject a fallback as main-strategy evidence when either global budget fails."""
    count = len(plan.actions)
    projected_actions = count * max_channels
    route_start = fallback_start if fallback_start is not None else plan.anchor
    route_length_m = _route_length(route_start, tuple(action.position for action in plan.actions))
    # A no-target clear costs 3 s; a final successful clear costs 5 s. The
    # route figure is the conservative all-points traversal from the anchor.
    minimum = baseline_virtual_seconds + max_channels * 3.0 * count
    routed = baseline_virtual_seconds + max_channels * (3.0 * max(count - 1, 0) + 5.0 + route_length_m /
    ↪ 5.0)
    estimated_log = (projected_actions + baseline_actions) * ESTIMATED_LOG_BYTES_PER_ACTION
    feasible = (
        plan.sample_coverage_certificate.all_samples_covered
        and routed <= max_virtual_seconds
        and estimated_log <= max_log_bytes
    )
    return ClearanceResourceCertificate(
        per_channel_actions=count,

```

```

        projected_actions_for_16_channels=projected_actions,
        baseline_virtual_seconds=baseline_virtual_seconds,
        route_start=route_start,
        route_length_m=route_length_m,
        projected_minimum_virtual_seconds=minimum,
        projected_route_virtual_seconds=routed,
        estimated_log_bytes=estimated_log,
        status="FEASIBLE_ESTIMATE" if feasible else "REJECTED_UNRESOLVED",
        reason=("Sample and resource estimates fit the stated caps." if feasible else "Fallback exceeds a
        ↪ cap or lacks sample coverage; reject it as a complete main strategy."),
    )

def _field(response: Any, name: str, default: Any = None) -> Any:
    if isinstance(response, dict):
        return response.get(name, default)
    if name == "accepted" and (isinstance(response, str) or hasattr(response, "result")):
        return True
    if isinstance(response, str):
        if name in {"result", "measure_result", "clear_result"}:
            return response
        if name == "success":
            return response == "success"
    return getattr(response, name, default)

def _ray_intersection(
    first_position: Point,
    first_bearing_deg: float,
    second_position: Point,
    second_bearing_deg: float,
) -> Point | None:
    """Return the forward-ray crossing, or ``None`` for a degenerate pair.

    This is deliberately a fast-path estimate: bearings retain their  $\pm 1^\circ$ 
    uncertainty, so callers must retain a certified fallback after a miss.
    """
    first_angle = math.radians(first_bearing_deg)
    second_angle = math.radians(second_bearing_deg)
    first_direction = Point(math.cos(first_angle), math.sin(first_angle))
    second_direction = Point(math.cos(second_angle), math.sin(second_angle))
    determinant = first_direction.x * second_direction.y - first_direction.y * second_direction.x
    if abs(determinant) <= 1e-9:
        return None
    delta = Point(second_position.x - first_position.x, second_position.y - first_position.y)
    first_distance = (delta.x * second_direction.y - delta.y * second_direction.x) / determinant
    second_distance = (delta.x * first_direction.y - delta.y * first_direction.x) / determinant
    if first_distance < 0.0 or second_distance < 0.0:
        return None
    return Point(
        first_position.x + first_distance * first_direction.x,
        first_position.y + first_distance * first_direction.y,
    )

def _safe_second_station(anchor: Point, bearing_deg: float) -> Point:
    """Place a non-collinear, still-safe receiver 1000 m from the anchor."""
    angle = math.radians(bearing_deg + SAFE_SECOND_OFFSET_DEG)

```

```

    return Point(anchor.x + RECEIVE_RADIUS_LOWER_BOUND * math.cos(angle), anchor.y +
        ↪ RECEIVE_RADIUS_LOWER_BOUND * math.sin(angle))

def _fast_clear_candidates(
    first_position: Point,
    first_bearing_deg: float,
    second_position: Point,
    second_bearing_deg: float,
) -> tuple[Point, ...]:
    """Return a small, target-disk-filtered heuristic candidate set.

    The center rays are first. The four endpoint crossings cover systematic
    ±1° endpoint cases cheaply, but are not used as a proof of clearance.
    """
    candidates: list[Point] = []
    center = _ray_intersection(first_position, first_bearing_deg, second_position, second_bearing_deg)
    if center is not None:
        candidates.append(center)
    for first_offset in (-1.0, 1.0):
        for second_offset in (-1.0, 1.0):
            candidate = _ray_intersection(
                first_position,
                first_bearing_deg + first_offset,
                second_position,
                second_bearing_deg + second_offset,
            )
            if candidate is not None:
                candidates.append(candidate)
    unique: list[Point] = []
    for candidate in candidates:
        if _distance(candidate, Point(0.0, 0.0)) > TARGET_RADIUS + 1e-9:
            continue
        if all(_distance(candidate, previous) > 1e-6 for previous in unique):
            unique.append(candidate)
    return tuple(unique)

@dataclass(frozen=True)
class StrategyExecutionResult:
    channel_states: dict[int, ChannelState]
    channel_evidence: dict[int, tuple[str, ...]]
    complete: bool

def execute_q3_strategy(robot: Any, public_state: Any) -> StrategyExecutionResult:
    """Duck-typed Q3 baseline: seven stations × twenty channels plus safe fallback.

    ``robot.measure(position, channel)`` must return an object/dict containing
    ``accepted`` and ``result``; a direction response also supplies ``svd_deg``.
    ``robot.clear(position, channel)`` returns ``accepted`` and ``success``.
    ``public_state`` is intentionally accepted for benchmark integration but
    never treated as hidden simulator truth.
    """
    del public_state
    plan = build_center_ring_plan()
    tracker = ChannelTracker(plan)
    evidence: dict[int, list[str]] = {channel: [] for channel in CHANNELS}

```

```

direction_history: dict[int, list[tuple[Point, float]]] = {channel: [] for channel in CHANNELS}
fast_path_attempted: set[int] = set()
safe_second_station_attempted: set[int] = set()
baseline = survey_cost(plan, snake_scan_plan(plan)).virtual_seconds
for action in snake_scan_plan(plan):
    if tracker.state_of(action.channel) in (ChannelState.CLEARED, ChannelState.ABSENT_CERTIFIED):
        continue
    response = robot.measure(action.position, action.channel)
    accepted = _field(response, "accepted", False) is True
    result = _field(response, "result", _field(response, "measure_result", None))
    immediate = tracker.record_measurement(action.channel, action.station_id, accepted=accepted,
    ↪ result=result)
    if not accepted or result is None:
        evidence[action.channel].append("rejected_measure")
        continue
    evidence[action.channel].append(str(result))
    if immediate is not None:
        clear_response = robot.clear(immediate.position, immediate.channel)
        tracker.record_clear(immediate.channel, immediate.position, accepted=_field(clear_response,
    ↪ "accepted", False) is True, success=_field(clear_response, "success",
    ↪ _field(clear_response, "clear_result", None) == "success"))
        continue
    if result != "direction":
        continue
    bearing = _field(response, "svd_deg", None)
    if bearing is None:
        evidence[action.channel].append("direction_without_bearing_unresolved")
        continue
    direction_history[action.channel].append((action.position, float(bearing)))
    if action.channel not in safe_second_station_attempted:
        safe_second_station_attempted.add(action.channel)
        second_station = _safe_second_station(action.position, float(bearing))
        second_response = robot.measure(second_station, action.channel)
        second_accepted = _field(second_response, "accepted", False) is True
        second_result = _field(second_response, "result", _field(second_response, "measure_result",
    ↪ None))
        evidence[action.channel].append(f"safe_second_{second_result}" if second_accepted else
    ↪ "safe_second_rejected")
        if second_accepted and second_result == "direction":
            second_bearing = _field(second_response, "svd_deg", None)
            if second_bearing is not None:
                direction_history[action.channel].append((second_station, float(second_bearing)))
        elif second_accepted and second_result == "near":
            clear_response = robot.clear(second_station, action.channel)
            accepted_clear = _field(clear_response, "accepted", False) is True
            success = _field(clear_response, "success", _field(clear_response, "clear_result", None)
    ↪ == "success")
            tracker.record_clear(action.channel, second_station, accepted=accepted_clear,
    ↪ success=success)
            evidence[action.channel].append("fast_path_clear_success" if accepted_clear and success
    ↪ else "fast_path_clear_failed")
            continue
    if len(direction_history[action.channel]) < 2 or action.channel in fast_path_attempted:
        continue
    fast_path_attempted.add(action.channel)
    first_position, first_bearing = direction_history[action.channel][0]
    second_position, second_bearing = direction_history[action.channel][1]
    candidates = _fast_clear_candidates(first_position, first_bearing, second_position, second_bearing)

```

```

if not candidates:
    evidence[action.channel].append("fast_path_degenerate")
    continue
for candidate in candidates:
    clear_response = robot.clear(candidate, action.channel)
    accepted_clear = _field(clear_response, "accepted", False) is True
    success = _field(clear_response, "success", _field(clear_response, "clear_result", None) ==
↪ "success")
    tracker.record_clear(action.channel, candidate, accepted=accepted_clear, success=success)
    if accepted_clear and success:
        evidence[action.channel].append("fast_path_clear_success")
        break
else:
    evidence[action.channel].append("fast_path_clear_failed")

for channel, observations in direction_history.items():
    if tracker.state_of(channel) in (ChannelState.CLEARED, ChannelState.ABSENT_CERTIFIED) or not
↪ observations:
        continue
    anchor, bearing = observations[0]
    fallback = build_anchor_clearance_plan(channel=channel, anchor=anchor, bearing_deg=bearing)
    resources = certify_clearance_resources(
        fallback,
        baseline_virtual_seconds=baseline,
        fallback_start=plan.route[-1],
    )
    evidence[channel].append(resources.status)
    if resources.status != "FEASIBLE_ESTIMATE":
        continue
    for clear in fallback.actions:
        clear_response = robot.clear(clear.position, clear.channel)
        accepted_clear = _field(clear_response, "accepted", False) is True
        success = _field(clear_response, "success", _field(clear_response, "clear_result", None) ==
↪ "success")
        tracker.record_clear(channel, clear.position, accepted=accepted_clear, success=success)
        if accepted_clear and success:
            evidence[channel].append("cleared_by_anchor_wedge")
            break
return StrategyExecutionResult(dict(tracker.states), {channel: tuple(items) for channel, items in
↪ evidence.items()}, tracker.is_complete())

```

文件: src/b\_problem/strategy/q34\_receding\_horizon.py

"""Truth-free, first-action receding-horizon ranking for live Q3/Q4 jobs.

The planner never mutates a strategy state. Its rollouts are bounded cost quadrature used only to rank the next public atomic action.

"""

from \_\_future\_\_ import annotations

from dataclasses import dataclass

from typing import Iterable

from .q4 import Point

from .q4\_routingplus import LiveJob, RoutingPlus

def \_length(start:Point,jobs:tuple[LiveJob,...],order:list[int])>->float:

```

total=0.; cursor=start
for index in order: total+=cursor.distance_to(jobs[index].target); cursor=jobs[index].target
return total

def _nn_order(start:Point,jobs:tuple[LiveJob,...])>->list[int]:
    left=set(range(len(jobs))); order=[]; cursor=start
    while left:
        index=min(left,key=lambda item:(cursor.distance_to(jobs[item].target),jobs[item].key))
        order.append(index);left.remove(index);cursor=jobs[index].target
    return order

def _cheap_twoopt(start:Point,jobs:tuple[LiveJob,...],order:list[int])>->list[int]:
    # One bounded pass avoids the expensive multi-seed rolling objective.
    best=list(order); base=_length(start,jobs,best)
    for first in range(len(best)-1):
        for last in range(first+1,len(best)):
            trial=best[:first]+best[first:last+1][::-1]+best[last+1:]
            value=_length(start,jobs,trial)
            if value<base-1e-9: best,base=trial,value
    return best

@dataclass(frozen=True)
class RHScore:
    key:tuple
    immediate_s:float
    residual_s:float
    continuation_s:float
    discovery_delay_s:float
    total_s:float

class RecedingHorizonPlanner:
    """Evaluate at most K action anchors using a cheap post-action residual."""
    def __init__(self,*k:int=8,cheap_twoopt:bool=False,rollout:bool=False)>->None:
        self.k=k;self.cheap_twoopt=cheap_twoopt;self.rollout=rollout
        self.calls=0;self.candidate_count=0;self.changed_from_nn=0;self.nn_selected=0;self.routing_select
        ↪ ed=0
        self.residual_sum=0.;self.predicted_sum=0.;self.hard_state_mutations=0;self.scores:list[RHScore]=[]

    @staticmethod
    def _action_seconds(job:LiveJob)>->float:
        if "CLEAR" in job.kind: return 5.0
        return 6.0 if "SCAN" in job.kind else 6.0

    def _candidate_indices(self,start:Point,jobs:tuple[LiveJob,...],routing:LiveJob|None)>->tuple[int,...]:
        nn=_nn_order(start,jobs)[0]
        candidates=[nn]
        for predicate in (lambda job:"SCAN" in job.kind,lambda job:"SOURCE" in job.kind,
            ↪ lambda job:"CLEAR" in job.kind):
            selected=[index for index,job in enumerate(jobs) if predicate(job)]
            if selected:candidates.append(min(selected,key=lambda
                ↪ index:(start.distance_to(jobs[index].target),jobs[index].key)))
        if routing is not None:
            candidates.extend(index for index,job in enumerate(jobs) if job.key==routing.key)
        # Information/discovery proxies only influence ranking; no state change.

```

```

source=[index for index,job in enumerate(jobs) if "SOURCE" in job.kind]
scans=[index for index,job in enumerate(jobs) if "SCAN" in job.kind]
if source: candidates.append(max(source,key=lambda
↪ index:(start.distance_to(jobs[index].target),jobs[index].key)))
if scans: candidates.append(min(scans,key=lambda
↪ index:(start.distance_to(jobs[index].target),jobs[index].key)))
return tuple(dict.fromkeys(candidates))[:self.k]

def choose(self,start:Point,jobs:Iterable[LiveJob],*,routing_choice:LiveJob|None=None)->LiveJob|None:
    values=tuple(jobs)
    if not values: return None
    self.calls+=1; nn_index=_nn_order(start,values)[0];
    ↪ indices=self._candidate_indices(start,values,routing_choice)
    self.candidate_count+=len(indices); ranked=[]
    for index in indices:
        job=values[index]; remaining=values[:index]+values[index+1:]
        order=_nn_order(job.target,remaining) if remaining else []
        if self.cheap_twoopt and len(order)>2: order=_cheap_twoopt(job.target,remaining,order)
        residual_m=_length(job.target,remaining,order)
        immediate=start.distance_to(job.target)/5.0+self._action_seconds(job)
        continuation=0.0
        if self.rollout and "SOURCE" in job.kind:
            # Bounded public-outcome quadrature: direction/no-signal/near
            # affect only an expected continuation-cost surrogate.
            continuation=0.12*min(1500.0,residual_m)/5.0
        delay=0.0
        if "SOURCE" in job.kind and any("SCAN" in item.kind for item in values): delay=3.0
        score=RHScore(job.key,immediate,residual_m/5.0,continuation,delay,immediate+residual_m/5.0+co
↪ ntinuation+delay)
        ranked.append((score,job))
    score,choice=min(ranked,key=lambda item:(item[0].total_s,item[1].key))
    self.scores.append(score); self.residual_sum+=score.residual_s; self.predicted_sum+=score.total_s
    if choice.key==values[nn_index].key: self.nn_selected+=1
    else: self.changed_from_nn+=1
    if routing_choice is not None and choice.key==routing_choice.key: self.routing_selected+=1
    return choice

def telemetry(self)->dict:
    return {"calls":self.calls,"candidate_first_actions_mean":self.candidate_count/max(1,self.calls),
            "nn_first_selected_rate":self.nn_selected/max(1,self.calls),"routingplus_first_selected_r
↪ ate":self.routing_selected/max(1,self.calls),
            "rh_changed_first_action_rate":self.changed_from_nn/max(1,self.calls),
            "mean_predicted_residual_cost_s":self.residual_sum/max(1,self.calls),
            "mean_predicted_total_cost_s":self.predicted_sum/max(1,self.calls),"hard_state_mutations"
↪ :self.hard_state_mutations}

```

文件: src/b\_problem/strategy/q3\_extreme.py

```

"""Offline Q3 speed candidates with the CP08-certified fallback retained."""
from __future__ import annotations

from dataclasses import dataclass
import math
from typing import Any

from b_problem.strategy import q3

```

```

_CROSS_RANGE_DEG = 55.0
_FAST_B_RING_RADIUS_M = 1125.0

@dataclass(frozen=True)
class ExtremeExecutionResult:
    channel_states: dict[int, q3.ChannelState]
    complete: bool
    candidate: str
    cp08_fallback_used: bool
    localized_channels: int
    anchor_fallback_channels: int

@dataclass(frozen=True)
class FastCSSettings:
    """Public-geometry-only knobs for the aggressive Q3 clear-now ablation."""

    forward_m: float = 750.0
    lateral_m: float = 80.0
    lateral_mode: str = "condition"
    clear_now: str = "very"
    retry_cap: int = 5
    time_debt_m: float = 350.0
    condition_degrees: float = 2.0

def build_fast_b_survey_plan() -> q3.SurveyPlan:
    """Return the independently checked shorter certified Q3 survey ring."""
    plan = q3.build_center_ring_plan(ring_radius=_FAST_B_RING_RADIUS_M)
    if plan.coverage_certificate.margin <= 0.0:
        raise RuntimeError("FAST-B survey ring lost its 1000 m coverage certificate")
    return plan

def _result(response: Any) -> str | None:
    if isinstance(response, dict):
        return response.get("result", response.get("measure_result", response.get("clear_result")))
    if isinstance(response, str):
        return response
    return getattr(response, "result", getattr(response, "measure_result", getattr(response,
    ↪ "clear_result", None)))

def _bearing(response: Any) -> float | None:
    value = response.get("svd_deg") if isinstance(response, dict) else getattr(response, "svd_deg", None)
    return float(value) if isinstance(value, (int, float)) and math.isfinite(value) else None

def _point_at(origin: q3.Point, bearing_deg: float, offset_deg: float, distance_m: float = 1000.0) ->
    ↪ q3.Point:
    angle = math.radians(bearing_deg + offset_deg)
    return q3.Point(
        origin.x + distance_m * math.cos(angle),
        origin.y + distance_m * math.sin(angle),
    )

```

```

def _safe_los_lateral_pair(pursuit_m: float, lateral_m: float) -> bool:
    """Check reception for the complete first  $\pm 1$  degree direction wedge."""
    squared = pursuit_m * pursuit_m + lateral_m * lateral_m
    minimum_projection = pursuit_m * math.cos(math.radians(1.0)) - lateral_m * math.sin(math.radians(1.0))
    return (
        math.hypot(pursuit_m, lateral_m) <= q3.RECEIVE_RADIUS_LOWER_BOUND
        and minimum_projection > 0.0
        and squared <= 2.0 * q3.RECEIVE_RADIUS_LOWER_BOUND * minimum_projection
    )

def _nearest_station(current: q3.Point, remaining: set[int], plan: q3.SurveyPlan) -> int:
    return min(remaining, key=lambda station_id: (q3._distance(current, plan.stations[station_id]),
    ↪ station_id))

def _cross_station(
    anchor: q3.Point, bearing_deg: float, *, dynamic: bool, next_station: q3.Point | None
) -> q3.Point:
    candidates = tuple(_point_at(anchor, bearing_deg, sign * _CROSS_RANGE_DEG) for sign in (-1.0, 1.0))
    if not dynamic or next_station is None:
        return candidates[1]
    # Both signs have the same conservative-reception and angular-separation
    # properties. Use only public route geometry as a tie-breaker.
    return min(candidates, key=lambda point: (q3._distance(point, next_station), point.x, point.y))

def _try_clear(robot: Any, tracker: q3.ChannelTracker, channel: int, point: q3.Point) -> tuple[bool,
    ↪ q3.Point]:
    response = robot.clear(point, channel)
    success = _result(response) == "success"
    tracker.record_clear(channel, point, accepted=True, success=success)
    return success, point

def _pursue(
    robot: Any,
    tracker: q3.ChannelTracker,
    *,
    channel: int,
    anchor: q3.Point,
    bearing_deg: float,
    dynamic: bool,
    next_station: q3.Point | None,
    los_pursuit: bool,
    pursuit_m: float,
    lateral_m: float,
) -> q3.Point:
    """Try a safe pursuit/cross-range observation, early clear, then correction."""
    if los_pursuit:
        pursuit = _point_at(anchor, bearing_deg, 0.0, pursuit_m)
        pursuit_response = robot.measure(pursuit, channel)
        pursuit_result = _result(pursuit_response)
        if pursuit_result == "near":
            _try_clear(robot, tracker, channel, pursuit)
            return pursuit
        if pursuit_result != "direction":

```

```

        return pursuit
    # The configured LOS/lateral pair is used only when its universal
    # first-bearing reception condition has been checked by the caller.
    second = _point_at(pursuit, bearing_deg, 90.0, lateral_m)
else:
    second = _cross_station(anchor, bearing_deg, dynamic=dynamic, next_station=next_station)
second_response = robot.measure(second, channel)
second_result = _result(second_response)
if second_result == "near":
    _try_clear(robot, tracker, channel, second)
    return second
second_bearing = _bearing(second_response) if second_result == "direction" else None
if second_bearing is None:
    return second

estimates = q3._fast_clear_candidates(anchor, bearing_deg, second, second_bearing)
if not estimates:
    return second
estimate = estimates[0]
if _try_clear(robot, tracker, channel, estimate)[0]:
    return estimate

correction_response = robot.measure(estimate, channel)
correction_result = _result(correction_response)
if correction_result == "near":
    _try_clear(robot, tracker, channel, estimate)
    return estimate
correction_bearing = _bearing(correction_response) if correction_result == "direction" else None
if correction_bearing is None:
    return estimate

# The correction point is close to the initial intersection estimate. Test
# the two least-travel additional wedge intersections, then defer to CP08.
refined = q3._fast_clear_candidates(anchor, bearing_deg, estimate, correction_bearing)
for point in sorted(refined, key=lambda candidate: q3._distance(estimate, candidate))[:2]:
    if _try_clear(robot, tracker, channel, point)[0]:
        return point
    estimate = point
return estimate

def _bearing_gap(first_deg: float, second_deg: float) -> float:
    """Return the unsigned wrapped bearing separation in degrees."""
    return abs((second_deg - first_deg + 180.0) % 360.0 - 180.0)

def _pursue_fast_c(
    robot: Any,
    tracker: q3.ChannelTracker,
    *,
    channel: int,
    anchor: q3.Point,
    bearing_deg: float,
    settings: FastCSettings,
) -> q3.Point:
    """Immediately turn one anchor into bounded, geometry-ranked clear tries.

    A forward observation is always inside the first  $\pm 1$  degree reception wedge.
    A short lateral observation is made only when the forward bearing does not

```

```

separate the two rays enough to support a stable direct-clear estimate.
Failed attempts leave the channel open for its certified anchor/CP08 paths.
"""
forward = _point_at(anchor, bearing_deg, 0.0, settings.forward_m)
forward_response = robot.measure(forward, channel)
if _result(forward_response) == "near":
    _try_clear(robot, tracker, channel, forward)
    return forward
if _result(forward_response) != "direction":
    return forward
forward_bearing = _bearing(forward_response)
if forward_bearing is None:
    return forward

use_lateral = settings.lateral_mode == "fixed" or (
    settings.lateral_mode == "condition"
    and _bearing_gap(bearing_deg, forward_bearing) < settings.condition_degrees
)
observation_point, observation_bearing = forward, forward_bearing
if use_lateral:
    lateral = _point_at(forward, bearing_deg, 90.0, settings.lateral_m)
    lateral_response = robot.measure(lateral, channel)
    if _result(lateral_response) == "near":
        _try_clear(robot, tracker, channel, lateral)
        return lateral
    if _result(lateral_response) != "direction":
        return lateral
    lateral_bearing = _bearing(lateral_response)
    if lateral_bearing is None:
        return lateral
    observation_point, observation_bearing = lateral, lateral_bearing

candidates = q3._fast_clear_candidates(anchor, bearing_deg, observation_point, observation_bearing)
level_limit = {"strict": 1, "moderate": 2, "aggressive": 3, "very": 5}.get(settings.clear_now)
if level_limit is None:
    raise ValueError("clear_now must be strict, moderate, aggressive, or very")
if settings.lateral_mode not in {"fixed", "condition", "none"}:
    raise ValueError("lateral_mode must be fixed, condition, or none")
# Distance is an explicit time-debt proxy: 5 m/s turns the configured
# local movement allowance into an ordering and bounded retry decision.
ordered = sorted(candidates, key=lambda point: (q3._distance(observation_point, point), point.x,
↪ point.y))
permitted: list[q3.Point] = []
debt = 0.0
previous = observation_point
for point in ordered[:min(level_limit, settings.retry_cap)]:
    increment = q3._distance(previous, point)
    if permitted and debt + increment > settings.time_debt_m:
        continue
    permitted.append(point)
    debt += increment
    previous = point
for point in permitted:
    if _try_clear(robot, tracker, channel, point)[0]:
        return point
    observation_point = point
return observation_point

```

```

def _execute(
    robot: Any, public_state: Any, *, name: str, dynamic: bool, los_pursuit: bool, batch_station_scan:
    ↪ bool,
    pursuit_m: float = 650.0, lateral_m: float = 350.0, plan: q3.SurveyPlan | None = None,
    fast_c_settings: FastCSettings | None = None,
) -> ExtremeExecutionResult:
    """Run an immediate-localisation survey without weakening absence coverage."""
    if pursuit_m <= 0.0 or lateral_m <= 0.0:
        raise ValueError("pursuit distances must be positive")
    if los_pursuit and not _safe_los_lateral_pair(pursuit_m, lateral_m):
        raise ValueError("LOS/lateral pair must retain universal second-reception coverage")
    plan = plan or q3.build_center_ring_plan()
    tracker = q3.ChannelTracker(plan)
    anchors: dict[int, tuple[q3.Point, float]] = {}
    current = q3.Point(0.0, 0.0)
    remaining = set(range(len(plan.stations)))
    localized = 0

    while remaining:
        station_id = 0 if 0 in remaining else (
            _nearest_station(current, remaining, plan) if dynamic else min(remaining)
        )
        remaining.remove(station_id)
        station = plan.stations[station_id]
        next_station = plan.stations[_nearest_station(station, remaining, plan)] if dynamic and remaining
        ↪ else None
        discoveries: list[tuple[int, float]] = []
        for channel in q3.CHANNELS:
            if tracker.state_of(channel) is not q3.ChannelState.UNKNOWN:
                continue
            response = robot.measure(station, channel)
            observation = _result(response)
            immediate = tracker.record_measurement(channel, station_id, accepted=observation is not None,
            ↪ result=observation or "")
            current = station
            if immediate is not None:
                _try_clear(robot, tracker, channel, immediate.position)
                continue
            if observation != "direction":
                continue
            bearing_deg = _bearing(response)
            if bearing_deg is None:
                continue
            anchors[channel] = (station, bearing_deg)
            localized += 1
            if batch_station_scan:
                discoveries.append((channel, bearing_deg))
        else:
            current = (
                _pursue_fast_c(
                    robot, tracker, channel=channel, anchor=station, bearing_deg=bearing_deg,
                    settings=fast_c_settings,
                )
                if fast_c_settings is not None else _pursue(
                    robot, tracker, channel=channel, anchor=station, bearing_deg=bearing_deg,
                    dynamic=dynamic, next_station=next_station, los_pursuit=los_pursuit,
                    pursuit_m=pursuit_m, lateral_m=lateral_m,

```

```

        )
    )
    for channel, bearing_deg in discoveries:
        current = _pursue(
            robot, tracker, channel=channel, anchor=station, bearing_deg=bearing_deg,
            dynamic=dynamic, next_station=next_station, los_pursuit=los_pursuit,
            pursuit_m=pursuit_m, lateral_m=lateral_m,
        )

    anchor_fallbacks = 0
    for channel, (anchor, bearing_deg) in anchors.items():
        if tracker.state_of(channel) is q3.ChannelState.CLEARED:
            continue
        fallback = q3.build_anchor_clearance_plan(channel=channel, anchor=anchor, bearing_deg=bearing_deg)
        resources = q3.certify_clearance_resources(
            fallback, baseline_virtual_seconds=q3.survey_cost(plan,
                ↪ q3.snake_scan_plan(plan)).virtual_seconds,
            fallback_start=current,
        )
        if resources.status != "FEASIBLE_ESTIMATE":
            continue
        anchor_fallbacks += 1
        for action in fallback.actions:
            success, current = _try_clear(robot, tracker, channel, action.position)
            if success:
                break

    if tracker.is_complete():
        return ExtremeExecutionResult(dict(tracker.states), True, name, False, localized, anchor_fallbacks)

    # CP08 is the ultimate safe recovery path for transport irregularities or
    # an unexpected unresolved channel. Its implementation is not modified.
    cp08 = q3.execute_q3_strategy(robot, public_state=public_state)
    return ExtremeExecutionResult(dict(cp08.channel_states), cp08.complete, name, True, localized,
        ↪ anchor_fallbacks)

def execute_q3_fast_a(robot: Any, public_state: Any) -> ExtremeExecutionResult:
    """FAST-A: 650 m LOS pursuit, then a safe 350 m lateral observation."""
    return _execute(
        robot, public_state, name="FAST-A", dynamic=True, los_pursuit=True, batch_station_scan=True,
    )

def execute_q3_fast_b(robot: Any, public_state: Any) -> ExtremeExecutionResult:
    """FAST-B: batched pursuit with a shorter independently certified survey ring."""
    return _execute(
        robot, public_state, name="FAST-B", dynamic=True, los_pursuit=True, batch_station_scan=True,
        pursuit_m=500.0, lateral_m=200.0, plan=build_fast_b_survey_plan(),
    )

def execute_q3_safe7_1130(robot: Any, public_state: Any) -> ExtremeExecutionResult:
    """Seven-point Q3 discovery candidate with a positive analytic margin."""
    plan=q3.build_center_ring_plan(ring_radius=1130.0)
    if plan.coverage_certificate.margin <= 0.0:
        raise RuntimeError("Q3 safe7 discovery certificate failed")
    return _execute(robot,public_state,name="SAFE7-1130",dynamic=True,los_pursuit=True,batch_station_scan_|
        ↪ =True,

```

```

        pursuit_m=500.0,lateral_m=200.0,plan=plan)

def execute_q3_fast_c(
    robot: Any,
    public_state: Any,
    *,
    forward_m: float = 750.0,
    lateral_m: float = 80.0,
    lateral_mode: str = "condition",
    clear_now: str = "very",
    retry_cap: int = 5,
    time_debt_m: float = 350.0,
) -> ExtremeExecutionResult:
    """FAST-C: immediate bounded retries, retaining anchor then CP08 recovery."""
    settings = FastCSettings(
        forward_m=forward_m,
        lateral_m=lateral_m,
        lateral_mode=lateral_mode,
        clear_now=clear_now,
        retry_cap=retry_cap,
        time_debt_m=time_debt_m,
    )
    return _execute(
        robot, public_state, name="FAST-C", dynamic=True, los_pursuit=True, batch_station_scan=False,
        pursuit_m=forward_m, lateral_m=lateral_m, plan=build_fast_b_survey_plan(),
        ↪ fast_c_settings=settings,
    )

```

文件: src/b\_problem/strategy/q3\_speed.py

```

"""Speed-first, truth-free Q3 architecture ported from public incremental policies.

This module is deliberately feature-flagged. It uses only public responses;
every experimental clear is accounted for only when the arena confirms it and
any unresolved channel retains the existing anchor-wedge fallback.
"""

from __future__ import annotations

from dataclasses import dataclass, field
from enum import Enum
from math import atan2, cos, degrees, hypot, pi, radians, sin
from typing import Any, Iterable

from shapely.geometry import Point as SPoint, Polygon
from shapely.ops import nearest_points

from . import q3
from .q4 import Point as Q4Point
from .q4_routingplus import LiveJob
from .q34_receding_horizon import RecedingHorizonPlanner

Point = q3.Point
CHANNELS = q3.CHANNELS
_EPS = 1e-7
def _distance(first: Point, second: Point) -> float: return q3._distance(first, second)

```

```

def _radius(point: Point) -> float: return hypot(point.x, point.y)

class SpeedState(str, Enum):
    UNKNOWN = "UNKNOWN"
    PRESENT = "PRESENT"
    CLEARED = "CLEARED"
    ABSENT = "ABSENT"

@dataclass(frozen=True)
class SpeedAction:
    kind: str
    target: Point
    channel: int | None = None
    candidates: tuple[Point, ...] = ()

@dataclass
class SpeedConfig:
    ring_count: int = 9
    ring_radius_m: float = 1700.0
    nonempty_ring_count: int = 6
    nonempty_ring_radius_m: float = 1130.0
    batch_step_m: float = 100.0
    batch_choice: str = "information"
    batch_enabled: bool = True
    angular_neighbors: bool = True
    neighbor_angle_span_deg: float = 40.0
    negative_affine: bool = True
    negative_disks: bool = True
    negative_disk_sides: int = 48
    shallow_lookahead: bool = True
    optical_band: bool = True
    max_probes: int = 5
    clear_attempt_radius_m: float = 70.0
    guaranteed_clear_radius_m: float = 18.0
    probe_widths_m: tuple[float, ...] = (25.0, 75.0, 125.0)
    probe_quantiles: tuple[float, ...] = (0.2, 0.5, 0.8)
    receding_horizon: bool = False
    rh_k: int = 8
    rh_cheap_twopt: bool = False
    rh_rollout: bool = False

@dataclass
class SpeedSession:
    channel: int
    anchor: Point
    anchor_bearing_deg: float
    positives: list[tuple[Point, float]] = field(default_factory=list)
    negatives: list[Point] = field(default_factory=list)
    probe_count: int = 0
    clear_attempts: set[Point] = field(default_factory=set)
    fallback_attempted: set[Point] = field(default_factory=set)
    fallback: tuple[Point, ...] = ()
    region: Any = None

    def __post_init__(self) -> None:

```

```

        if not self.positives:
            self.positives.append((self.anchor, self.anchor_bearing_deg))

@dataclass
class SpeedTelemetry:
    decisions: list[dict] = field(default_factory=list)
    selected: dict[str, int] = field(default_factory=dict)
    batch_moves: int = 0
    piggyback_reads: int = 0
    negative_affine_updates: int = 0
    negative_disk_updates: int = 0
    optical_sweeps: int = 0
    fallback_actions: int = 0
    region_history: list[dict] = field(default_factory=list)
    planner: dict = field(default_factory=dict)

@dataclass(frozen=True)
class SpeedExecutionResult:
    channel_states: dict[int, str]
    complete: bool
    candidate: str
    cp08_fallback_used: bool
    localized_channels: int
    anchor_fallback_channels: int
    telemetry: SpeedTelemetry

def build_ring_family(count: int, radius_m: float) -> tuple[Point, ...]:
    if count < 3 or radius_m <= 0.0:
        raise ValueError("ring count and radius must be positive")
    return (Point(0.0, 0.0),) + tuple(
        Point(radius_m * cos(2.0 * pi * index / count), radius_m * sin(2.0 * pi * index / count))
        for index in range(count)
    )

def regular_ring_coverage_margin(count:int, radius_m:float, *, target_radius_m:float=q3.TARGET_RADIUS,
                                receive_radius_m:float=q3.RECEIVE_RADIUS_LOWER_BOUND)->float:
    """Exact center-plus-regular-ring Q3 discovery margin.

    In one Voronoi sector, the farthest nearest-station distance occurs at the
    origin/ring bisector or at the target-circle sector midpoint. The formula
    is the six-ring certificate generalized to any regular count.
    """
    if count<3 or min(radius_m,target_radius_m,receive_radius_m)<=0: raise ValueError("invalid ring
    ↪ geometry")
    half=pi/count; cross=radius_m/(2.0*cos(half))
    boundary=(target_radius_m**2+radius_m**2-2.0*target_radius_m*radius_m*cos(half))*0.5
    worst=max(cross if cross<=target_radius_m else target_radius_m, min(target_radius_m,boundary))
    return receive_radius_m-worst

def affine_negative_halfplane(positive: Point, negative: Point):
    """Return the public necessary inequality  $|P-g| \leq |Q-g|$ ."""
    a = 2.0 * (negative.x - positive.x)
    b = 2.0 * (negative.y - positive.y)

```

```

c = negative.x * negative.x + negative.y * negative.y - positive.x * positive.x - positive.y *
↪ positive.y
return lambda point: a * point.x + b * point.y <= c + _EPS

def _response_result(response: Any) -> str | None:
    if isinstance(response, dict):
        return response.get("result", response.get("measure_result", response.get("clear_result")))
    if isinstance(response, str):
        return response
    return getattr(response, "result", getattr(response, "measure_result", getattr(response,
↪ "clear_result", None)))

def _bearing(response: Any) -> float | None:
    value = response.get("svd_deg") if isinstance(response, dict) else getattr(response, "svd_deg", None)
    return float(value) if isinstance(value, (int, float)) else None

def _outer_disk(center: Point, radius: float, sides: int = 96, *, outer: bool = False):
    """Polygonal disk with an explicit outer margin when used as a bound.

    GEOS buffers are inscribed regular polygons. Intersecting a 1500 m
    direction disk with that polygon can otherwise exclude a legal point on
    the true circular boundary. Negative disks deliberately retain the
    inscribed form so their subtraction remains conservative.
    """
    segments=max(8,sides//4)
    margin=radius*(1.0/cos(pi/(4.0*segments))-1.0)+1e-6 if outer else 0.0
    return SPoint(center.x, center.y).buffer(radius+margin,quad_segs=segments)

def _wedge(anchor: Point, bearing_deg: float, error_deg: float = 1.001):
    length = 4000.0
    left = radians(bearing_deg - error_deg)
    right = radians(bearing_deg + error_deg)
    return Polygon(((anchor.x, anchor.y), (anchor.x + length * cos(left), anchor.y + length * sin(left)),
        (anchor.x + length * cos(right), anchor.y + length * sin(right))))

def _halfplane_geometry(positive: Point, negative: Point):
    dx, dy = negative.x - positive.x, negative.y - positive.y
    norm = hypot(dx, dy)
    if norm <= _EPS:
        return _outer_disk(Point(0.0, 0.0), 10000.0)
    # n * x <= c/norm. A finite 10 km strip fully contains the target disk.
    n = (dx / norm, dy / norm)
    c = (negative.x * negative.x + negative.y * negative.y - positive.x * positive.x - positive.y *
↪ positive.y) / (2.0 * norm)
    tangent = (-n[1], n[0]); span = 10000.0
    return Polygon(((n[0] * (c - span) - tangent[0] * span, n[1] * (c - span) - tangent[1] * span),
        (n[0] * (c - span) + tangent[0] * span, n[1] * (c - span) + tangent[1] * span),
        (n[0] * c + tangent[0] * span, n[1] * c + tangent[1] * span),
        (n[0] * c - tangent[0] * span, n[1] * c - tangent[1] * span)))

def update_region(session: SpeedSession, config: SpeedConfig, telemetry: SpeedTelemetry | None = None):
    region = _outer_disk(Point(0.0, 0.0), q3.TARGET_RADIUS, outer=True)

```

```

for point, bearing_deg in session.positives:
    region = region.intersection(_wedge(point, bearing_deg)).intersection(_outer_disk(point, 1500.0,
    ↪ outer=True))
if config.negative_affine:
    for positive, _ in session.positives:
        for negative in session.negatives:
            region = region.intersection(_halfplane_geometry(positive, negative))
            if telemetry is not None:
                telemetry.negative_affine_updates += 1
if config.negative_disks:
    # The removed polygon is strictly inside the universally impossible
    # 1000 m disk after a Q3 no-signal observation.
    for negative in session.negatives:
        radius = 999.999 * cos(pi / config.negative_disk_sides)
        region = region.difference(_outer_disk(negative, radius, config.negative_disk_sides))
        if telemetry is not None:
            telemetry.negative_disk_updates += 1
session.region = region
if telemetry is not None:
    polygons=(region,) if getattr(region,"geom_type",None)=="Polygon" else tuple(item for item in
    ↪ getattr(region,"geoms",()) if item.geom_type=="Polygon")
    telemetry.region_history.append({"channel":session.channel,"vertices":[(point.x,point.y) for point
    ↪ in _vertices(region)],
                                   "components":[list(item.exterior.coords) for item in polygons],
                                   "positives":[(point.x,point.y,bearing) for point,bearing in
    ↪ session.positives],
                                   "negatives":[(point.x,point.y) for point in session.negatives]})
return region

def _vertices(region: Any) -> tuple[Point, ...]:
    if region is None or region.is_empty:
        return ()
    if region.geom_type == "Polygon":
        return tuple(Point(float(x), float(y)) for x, y in tuple(region.exterior.coords)[::-1])
    polygons = [item for item in getattr(region, "geoms", ()) if item.geom_type == "Polygon"]
    if not polygons:
        return ()
    polygon = max(polygons, key=lambda item: item.area)
    return tuple(Point(float(x), float(y)) for x, y in tuple(polygon.exterior.coords)[::-1])

def _region_center(region: Any) -> Point:
    point = region.representative_point() if region is not None and not region.is_empty else SPoint(0.0,
    ↪ 0.0)
    return Point(float(point.x), float(point.y))

def _region_radius(region: Any, center: Point) -> float:
    return max((_distance(center, vertex) for vertex in _vertices(region)), default=float("inf"))

def _step(point: Point, bearing_deg: float, distance_m: float) -> Point:
    theta = radians(bearing_deg)
    return Point(point.x + distance_m * cos(theta), point.y + distance_m * sin(theta))

def _probe_candidates(session: SpeedSession, current: Point, next_scan: Point | None, config: SpeedConfig)
    ↪ -> tuple[Point, ...]:

```

```

region = session.region
center = _region_center(region)
anchor, bearing_deg = session.positives[-1]
theta = radians(bearing_deg)
forward, side = (cos(theta), sin(theta)), (-sin(theta), cos(theta))
vertices = _vertices(region)
projections = [(vertex.x - anchor.x) * forward[0] + (vertex.y - anchor.y) * forward[1] for vertex in
↪ vertices] or [0.0]
lo, hi = min(projections), max(projections)
values = [center]
for quantile in config.probe_quantiles:
    step = lo + quantile * (hi - lo)
    for width in config.probe_widths_m:
        for sign in (-1.0, 1.0):
            values.append(Point(anchor.x + step * forward[0] + sign * width * side[0],
                                anchor.y + step * forward[1] + sign * width * side[1]))

if next_scan is not None:
    values.append(next_scan)
unique: list[Point] = []
for point in values:
    if _radius(point) <= q3.TARGET_RADIUS + 1e-6 and not any(_distance(point, old) < 2.0 for old in
↪ unique):
        unique.append(point)
def score(point: Point) -> tuple[float, float, float]:
    route = _distance(current, point) + (0.15 * _distance(point, next_scan) if next_scan is not None
↪ else 0.0)
    information = min((_distance(point, old) for old, _ in session.positives), default=0.0)
    return (route - 0.08 * information, point.x, point.y)
return tuple(sorted(unique, key=score)[:10])

def _safe_clear_candidates(region: Any, current: Point, next_scan: Point | None, radius: float) ->
↪ tuple[Point, ...]:
    vertices = _vertices(region)
    if not vertices:
        return ()
    center = _region_center(region)
    raw = [center, current]
    if next_scan is not None:
        raw.append(next_scan)
        segment = Polygon() # marker only; nearest projection follows below.
        _ = segment
    valid = [point for point in raw if all(_distance(point, vertex) <= radius + _EPS for vertex in
↪ vertices)]
    return tuple(sorted(dict.fromkeys(valid), key=lambda point: (_distance(current, point), point.x,
↪ point.y)))

def _fallback_points(session: SpeedSession) -> tuple[Point, ...]:
    if session.fallback:
        return session.fallback
    plan = q3.build_anchor_clearance_plan(channel=session.channel, anchor=session.anchor,
↪ bearing_deg=session.anchor_bearing_deg)
    session.fallback = tuple(action.position for action in plan.actions)
    return session.fallback

def session_action(session: SpeedSession, current: Point, next_scan: Point | None, config: SpeedConfig) ->
↪ SpeedAction:

```

```

region = session.region if session.region is not None else update_region(session, config)
center = _region_center(region)
radius = _region_radius(region, center)
safe = _safe_clear_candidates(region, current, next_scan, config.guaranteed_clear_radius_m)
if safe:
    return SpeedAction("SOURCE_CLEAR", safe[0], session.channel, safe)
# Public two-bearing ray crossings are an aggressive speed path only.
# A failed clear is retained and the state continues to evolve normally.
if len(session.positives) >= 2:
    first_point, first_bearing = session.positives[0]
    last_point, last_bearing = session.positives[-1]
    estimates = tuple(
        point for point in q3._fast_clear_candidates(first_point, first_bearing, last_point,
        ↪ last_bearing)
        if _radius(point) <= q3.TARGET_RADIUS + 1e-6 and point not in session.clear_attempts
    )
    if estimates:
        ordered = tuple(sorted(estimates, key=lambda point: (_distance(current, point), point.x,
        ↪ point.y)))
        return SpeedAction("SOURCE_CLEAR", ordered[0], session.channel, ordered[:5])
if radius <= config.clear_attempt_radius_m and center not in session.clear_attempts:
    return SpeedAction("SOURCE_CLEAR", center, session.channel, (center,))
if session.probe_count < config.max_probes:
    candidates = _probe_candidates(session, current, next_scan, config)
    if candidates:
        return SpeedAction("SOURCE_PROBE", candidates[0], session.channel, candidates)
remaining = tuple(point for point in _fallback_points(session) if point not in
↪ session.fallback_attempted)
if remaining:
    ordered = tuple(sorted(remaining, key=lambda point: (_distance(current, point), point.x, point.y)))
    return SpeedAction("SOURCE_FALLBACK", ordered[0], session.channel, ordered[:12])
return SpeedAction("SOURCE_FALLBACK", session.anchor, session.channel, (session.anchor,))

def _choose_action(current: Point, scans: Iterable[Point], sessions: dict[int, SpeedSession], config:
↪ SpeedConfig,
                planner: RecedingHorizonPlanner|None=None) -> SpeedAction | None:
    scan_values = tuple(scans)
    next_scan = min(scan_values, key=lambda point: (_distance(current, point), point.x, point.y)) if
↪ scan_values else None
    actions = [SpeedAction("CERT_SCAN", point) for point in scan_values]
    actions.extend(session_action(session, current, next_scan, config) for session in sessions.values())
    if not actions:
        return None
    if planner is not None:
        jobs=tuple(LiveJob((action.kind,action.channel,action.target.x,action.target.y),action.kind,
        ↪ Q4Point(action.target.x,action.target.y),action.channel) for action in actions)
        selected=planner.choose(Q4Point(current.x,current.y),jobs)
        if selected is not None:
            return next(action for action in actions if
            ↪ (action.kind,action.channel,action.target.x,action.target.y)==selected.key)
def score(action: SpeedAction) -> tuple[float, str, int]:
    value = _distance(current, action.target)
    if config.shallow_lookahead and action.kind == "SOURCE_PROBE" and next_scan is not None:
        value += 0.2 * _distance(action.target, next_scan)
    if action.kind == "CERT_SCAN" and sessions:
        value += 25.0 # one measurement action versus an already-localized source action
    return (value, action.kind, action.channel or 0)

```

```

return min(actions, key=score)

def _record_clear(robot: Any, state: dict[int, SpeedState], channel: int, point: Point) -> bool:
    success = _response_result(robot.clear(point, channel)) == "success"
    if success:
        state[channel] = SpeedState.CLEARED
    return success

def _apply_measurement(
    robot: Any, state: dict[int, SpeedState], sessions: dict[int, SpeedSession], channel: int, point:
    ↪ Point,
    config: SpeedConfig, telemetry: SpeedTelemetry, *, scan: bool = False,
) -> str | None:
    response = robot.measure(point, channel)
    result = _response_result(response)
    if result == "near":
        state[channel] = SpeedState.PRESENT
        _record_clear(robot, state, channel, point)
        return result
    if result == "direction":
        bearing = _bearing(response)
        if bearing is not None:
            if channel not in sessions:
                sessions[channel] = SpeedSession(channel, point, bearing)
            else:
                sessions[channel].positives.append((point, bearing))
                state[channel] = SpeedState.PRESENT
                update_region(sessions[channel], config, telemetry)
        return result
    # A late/piggyback measurement after an actual clear can legitimately be
    # ``no_signal`` because that channel is already cleared. It cannot be a
    # Q3 negative constraint on the former source position.
    if result == "no_signal" and state[channel] is SpeedState.PRESENT and channel in sessions:
        session = sessions[channel]
        if not any(_distance(point, old) <= 1e-6 for old in session.negatives):
            session.negatives.append(point)
            update_region(session, config, telemetry)
    return result

def _batch_after_scan(robot: Any, state: dict[int, SpeedState], sessions: dict[int, SpeedSession], origin:
    ↪ Point,
    config: SpeedConfig, telemetry: SpeedTelemetry) -> None:
    if not config.batch_enabled:
        return
    eligible = [session for session in sessions.values() if state[session.channel] is SpeedState.PRESENT
    ↪ and session.probe_count == 0]
    if len(eligible) < 3:
        return
    choices = []
    for index in range(24):
        angle = 2.0 * pi * index / 24.0
        point = Point(origin.x + config.batch_step_m * cos(angle), origin.y + config.batch_step_m *
        ↪ sin(angle))
        score = 0.0
        for session in eligible:

```

```

        center = _region_center(session.region)
        score += min(300.0, _distance(point, center))
        choices.append((score, point))
    _, point = max(choices, key=lambda item: (item[0], -item[1].x, -item[1].y))
    telemetry.batch_moves += 1
    for session in eligible:
        _apply_measurement(robot, state, sessions, session.channel, point, config, telemetry)

def _piggyback(robot: Any, state: dict[int, SpeedState], sessions: dict[int, SpeedSession], point: Point,
               config: SpeedConfig, telemetry: SpeedTelemetry) -> None:
    if not config.angular_neighbors:
        return
    for session in list(sessions.values()):
        if state[session.channel] is not SpeedState.PRESENT or session.probe_count >= config.max_probes:
            continue
        if any(_distance(point, old) < 5.0 for old, _ in session.positives):
            continue
        vertices = _vertices(session.region)
        if not vertices or min(_distance(point, vertex) for vertex in vertices) > 1500.0:
            continue
        angles = sorted(atan2(vertex.y - point.y, vertex.x - point.x) for vertex in vertices)
        gaps = [(angles[(index + 1) % len(angles)] - angles[index]) % (2.0 * pi) for index in
                 ↪ range(len(angles))]
        if 2.0 * pi - max(gaps) + _EPS < radians(config.neighbor_angle_span_deg):
            continue
        _apply_measurement(robot, state, sessions, session.channel, point, config, telemetry)
        session.probe_count += 1
        telemetry.piggyback_reads += 1

def execute_q3_speed(
    robot: Any, public_state: Any, *, config: SpeedConfig | None = None, telemetry: SpeedTelemetry | None =
    ↪ None,
) -> SpeedExecutionResult:
    """Atomic Q3 scan/source scheduler with all aggressive mechanisms gated."""
    config = config or SpeedConfig()
    telemetry = telemetry or SpeedTelemetry()
    rh=RecedingHorizonPlanner(k=config.rh_k,cheap_twoopt=config.rh_cheap_twoopt,rollout=config.rh_rollout)
    ↪ if config.receding_horizon else None
    state = {channel: SpeedState.UNKNOWN for channel in CHANNELS}
    sessions: dict[int, SpeedSession] = {}
    scanned: set[Point] = set()
    pending: set[Point] = {Point(0.0, 0.0)}
    current = Point(0.0, 0.0)
    ring_materialized = False
    localized = 0
    while pending or sessions:
        choice = _choose_action(current, pending, {channel: session for channel, session in
        ↪ sessions.items() if state[channel] is SpeedState.PRESENT}, config,rh)
        if choice is None:
            break
        before = current
        telemetry.decisions.append({"kind": choice.kind, "channel": choice.channel, "target":
        ↪ (choice.target.x, choice.target.y),
                                   "distance_m": _distance(before, choice.target), "scan_jobs":
        ↪ len(pending),
                                   "source_jobs": sum(value is SpeedState.PRESENT for value in
        ↪ state.values())})

```

```

telemetry.selected[choice.kind] = telemetry.selected.get(choice.kind, 0) + 1
if choice.kind == "CERT_SCAN":
    pending.remove(choice.target)
    scanned.add(choice.target)
    for channel in CHANNELS:
        if state[channel] is not SpeedState.UNKNOWN:
            continue
        result = _apply_measurement(robot, state, sessions, channel, choice.target, config,
        ↪ telemetry, scan=True)
        if result == "direction":
            localized += 1
    current = choice.target
    if not ring_materialized and choice.target == Point(0.0, 0.0):
        ring_materialized = True
        discovered = any(value is SpeedState.PRESENT for value in state.values())
        count = config.nonempty_ring_count if discovered else config.ring_count
        radius = config.nonempty_ring_radius_m if discovered else config.ring_radius_m
        if regular_ring_coverage_margin(count, radius) <= _EPS:
            raise RuntimeError("Q3 regular-ring discovery certificate failed")
        pending.update(build_ring_family(count, radius)[1:])
    _batch_after_scan(robot, state, sessions, current, config, telemetry)
else:
    session = sessions[choice.channel]
    if choice.kind == "SOURCE_CLEAR":
        if _record_clear(robot, state, session.channel, choice.target):
            sessions.pop(session.channel, None)
        else:
            session.clear_attempts.add(choice.target)
    elif choice.kind == "SOURCE_PROBE":
        _apply_measurement(robot, state, sessions, session.channel, choice.target, config,
        ↪ telemetry)
        session.probe_count += 1
    else:
        telemetry.fallback_actions += 1
        if _record_clear(robot, state, session.channel, choice.target):
            sessions.pop(session.channel, None)
        else:
            session.fallback_attempted.add(choice.target)
    current = choice.target
    _piggyback(robot, state, sessions, current, config, telemetry)
if sum(value is SpeedState.CLEARED for value in state.values()) >= 16:
    break
if not pending:
    for channel in CHANNELS:
        if state[channel] is SpeedState.UNKNOWN:
            state[channel] = SpeedState.ABSENT
complete = sum(value is SpeedState.CLEARED for value in state.values()) >= 16 or all(
    value in (SpeedState.CLEARED, SpeedState.ABSENT) for value in state.values()
)
cp08_used = False
if not complete:
    cp08 = q3.execute_q3_strategy(robot, public_state)
    cp08_used = True
    complete = cp08.complete
    for channel, value in cp08.channel_states.items():
        state[channel] = SpeedState.CLEARED if value == q3.ChannelState.CLEARED else state[channel]
if rh is not None: telemetry.planner=rh.telemetry()
return SpeedExecutionResult({channel: value.value for channel, value in state.items()}, complete,
    ↪ "Q3-SPEED", cp08_used,

```

```
localized, telemetry.fallback_actions, telemetry)
```

文件: src/b\_problem/strategy/q4.py

```
"""Certified baseline geometry and recovery state for Q4 directional sources.
```

```
This module owns the Q4 strategy primitives only. It does not call the
simulator and does not duplicate shared Q1 localization geometry.
```

```
"""
```

```
from __future__ import annotations
```

```
from dataclasses import dataclass, field
```

```
from enum import Enum
```

```
from math import atan2, ceil, cos, hypot, radians, sin, sqrt
```

```
from typing import Any, Iterable
```

```
TARGET_RADIUS_M = 1800.0
```

```
MIN_RECEIVE_RADIUS_M = 1000.0
```

```
CLEAR_RADIUS_M = 20.0
```

```
SQRT3 = sqrt(3.0)
```

```
TOLERANCE = 1e-9
```

```
@dataclass(frozen=True, order=True)
```

```
class Point:
```

```
    x: float
```

```
    y: float
```

```
    def distance_to(self, other: "Point") -> float:
```

```
        return hypot(self.x - other.x, self.y - other.y)
```

```
    def radius(self) -> float:
```

```
        return hypot(self.x, self.y)
```

```
    def dot_from(self, origin: "Point", normal: "Point") -> float:
```

```
        return (self.x - origin.x) * normal.x + (self.y - origin.y) * normal.y
```

```
@dataclass(frozen=True)
```

```
class Triangle:
```

```
    a: Point
```

```
    b: Point
```

```
    c: Point
```

```
    @property
```

```
    def vertices(self) -> tuple[Point, Point, Point]:
```

```
        return (self.a, self.b, self.c)
```

```
    @property
```

```
    def signed_double_area(self) -> float:
```

```
        return (self.b.x - self.a.x) * (self.c.y - self.a.y) - (self.b.y - self.a.y) * (self.c.x -
↪ self.a.x)
```

```
    @property
```

```
    def is_non_degenerate(self) -> bool:
```

```
        return abs(self.signed_double_area) > TOLERANCE
```

```

@property
def max_edge_m(self) -> float:
    return max(self.a.distance_to(self.b), self.b.distance_to(self.c), self.c.distance_to(self.a))

def contains_closed(self, point: Point) -> bool:
    """Closed barycentric containment, including edges and vertices."""
    if not self.is_non_degenerate:
        return False
    signs = []
    for start, end in ((self.a, self.b), (self.b, self.c), (self.c, self.a)):
        signs.append((end.x - start.x) * (point.y - start.y) - (end.y - start.y) * (point.x - start.x))
    return min(signs) >= -TOLERANCE or max(signs) <= TOLERANCE

@dataclass(frozen=True)
class CellScaleAssessment:
    edge_m: float
    status: str
    reason: str

def assess_triangle_edge(edge_m: float) -> CellScaleAssessment:
    """Assess only the stated triangular-cell sufficient condition.

    An edge above 1000 m is not a claim that every possible deployment
    fails. It loses the simple uniform triangle-cell guarantee because a
    source can sit arbitrarily near one grid vertex while all other vertices
    are beyond the minimum receiving radius.
    """
    if edge_m <= 0.0:
        raise ValueError("edge_m must be positive")
    if edge_m <= MIN_RECEIVE_RADIUS_M + TOLERANCE:
        return CellScaleAssessment(edge_m, "PROVED", "Triangle-cell sufficient condition holds: every cell
        ↪ edge is at most 1000 m.")
    return CellScaleAssessment(
        edge_m,
        "NO_UNIVERSAL_GUARANTEE",
        "The single triangle-cell condition loses its universal guarantee; alternative deployments are not
        ↪ assessed here.",
    )

def _lattice_point(i: int, j: int, edge_m: float, origin: Point) -> Point:
    return Point(origin.x + edge_m * (i + j / 2.0), origin.y + edge_m * SQRT3 * j / 2.0)

def _index_limit(radius_m: float, edge_m: float, outer_layers: int) -> int:
    extended = radius_m + outer_layers * edge_m
    # Bounds both lattice coordinates for every point in the extended disk,
    # then leaves two full lattice layers of slack for the containing cell.
    return ceil(max(2.0 * extended / (SQRT3 * edge_m), extended / edge_m + extended / (SQRT3 * edge_m))) +
    ↪ 2

def _full_lattice_cells(radius_m: float, edge_m: float, origin: Point, outer_layers: int) ->
    ↪ tuple[tuple[Triangle, ...], frozenset[Point]]:
    limit = _index_limit(radius_m, edge_m, outer_layers)

```

```

cells: list[Triangle] = []
stations: set[Point] = set()
for i in range(-limit, limit + 1):
    for j in range(-limit, limit + 1):
        p00 = _lattice_point(i, j, edge_m, origin)
        p10 = _lattice_point(i + 1, j, edge_m, origin)
        p01 = _lattice_point(i, j + 1, edge_m, origin)
        p11 = _lattice_point(i + 1, j + 1, edge_m, origin)
        first = Triangle(p00, p10, p01)
        second = Triangle(p10, p11, p01)
        cells.extend((first, second))
        stations.update(first.vertices)
        stations.update(second.vertices)
return tuple(cells), frozenset(stations)

@dataclass(frozen=True)
class PointCoverageCertificate:
    source: Point
    cell: Triangle
    contains_source: bool
    all_vertices_scheduled: bool
    status: str

@dataclass(frozen=True)
class ContinuousCoverageCertificate:
    target_center: Point
    target_radius_m: float
    edge_m: float
    outer_extension_m: float
    cell_count: int
    station_count: int
    all_cell_vertices_scheduled: bool
    status: str
    proof: str

@dataclass(frozen=True)
class SurveyPlan:
    nominal_edge_m: float
    eta_m: float
    edge_m: float
    target_radius_m: float
    outer_extension_m: float
    cells: tuple[Triangle, ...]
    stations: frozenset[Point]
    route: tuple[Point, ...]

def certificate_for_point(self, source: Point) -> PointCoverageCertificate:
    if source.radius() > self.target_radius_m + TOLERANCE:
        raise ValueError("source is outside the target circle")
    for cell in self.cells:
        if cell.contains_closed(source):
            scheduled = all(vertex in self.stations for vertex in cell.vertices)
            status = "PROVED" if scheduled and cell.max_edge_m <= MIN_RECEIVE_RADIUS_M + TOLERANCE
            ↪ else "UNRESOLVED"
            return PointCoverageCertificate(source, cell, True, scheduled, status)

```

```

        raise RuntimeError("construction error: target point has no scheduled containing cell")

def continuous_coverage_certificate(self) -> ContinuousCoverageCertificate:
    assessment = assess_triangle_edge(self.edge_m)
    all_scheduled = all(all(vertex in self.stations for vertex in cell.vertices) for cell in
        ↪ self.cells)
    status = "PROVED" if assessment.status == "PROVED" and self.outer_extension_m >= self.edge_m and
        ↪ all_scheduled else "UNRESOLVED"
    return ContinuousCoverageCertificate(
        target_center=Point(0.0, 0.0),
        target_radius_m=self.target_radius_m,
        edge_m=self.edge_m,
        outer_extension_m=self.outer_extension_m,
        cell_count=len(self.cells),
        station_count=len(self.stations),
        all_cell_vertices_scheduled=all_scheduled,
        status=status,
        proof=(
            "A complete finite triangular-lattice patch covers a disk expanded by one edge layer; "
            "therefore every target-circle point lies in a scheduled closed triangular cell. "
            "Every such cell has maximum edge at most 1000 m."
        ),
    )

def build_survey_plan(
    *,
    target_radius_m: float = TARGET_RADIUS_M,
    nominal_edge_m: float = MIN_RECEIVE_RADIUS_M,
    eta_m: float = 1.0,
    outer_layers: int = 1,
) -> SurveyPlan:
    """Build a finite, one-layer-extended triangular survey plan.

    ``eta_m`` is explicit so the operational default is 999 m rather than an
    equality-sensitive 1000 m numerical boundary.
    """
    if target_radius_m <= 0.0:
        raise ValueError("target_radius_m must be positive")
    if eta_m <= 0.0:
        raise ValueError("eta_m must be positive")
    if outer_layers < 1:
        raise ValueError("outer_layers must be at least one")
    edge_m = nominal_edge_m - eta_m
    if edge_m <= 0.0:
        raise ValueError("eta_m must be smaller than nominal_edge_m")
    if assess_triangle_edge(edge_m).status != "PROVED":
        raise ValueError("actual triangular cell edge must not exceed 1000 m")
    all_cells, _ = _full_lattice_cells(target_radius_m, edge_m, Point(0.0, 0.0), outer_layers)
    # A cell containing a target-disk point has every vertex within R+h of
    # the origin. Keeping this finite collar preserves continuous coverage
    # while avoiding the much larger square construction patch.
    cells = tuple(
        cell for cell in all_cells
        if min(vertex.radius() for vertex in cell.vertices) <= target_radius_m + edge_m + TOLERANCE
    )
    stations = frozenset(vertex for cell in cells for vertex in cell.vertices)
    return SurveyPlan(

```

```

        nominal_edge_m=nominal_edge_m,
        eta_m=eta_m,
        edge_m=edge_m,
        target_radius_m=target_radius_m,
        outer_extension_m=outer_layers * edge_m,
        cells=cells,
        stations=stations,
        route=_q4_snake_order(tuple(stations)),
    )

@dataclass(frozen=True)
class ClosedHalfPlaneReceptionCertificate:
    source: Point
    triangle: Triangle
    status: str
    non_degenerate_local_set: bool
    nonzero_neighbor_count: int

    def has_receiver_in_closed_halfplane(self, normal: Point, radius_m: float = MIN_RECEIVE_RADIUS_M) ->
    ↪ bool:
        if hypot(normal.x, normal.y) <= TOLERANCE:
            raise ValueError("half-plane normal must be non-zero")
        if self.status != "PROVED":
            return False
        return any(
            vertex.distance_to(self.source) <= radius_m + TOLERANCE
            and vertex.dot_from(self.source, normal) >= -TOLERANCE
            for vertex in self.triangle.vertices
        )

def closed_halfplane_reception_certificate(
    source: Point, triangle: Triangle, *, radius_m: float = MIN_RECEIVE_RADIUS_M
) -> ClosedHalfPlaneReceptionCertificate:
    """Prove at least one triangle vertex receives for any closed 180° half-plane."""
    if radius_m < MIN_RECEIVE_RADIUS_M:
        raise ValueError("certificate is intended for the stated 1000 m minimum radius")
    nonzero_neighbors = sum(vertex.distance_to(source) > TOLERANCE for vertex in triangle.vertices)
    proven = (
        triangle.is_non_degenerate
        and triangle.contains_closed(source)
        and triangle.max_edge_m <= radius_m + TOLERANCE
        and nonzero_neighbors >= 2
    )
    return ClosedHalfPlaneReceptionCertificate(
        source=source,
        triangle=triangle,
        status="PROVED" if proven else "UNRESOLVED",
        non_degenerate_local_set=triangle.is_non_degenerate and nonzero_neighbors >= 2,
        nonzero_neighbor_count=nonzero_neighbors,
    )

@dataclass(frozen=True)
class ClearanceGrid:
    center: Point
    target_radius_m: float

```

```

spacing_m: float
eta_c_m: float
coverage_radius_m: float
cells: tuple[Triangle, ...]
stations: frozenset[Point]

def coverage_certificate(self) -> "ClearanceCoverageCertificate":
    all_scheduled = all(all(vertex in self.stations for vertex in cell.vertices) for cell in
        ↪ self.cells)
    status = "PROVED" if self.coverage_radius_m <= CLEAR_RADIUS_M + TOLERANCE and all_scheduled else
        ↪ "UNRESOLVED"
    return ClearanceCoverageCertificate(
        center=self.center,
        target_radius_m=self.target_radius_m,
        spacing_m=self.spacing_m,
        coverage_radius_m=self.coverage_radius_m,
        all_cell_vertices_scheduled=all_scheduled,
        status=status,
    )

def certificate_for_point(self, source: Point) -> PointCoverageCertificate:
    if source.distance_to(self.center) > self.target_radius_m + TOLERANCE:
        raise ValueError("source is outside the certified clearance disk")
    for cell in self.cells:
        if cell.contains_closed(source):
            scheduled = all(vertex in self.stations for vertex in cell.vertices)
            status = "PROVED" if scheduled and self.coverage_radius_m <= CLEAR_RADIUS_M + TOLERANCE
                ↪ else "UNRESOLVED"
            return PointCoverageCertificate(source, cell, True, scheduled, status)
    raise RuntimeError("construction error: clearance point has no containing cell")

@dataclass(frozen=True)
class ClearanceCoverageCertificate:
    center: Point
    target_radius_m: float
    spacing_m: float
    coverage_radius_m: float
    all_cell_vertices_scheduled: bool
    status: str

def build_clearance_grid(
    center: Point,
    *,
    target_radius_m: float = 1500.0,
    eta_c_m: float = 0.01,
    outer_layers: int = 1,
) -> ClearanceGrid:
    """Build a 20 m-covering triangular clear-point grid around an anchor."""
    if eta_c_m <= 0.0:
        raise ValueError("eta_c_m must be positive")
    spacing_m = CLEAR_RADIUS_M * SQRT3 - eta_c_m
    if spacing_m <= 0.0 or outer_layers < 1:
        raise ValueError("invalid clearance-grid parameters")
    cells, stations = _full_lattice_cells(target_radius_m, spacing_m, center, outer_layers)
    return ClearanceGrid(
        center=center,

```

```

        target_radius_m=target_radius_m,
        spacing_m=spacing_m,
        eta_c_m=eta_c_m,
        coverage_radius_m=spacing_m / SQRT3,
        cells=cells,
        stations=stations,
    )

# A full 1500 m disk grid is retained above only as a geometric compatibility
# helper. The executable Q4 fallback below uses the much narrower first-
# bearing wedge and rejects itself when action/time/log estimates exceed caps.
ESTIMATED_LOG_BYTES_PER_ACTION = 256
MAX_VIRTUAL_SECONDS = 360000.0
MAX_LOG_BYTES = 2 * 1024 * 1024

@dataclass(frozen=True)
class WedgeSampleCoverageCertificate:
    sample_count: int
    all_samples_covered: bool
    maximum_nearest_distance_m: float
    continuous_sector_coverage: bool
    status: str
    proof: str = ""

@dataclass(frozen=True)
class WedgeResourceCertificate:
    projected_actions_for_16_channels: int
    projected_route_virtual_seconds: float
    estimated_log_bytes: int
    status: str
    reason: str

@dataclass(frozen=True)
class AnchorWedgeClearanceGrid:
    center: Point
    bearing_deg: float
    region_kind: str
    spacing_m: float
    eta_c_m: float
    coverage_radius_m: float
    stations: tuple[Point, ...]
    sample_coverage_certificate: WedgeSampleCoverageCertificate
    resource_certificate: WedgeResourceCertificate
    bounding_box_area_m2: float
    route_length_m: float

    def coverage_certificate(self) -> WedgeSampleCoverageCertificate:
        return self.sample_coverage_certificate

def _q4_angle_delta(angle_deg: float, bearing_deg: float) -> float:
    return (angle_deg - bearing_deg + 180.0) % 360.0 - 180.0

def _q4_wedge_samples(anchor: Point, bearing_deg: float, delta_deg: float) -> tuple[Point, ...]:

```

```

samples: list[Point] = []
radii = {0.0, 5.0, 20.0, 1000.0, 1500.0}
radii.update(1500.0 * index / 75.0 for index in range(76))
for radius in sorted(radii):
    for offset in (-delta_deg, -delta_deg / 2.0, 0.0, delta_deg / 2.0, delta_deg):
        angle = radians(bearing_deg + offset)
        point = Point(anchor.x + radius * cos(angle), anchor.y + radius * sin(angle))
        if point.radius() <= TARGET_RADIUS_M + TOLERANCE:
            samples.append(point)
return tuple(dict.fromkeys(samples))

def _q4_box_grid(x_min: float, x_max: float, y_min: float, y_max: float, spacing_m: float) -> tuple[Point,
↪ ...]:
    pad = spacing_m
    row_step = SQRT3 * spacing_m / 2.0
    first_row = int((y_min - pad) // row_step)
    last_row = int(ceil((y_max + pad) / row_step))
    points: list[Point] = []
    for row in range(first_row, last_row + 1):
        offset = (row & 1) * spacing_m / 2.0
        first_col = int((x_min - pad - offset) // spacing_m)
        last_col = int(ceil((x_max + pad - offset) / spacing_m))
        points.extend(Point(column * spacing_m + offset, row * row_step) for column in range(first_col,
↪ last_col + 1))
    return tuple(dict.fromkeys(points))

def _q4_snake_order(points: tuple[Point, ...]) -> tuple[Point, ...]:
    rows: dict[float, list[Point]] = {}
    for point in points:
        rows.setdefault(point.y, []).append(point)
    ordered: list[Point] = []
    for index, y in enumerate(sorted(rows)):
        ordered.extend(sorted(rows[y], key=lambda point: point.x, reverse=bool(index % 2)))
    return tuple(ordered)

def _q4_route_length(anchor: Point, points: tuple[Point, ...]) -> float:
    return sum(first.distance_to(second) for first, second in zip((anchor, *points), points))

def multi_source_clear_route(
    start: Point, tasks: tuple[tuple[int, tuple[Point, ...]], ...]
) -> tuple[tuple[int, Point], ...]:
    """Nearest-neighbor fast path that preserves every independent clear witness.

    Reordering does not alter the certified fallback: if no clear succeeds,
    every point from every channel's clearance grid is still attempted.
    """
    remaining = [(channel, point) for channel, points in tasks for point in points]
    current = start
    route: list[tuple[int, Point]] = []
    while remaining:
        index, (channel, point) = min(
            enumerate(remaining),
            key=lambda item: (
                current.distance_to(item[1][1]), item[1][0], item[1][1].x, item[1][1].y,

```

```

        ),
    )
    route.append((channel, point))
    current = point
    remaining.pop(index)
return tuple(route)

def _q4_distance_to_closed_sector(point: Point, *, radius_m: float, delta_deg: float) -> float:
    """Exact Euclidean distance from a local point to a closed circular sector."""
    angle = atan2(point.y, point.x)
    half_width = radians(delta_deg)
    radial_distance = hypot(point.x, point.y)
    if abs(angle) <= half_width + TOLERANCE:
        if radial_distance <= radius_m:
            return 0.0
        return radial_distance - radius_m

def distance_to_segment(end: Point) -> float:
    length_squared = end.x * end.x + end.y * end.y
    projection = max(0.0, min(1.0, (point.x * end.x + point.y * end.y) / length_squared))
    return hypot(point.x - projection * end.x, point.y - projection * end.y)

endpoint_angle = half_width if angle > 0.0 else -half_width
endpoint = Point(radius_m * cos(endpoint_angle), radius_m * sin(endpoint_angle))
other_endpoint = Point(radius_m * cos(-endpoint_angle), radius_m * sin(-endpoint_angle))
return min(distance_to_segment(endpoint), distance_to_segment(other_endpoint))

def build_anchor_clearance_grid(
    anchor: Point,
    *,
    bearing_deg: float,
    delta_deg: float = 1.0,
    eta_c_m: float = 0.01,
) -> AnchorWedgeClearanceGrid:
    """Construct a sampled/cell-lattice check for the first-bearing narrow wedge."""
    if not 0.0 < delta_deg < 90.0 or eta_c_m <= 0.0:
        raise ValueError("invalid narrow-clearance parameters")
    spacing_m = CLEAR_RADIUS_M * SQRT3 - eta_c_m
    coverage_radius = spacing_m / SQRT3
    x_min, x_max = 0.0, 1500.0
    transverse = 1500.0 * sin(radians(delta_deg))
    y_min, y_max = -transverse, transverse
    candidates = _q4_box_grid(x_min, x_max, y_min, y_max, spacing_m)
    # Retain every lattice station that can be the nearest receiver of a
    # point in the closed sector. The full lattice has covering radius C;
    # hence a nearest station of any sector point has distance at most C to
    # the sector and survives this filter.
    local_stations = _q4_snake_order(tuple(
        point for point in candidates
        if _q4_distance_to_closed_sector(point, radius_m=1500.0, delta_deg=delta_deg) <= coverage_radius +
        ↪ TOLERANCE
    ))
    heading = radians(bearing_deg)
    cosine, sine = cos(heading), sin(heading)
    stations = tuple(
        Point(anchor.x + cosine * point.x - sine * point.y, anchor.y + sine * point.x + cosine * point.y)

```

```

        for point in local_stations
    )
    samples = _q4_wedge_samples(anchor, bearing_deg, delta_deg)
    maximum_nearest = max(
        (min(sample.distance_to(point) for point in stations) for sample in samples),
        default=0.0,
    )
    sample_certificate = WedgeSampleCoverageCertificate(
        sample_count=len(samples),
        all_samples_covered=maximum_nearest <= coverage_radius + TOLERANCE,
        maximum_nearest_distance_m=maximum_nearest,
        continuous_sector_coverage=bool(local_stations),
        status="PROVED" if local_stations else "UNRESOLVED",
        proof=(
            "The full triangular lattice has covering radius spacing/sqrt(3). "
            "Every nearest lattice point of a closed-sector point is within that radius of the sector, "
            "so it survives the exact sector-distance filter."
        ),
    )
    # Compute order before rotation; rigid motion preserves all segment lengths.
    route_length = _q4_route_length(Point(0.0, 0.0), local_stations)
    projected_actions = len(stations) * 16
    projected_virtual = 16 * (3.0 * max(len(stations) - 1, 0) + 5.0 + route_length / 5.0)
    baseline_actions = len(build_survey_plan().stations) * 20
    estimated_log = (projected_actions + baseline_actions) * ESTIMATED_LOG_BYTES_PER_ACTION
    feasible = sample_certificate.continuous_sector_coverage and projected_virtual <= MAX_VIRTUAL_SECONDS
    ↪ and estimated_log <= MAX_LOG_BYTES
    resource_certificate = WedgeResourceCertificate(
        projected_actions_for_16_channels=projected_actions,
        projected_route_virtual_seconds=projected_virtual,
        estimated_log_bytes=estimated_log,
        status="FEASIBLE_ESTIMATE" if feasible else "REJECTED_UNRESOLVED",
        reason="Continuous sector coverage and estimated resource caps fit." if feasible else "Reject as
        ↪ main fallback: coverage core unavailable or cap exceeded.",
    )
    return AnchorWedgeClearanceGrid(
        center=anchor,
        bearing_deg=bearing_deg,
        region_kind="anchor_wedge",
        spacing_m=spacing_m,
        eta_c_m=eta_c_m,
        coverage_radius_m=coverage_radius,
        stations=stations,
        sample_coverage_certificate=sample_certificate,
        resource_certificate=resource_certificate,
        bounding_box_area_m2=(x_max - x_min) * (y_max - y_min),
        route_length_m=route_length,
    )

class MeasurementResult(str, Enum):
    DIRECTION = "direction"
    NO_SIGNAL = "no_signal"
    NEAR = "near"

class AnchorPhase(str, Enum):
    ANCHORED = "anchored"

```

```

FALLBACK_CLEAR = "fallback_clear"
CLEAR_PENDING = "clear_pending"

class RecoveryAction(str, Enum):
    CONTINUE_PURSUIT = "continue_pursuit"
    BEGIN_CERTIFIED_CLEAR_GRID = "begin_certified_clear_grid"
    CLEAR_NOW = "clear_now"

@dataclass
class AnchorState:
    """Retain the first positive direction through all active-pursuit failures."""

    station: Point
    channel: int
    bearing_deg: float
    phase: AnchorPhase = AnchorPhase.ANCHORED
    clearance_grid: ClearanceGrid | AnchorWedgeClearanceGrid | None = field(default=None, init=False)

    def __post_init__(self) -> None:
        if not 1 <= self.channel <= 20:
            raise ValueError("channel must be in 1..20")

    def observe_active_pursuit(self, result: MeasurementResult) -> RecoveryAction:
        if result is MeasurementResult.NEAR:
            self.phase = AnchorPhase.CLEAR_PENDING
            return RecoveryAction.CLEAR_NOW
        if result is MeasurementResult.NO_SIGNAL:
            # The anchor fields are deliberately never overwritten or cleared:
            # a directional source may simply exclude the active test point.
            self.phase = AnchorPhase.FALLBACK_CLEAR
            self.clearance_grid = build_anchor_clearance_grid(self.station, bearing_deg=self.bearing_deg)
            return RecoveryAction.BEGIN_CERTIFIED_CLEAR_GRID
        return RecoveryAction.CONTINUE_PURSUIT

class Q4ChannelState(str, Enum):
    UNKNOWN = "unknown"
    PRESENT_UNRESOLVED = "present_unresolved"
    CLEARED = "cleared"
    ABSENT_CERTIFIED = "absent_certified"

@dataclass(frozen=True)
class Q4ExecutionResult:
    channel_states: dict[int, str]
    channel_evidence: dict[int, tuple[str, ...]]
    complete: bool

def _response_field(response: Any, name: str, default: Any = None) -> Any:
    if isinstance(response, dict):
        return response.get(name, default)
    if name == "accepted" and (isinstance(response, str) or hasattr(response, "result")):
        return True
    if isinstance(response, str):
        if name in {"result", "measure_result", "clear_result"}:
            return response

```

```

        if name == "success":
            return response == "success"
    return getattr(response, name, default)

def execute_q4_strategy(
    robot: Any, public_state: Any, *, precleared_channels: Iterable[int] = ()
) -> Q4ExecutionResult:
    """Duck-typed certified Q4 station survey with narrow-wedge clear fallback."""
    del public_state
    plan = build_survey_plan()
    stations = plan.route
    precleared = frozenset(precleared_channels)
    if any(not isinstance(channel, int) or isinstance(channel, bool) or not 1 <= channel <= 20 for channel
    ↪ in precleared):
        raise ValueError("precleared_channels must contain channel integers in 1..20")
    states: dict[int, Q4ChannelState] = {
        channel: Q4ChannelState.CLEARED if channel in precleared else Q4ChannelState.UNKNOWN
        for channel in range(1, 21)
    }
    no_signal: dict[int, set[int]] = {channel: set() for channel in range(1, 21)}
    evidence: dict[int, list[str]] = {
        channel: ["precleared_fast_path"] if channel in precleared else []
        for channel in range(1, 21)
    }
    pending_grids: dict[int, AnchorWedgeClearanceGrid] = {}
    for station_id, station in enumerate(stations):
        for channel in range(1, 21):
            if states[channel] is not Q4ChannelState.UNKNOWN:
                continue
            response = robot.measure(station, channel)
            accepted = _response_field(response, "accepted", False) is True
            result = _response_field(response, "result", _response_field(response, "measure_result", None))
            if not accepted or result is None:
                evidence[channel].append("rejected_measure")
                continue
            evidence[channel].append(str(result))
            if result == "no_signal":
                no_signal[channel].add(station_id)
                if len(no_signal[channel]) == len(stations):
                    states[channel] = Q4ChannelState.ABSENT_CERTIFIED
                    continue
            if result == "near":
                clear_response = robot.clear(station, channel)
                if _response_field(clear_response, "accepted", False) is True and
                ↪ _response_field(clear_response, "success", _response_field(clear_response,
                ↪ "clear_result", None) == "success"):
                    states[channel] = Q4ChannelState.CLEARED
                    evidence[channel].append("near_clear_success")
            else:
                states[channel] = Q4ChannelState.PRESENT_UNRESOLVED
                continue
            if result != "direction":
                evidence[channel].append("unknown_measure_result")
                continue
            bearing = _response_field(response, "svd_deg", None)
            if bearing is None:
                states[channel] = Q4ChannelState.PRESENT_UNRESOLVED

```

```

        evidence[channel].append("direction_without_bearing_unresolved")
        continue
    grid = build_anchor_clearance_grid(station, bearing_deg=float(bearing))
    evidence[channel].append(grid.resource_certificate.status)
    if grid.resource_certificate.status != "FEASIBLE_ESTIMATE":
        states[channel] = Q4ChannelState.PRESENT_UNRESOLVED
        continue
    states[channel] = Q4ChannelState.PRESENT_UNRESOLVED
    pending_grids[channel] = grid
clear_start = stations[-1] if stations else Point(0.0, 0.0)
tasks = tuple((channel, grid.stations) for channel, grid in sorted(pending_grids.items()))
for channel, point in multi_source_clear_route(clear_start, tasks):
    if states[channel] is not Q4ChannelState.PRESENT_UNRESOLVED:
        continue
    clear_response = robot.clear(point, channel)
    if _response_field(clear_response, "accepted", False) is True and _response_field(clear_response,
↪ "success", _response_field(clear_response, "clear_result", None) == "success"):
        states[channel] = Q4ChannelState.CLEARED
        evidence[channel].append("cleared_by_anchor_wedge")
complete = sum(state is Q4ChannelState.CLEARED for state in states.values()) >= 16 or all(
    state in (Q4ChannelState.CLEARED, Q4ChannelState.ABSENT_CERTIFIED) for state in states.values()
)
return Q4ExecutionResult(
    channel_states={channel: state.value for channel, state in states.items()},
    channel_evidence={channel: tuple(items) for channel, items in evidence.items()},
    complete=complete,
)

```

文件: src/b\_problem/strategy/q4\_batcr.py

```

"""Future-only BATCR / GTSP-lite routing for grouped Q4 clear representatives.

```

```

This module deliberately has no strategy entry point. It plans metadata-rich
clear tours for a future integration without changing CERT-D or CP08.
"""

```

```

from __future__ import annotations

```

```

from dataclasses import dataclass
from math import inf
from typing import Iterable

```

```

from .q4 import Point

```

```

@dataclass(frozen=True)
class ClearRepresentative:
    channel: int
    candidate_id: str
    point: Point
    reason: str
    state: str

```

```

@dataclass(frozen=True)
class ClearSourceCluster:
    channel: int
    candidates: tuple[ClearRepresentative, ...]

```

```

def __post_init__(self) -> None:
    if not 2 <= len(self.candidates) <= 5:
        raise ValueError("a clear-source cluster needs two through five representatives")
    if any(candidate.channel != self.channel for candidate in self.candidates):
        raise ValueError("every representative must preserve its source channel")
    if len({candidate.candidate_id for candidate in self.candidates}) != len(self.candidates):
        raise ValueError("representative ids must be unique within a source cluster")

@dataclass(frozen=True)
class BatcrOpenTour:
    greedy_route: tuple[ClearRepresentative, ...]
    nn_route: tuple[ClearRepresentative, ...]
    multi_start_nn_route: tuple[ClearRepresentative, ...]
    two_opt_route: tuple[ClearRepresentative, ...]
    deferred_clusters: tuple[ClearSourceCluster, ...]
    greedy_length_m: float
    nn_length_m: float
    multi_start_nn_length_m: float
    two_opt_length_m: float

def open_insertion_detour(
    previous: Point, following: Point | None, candidate: ClearRepresentative
) -> float:
    """Exact added distance for inserting a task into an open path."""
    if following is None:
        return previous.distance_to(candidate.point)
    return (
        previous.distance_to(candidate.point)
        + candidate.point.distance_to(following)
        - previous.distance_to(following)
    )

def _route_length(start: Point, route: Iterable[ClearRepresentative]) -> float:
    current = start
    total = 0.0
    for task in route:
        total += current.distance_to(task.point)
        current = task.point
    return total

def _best_insertion(
    start: Point, route: tuple[ClearRepresentative, ...], cluster: ClearSourceCluster
) -> tuple[float, int, ClearRepresentative]:
    best = (inf, 0, cluster.candidates[0])
    for index in range(len(route) + 1):
        previous = start if index == 0 else route[index - 1].point
        following = None if index == len(route) else route[index].point
        for candidate in cluster.candidates:
            proposal = (open_insertion_detour(previous, following, candidate), index, candidate)
            if (proposal[0], candidate.channel, candidate.candidate_id, candidate.point.x,
                ↪ candidate.point.y) < (
                best[0], best[2].channel, best[2].candidate_id, best[2].point.x, best[2].point.y
            ):

```

```

        best = proposal
    return best

def _nearest_neighbor(start: Point, tasks: Iterable[ClearRepresentative]) -> tuple[ClearRepresentative,
↳ ...]:
    remaining = list(tasks)
    current = start
    route: list[ClearRepresentative] = []
    while remaining:
        index, task = min(
            enumerate(remaining),
            key=lambda item: (
                current.distance_to(item[1].point), item[1].channel, item[1].candidate_id,
            ),
        )
        route.append(task)
        current = task.point
        remaining.pop(index)
    return tuple(route)

def _multi_start_nn(start: Point, tasks: tuple[ClearRepresentative, ...]) -> tuple[ClearRepresentative,
↳ ...]:
    if not tasks:
        return ()
    candidates = [_nearest_neighbor(start, tasks)]
    for first in tasks:
        candidates.append((first, *_nearest_neighbor(first.point, (task for task in tasks if task !=
↳ first))))
    return min(
        candidates,
        key=lambda route: (_route_length(start, route), tuple((task.channel, task.candidate_id) for task
↳ in route)),
    )

def _open_two_opt(start: Point, route: Iterable[ClearRepresentative]) -> tuple[ClearRepresentative, ...]:
    best = tuple(route)
    best_length = _route_length(start, best)
    improved = True
    while improved:
        improved = False
        for first in range(len(best) - 1):
            for last in range(first + 1, len(best)):
                candidate = (*best[:first], *reversed(best[first:last + 1]), *best[last + 1:])
                candidate_length = _route_length(start, candidate)
                if candidate_length + 1e-9 < best_length:
                    best, best_length, improved = candidate, candidate_length, True
                    break
            if improved:
                break
    return best

def plan_batcr_open_tour(
    start: Point, clusters: Iterable[ClearSourceCluster], *, defer_detour_m: float
) -> BatcrOpenTour:

```

```

"""Greedily fuse one representative per source, deferring excessive detours."""
if defer_detour_m < 0.0:
    raise ValueError("defer_detour_m must be non-negative")
remaining = list(clusters)
if len({cluster.channel for cluster in remaining}) != len(remaining):
    raise ValueError("source clusters must have unique channels")
greedy: list[ClearRepresentative] = []
deferred: list[ClearSourceCluster] = []
while remaining:
    proposals = [(*_best_insertion(start, tuple(greedy), cluster), cluster) for cluster in remaining]
    detour, index, representative, cluster = min(
        proposals,
        key=lambda item: (item[0], item[3].channel, item[2].candidate_id),
    )
    remaining.remove(cluster)
    if detour > defer_detour_m:
        deferred.append(cluster)
        continue
    greedy.insert(index, representative)
greedy_route = tuple(greedy)
nn_route = _nearest_neighbor(start, greedy_route)
multi_start = _multi_start_nn(start, greedy_route)
two_opt = _open_two_opt(start, multi_start)
return BatcrOpenTour(
    greedy_route=greedy_route,
    nn_route=nn_route,
    multi_start_nn_route=multi_start,
    two_opt_route=two_opt,
    deferred_clusters=tuple(deferred),
    greedy_length_m=_route_length(start, greedy_route),
    nn_length_m=_route_length(start, nn_route),
    multi_start_nn_length_m=_route_length(start, multi_start),
    two_opt_length_m=_route_length(start, two_opt),
)

```

文件: src/b\_problem/strategy/q4\_batcr\_a.py

```

"""BATCR-A: CERT-D survey safety with optional shared second-bearing shortcuts.

```

```

This independent candidate never changes CERT-D or CP08. A failed or absent
CERT-D certificate delegates immediately to exact CP08; a failed shared-B
shortcut leaves each original anchor-wedge clearance witness intact.
"""

```

```

from __future__ import annotations

```

```

from dataclasses import dataclass
from math import atan2, degrees, hypot
from typing import Any, Callable, Iterable

```

```

from b_problem.batch_geometry import shared_second_bearing

```

```

from . import q4, q4_extreme
from .q4_cert_d import CERT_D_EDGE_M, CERT_D_STATION_COUNT, CertDStationCertificate
from .q4_static_routing import FutureRouteTrace, nearest_neighbor_route, open_two_opt

```

```

@dataclass(frozen=True)

```

```

class BatcrBatchTrace:
    station: q4.Point
    point_b: q4.Point | None
    channels_served: tuple[int, ...]
    reason: str

@dataclass(frozen=True)
class BatcrAExecutionResult:
    execution: q4.Q4ExecutionResult
    route_trace: tuple[FutureRouteTrace, ...]
    batch_trace: tuple[BatcrBatchTrace, ...]
    used_batcr_a: bool

def _load_cert_d_certificate() -> CertDStationCertificate | None:
    """Copy CERT-D's exact 31-station certificate gate without changing CERT-D."""
    try:
        from b_problem.certificate_experiments.q4_station_selection import (
            radial_support_candidate,
            required_radial_support,
        )
        candidate = radial_support_candidate(CERT_D_EDGE_M)
        required = required_radial_support(CERT_D_EDGE_M)
    except (ImportError, AttributeError, ValueError):
        return None
    if (
        candidate.edge_m != CERT_D_EDGE_M
        or len(candidate.stations) != CERT_D_STATION_COUNT
        or required != candidate.stations
    ):
        return None
    return CertDStationCertificate(candidate.stations, candidate.edge_m, "PROVED")

def _runtime_certificate_is_valid(certificate: CertDStationCertificate | None) -> bool:
    return (
        certificate is not None
        and certificate.analytic_status == "PROVED"
        and certificate.edge_m == CERT_D_EDGE_M
        and len(certificate.stations) == CERT_D_STATION_COUNT
        and all(isinstance(point, q4.Point) for point in certificate.stations)
    )

def _next_station(current: q4.Point, unvisited: Iterable[q4.Point]) -> q4.Point:
    points = tuple(unvisited)
    if not points:
        raise ValueError("unvisited stations must not be empty")
    return open_two_opt(current, nearest_neighbor_route(current, points))[0]

def _route_directions(origin: q4.Point, unvisited: Iterable[q4.Point]) -> tuple[float, ...]:
    headings = {
        degrees(atan2(point.y - origin.y, point.x - origin.x)) % 360.0
        for point in unvisited
        if point.distance_to(origin) > q4.TOLERANCE
    }
    return tuple(sorted(headings))

```

```

def _batch_anchor(anchor: q4_extreme.PositiveAnchor) -> shared_second_bearing.BatchAnchor:
    return shared_second_bearing.BatchAnchor(anchor.channel, anchor.bearing_deg, anchor.station.x,
    ↪ anchor.station.y)

def _try_shared_b(
    robot: Any,
    station: q4.Point,
    batch: tuple[q4_extreme.PositiveAnchor, ...],
    route_directions_deg: tuple[float, ...],
) -> tuple[BatcrBatchTrace, frozenset[int]]:
    """Attempt exactly one geometrically shared B, retaining all failed anchors."""
    if len(batch) < 2:
        return BatcrBatchTrace(station, None, (), "insufficient_batch_fast_c"), frozenset()
    best = shared_second_bearing.best_shared_second_bearing(
        tuple(_batch_anchor(anchor) for anchor in batch), route_directions_deg=route_directions_deg,
    )
    if len(best.served_channels) < 2 or best.nonparallel_pair_count < 1:
        return BatcrBatchTrace(station, None, (), "insufficient_viable_geometry_fast_c"), frozenset()
    point_b = q4.Point(best.x, best.y)
    by_channel = {anchor.channel: anchor for anchor in batch}
    cleared: set[int] = set()
    for channel in best.served_channels:
        anchor = by_channel[channel]
        response = robot.measure(point_b, channel)
        if not q4_extreme._accepted(response): # noqa: SLF001 - shared public response adapter.
            continue
        result = q4_extreme._field(response, "result", "measure_result") # noqa: SLF001
        if result == "near":
            if q4_extreme._try_fast_clear(robot, channel, point_b, corrections=True): # noqa: SLF001 -
            ↪ bounded existing clear routine.
                cleared.add(channel)
            continue
        if result != "direction":
            continue
        second_bearing = q4_extreme._field(response, "svd_deg") # noqa: SLF001
        if second_bearing is None:
            continue
        crossing = shared_second_bearing.forward_ray_intersection(
            _batch_anchor(anchor), second_x=point_b.x, second_y=point_b.y,
            ↪ second_bearing_deg=float(second_bearing),
        )
        if crossing is None:
            continue
        estimate = q4.Point(*crossing)
        if estimate.radius() <= q4.TARGET_RADIUS_M + q4.TOLERANCE and q4_extreme._try_fast_clear( # noqa:
            ↪ SLF001
            robot, channel, estimate, corrections=True
        ):
            cleared.add(channel)
    return BatcrBatchTrace(station, point_b, best.served_channels, "shared_b_attempt"), frozenset(cleared)

def _try_single_anchor_fast_c(robot: Any, anchor: q4_extreme.PositiveAnchor) -> bool:
    """Use the existing per-source local path only when a shared B is unavailable."""
    return q4_extreme._direct_cross_range_pursuit(robot, anchor) # noqa: SLF001 - unchanged FAST-C
    ↪ fallback path.

```

```

def execute_q4_batcr_a_strategy(
    robot: Any,
    public_state: Any,
    *,
    certificate_provider: Callable[[], CertDStationCertificate | None] = _load_cert_d_certificate,
    channels: Iterable[int] = range(1, 21),
) -> BatcrAExecutionResult:
    """Run certified 31-station survey, shared B when viable, anchor wedges otherwise."""
    try:
        certificate = certificate_provider()
    except Exception:
        certificate = None
    if not _runtime_certificate_is_valid(certificate):
        return BatcrAExecutionResult(q4.execute_q4_strategy(robot, public_state), (), (), False)

    selected = tuple(channels)
    states = {channel: q4.Q4ChannelState.UNKNOWN for channel in selected}
    evidence = {channel: [] for channel in selected}
    no_signal = {channel: set() for channel in selected}
    pending_grids: dict[int, q4.AnchorWedgeClearanceGrid] = {}
    route_trace: list[FutureRouteTrace] = []
    batch_trace: list[BatcrBatchTrace] = []
    unvisited = set(certificate.stations)
    station_id = 0

    while unvisited:
        current = q4_extreme._robot_position(robot, q4.Point(0.0, 0.0)) # noqa: SLF001
        station = _next_station(current, unvisited)
        route_trace.append(FutureRouteTrace(
            from_point=current, to_point=station, reason="batcr_a_cert_d_rolling_open_2opt",
            channel=None, task_type="survey", state="station_scan",
        ))
        unvisited.remove(station)
        batch: list[q4_extreme.PositiveAnchor] = []
        for channel in selected:
            if states[channel] is not q4.Q4ChannelState.UNKNOWN:
                continue
            response = robot.measure(station, channel)
            if not q4_extreme._accepted(response): # noqa: SLF001
                continue
            result = q4_extreme._field(response, "result", "measure_result") # noqa: SLF001
            if result == "no_signal":
                no_signal[channel].add(station_id)
                if len(no_signal[channel]) == CERT_D_STATION_COUNT:
                    states[channel] = q4.Q4ChannelState.ABSENT_CERTIFIED
                    evidence[channel].append("batcr_a_absent_certified")
                continue
            if result == "near":
                if q4_extreme._clear_succeeded(robot.clear(station, channel)): # noqa: SLF001
                    states[channel] = q4.Q4ChannelState.CLEARED
                    evidence[channel].append("batcr_a_cleared_at_cert_d_station")
                else:
                    states[channel] = q4.Q4ChannelState.PRESENT_UNRESOLVED
                continue
            if result != "direction":
                continue
            bearing = q4_extreme._field(response, "svd_deg") # noqa: SLF001

```

```

        if bearing is None:
            continue
        anchor = q4_extreme.PositiveAnchor(channel, station, float(bearing))
        grid = q4.build_anchor_clearance_grid(station, bearing_deg=float(bearing))
        states[channel] = q4.Q4ChannelState.PRESENT_UNRESOLVED
        pending_grids[channel] = grid
        batch.append(anchor)
        evidence[channel].append("batcr_a_positive_anchor_retained")

    if batch:
        trace, cleared = _try_shared_b(robot, station, tuple(batch), _route_directions(station,
        ↪ unvisited))
        batch_trace.append(trace)
        if trace.reason == "shared_b_attempt":
            for channel in trace.channels_served:
                if channel in cleared:
                    states[channel] = q4.Q4ChannelState.CLEARED
                    pending_grids.pop(channel, None)
                    evidence[channel].append("batcr_a_shared_b_clear")
                else:
                    evidence[channel].append("batcr_a_shared_b_anchor_wedge_fallback")
            for anchor in batch:
                if anchor.channel not in trace.channels_served:
                    evidence[anchor.channel].append("batcr_a_not_served_anchor_wedge_fallback")
            else:
                for anchor in q4_extreme.order_positive_anchors(station, batch): # noqa: SLF001
                    if _try_single_anchor_fast_c(robot, anchor):
                        states[anchor.channel] = q4.Q4ChannelState.CLEARED
                        pending_grids.pop(anchor.channel, None)
                        evidence[anchor.channel].append("batcr_fast_c_single_anchor_clear")
                    else:
                        evidence[anchor.channel].append("batcr_fast_c_single_anchor_anchor_wedge_fallback")

    station_id += 1

tasks = tuple(
    (channel, grid.stations)
    for channel, grid in sorted(pending_grids.items())
    if states[channel] is q4.Q4ChannelState.PRESENT_UNRESOLVED
)
fallback_start = q4_extreme._robot_position(robot, q4.Point(0.0, 0.0)) # noqa: SLF001
for channel, point in q4.multi_source_clear_route(fallback_start, tasks):
    if states[channel] is not q4.Q4ChannelState.PRESENT_UNRESOLVED:
        continue
    if q4_extreme._clear_succeeded(robot.clear(point, channel)): # noqa: SLF001
        states[channel] = q4.Q4ChannelState.CLEARED
        evidence[channel].append("cleared_by_anchor_wedge")
complete = sum(state is q4.Q4ChannelState.CLEARED for state in states.values()) >= 16 or all(
    state in (q4.Q4ChannelState.CLEARED, q4.Q4ChannelState.ABSENT_CERTIFIED)
    for state in states.values()
)
execution = q4.Q4ExecutionResult(
    channel_states={channel: state.value for channel, state in states.items()},
    channel_evidence={channel: tuple(items) for channel, items in evidence.items()},
    complete=complete,
)
return BatcrAExecutionResult(execution, tuple(route_trace), tuple(batch_trace), True)

```

文件: src/b\_problem/strategy/q4\_cert\_d.py

```

"""Explicit CERT-D Q4 candidate: a certified reduced survey with CP08 fallback."""
from __future__ import annotations

from dataclasses import dataclass
from math import cos, pi, sin
from typing import Any, Callable, Iterable
from types import SimpleNamespace

from . import q4, q4_extreme
from .q4_mbfc import build_mbfc_cover
from .q4_static_routing import FutureRouteTrace, nearest_neighbor_route, open_two_opt
from .q4_telemetry import RecoveryTelemetry

CERT_D_EDGE_M = 925.0
CERT_D_STATION_COUNT = 31

@dataclass(frozen=True)
class CertDStationCertificate:
    stations: frozenset[q4.Point]
    edge_m: float
    analytic_status: str

@dataclass(frozen=True)
class CertDExecutionResult:
    execution: q4.Q4ExecutionResult
    route_trace: tuple[FutureRouteTrace, ...]
    used_cert_d: bool

class _ObservationRobot:
    """Transparent public-interface recorder used only during a fast pursuit."""
    def __init__(self, robot: Any, channel: int, observations: list[tuple[q4.Point, float]]) -> None:
        self._robot, self._channel, self._observations = robot, channel, observations
    @property
    def public_state(self):
        return self._robot.public_state
    @property
    def _telemetry(self):
        return getattr(self._robot, "_telemetry", None)
    def measure(self, position: q4.Point, channel: int):
        response = self._robot.measure(position, channel)
        if channel == self._channel and q4_extreme._field(response, "result", "measure_result") ==
        ↪ "direction":
            bearing = q4_extreme._field(response, "svd_deg")
            if bearing is not None:
                self._observations.append((q4.Point(float(position.x), float(position.y)), float(bearing)))
                if self._telemetry is not None:
                    self._telemetry.source_observations[self._channel] = self._observations
        return response
    def clear(self, position: q4.Point, channel: int):
        return self._robot.clear(position, channel)

def _load_cert_d_certificate() -> CertDStationCertificate | None:
    try:
        from b_problem.certificate_experiments.q4_station_selection import (

```

```

        radial_support_candidate,
        required_radial_support,
    )

    candidate = radial_support_candidate(CERT_D_EDGE_M)
    required = required_radial_support(CERT_D_EDGE_M)
except (ImportError, AttributeError, ValueError):
    return None
if (
    candidate.edge_m != CERT_D_EDGE_M
    or len(candidate.stations) != CERT_D_STATION_COUNT
    or required != candidate.stations
):
    return None
return CertDStationCertificate(candidate.stations, candidate.edge_m, "PROVED")

def _runtime_certificate_is_valid(certificate: CertDStationCertificate | None) -> bool:
    if certificate is None or not all(isinstance(point, q4.Point) for point in certificate.stations):
        return False
    if certificate.analytic_status == "PROVED":
        return certificate.edge_m == CERT_D_EDGE_M and len(certificate.stations) == CERT_D_STATION_COUNT
    # A separate finite oriented-pair/Voronoi proof, re-run by its constructor.
    return certificate.analytic_status == "PROVED_21POINT" and len(certificate.stations) == 21

def rolling_open_route_next(current: q4.Point, unvisited: Iterable[q4.Point]) -> q4.Point:
    """Recompute an anchored open 2-opt path and take its first station."""
    points = tuple(unvisited)
    if not points:
        raise ValueError("unvisited stations must not be empty")
    return open_two_opt(current, nearest_neighbor_route(current, points))[0]

def _shared_baseline(
    robot: Any, batch: list[q4_extreme.PositiveAnchor], states: dict[int, Any],
    positive_history: dict[int, list[tuple[q4.Point, float]]], evidence: dict[int, list[str]],
    pending_grids: dict[int, Any],
) -> tuple[int, int]:
    """One optional 50 m move to add safe positive bearings for fresh sources.

    The score is route-only heuristic. A direction appends an ordinary
    conservative bearing; generic no_signal is logged and changes no region.
    """
    fresh = [a for a in batch if states[a.channel] is q4.Q4ChannelState.PRESENT_UNRESOLVED]
    if len(fresh) < 2:
        return 0, 0
    origin = q4_extreme._robot_position(robot, fresh[0].station)
    options: list[tuple[float, int, q4.Point, list[q4_extreme.PositiveAnchor]]] = []
    for index in range(24):
        angle = 2.0 * pi * index / 24.0
        point = q4.Point(origin.x + 50.0 * cos(angle), origin.y + 50.0 * sin(angle))
        selected: list[q4_extreme.PositiveAnchor] = []
        score = 0.0
        for anchor in fresh:
            estimate = q4.Point(anchor.station.x + 1000.0 * cos(anchor.bearing_deg * pi / 180.0),
                                anchor.station.y + 1000.0 * sin(anchor.bearing_deg * pi / 180.0))
            first = q4.Point(anchor.station.x - estimate.x, anchor.station.y - estimate.y)
            second = q4.Point(point.x - estimate.x, point.y - estimate.y)

```

```

        denominator = first.radius() * second.radius()
        parallax = abs(first.x * second.y - first.y * second.x) / denominator if denominator > 1e-9
        ↪ else 0.0
        if parallax >= 0.025:
            selected.append(anchor)
            score += 1000.0 * parallax
        score -= origin.distance_to(point)
        if selected:
            options.append((score, index, point, selected))
    if not options:
        return 0, 0
    score, _index, point, selected = max(options, key=lambda item: (item[0], -item[1]))
    if score <= 0.0:
        return 0, 0
    current_channel = getattr(getattr(robot, "public_state", None), "measure_channel", None)
    selected.sort(key=lambda anchor: (anchor.channel != current_channel, anchor.channel))
    reads = 0
    for anchor in selected:
        if states[anchor.channel] is not q4.Q4ChannelState.PRESENT_UNRESOLVED:
            continue
        response = robot.measure(point, anchor.channel)
        if not q4_extreme._accepted(response):
            continue
        result = q4_extreme._field(response, "result", "measure_result")
        reads += 1
        if result == "direction":
            bearing = q4_extreme._field(response, "svd_deg")
            if bearing is not None:
                positive_history[anchor.channel].append((point, float(bearing)))
                evidence[anchor.channel].append("shared50_positive_bearing")
            elif result == "near":
                if q4_extreme._clear_succeeded(robot.clear(point, anchor.channel)):
                    states[anchor.channel] = q4.Q4ChannelState.CLEARED
                    pending_grids.pop(anchor.channel, None)
                    evidence[anchor.channel].append("shared50_near_clear")
                else:
                    evidence[anchor.channel].append("shared50_near_failed_clear_retained")
            elif result == "no_signal":
                evidence[anchor.channel].append("shared50_no_signal_record_only")
    return 1, reads

def execute_q4_cert_d_strategy(
    robot: Any,
    public_state: Any,
    *,
    certificate_provider: Callable[[], CertDStationCertificate | None] = _load_cert_d_certificate,
    channels: Iterable[int] = range(1, 21),
    local_pursuit: Callable[[Any, q4_extreme.PositiveAnchor], bool] =
    ↪ q4_extreme._direct_cross_range_pursuit,
    allow_prevalidated_certificate: bool = False,
    mbfc_enabled: bool = False,
    shared_baseline: bool = False,
    telemetry: RecoveryTelemetry | None = None,
) -> CertDExecutionResult:
    """Execute CERT-D only after all reduced-survey proof invariants are present."""
    try:
        certificate = certificate_provider()

```

```

except Exception:
    certificate = None
certificate_valid = _runtime_certificate_is_valid(certificate) or (
    allow_prevalidated_certificate
    and certificate is not None
    and certificate.analytic_status == "PROVED"
    and len(certificate.stations) >= 3
)
if not certificate_valid:
    return CertDExecutionResult(q4.execute_q4_strategy(robot, public_state), (), False)

# Optional wrapper is observational only: all public calls and arguments
# remain exactly those selected by the pre-existing control flow.
if telemetry is not None:
    robot = telemetry.wrap(robot)

selected = tuple(channels)
states = {channel: "UNKNOWN" for channel in selected}
evidence = {channel: [] for channel in selected}
no_signal = {channel: set() for channel in selected}
pending_grids: dict[int, q4.AnchorWedgeClearanceGrid] = {}
positive_history: dict[int, list[tuple[q4.Point, float]]] = {}
route_trace: list[FutureRouteTrace] = []
unvisited = set(certificate.stations)
station_id = 0

while unvisited:
    current = q4_extreme._robot_position(robot, q4.Point(0.0, 0.0)) # noqa: SLF001 - read-only
    ↪ position adapter.
    station = rolling_open_route_next(current, unvisited)
    if telemetry is not None:
        telemetry.context(global_phase="CERT_ROUTE", source_channel=None, state="SCAN")
        telemetry.snapshot(robot, kind="GLOBAL_SCHEDULER", states=states,
            ↪ observations=positive_history,
                route_remaining=len(unvisited), current_route_target=station,
                jobs=[{"job_id": f"cert_scan_{station_id}", "job_type": "CERT_SCAN",
                    ↪ "target_position": [station.x, station.y],
                        "reason": "cert_d_rolling_open_2opt", "currently_eligible": True,
                        "distance_from_robot": current.distance_to(station),
                            ↪ "estimated_move_time": current.distance_to(station) / 5,
                                "route_station_index": station_id, "correctness_dependency":
                                    ↪ "directional_discovery_certificate"}])
        route_trace.append(FutureRouteTrace(
            from_point=current, to_point=station, reason="cert_d_rolling_open_2opt",
            channel=None, task_type="survey", state="station_scan",
        ))
    unvisited.remove(station)
    batch: list[q4_extreme.PositiveAnchor] = []
    for channel in selected:
        if states[channel] != "UNKNOWN":
            continue
        response = robot.measure(station, channel)
        if not q4_extreme._accepted(response): # noqa: SLF001 - same public-response adapter as
            ↪ FAST-C.
            continue
        result = q4_extreme._field(response, "result", "measure_result") # noqa: SLF001
        if result == "no_signal":
            no_signal[channel].add(station_id)

```

```

        if len(no_signal[channel]) == len(certificate.stations):
            states[channel] = q4.Q4ChannelState.ABSENT_CERTIFIED
            evidence[channel].append("cert_d_absent_certified")
            continue
    if result == "near":
        if q4_extreme._clear_succeeded(robot.clear(station, channel)): # noqa: SLF001
            states[channel] = q4.Q4ChannelState.CLEARED
            evidence[channel].append("cleared_at_cert_d_station")
        else:
            states[channel] = q4.Q4ChannelState.PRESENT_UNRESOLVED
            continue
    if result != "direction":
        continue
    bearing = q4_extreme._field(response, "svd_deg") # noqa: SLF001
    if bearing is None:
        continue
    anchor = q4_extreme.PositiveAnchor(channel, station, float(bearing))
    positive_history[channel] = [(station, float(bearing))]
    if telemetry is not None:
        telemetry.source_observations[channel] = positive_history[channel]
    grid = q4.build_anchor_clearance_grid(station, bearing_deg=float(bearing))
    if grid is None:
        continue
    states[channel] = q4.Q4ChannelState.PRESENT_UNRESOLVED
    pending_grids[channel] = grid
    evidence[channel].append("cert_d_positive_anchor_retained")
    batch.append(anchor)
if shared_baseline:
    _shared_baseline(robot, batch, states, positive_history, evidence, pending_grids)
for anchor in q4_extreme.order_positive_anchors(station, batch):
    if states[anchor.channel] is not q4.Q4ChannelState.PRESENT_UNRESOLVED:
        continue
    observed = positive_history[anchor.channel]
    if telemetry is not None:
        telemetry.context(global_phase="SOURCE_HANDLER", source_channel=anchor.channel,
            ↳ state="SECOND_BEARING")
        telemetry.snapshot(robot, kind="SOURCE_HANDLER", states=states,
            ↳ observations=positive_history,
                route_remaining=len(unvisited), current_route_target=station,
                jobs=[{"job_id": f"source_{anchor.channel}", "job_type":
                    ↳ "SOURCE_LOCALIZATION", "source_channel": anchor.channel,
                        "target_position": [anchor.station.x, anchor.station.y],
                            ↳ "reason": "fast_c_cross_range", "currently_eligible": True,
                                "distance_from_robot": q4_extreme._robot_position(robot,
                                    ↳ station).distance_to(anchor.station),
                                        "estimated_move_time": q4_extreme._robot_position(robot,
                                            ↳ station).distance_to(anchor.station)/5,
                                                "source_state": "PRESENT_UNRESOLVED", "correctness_dependency":
                                                    ↳ "CP08_anchor_fallback_retained"}])
    if local_pursuit(_ObservationRobot(robot, anchor.channel, observed), anchor):
        states[anchor.channel] = q4.Q4ChannelState.CLEARED
        pending_grids.pop(anchor.channel, None)
        evidence[anchor.channel].append("cert_d_fast_c_clear")
    else:
        if mbfc_enabled:
            baseline = pending_grids[anchor.channel].stations
            mbfc = build_mbfc_cover(tuple(observed), baseline)
            if mbfc.enabled:

```

```

        pending_grids[anchor.channel] = SimpleNamespace(stations=mbfc.candidates) # type:
        ↪ ignore[assignment]
        evidence[anchor.channel].append("mbfc_enabled")
    else:
        evidence[anchor.channel].append(f"mbfc_revert_{mbfc.revert}")
        evidence[anchor.channel].append("cert_d_anchor_wedge_fallback")
    if telemetry is not None:
        telemetry.context(global_phase="CERT_ROUTE", source_channel=None, state="SCAN")
    station_id += 1

tasks = tuple(
    (channel, grid.stations)
    for channel, grid in sorted(pending_grids.items())
    if states[channel] is q4.Q4ChannelState.PRESENT_UNRESOLVED
)
fallback_start = q4_extreme._robot_position(robot, q4.Point(0.0, 0.0)) # noqa: SLF001
for channel, point in q4.multi_source_clear_route(fallback_start, tasks):
    if states[channel] is not q4.Q4ChannelState.PRESENT_UNRESOLVED:
        continue
    if telemetry is not None:
        telemetry.context(global_phase="FALLBACK", source_channel=channel, state="SOURCE_FALLBACK")
        telemetry.snapshot(robot, kind="CLEAR_DECISION", states=states, observations=positive_history,
        ↪ route_remaining=0,
        jobs=[{"job_id": f"fallback_{channel}", "job_type": "SOURCE_FALLBACK",
        ↪ "source_channel": channel,
        "target_position": [point.x, point.y], "reason":
        ↪ "anchor_wedge_certified_clear", "currently_eligible": True,
        "distance_from_robot": q4_extreme._robot_position(robot,
        ↪ point).distance_to(point),
        "estimated_move_time": q4_extreme._robot_position(robot,
        ↪ point).distance_to(point)/5,
        "source_state": "PRESENT_UNRESOLVED", "correctness_dependency":
        ↪ "anchor_wedge_cover"}])
    if q4_extreme._clear_succeeded(robot.clear(point, channel)): # noqa: SLF001
        states[channel] = q4.Q4ChannelState.CLEARED
        evidence[channel].append("cleared_by_anchor_wedge")
complete = sum(state is q4.Q4ChannelState.CLEARED for state in states.values()) >= 16 or all(
    state in (q4.Q4ChannelState.CLEARED, q4.Q4ChannelState.ABSENT_CERTIFIED)
    for state in states.values()
)
)
execution = q4.Q4ExecutionResult(
    channel_states={channel: state.value for channel, state in states.items()},
    channel_evidence={channel: tuple(items) for channel, items in evidence.items()},
    complete=complete,
)
return CertDExecutionResult(execution, tuple(route_trace), True)

```

文件: src/b\_problem/strategy/q4\_extreme.py

"""Survey-integrated Q4 acceleration candidates with the CP08 proof path intact.

Both candidates execute the same certified CP08 station survey. They only add bounded actions after a public ``direction`` observation; a failed shortcut is left for CP08's existing anchor-wedge clearance route.

"""

from \_\_future\_\_ import annotations

```

from dataclasses import dataclass
from math import cos, radians, sin
from typing import Any, Iterable

from b_problem.localization.q4_all_bearing import BearingObservation, all_positive_bearing_fit,
↳ point_satisfies_bearing_set

from . import q4

Point = q4.Point
LOS_PURSUIT_M = 400.0
CROSS_RANGE_M = 300.0
FAST_C_PURSUIT_M = 300.0
LOCAL_CORRECTION_M = 15.0
MAX_LOCAL_CORRECTIONS = 4

@dataclass(frozen=True)
class PositiveAnchor:
    """One CP08-certified direction observation retained for fallback."""

    channel: int
    station: Point
    bearing_deg: float

def _field(response: Any, *names: str) -> Any:
    if isinstance(response, dict):
        for name in names:
            if name in response:
                return response[name]
    for name in names:
        if hasattr(response, name):
            return getattr(response, name)
    if isinstance(response, str) and any(name in {"result", "measure_result", "clear_result"} for name in
↳ names):
        return response
    return None

def _accepted(response: Any) -> bool:
    return _field(response, "accepted") is not False

def _step(point: Point, bearing_deg: float, distance_m: float) -> Point:
    angle = radians(bearing_deg)
    return Point(point.x + distance_m * cos(angle), point.y + distance_m * sin(angle))

def _cross(first: Point, second: Point) -> float:
    return first.x * second.y - first.y * second.x

def triangulated_early_estimate(
    anchor: PositiveAnchor, second_station: Point, second_bearing_deg: float
) -> Point | None:
    """Intersect two forward public bearing rays, rejecting parallel/backward rays."""

```

```

first_direction = _step(Point(0.0, 0.0), anchor.bearing_deg, 1.0)
second_direction = _step(Point(0.0, 0.0), second_bearing_deg, 1.0)
denominator = _cross(first_direction, second_direction)
if abs(denominator) <= 1e-9:
    return None
delta = Point(second_station.x - anchor.station.x, second_station.y - anchor.station.y)
first_scale = _cross(delta, second_direction) / denominator
second_scale = _cross(delta, first_direction) / denominator
if first_scale < 0.0 or second_scale < 0.0:
    return None
estimate = Point(
    anchor.station.x + first_scale * first_direction.x,
    anchor.station.y + first_scale * first_direction.y,
)
return estimate if estimate.radius() <= q4.TARGET_RADIUS_M + 1e-6 else None

def _clear_succeeded(response: Any) -> bool:
    return _accepted(response) and _field(response, "success", "clear_result") in (True, "success")

def order_positive_anchors(start: Point, anchors: Iterable[PositiveAnchor]) -> tuple[PositiveAnchor, ...]:
    """Rolling nearest-anchor ordering used by the batched candidate.

    The ordering changes only shortcut travel; every retained anchor remains in
    the CP08 fallback set regardless of its position here.
    """
    remaining = list(anchors)
    ordered: list[PositiveAnchor] = []
    current = start
    while remaining:
        index, chosen = min(
            enumerate(remaining),
            key=lambda item: (current.distance_to(item[1].station), item[1].channel),
        )
        ordered.append(chosen)
        current = chosen.station
        remaining.pop(index)
    return tuple(ordered)

def _try_fast_clear(robot: Any, channel: int, point: Point, *, corrections: bool) -> bool:
    """Early clear, with local corrections reserved for a ``near`` witness."""
    attempts = [point]
    if corrections:
        for dx, dy in ((LOCAL_CORRECTION_M, 0.0), (-LOCAL_CORRECTION_M, 0.0),
            (0.0, LOCAL_CORRECTION_M), (0.0, -LOCAL_CORRECTION_M))[:MAX_LOCAL_CORRECTIONS]:
            attempts.append(Point(point.x + dx, point.y + dy))
    return any(_clear_succeeded(robot.clear(candidate, channel)) for candidate in attempts)

def _los_pursuit(robot: Any, anchor: PositiveAnchor) -> bool:
    """FAST-A's deliberately weaker co-linear control, retained as a comparator."""
    probe = _step(anchor.station, anchor.bearing_deg, LOS_PURSUIT_M)
    response = robot.measure(probe, anchor.channel)
    if not _accepted(response):
        return False
    result = _field(response, "result", "measure_result")
    if result == "near":

```

```

        return _try_fast_clear(robot, anchor.channel, probe, corrections=True)
    if result != "direction":
        return False
    bearing = _field(response, "svd_deg")
    if bearing is None:
        return False
    return _try_fast_clear(
        robot, anchor.channel, _step(probe, float(bearing), LOS_PURSUIT_M), corrections=False
    )

def _cross_range_pursuit(robot: Any, anchor: PositiveAnchor, *, speculative: bool) -> bool:
    """FAST-B/C: short pursuit, one lateral bearing, and bounded clear candidates."""
    pursuit = _step(anchor.station, anchor.bearing_deg, LOS_PURSUIT_M)
    pursuit_response = robot.measure(pursuit, anchor.channel)
    if not _accepted(pursuit_response):
        return False
    pursuit_result = _field(pursuit_response, "result", "measure_result")
    if pursuit_result == "no_signal":
        # One inexpensive return-to-certified-anchor recovery, then preserve
        # the existing wedge proof instead of widening an unobserved search.
        robot.measure(anchor.station, anchor.channel)
        return False
    if pursuit_result == "near":
        return _try_fast_clear(robot, anchor.channel, pursuit, corrections=True)
    if pursuit_result != "direction":
        return False
    pursuit_bearing = _field(pursuit_response, "svd_deg")
    if pursuit_bearing is None:
        return False
    if speculative and _try_fast_clear(robot, anchor.channel, pursuit, corrections=False):
        return True
    # Direction alone carries no range. A lateral second station produces the
    # non-parallel ray needed for an independently derived point estimate.
    for sign in (1.0, -1.0):
        lateral = _step(pursuit, float(pursuit_bearing) + sign * 90.0, CROSS_RANGE_M)
        lateral_response = robot.measure(lateral, anchor.channel)
        if not _accepted(lateral_response):
            continue
        lateral_result = _field(lateral_response, "result", "measure_result")
        if lateral_result == "no_signal":
            continue
        if lateral_result == "near" and _try_fast_clear(robot, anchor.channel, lateral, corrections=True):
            return True
        if lateral_result != "direction":
            continue
        lateral_bearing = _field(lateral_response, "svd_deg")
        if lateral_bearing is None:
            continue
        estimate = triangulated_early_estimate(anchor, lateral, float(lateral_bearing))
        if estimate is not None and _try_fast_clear(robot, anchor.channel, estimate, corrections=True):
            return True
    return False

def _direct_cross_range_pursuit(robot: Any, anchor: PositiveAnchor) -> bool:
    """FAST-C: omit the co-linear remeasure and acquire the lateral ray directly."""
    pursuit = _step(anchor.station, anchor.bearing_deg, FAST_C_PURSUIT_M)
    for sign in (1.0, -1.0):

```

```

lateral = _step(pursuit, anchor.bearing_deg + sign * 90.0, CROSS_RANGE_M)
telemetry = getattr(robot, "_telemetry", None)
if telemetry is not None:
    telemetry.context(global_phase="SOURCE_HANDLER", source_channel=anchor.channel,
                      state="SECOND_BEARING" if sign > 0 else "LOCALIZATION_RECOVERY",
                      trigger_function="_direct_cross_range_pursuit",
                      trigger_reason="scheduled_lateral_bearing",
                      trigger_values={"lateral_sign": sign, "cross_range_m": CROSS_RANGE_M})
    telemetry.snapshot(robot, kind="RECOVERY_DECISION" if sign < 0 else "SOURCE_HANDLER",
                      current_route_target=lateral,
                      jobs=[{"job_id": f"lateral_{anchor.channel}_{sign}", "job_type":
↪ "SOURCE_LOCALIZATION",
↪ "source_channel": anchor.channel, "target_position": [lateral.x,
↪ lateral.y],
↪ "reason": "fast_c_lateral_bearing", "currently_eligible": True,
↪ "distance_from_robot": _robot_position(robot,
↪ lateral).distance_to(lateral),
↪ "estimated_move_time": _robot_position(robot,
↪ lateral).distance_to(lateral)/5,
↪ "source_state": "PRESENT_UNRESOLVED", "correctness_dependency":
↪ "CP08_anchor_fallback_retained"}])
response = robot.measure(lateral, anchor.channel)
if not _accepted(response):
    continue
result = _field(response, "result", "measure_result")
if result == "no_signal":
    continue
if result == "near" and _try_fast_clear(robot, anchor.channel, lateral, corrections=True):
    return True
if result != "direction":
    continue
bearing = _field(response, "svd_deg")
if bearing is None:
    continue
estimate = triangulated_early_estimate(anchor, lateral, float(bearing))
if estimate is not None:
    if telemetry is not None:
        telemetry.context(global_phase="SOURCE_HANDLER", source_channel=anchor.channel,
↪ state="CLEAR_APPROACH",
↪ trigger_function="triangulated_early_estimate",
↪ trigger_reason="two_positive_bearings",
↪ trigger_values={"lateral_sign": sign})
        telemetry.snapshot(robot, kind="CLEAR_DECISION", current_route_target=estimate,
↪ jobs=[{"job_id": f"clear_{anchor.channel}", "job_type": "SOURCE_CLEAR",
↪ "source_channel": anchor.channel,
↪ "target_position": [estimate.x, estimate.y], "reason":
↪ "triangulated_early_estimate",
↪ "currently_eligible": True, "distance_from_robot":
↪ _robot_position(robot, estimate).distance_to(estimate),
↪ "estimated_move_time": _robot_position(robot,
↪ estimate).distance_to(estimate)/5,
↪ "source_state": "PRESENT_UNRESOLVED", "correctness_dependency":
↪ "CP08_anchor_fallback_retained"}])
    if estimate is not None and _try_fast_clear(robot, anchor.channel, estimate, corrections=True):
        return True
return False

```

```

def _direct_cross_range_pursuit_all_bearing(robot: Any, anchor: PositiveAnchor) -> bool:
    """SMAR-0: reuse both scheduled lateral measurements before one heuristic clear.

    There is no new observation waypoint here. The conservative anchor wedge
    remains untouched and is used by CERT-D if the numeric shortcut fails.
    """
    pursuit = _step(anchor.station, anchor.bearing_deg, FAST_C_PURSUIT_M)
    observations = [BearingObservation(anchor.station, anchor.bearing_deg)]
    for sign in (1.0, -1.0):
        lateral = _step(pursuit, anchor.bearing_deg + sign * 90.0, CROSS_RANGE_M)
        response = robot.measure(lateral, anchor.channel)
        if not _accepted(response):
            continue
        result = _field(response, "result", "measure_result")
        if result == "near" and _try_fast_clear(robot, anchor.channel, lateral, corrections=True):
            return True
        if result != "direction":
            continue
        bearing = _field(response, "svd_deg")
        if bearing is not None:
            observations.append(BearingObservation(lateral, float(bearing)))
    fit = all_positive_bearing_fit(observations)
    if fit.degenerate or fit.estimate is None or fit.estimate.radius() > q4.TARGET_RADIUS_M + 1e-6:
        return False
    # This is a public-wedge/range gate, not a probabilistic confidence claim.
    # It prevents a numerical fit outside the broadest possible receive domain;
    # exact 1000 m versus 1500 m radius remains unknown until a clear response.
    if not point_satisfies_bearing_set(fit.estimate, observations, max_range_m=1500.0):
        return False
    return _try_fast_clear(robot, anchor.channel, fit.estimate, corrections=True)

def _robot_position(robot: Any, fallback: Point) -> Point:
    state = getattr(robot, "public_state", None)
    if callable(state):
        state = state()
    position = getattr(state, "position", None)
    if position is None:
        return fallback
    return Point(float(position.x), float(position.y))

def execute_q4_survey_speed_strategy(
    robot: Any,
    public_state: Any,
    *,
    candidate: str = "fast_b",
    channels: Iterable[int] = range(1, 21),
    max_shortcuts: int | None = None,
) -> q4.Q4ExecutionResult:
    """Run FAST-A/FAST-B inside CP08's certified survey and clearance route."""
    if candidate not in {"fast_a", "fast_b", "fast_c"}:
        raise ValueError("candidate must be fast_a, fast_b, or fast_c")
    if max_shortcuts is not None and max_shortcuts < 0:
        raise ValueError("max_shortcuts must be non-negative or None")
    selected = tuple(channels)
    plan = q4.build_survey_plan()
    states = {channel: "UNKNOWN" for channel in selected}
    evidence = {channel: [] for channel in selected}

```

```

no_signal = {channel: set() for channel in selected}
pending_grids: dict[int, q4.AnchorWedgeClearanceGrid] = {}
shortcut_attempts = 0

for station_id, station in enumerate(plan.route):
    batch: list[PositiveAnchor] = []
    for channel in selected:
        if states[channel] != "UNKNOWN":
            continue
        response = robot.measure(station, channel)
        if not _accepted(response):
            continue
        result = _field(response, "result", "measure_result")
        if result == "no_signal":
            no_signal[channel].add(station_id)
            if len(no_signal[channel]) == len(plan.route):
                states[channel] = q4.Q4ChannelState.ABSENT_CERTIFIED
                evidence[channel].append("absent_certified")
            continue
        if result == "near":
            if _clear_succeeded(robot.clear(station, channel)):
                states[channel] = q4.Q4ChannelState.CLEARED
                evidence[channel].append("cleared_at_survey_station")
            else:
                states[channel] = q4.Q4ChannelState.PRESENT_UNRESOLVED
            continue
        if result != "direction":
            continue
        bearing = _field(response, "svd_deg")
        if bearing is None:
            continue
        anchor = PositiveAnchor(channel, station, float(bearing))
        grid = q4.build_anchor_clearance_grid(station, bearing_deg=float(bearing))
        if grid is None:
            continue
        states[channel] = q4.Q4ChannelState.PRESENT_UNRESOLVED
        pending_grids[channel] = grid
        evidence[channel].append("positive_anchor_retained")
        if candidate == "fast_a":
            if max_shortcuts is not None and shortcut_attempts >= max_shortcuts:
                evidence[channel].append("fast_a_time_debt_fallback")
            elif _los_pursuit(robot, anchor):
                shortcut_attempts += 1
                states[channel] = q4.Q4ChannelState.CLEARED
                pending_grids.pop(channel, None)
                evidence[channel].append("fast_a_los_clear")
            else:
                shortcut_attempts += 1
                evidence[channel].append("fast_a_anchor_wedge_fallback")
        else:
            batch.append(anchor)
    if candidate in {"fast_b", "fast_c"}:
        for anchor in order_positive_anchors(station, batch):
            if states[anchor.channel] is not q4.Q4ChannelState.PRESENT_UNRESOLVED:
                continue
            if max_shortcuts is not None and shortcut_attempts >= max_shortcuts:
                evidence[anchor.channel].append(f"{candidate}_time_debt_fallback")
            elif (
                _direct_cross_range_pursuit(robot, anchor)

```

```

        if candidate == "fast_c"
        else _cross_range_pursuit(robot, anchor, speculative=False)
    ):
        shortcut_attempts += 1
        states[anchor.channel] = q4.Q4ChannelState.CLEARED
        pending_grids.pop(anchor.channel, None)
        evidence[anchor.channel].append(f"{candidate}_cross_range_clear")
    else:
        shortcut_attempts += 1
        evidence[anchor.channel].append(f"{candidate}_anchor_wedge_fallback")

tasks = [
    (channel, grid.stations)
    for channel, grid in sorted(pending_grids.items())
    if states[channel] is q4.Q4ChannelState.PRESENT_UNRESOLVED
]
for channel, point in q4.multi_source_clear_route(_robot_position(robot, plan.route[-1]),
↳ tuple(tasks)):
    if states[channel] is not q4.Q4ChannelState.PRESENT_UNRESOLVED:
        continue
    if _clear_succeeded(robot.clear(point, channel)):
        states[channel] = q4.Q4ChannelState.CLEARED
        evidence[channel].append("cleared_by_anchor_wedge")
complete = sum(state is q4.Q4ChannelState.CLEARED for state in states.values()) >= 16 or all(
    state in (q4.Q4ChannelState.CLEARED, q4.Q4ChannelState.ABSENT_CERTIFIED)
    for state in states.values()
)
return q4.Q4ExecutionResult(
    channel_states={channel: state.value for channel, state in states.items()},
    channel_evidence={channel: tuple(items) for channel, items in evidence.items()},
    complete=complete,
)

```

文件: src/b\_problem/strategy/q4\_mbfc.py

```

"""Conservative, static-entry multi-bearing fallback cover (MBFC-A)."""
from __future__ import annotations
from dataclasses import dataclass
from math import ceil, cos, floor, pi, radians, sin, sqrt
from shapely.geometry import Point as SPoint, Polygon
from .q4 import CLEAR_RADIUS_M, Point, TARGET_RADIUS_M

RANGE_M=1500.0; ERR_DEG=1.0; EPS=0.01; SPACING=CLEAR_RADIUS_M*sqrt(3)-0.01; COVER=SPACING/sqrt(3)
@dataclass(frozen=True)
class MBFCResult:
    candidates: tuple[Point,...]; first_area_m2: float; multi_area_m2: float; enabled: bool; revert:
↳ str|None
def _circle(c:Point,r:float,n:int=720):
    R=(r+EPS)/cos(pi/n);return Polygon([(c.x+R*cos(2*pi*i/n),c.y+R*sin(2*pi*i/n)) for i in range(n)])
def _wedge(c:Point,b:float):
    a=radians(b-ERR_DEG-EPS);z=radians(b+ERR_DEG+EPS);L=5000.;return
↳ Polygon([(c.x,c.y),(c.x+L*cos(a),c.y+L*sin(a)),(c.x+L*cos(z),c.y+L*sin(z))])
def _region(observations:tuple[tuple[Point,float],...]):
    g=_circle(Point(0.,0.),TARGET_RADIUS_M)
    for p,b in observations:g=g.intersection(_wedge(p,b)).intersection(_circle(p,RANGE_M))
    return g
def _cover(g):

```

```

if g.is_empty: return ()
x0,y0,x1,y1=g.bounds;dy=sqrt(3)*SPACING/2;out=[]
for row in range(floor((y0-COVER)/dy),ceil((y1+COVER)/dy)+1):
    y=row*dy;off=(row&1)*SPACING/2
    for col in range(floor((x0-COVER-off)/SPACING),ceil((x1+COVER-off)/SPACING)+1):
        x=col*SPACING+off
        if g.distance(SPoint(x,y))<=COVER+EPS:out.append(Point(x,y))
return tuple(out)
def build_mbfc_cover(observations:tuple[tuple[Point,float],...], baseline:tuple[Point,...])>->MBFCResult:
    """Fail closed: outer geometry or coverage anomaly returns baseline unchanged."""
    try:
        if len(observations)<2:return MBFCResult(baseline,0.,0.,False,'no_benefit')
        first=_region(observations[:1]);multi=_region(observations)
        if multi.is_empty or not multi.is_valid:return MBFCResult(baseline,first.area,0.,False,'empty_region')
        candidates=_cover(multi)
        if not candidates:return MBFCResult(baseline,first.area,multi.area,False,'coverage')
        # The full lattice covering radius is COVER<20; retaining every center
        # within COVER of the outer region is the analytic coverage certificate.
        if len(candidates)>=len(baseline):return MBFCResult(baseline,first.area,multi.area,False,'no_benefit')
        return MBFCResult(candidates,first.area,multi.area,True,None)
    except Exception:return MBFCResult(baseline,0.,0.,False,'geometry')

```

文件: src/b\_problem/strategy/q4\_mincert.py

```

"""Runtime adapter for a prevalidated MINCERT station set only."""
from __future__ import annotations

from math import cos, radians, sin
from typing import Any, Iterable

from b_problem.certificate_experiments.q4_mincert import validate_directional_certificate
from b_problem.certificate_experiments.q4_21_certificate import candidate_21, certify_21

from . import q4
from .q4_cert_d import CertDExecutionResult, CertDStationCertificate, execute_q4_cert_d_strategy
from .q4_telemetry import RecoveryTelemetry

def mincert_certificate(stations: Iterable[q4.Point]) -> CertDStationCertificate:
    """Fail closed unless the offline conservative certificate passes."""
    frozen = frozenset(stations)
    audit = validate_directional_certificate(frozen)
    if audit.status != "PASS":
        raise ValueError(f"MINCERT validator rejected station set: {audit.detail}")
    return CertDStationCertificate(frozen, audit.max_active_triangle_edge_m or 0.0, "PROVED")

def certificate_21point() -> CertDStationCertificate:
    """Fail closed unless the finite 21-point certificate passes in-process."""
    stations = frozenset(candidate_21())
    audit = certify_21(tuple(stations))
    if not audit.certified or audit.oriented_pair_checks != 408:
        raise ValueError("21-point directional discovery certificate did not pass")
    return CertDStationCertificate(stations, audit.worst_receive_radius_m, "PROVED_21POINT")

def rotated_21point_certificate(angle_deg: float) -> CertDStationCertificate:

```

```

"""Re-run the finite certificate for a rigidly rotated 21-point layout.

Rotation preserves the circular arena and all distance/closed-half-plane
predicates, but we still execute the independent finite validator instead
of relying on that observation at runtime.
"""
theta=radians(angle_deg); c,s=cos(theta),sin(theta)
stations=frozenset(q4.Point(c*p.x-s*p.y,s*p.x+c*p.y) for p in candidate_21())
audit=certify_21(tuple(stations))
if not audit.certified or audit.oriented_pair_checks != 408:
    raise ValueError("rotated 21-point certificate did not pass")
return CertDStationCertificate(stations,audit.worst_receive_radius_m,"PROVED_21POINT_ROTATED")

def execute_q4_mincert_strategy(
    robot: Any, public_state: Any, certificate: CertDStationCertificate, *, channels: Iterable[int] =
    ↪ range(1, 21)
) -> CertDExecutionResult:
    """CERT-D execution policy with only its static station set replaced."""
    return execute_q4_cert_d_strategy(
        robot, public_state, certificate_provider=lambda: certificate, channels=channels,
        ↪ allow_prevalidated_certificate=True
    )

def execute_q4_mincert_mbfc_strategy(
    robot: Any, public_state: Any, certificate: CertDStationCertificate, *, channels: Iterable[int] =
    ↪ range(1, 21),
    telemetry: RecoveryTelemetry | None = None, shared_baseline: bool = False,
) -> CertDExecutionResult:
    """Experimental MBFC-A adapter; the frozen champion entry point stays unchanged."""
    return execute_q4_cert_d_strategy(
        robot, public_state, certificate_provider=lambda: certificate, channels=channels,
        allow_prevalidated_certificate=True, mbfc_enabled=True, telemetry=telemetry,
        ↪ shared_baseline=shared_baseline,
    )

def execute_q4_full_port_strategy(robot: Any, public_state: Any, certificate: CertDStationCertificate, *,
    routingplus: bool = True, shared_baseline: bool = True, telemetry=None,
    region_clear: bool = False, region_probe: bool = False, region_fallback:
    ↪ bool = False, alternating_region: bool = False,
    paired_negative: bool = False, adaptive_certificate_rotation: bool =
    ↪ False, early16: bool = True,
    receding_horizon: bool = False, rh_k: int = 8, rh_cheap_twoopt: bool =
    ↪ False, rh_rollout: bool = False):
    """Feature-flagged atomic mixed runtime; the frozen champion is unchanged."""
    from .q4_mixed_atomic import execute_q4_mixed_atomic
    return execute_q4_mixed_atomic(robot, public_state, certificate, routingplus=routingplus,
        shared_baseline=shared_baseline, telemetry=telemetry,
        region_clear=region_clear, region_probe=region_probe,
        ↪ region_fallback=region_fallback,
        ↪ alternating_region=alternating_region,
        paired_negative=paired_negative,
        ↪ adaptive_certificate_rotation=adaptive_certificate_rotation,
        ↪ early16=early16,
        receding_horizon=receding_horizon, rh_k=rh_k, rh_cheap_twoopt=rh_cheap_t
        ↪ woapt, rh_rollout=rh_rollout)

```

文件: src/b\_problem/strategy/q4\_mixed\_atomic.py

```
"""Atomic mixed SCAN/SOURCE runtime for the certified Q4 experiment line."""
from __future__ import annotations
from dataclasses import dataclass, field
from typing import Any, Iterable
from types import SimpleNamespace
from . import q4, q4_extreme
from .q4_cert_d import CertDExecutionResult, CertDStationCertificate, _runtime_certificate_is_valid,
↳ _shared_baseline
from .q4_mbfc import build_mbfc_cover
from .q4_routingplus import LiveJob, RoutingPlus, RegionNN
from .q4_region_speed import clear_service_candidates, localization_candidates, positive_outer_vertices
from .q4_static_routing import FutureRouteTrace
from b_problem.localization.q4_pair_certificate import PairBinding
from .q34_receding_horizon import RecedingHorizonPlanner

@dataclass
class SourceSession:
    anchor: q4_extreme.PositiveAnchor
    observations: list[tuple[q4.Point, float]]
    fallback: tuple[q4.Point, ...]
    phase: str = "LOCALIZE"
    lateral_sign: int = 1
    target: q4.Point | None = None
    fallback_index: int = 0
    fallback_attempted: set[q4.Point] = field(default_factory=set)
    snapshot_version: int = 0
    pair_binding: PairBinding | None = None
    failed_clear_exclusions: set[q4.Point] = field(default_factory=set)

@dataclass
class MixedTelemetry:
    decisions: list[dict] = field(default_factory=list)
    selected: dict[str, int] = field(default_factory=dict)
    planner: dict = field(default_factory=dict)
    region_history: list[dict] = field(default_factory=list)

def _lateral(session: SourceSession) -> q4.Point:
    pursuit = q4_extreme._step(session.anchor.station, session.anchor.bearing_deg, q4_extreme.FAST_C_PURSUIT_M)
    return q4_extreme._step(pursuit, session.anchor.bearing_deg + 90.0 * session.lateral_sign, q4_extreme.CROSS |
↳ _RANGE_M)

def _source_job(session: SourceSession, ch: int, *, current: q4.Point | None = None, next_scan: q4.Point | None = None, re
↳ gion_clear: bool = False, region_probe: bool = False, region_fallback: bool = False) -> LiveJob | None:
    if session.phase == "LOCALIZE":
        candidates = localization_candidates(session.anchor, route_goal=next_scan) if region_probe else ()
        return LiveJob(("source", ch, "localize", session.lateral_sign), "SOURCE_LOCALIZATION_STEP", _lateral(
↳ session), ch, candidates)
    if session.phase == "CLEAR" and session.target is not None:
        candidates = clear_service_candidates(positive_outer_vertices(session.observations), current=current
↳ , next_goal=next_scan) if region_clear and current is not None else ()
        return LiveJob(("source", ch, "clear"), "SOURCE_CLEAR_STEP", session.target, ch, candidates)
    if session.phase == "FALLBACK":
        remaining = tuple(point for point in session.fallback if point not in session.fallback_attempted)
        if not remaining: return None
        # Failed clears form an auxiliary non-convex ledger. It ranks cover
        # witnesses away from already disproved 20 m disks but never cuts the
```

```

    # convex hard source representation or certifies a result.
    ordered=tuple(sorted(remaining,key=lambda point:(-min((point.distance_to(old) for old in
    ↪ session.failed_clear_exclusions),default=0.0),current.distance_to(point),point.x,point.y))) if
    ↪ region_fallback and current is not None else remaining
    return LiveJob(("source",ch,"fallback",len(session.fallback_attempted)), "SOURCE_FALLBACK_STEP",or
    ↪ dered[0],ch,ordered[:4] if region_fallback else ())
return None

def _prepare_fallback(session:SourceSession)->None:
    mbfc=build_mbfc_cover(tuple(session.observations),session.fallback)
    if mbfc.enabled:session.fallback=mbfc.candidates
    session.phase="FALLBACK"

def execute_q4_mixed_atomic(
    robot:Any, public_state:Any, certificate:CertDStationCertificate, *,
    ↪ channels:Iterable[int]=range(1,21),
    routingplus:bool=True, shared_baseline:bool=True, telemetry:MixedTelemetry|None=None,
    region_clear:bool=False, region_probe:bool=False, region_fallback:bool=False,
    ↪ alternating_region:bool=False,
    paired_negative:bool=False, adaptive_certificate_rotation:bool=False, early16:bool=True,
    receding_horizon:bool=False, rh_k:int=8, rh_cheap_twoopt:bool=False, rh_rollout:bool=False,
)->CertDExecutionResult:
    """Execute one live job at a time; source jobs issue exactly one public action."""
    if not _runtime_certificate_is_valid(certificate):
        return CertDExecutionResult(q4.execute_q4_strategy(robot,public_state),(),False)
    # The directional-shadow position cut remains disabled until independently
    # derived for this API. ``paired_negative`` therefore enables only the
    # identity-bound evidence ledger, never a Q4 source-position contraction.
    if adaptive_certificate_rotation:
        from .q4_mincert import rotated_21point_certificate
        current=q4_extreme._robot_position(robot,q4.Point(0.,0.))
        candidates=tuple(rotated_21point_certificate(angle) for angle in range(0,30,3))
        certificate=min(candidates,key=lambda item:min(current.distance_to(point) for point in
        ↪ item.stations))
    selected=tuple(channels); states={ch:"UNKNOWN" for ch in selected}; evidence={ch:[] for ch in selected}
    no_signal={ch:set() for ch in selected}; sessions=dict[int,SourceSession]={};
    ↪ positive=dict[int,list[tuple[q4.Point,float]]]={}
    unvisited=set(certificate.stations); planner=RoutingPlus();
    ↪ region_planner=RegionNN(alternating=alternating_region,rounds=2)
    rh=RecedingHorizonPlanner(k=rh_k,cheap_twoopt=rh_cheap_twoopt,rollout=rh_rollout);
    ↪ trace:list[FutureRouteTrace]=[]; tel=telemetry or MixedTelemetry(); scan_index=0
    previous_route:tuple[LiveJob,...]=()
    while unvisited or any(states[ch] is q4.Q4ChannelState.PRESENT_UNRESOLVED for ch in selected):
        if early16 and sum(s is q4.Q4ChannelState.CLEARED for s in states.values())>=16:
            break
        current=q4_extreme._robot_position(robot,q4.Point(0.,0.)); jobs=[]
        jobs.extend(LiveJob(("scan",p.x,p.y),"CERT_SCAN",p) for p in unvisited)
        next_scan=min(unvisited,key=lambda point:(current.distance_to(point),point.x,point.y)) if
        ↪ unvisited else None
        jobs.extend(job for ch,s in sorted(sessions.items()) if states[ch] is
        ↪ q4.Q4ChannelState.PRESENT_UNRESOLVED for job in
        ↪ (_source_job(s,ch,current=current,next_scan=next_scan,region_clear=region_clear,region_probe=
        ↪ region_probe,region_fallback=region_fallback),) if job)
        if not jobs:break
        nscan=sum(j.kind=="CERT_SCAN" for j in jobs); nsource=len(jobs)-nscan
        nn_choice=min(jobs,key=lambda job:(current.distance_to(job.target),job.key))
        routing_choice=planner.choose(current,jobs) if (routingplus and receding_horizon) else None
        if receding_horizon:chosen=rh.choose(current,jobs,routing_choice=routing_choice)

```

```

elif region_clear or region_probe or region_fallback or
↳ alternating_region:chosen=region_planner.choose(current,jobs)
elif routingplus:chosen=planner.choose(current,jobs)
else:chosen=min(jobs,key=lambda j:(current.distance_to(j.target),j.key))
assert chosen is not None
live_by_key={job.key:job for job in jobs}
previous_suffix=previous_route[1:]
surviving=[job for job in previous_suffix if job.key in live_by_key]
source_displacements=[]
for old in previous_suffix:
    if old.channel is None: continue
    latest=next((job for job in jobs if job.channel==old.channel and job.kind==old.kind),None)
    if latest is not None: source_displacements.append(old.target.distance_to(latest.target))
tel.decisions.append({"n_scan_jobs":nscan,"n_source_jobs":nsource,"pool_size":len(jobs),"mixed_po_
↳ ol":nscan>0 and nsource>0,
                        "multi_source_pool":nsource>2,"selected_job_type":chosen.kind,"selected_ch_
↳ annel":chosen.channel,
                        "distance_to_selected_job":current.distance_to(chosen.target),"nn_job_key":_
↳ repr(nn_choice.key),
                        "routing_job_key":repr(routing_choice.key) if routing_choice else
↳ None,"selected_job_key":repr(chosen.key),
                        "changed_from_nn":chosen.key!=nn_choice.key,
                        "suffix_survival_rate":len(surviving)/max(1,len(previous_suffix)),
                        "mean_source_anchor_displacement_m":sum(source_displacements)/max(1,len(sou_
↳ rce_displacements)))})
tel.selected[chosen.kind]=tel.selected.get(chosen.kind,0)+1
trace.append(FutureRouteTrace(current,chosen.target,"mixed_atomic_routingplus" if routingplus else
↳ "mixed_atomic_nn",chosen.channel,
                        "survey" if chosen.kind=="CERT_SCAN" else "source",chosen.kind))
if chosen.kind=="CERT_SCAN":
    station=chosen.target;unvisited.remove(station);batch=[]
    for ch in selected:
        if states[ch]!="UNKNOWN":continue
        response=robot.measure(station,ch)
        if not q4_extreme._accepted(response):continue
        result=q4_extreme._field(response,"result","measure_result")
        if result=="no_signal":
            no_signal[ch].add(scan_index)
            if len(no_signal[ch])==len(certificate.stations):states[ch]=q4.Q4ChannelState.ABSENT_
↳ CERTIFIED;evidence[ch].append("21point_absent_certified")
        elif result=="near":
            if q4_extreme._clear_succeeded(robot.clear(station,ch)):states[ch]=q4.Q4ChannelState._
↳ CLEARED;evidence[ch].append("scan_near_clear")
            else:states[ch]=q4.Q4ChannelState.PRESENT_UNRESOLVED
        elif result=="direction":
            bearing=q4_extreme._field(response,"svd_deg")
            if bearing is None:continue
            anchor=q4_extreme.PositiveAnchor(ch,station,float(bearing));grid=q4.build_anchor_clea_
↳ rance_grid(station,bearing_deg=float(bearing))
            if grid is None:continue
            states[ch]=q4.Q4ChannelState.PRESENT_UNRESOLVED;positive[ch]=[(station,float(bearing))]
            session=SourceSession(anchor,positive[ch],grid.stations)
            if paired_negative:
                # No measurement is scheduled until the unresolved
                # directional cut has a production proof. Keeping the
                # ledger object preserves generation/snapshot binding
                # and makes accidental generic no_signal use impossible.
                session.pair_binding=PairBinding(0,ch,0,(station,station),(station,))

```

```

        evidence[ch].append("pair_ledger_created_cut_disabled")
        sessions[ch]=session;batch.append(anchor);evidence[ch].append("anchor_retained")
    if shared_baseline:
        before=q4_extreme._robot_position(robot,station)
        moves,_reads=_shared_baseline(robot,batch,states,positive,evidence,{ch:SimpleNamespace(st
        ↪ ations=s.fallback) for ch,s in sessions.items()})
        after=q4_extreme._robot_position(robot,before)
        if moves:trace.append(FutureRouteTrace(before,after,"shared50",None,"source","SHARED_BASE
        ↪ LINE"))
        scan_index+=1
    else:
        ch=chosen.channel;assert ch is not None;session=sessions[ch]
        if chosen.kind=="SOURCE_LOCALIZATION_STEP":
            response=robot.measure(chosen.target,ch);result=q4_extreme._field(response,"result","meas
            ↪ ure_result")
            if result=="direction":
                bearing=q4_extreme._field(response,"svd_deg")
                if bearing is not None:
                    session.observations.append((chosen.target,float(bearing)));session.snapshot_vers
                    ↪ ion+=1
                    tel.region_history.append({"channel":ch,"observations":tuple(session.observations
                    ↪ ),"snapshot_version":session.snapshot_version})
                    estimate=q4_extreme.triangulated_early_estimate(session.anchor,chosen.target,floa
                    ↪ t(bearing))
                    if estimate is not None:session.target=estimate;session.phase="CLEAR"
                    else:_prepare_fallback(session)
                elif result=="near":session.target=chosen.target;session.phase="CLEAR"
                elif session.lateral_sign>0:session.lateral_sign=-1
                else:_prepare_fallback(session)
            else:
                response=robot.clear(chosen.target,ch)
                if q4_extreme._clear_succeeded(response):states[ch]=q4.Q4ChannelState.CLEARED;evidence[ch]
                ↪ ].append("atomic_clear_success")
                else:
                    evidence[ch].append("atomic_clear_failed_retained")
                    session.failed_clear_exclusions.add(chosen.target)
                    if chosen.kind=="SOURCE_CLEAR_STEP":_prepare_fallback(session)
                    else:session.fallback_attempted.add(chosen.target);session.fallback_index+=1
        # This is telemetry only: a deterministic residual route gives a
        # comparable "plan survival" denominator at the next replan.
        previous_route=tuple(sorted(jobs,key=lambda job:(job is not
        ↪ chosen,current.distance_to(job.target),job.key)))
        complete=sum(s is q4.Q4ChannelState.CLEARED for s in states.values())>=16 or all(s in
        ↪ (q4.Q4ChannelState.CLEARED,q4.Q4ChannelState.ABSENT_CERTIFIED) for s in states.values())
        mixed=sum(bool(row["mixed_pool"]) for row in tel.decisions); multi=sum(bool(row["multi_source_pool"])
        ↪ for row in tel.decisions)
        base={"mixed_pool_rate":mixed/max(1,len(tel.decisions)),"multi_source_pool_rate":multi/max(1,len(tel.
        ↪ decisions)),
            "selected_scan_count":tel.selected.get("CERT_SCAN",0),"selected_source_count":sum(value for
            ↪ key,value in tel.selected.items() if key!="CERT_SCAN"),
            "paired_negative":"BOUND_LEDGER_ONLY_DIRECTIONAL_CUT_DISABLED" if paired_negative else
            ↪ "DISABLED","adaptive_rotation":adaptive_certificate_rotation,
            "early16":early16}
        base={"mean_suffix_survival_rate":sum(row["suffix_survival_rate"] for row in
        ↪ tel.decisions)/max(1,len(tel.decisions)),
            "mean_source_anchor_displacement_m":sum(row["mean_source_anchor_displacement_m"] for row in
            ↪ tel.decisions)/max(1,len(tel.decisions)),

```

```

        "actual_changed_from_nn_rate":sum(bool(row["changed_from_nn"]) for row in
        ↪ tel.decisions)/max(1,len(tel.decisions))}
planner_values={"calls":region_planner.calls,"twoopt_gain_m":0.,"oropt_gain_m":0.,"warm_uses":0,"serv
↪ ice_point_gain_m":region_planner.service_gain_m} if (region_clear or region_probe or
↪ region_fallback or alternating_region) and not receding_horizon else
↪ {"calls":planner.calls,"twoopt_gain_m":planner.twoopt_gain,"oropt_gain_m":planner.oropt_gain,"warm
↪ _uses":planner.warm_uses,"service_point_gain_m":0.}
if receding_horizon:planner_values|=rh.telemetry()
tel.planner=planner_values|base
execution=q4.Q4ExecutionResult({ch:getattr(s,"value",s) for ch,s in states.items()}},{ch:tuple(v) for
↪ ch,v in evidence.items()}),complete)
return CertDExecutionResult(execution,tuple(trace),True)

```

## 文件: src/b\_problem/strategy/q4\_region\_speed.py

```

"""Finite service-point candidates for experimental Q4 speed routing."""
from __future__ import annotations

from math import cos, hypot, radians, sin, sqrt
from typing import Iterable

from shapely.geometry import LineString, Point as SPoint
from shapely.ops import nearest_points

from b_problem.geometry import Point as GeometryPoint, minimum_enclosing_circle

from . import q4
from .q4_extreme import PositiveAnchor
from .q4_mbfc import _region

CLEAR_SERVICE_MARGIN_M = 2.0
CLEAR_SERVICE_RADIUS_M = q4.CLEAR_RADIUS_M - CLEAR_SERVICE_MARGIN_M

def positive_outer_vertices(observations: Iterable[tuple[q4.Point, float]]) -> tuple[q4.Point, ...]:
    try:
        geometry = _region(tuple(observations))
        if geometry.is_empty or geometry.geom_type != "Polygon":
            return ()
        return tuple(q4.Point(float(x), float(y)) for x, y in tuple(geometry.exterior.coords)[-1])
    except Exception:
        return ()

def is_clear_service_point(point: q4.Point, vertices: Iterable[q4.Point]) -> bool:
    values = tuple(vertices)
    return bool(values) and all(point.distance_to(vertex) <= CLEAR_SERVICE_RADIUS_M + 1e-7 for vertex in
    ↪ values)

def _intersections(a: q4.Point, b: q4.Point) -> tuple[q4.Point, ...]:
    distance = a.distance_to(b)
    radius = CLEAR_SERVICE_RADIUS_M
    if distance <= 1e-8 or distance > 2.0 * radius + 1e-8:
        return ()
    mid = q4.Point((a.x + b.x) / 2.0, (a.y + b.y) / 2.0)

```

```

height = sqrt(max(0.0, radius * radius - distance * distance / 4.0))
nx, ny = -(b.y - a.y) / distance, (b.x - a.x) / distance
return (q4.Point(mid.x + height * nx, mid.y + height * ny), q4.Point(mid.x - height * nx, mid.y -
↪ height * ny))

def clear_service_candidates(
    vertices: Iterable[q4.Point], *, current: q4.Point, next_goal: q4.Point | None = None
) -> tuple[q4.Point, ...]:
    values = tuple(vertices)
    if not values or max((a.distance_to(b) for i, a in enumerate(values) for b in values[i + 1 :]),
↪ default=0.0) > 2.0 * CLEAR_SERVICE_RADIUS_M + 1e-7:
        return ()
    try:
        service = SPoint(values[0].x, values[0].y).buffer(CLEAR_SERVICE_RADIUS_M, quad_segs=48)
        for vertex in values[1:]:
            service = service.intersection(SPoint(vertex.x, vertex.y).buffer(CLEAR_SERVICE_RADIUS_M,
↪ quad_segs=48))
            if service.is_empty:
                return ()
        raw = []
        nearest = nearest_points(SPoint(current.x, current.y), service)[1]
        raw.append(q4.Point(float(nearest.x), float(nearest.y)))
        if next_goal is not None:
            route = LineString(((current.x, current.y), (next_goal.x, next_goal.y)))
            point = nearest_points(route, service)[1]
            raw.append(q4.Point(float(point.x), float(point.y)))
        mec = minimum_enclosing_circle(tuple(GeometryPoint(vertex.x, vertex.y) for vertex in values))
        raw.append(q4.Point(mec.center.x, mec.center.y))
        for vertex in values:
            dx, dy = current.x - vertex.x, current.y - vertex.y
            length = hypot(dx, dy)
            if length > 1e-8:
                raw.append(q4.Point(vertex.x + CLEAR_SERVICE_RADIUS_M * dx / length, vertex.y +
↪ CLEAR_SERVICE_RADIUS_M * dy / length))
        for i, a in enumerate(values):
            for b in values[i + 1 :]:
                raw.extend(_intersections(a, b))
        result = []
        for point in raw:
            if is_clear_service_point(point, values) and not any(point.distance_to(old) < 1e-6 for old in
↪ result):
                result.append(point)
        return tuple(sorted(result, key=lambda point: (current.distance_to(point), point.x, point.y)))
    except Exception:
        return ()

def localization_candidates(anchor: PositiveAnchor, *, route_goal: q4.Point | None = None) ->
↪ tuple[q4.Point, ...]:
    """Small public-data-only probe family for one atomic source job."""
    values: list[q4.Point] = []
    for forward in (150.0, 300.0, 450.0):
        theta = radians(anchor.bearing_deg)
        pursuit = q4.Point(anchor.station.x + forward * cos(theta), anchor.station.y + forward *
↪ sin(theta))
        for sign in (1.0, -1.0):
            lateral = radians(anchor.bearing_deg + sign * 90.0)

```

```

        values.append(q4.Point(pursuit.x + 300.0 * cos(lateral), pursuit.y + 300.0 * sin(lateral)))
    if route_goal is not None:
        theta = radians(anchor.bearing_deg)
        projection = (route_goal.x - anchor.station.x) * cos(theta) + (route_goal.y - anchor.station.y) *
        ↪ sin(theta)
        forward = min(450.0, max(0.0, projection))
        for sign in (1.0, -1.0):
            lateral = radians(anchor.bearing_deg + sign * 90.0)
            values.append(q4.Point(anchor.station.x + forward * cos(theta) + 300.0 * cos(lateral),
                                   anchor.station.y + forward * sin(theta) + 300.0 * sin(lateral)))
    unique: list[q4.Point] = []
    for point in values:
        if not any(point.distance_to(old) < 1e-7 for old in unique):
            unique.append(point)
    return tuple(unique)

```

文件: src/b\_problem/strategy/q4\_routingplus.py

```

"""Deterministic RoutingPlus planner for live Q4 jobs.

Pure routing state only: it cannot alter feasible regions or channel states.
"""

from __future__ import annotations
from dataclasses import dataclass, replace
from math import atan2
from typing import Iterable
from .q4 import Point

@dataclass(frozen=True)
class LiveJob:
    key: tuple
    kind: str
    target: Point
    channel: int | None = None
    candidates: tuple[Point, ...] = ()

def _dist(a: Point, b: Point) -> float: return a.distance_to(b)
def _length(start: Point, jobs: tuple[LiveJob, ...], route: list[int]) -> float:
    total=0.; cur=start
    for i in route: total+=_dist(cur, jobs[i].target); cur=jobs[i].target
    return total

class RoutingPlus:
    def __init__(self) -> None:
        self.previous: tuple[tuple, ...] = ()
        self.calls=0; self.twoopt_gain=0.; self.oroft_gain=0.; self.warm_uses=0
    def _nn(self, start: Point, jobs: tuple[LiveJob, ...]) -> list[int]:
        left=set(range(len(jobs))); out=[]; cur=start
        while left:
            i=min(left, key=lambda
            ↪ k: (_dist(cur, jobs[k].target), jobs[k].key)); out.append(i); left.remove(i); cur=jobs[i].target
        return out
    def _insert(self, start: Point, jobs: tuple[LiveJob, ...]) -> list[int]:
        left=set(range(len(jobs))); route=[max(left, key=lambda
        ↪ i: (_dist(start, jobs[i].target), jobs[i].key)); left.remove(route[0])
        while left:
            cand=[]

```

```

        for i in left:
            for p in range(len(route)+1):
                a=start if p==0 else jobs[route[p-1]].target;b=None if p==len(route) else
                ↪ jobs[route[p]].target
                cand.append((_dist(a,jobs[i].target)+(0 if b is None else
                ↪ _dist(jobs[i].target,b)-_dist(a,b)),i,p))
            _,i,p=min(cand);route.insert(p,i);left.remove(i)
    return route
def _twoopt(self,start:Point,jobs:tuple[LiveJob,...],route:list[int])>->list[int]:
    route=list(route); improved=True
    while improved:
        improved=False;base=_length(start,jobs,route);best=route
        for i in range(len(route)-1):
            for j in range(i+1,len(route)):
                trial=route[:i]+route[i:j+1][::-1]+route[j+1:]
                if _length(start,jobs,trial)<_length(start,jobs,best)-1e-9:best=trial
            if _length(start,jobs,best)<base-1e-9:route=best;improved=True
    return route
def _oropt(self,start:Point,jobs:tuple[LiveJob,...],route:list[int])>->list[int]:
    route=list(route)
    for _ in range(12):
        baseline=_length(start,jobs,route); best=baseline; winner=None; n=len(route)
        for size in range(1,min(3,n-1)+1):
            for first in range(n-size+1):
                chain=route[first:first+size]; rest=route[:first]+route[first+size:]
                for insert in range(len(rest)+1):
                    variants=(chain,chain[::-1]) if size>1 else (chain,)
                    for variant in variants:
                        trial=rest[:insert]+variant+rest[insert:]
                        length=_length(start,jobs,trial)
                        if length<best-1e-9:best=length,trial=variant
                if winner is None:break
        route=self._twoopt(start,jobs,winner)
    return route
def choose(self,start:Point,jobs:Iterable[LiveJob])>->LiveJob|None:
    jobs=tuple(jobs)
    if not jobs:return None
    self.calls+=1
    angular=sorted(range(len(jobs)),key=lambda
    ↪ i:(atan2(jobs[i].target.y,jobs[i].target.x),jobs[i].key))
    if angular:
        cut=min(range(len(angular)),key=lambda
        ↪ i:_dist(start,jobs[angular[i]].target));angular=angular[cut:]+angular[:cut]
    seeds=[self._nn(start,jobs),self._insert(start,jobs),angular,[angular[0]]+angular[:0:-1] if
    ↪ len(angular)>1 else angular]
    keys={job.key:i for i,job in enumerate(jobs)}
    warm=[keys[k] for k in self.previous if k in keys]
    if warm:
        warm += [i for i in range(len(jobs)) if i not in warm];seeds.append(warm);self.warm_uses+=1
    initial=min(_length(start,jobs,s) for s in seeds)
    route=min((self._twoopt(start,jobs,s) for s in seeds),key=lambda r:_length(start,jobs,r))
    after_two=_length(start,jobs,route);self.twoopt_gain+=max(0.,initial-after_two)
    route=self._oropt(start,jobs,route);self.oropt_gain+=max(0.,after_two-_length(start,jobs,route))
    self.previous=tuple(jobs[i].key for i in route);return jobs[route[0]]

```

```

class RegionNN:
    """Select one live job and one legal finite service point."""

```

```

def __init__(self, *, alternating: bool = False, rounds: int = 3) -> None:
    self.alternating=alternating;self.rounds=rounds;self.calls=0;self.service_gain_m=0.0
    @staticmethod
    def _points(job: LiveJob) -> tuple[Point, ...]: return job.candidates or (job.target,)
    def choose(self, start: Point, jobs: Iterable[LiveJob]) -> LiveJob | None:
        values=tuple(jobs)
        if not values:return None
        self.calls+=1
        if not self.alternating:
            job,point=min(((job,point) for job in values for point in self._points(job)),key=lambda
                ↪ item:(_dist(start,item[1]),item[0].key,item[1].x,item[1].y))
            return replace(job,target=point)
        working=tuple(replace(job,target=min(self._points(job),key=lambda
            ↪ point:(_dist(start,point),point.x,point.y))) for job in values)
        helper=RoutingPlus();route=helper._nn(start,working)
        for _ in range(self.rounds):
            before=_length(start,working,route);mutable=list(working)
            for pos,index in enumerate(route):
                prev=start if pos==0 else mutable[route[pos-1]].target
                nxt=None if pos+1==len(route) else mutable[route[pos+1]].target
                target=min(self._points(values[index]),key=lambda point:(_dist(prev,point)+(0. if nxt is
                    ↪ None else _dist(point,nxt)),point.x,point.y))
                mutable[index]=replace(values[index],target=target)
            working=tuple(mutable);self.service_gain_m+=max(0.,before-_length(start,working,route));route =
            ↪ =helper._twoopt(start,working,route);route=helper._oropt(start,working,route)
        return working[route[0]]

```

文件: src/b\_problem/strategy/q4\_smar.py

```

"""Isolated SMAR candidates; CERT-D remains unchanged by default."""
from __future__ import annotations

from typing import Any, Iterable

from .q4_cert_d import CertDExecutionResult, execute_q4_cert_d_strategy
from .q4_extreme import _direct_cross_range_pursuit_all_bearing

def execute_q4_smar_0_strategy(robot: Any, public_state: Any, *, channels: Iterable[int] = range(1, 21))
    ↪ -> CertDExecutionResult:
    """Run the zero-new-observation all-positive-bearing candidate offline."""
    return execute_q4_cert_d_strategy(
        robot, public_state, channels=channels, local_pursuit=_direct_cross_range_pursuit_all_bearing
    )

```

文件: src/b\_problem/strategy/q4\_static\_routing.py

```

"""Future-only static Q4 route certificates and task-routing scaffolding.

Nothing in this module is called by the Q4 execution strategy. It analyses
the already certified station set and provides auditable building blocks for a
future route integration.
"""
from __future__ import annotations

```

```

from dataclasses import dataclass
from math import inf
from typing import Iterable

from .q4 import Point

def open_route_length(start: Point, route: Iterable[Point]) -> float:
    current = start
    total = 0.0
    for point in route:
        total += current.distance_to(point)
        current = point
    return total

def nearest_neighbor_route(start: Point, stations: Iterable[Point]) -> tuple[Point, ...]:
    remaining = list(stations)
    current = start
    route: list[Point] = []
    while remaining:
        index, point = min(
            enumerate(remaining), key=lambda item: (current.distance_to(item[1]), item[1].x, item[1].y)
        )
        route.append(point)
        current = point
        remaining.pop(index)
    return tuple(route)

def multi_start_nearest_neighbor_route(start: Point, stations: Iterable[Point]) -> tuple[Point, ...]:
    points = tuple(stations)
    if not points:
        return ()
    candidates = [nearest_neighbor_route(start, points)]
    for first in points:
        remaining = tuple(point for point in points if point != first)
        candidates.append((first, *nearest_neighbor_route(first, remaining)))
    return min(candidates, key=lambda route: (open_route_length(start, route), route))

def open_two_opt(start: Point, route: Iterable[Point]) -> tuple[Point, ...]:
    """Improve an anchored open path; deliberately never adds a return edge."""
    best = tuple(route)
    best_length = open_route_length(start, best)
    improved = True
    while improved:
        improved = False
        for first in range(len(best) - 1):
            for last in range(first + 1, len(best)):
                candidate = (*best[:first], *reversed(best[first:last + 1]), *best[last + 1:])
                candidate_length = open_route_length(start, candidate)
                if candidate_length + 1e-9 < best_length:
                    best, best_length, improved = candidate, candidate_length, True
                    break
            if improved:
                break
    return best

```

```

def mst_lower_bound(start: Point, stations: Iterable[Point]) -> float:
    """Prim MST weight on the anchored station graph: a lower bound for any open visit path."""
    nodes = (start, *stations)
    if len(nodes) < 2:
        return 0.0
    connected = {0}
    best = [inf] * len(nodes)
    for index in range(1, len(nodes)):
        best[index] = nodes[0].distance_to(nodes[index])
    total = 0.0
    while len(connected) < len(nodes):
        index = min((item for item in range(len(nodes)) if item not in connected), key=best.__getitem__)
        total += best[index]
        connected.add(index)
        for other in range(len(nodes)):
            if other not in connected:
                best[other] = min(best[other], nodes[index].distance_to(nodes[other]))
    return total

@dataclass(frozen=True)
class CertAOpenRouteCertificate:
    snake_route: tuple[Point, ...]
    nn_route: tuple[Point, ...]
    multi_start_nn_route: tuple[Point, ...]
    two_opt_route: tuple[Point, ...]
    snake_length_m: float
    nn_length_m: float
    multi_start_nn_length_m: float
    two_opt_length_m: float
    mst_lower_bound_m: float
    snake_to_mst_ratio: float
    two_opt_to_mst_ratio: float

def cert_a_open_route_certificate(
    start: Point, stations: Iterable[Point], snake_route: Iterable[Point]
) -> CertAOpenRouteCertificate:
    """Compare snake/NN/multi-start/2-opt over exactly the supplied station set."""
    points = tuple(stations)
    snake = tuple(snake_route)
    if set(snake) != set(points) or len(snake) != len(points):
        raise ValueError("snake_route must visit every certified station exactly once")
    nn = nearest_neighbor_route(start, points)
    multi_start = multi_start_nearest_neighbor_route(start, points)
    refined = open_two_opt(start, multi_start)
    lower_bound = mst_lower_bound(start, points)
    ratio = lambda length: length / lower_bound if lower_bound else 1.0
    return CertAOpenRouteCertificate(
        snake_route=snake,
        nn_route=nn,
        multi_start_nn_route=multi_start,
        two_opt_route=refined,
        snake_length_m=open_route_length(start, snake),
        nn_length_m=open_route_length(start, nn),
        multi_start_nn_length_m=open_route_length(start, multi_start),
        two_opt_length_m=open_route_length(start, refined),
        mst_lower_bound_m=lower_bound,
    )

```

```

        snake_to_mst_ratio=ratio(open_route_length(start, snake)),
        two_opt_to_mst_ratio=ratio(open_route_length(start, refined)),
    )

@dataclass(frozen=True)
class StaticRouteTask:
    task_type: str
    point: Point
    reason: str
    channel: int | None
    state: str

def open_task_route(start: Point, tasks: Iterable[StaticRouteTask]) -> tuple[StaticRouteTask, ...]:
    """Future-only nearest-neighbor ordering that preserves task identity."""
    remaining = list(tasks)
    current = start
    route: list[StaticRouteTask] = []
    while remaining:
        index, task = min(
            enumerate(remaining),
            key=lambda item: (
                current.distance_to(item[1].point), item[1].task_type,
                item[1].channel if item[1].channel is not None else -1,
            ),
        )
        route.append(task)
        current = task.point
        remaining.pop(index)
    return tuple(route)

@dataclass(frozen=True)
class FutureRouteTrace:
    from_point: Point
    to_point: Point
    reason: str
    channel: int | None
    task_type: str
    state: str

    @property
    def distance_m(self) -> float:
        return self.from_point.distance_to(self.to_point)

    def as_dict(self) -> dict[str, object]:
        return {
            "from": {"x": self.from_point.x, "y": self.from_point.y},
            "to": {"x": self.to_point.x, "y": self.to_point.y},
            "distance": self.distance_m,
            "reason": self.reason,
            "channel": self.channel,
            "task_type": self.task_type,
            "state": self.state,
        }

```

文件: src/b\_problem/strategy/q4\_telemetry.py

```

"""Optional, passive telemetry for frozen Q4 strategy replay.

The wrapper delegates every public action unchanged. It is intentionally not
imported by normal runtime entries, so telemetry cannot participate in a
decision.
"""
from __future__ import annotations

from dataclasses import asdict
from math import hypot
from typing import Any

from shapely import get_coordinates, minimum_bounding_circle
from shapely.geometry import Point as ShapelyPoint

from . import q4, q4_extreme
from .q4_mbfc import _region

class RecoveryTelemetry:
    def __init__(self, seed: int) -> None:
        self.seed = seed
        self.snapshots: list[dict[str, Any]] = []
        self.actions: list[dict[str, Any]] = []
        self._context: dict[str, Any] = {"global_phase": "INIT", "source_channel": None, "state": None}
        self.source_observations: dict[int, list[tuple[q4.Point, float]]] = {}
        self._index = 0

    def wrap(self, robot: Any) -> "TelemetryRobot":
        return TelemetryRobot(robot, self)

    def context(self, **values: Any) -> None:
        self._context.update(values)

    def feasible(self, observations: list[tuple[q4.Point, float]] | None) -> dict[str, Any] | None:
        if not observations:
            return None
        region = _region(tuple(observations))
        if region.is_empty:
            return {"representation": "MBFC_A_OUTER_SHAPELY_POLYGON", "empty": True}
        coords = get_coordinates(region)
        # Shapely's enclosing circle is an enclosing disk of the runtime's
        # actual outer polygon. The additive tolerance is conservative.
        circle = minimum_bounding_circle(region)
        cx, cy = circle.centroid.x, circle.centroid.y
        max_vertex = max((hypot(x - cx, y - cy) for x, y in coords), default=0.0)
        radius = max_vertex + 1e-8
        x0, y0, x1, y1 = region.bounds
        return {"representation": "MBFC_A_OUTER_SHAPELY_POLYGON", "vertex_count": len(coords),
                "vertices": [[float(x), float(y)] for x, y in coords], "area_m2": region.area,
                "diameter_upper_m": hypot(x1 - x0, y1 - y0), "centroid": [region.centroid.x,
                ↪ region.centroid.y],
                "bounding_box": [x0, y0, x1, y1], "mec_algorithm": "shapely.minimum_bounding_circle",
                "mec_tolerance_m": 1e-8, "mec_center": [cx, cy], "mec_radius": radius,
                "certified_mec_radius_upper_bound": radius,
                "mec_encloses_every_serialized_vertex": True, "maximum_vertex_to_center_distance":
                ↪ max_vertex}

```

```

def snapshot(self, robot: Any, *, kind: str, states: dict[int, Any] | None = None,
             observations: dict[int, list[tuple[q4.Point, float]]] | None = None,
             route_remaining: int | None = None, current_route_target: q4.Point | None = None,
             jobs: list[dict[str, Any]] | None = None, **extra: Any) -> None:
    state = robot.public_state
    previous = self.actions[-1] if self.actions else None
    resolved = states or {}
    source = self._context.get("source_channel")
    observation_map = observations if observations is not None else self.source_observations
    obs = observation_map.get(source, []) if source is not None else []
    self.snapshots.append({"seed": self.seed, "decision_index": self._index, "decision_kind": kind,
                          "virtual_time_s": state.virtual_time_s, "robot_position":
                          ↪ [state.position.x, state.position.y],
                          "current_measure_channel": state.measure_channel, "global_phase":
                          ↪ self._context.get("global_phase"),
                          "previous_action_type": None if previous is None else
                          ↪ previous["action_type"],
                          "previous_action_position": None if previous is None else
                          ↪ previous["to_position"],
                          "previous_action_channel": None if previous is None else
                          ↪ previous["channel"],
                          "source_currently_handled": source, "source_state":
                          ↪ self._context.get("state"),
                          "unresolved_channels": [c for c, v in resolved.items() if getattr(v,
                          ↪ "value", v) == "PRESENT_UNRESOLVED"],
                          "discovered_channels": sorted(observation_map.keys()),
                          "cleared_channels": [c for c, v in resolved.items() if getattr(v, "value",
                          ↪ v) == "CLEARED"],
                          "certificate_stations_remaining": route_remaining,
                          ↪ "current_certificate_route_index": None,
                          "fallback_active": self._context.get("global_phase") == "FALLBACK",
                          ↪ "final_certificate_active": False,
                          "current_route_target": None if current_route_target is None else
                          ↪ [current_route_target.x, current_route_target.y],
                          "pending_jobs": jobs or [], "serializable_multi_job_point":
                          ↪ sum(x.get("currently_eligible", False) for x in (jobs or [])) >= 2,
                          "source_snapshot": None if source is None else {"channel": source,
                          ↪ "positive_bearing_count": len(obs),
                          "positive_bearing_observations": [[p.x, p.y], bearing] for p, bearing
                          ↪ in obs],
                          "feasible_region": self.feasible(obs)}, **extra})

    self._index += 1

```

```

class TelemetryRobot:

```

```

    def __init__(self, robot: Any, telemetry: RecoveryTelemetry) -> None:
        self._robot, self._telemetry = robot, telemetry

```

```

    @property

```

```

    def public_state(self):
        return self._robot.public_state

```

```

    def _record(self, kind: str, position: q4.Point, channel: int, call):

```

```

        before = self.public_state
        response = call()
        after = self.public_state
        result = q4_extreme._field(response, "result", "measure_result", "clear_result", "success")

```

```

self._telemetry.actions.append({"seed": self._telemetry.seed, "action_index":
↪ len(self._telemetry.actions),
    "decision_context": dict(self._telemetry._context), "action_type": kind,
    "from_position": [before.position.x, before.position.y], "to_position": [position.x,
↪ position.y],
    "move_distance_m": hypot(position.x - before.position.x, position.y - before.position.y),
    "move_time_s": hypot(position.x - before.position.x, position.y - before.position.y) / 5.0,
    "channel": channel, "result": result, "virtual_time_before_s": before.virtual_time_s,
    "virtual_time_after_s": after.virtual_time_s})
return response

def measure(self, position: q4.Point, channel: int):
    return self._record("measure", position, channel, lambda: self._robot.measure(position, channel))

def clear(self, position: q4.Point, channel: int):
    return self._record("clear", position, channel, lambda: self._robot.clear(position, channel))

```

## 文件: vendor/portable\_sandbox/ablation\_preview.py

```

import sys,json,time,math,argparse
sys.path.insert(0,'/mnt/data/extbench/lib')
import numpy as np
from sim import generate_case,Simulator
from strategy2 import Params2,RobotV2
from geom import norm,diameter_rotating_calipers_pair,polygon_area,polygon_centroid,mec,unit

STRICT3=[[0.0,0.0],[1080.8618318011463,0.0],[673.9063293463646,845.0518083002004],[-240.51438429012381,10
↪ 53.762369034913],[-973.8228609568137,468.96837305133465],[-973.8228609568138,-468.9683730513344],[-24
↪ 0.51438429012407,-1053.762369034913],[673.9063293463643,-845.0518083002006]]
STRICT4=[[0.0,0.0],[-500.0,-866.0254037844386],[-1000.0,-1732.0508075688772],[-1500.0,-866.0254037844386]
↪ ,[-2000.0,0.0],[-1500.0,866.0254037844386],[-1000.0,0.0],[-500.0,866.0254037844386],[-1000.0,1732.050
↪ 8075688772],[0.0,1732.0508075688772],[500.0,866.0254037844386],[1000.0,0.0],[1500.0,-866.0254037844386]
↪ ],[0.0,-1732.0508075688772],[1000.0,-1732.0508075688772],[1500.0,-866.0254037844386],[2000.0,0.0],[15
↪ 00.0,866.0254037844386],[1000.0,1732.0508075688772]]

class PreviewTargetRHO(RobotV2):
    def _preview_next_measure_point(self,ch,poly):
        # pure version of parent _next_measure_point: no self.side mutation
        if self.problem==4 and ch in self.ray:
            apex,th=self.ray[ch];u=unit(th);n=np.array([-u[1],u[0]])
            tmax=max(float((np.asarray(p,float)-apex)@u) for p in
↪ poly);tmax=max(tmax,50.0);t_now=float((self.sim.pos-apex)@u)
            t=min(max(t_now+200.0,200.0),tmax);sgn=self.side.get(ch,1.0);lat=25.0*sgn if
↪ self.nosig.get(ch,0)==0 else 12.0*sgn
            return apex+t*u+lat*n
        D,(a,b)=diameter_rotating_calipers_pair(poly);area=polygon_area(poly);width=area/D if D>1e-9 else
↪ 0.0
        if width<=45.0 and D>80.0:
            u=(b-a)/D;s=float(np.clip((self.sim.pos-a)@u,0.0,D));t=s+250.0
            if t>D-40.0:t=max(D*0.5,s-250.0) if s>D*0.5 else D
            lat=0.0 if self.params.lateral_frac<=0 else
↪ 12.0;sgn=self.side.get(ch,1.0);n=np.array([-u[1],u[0]])
            return a+float(np.clip(t,0.0,D))*u+sgn*lat*n
        c=polygon_centroid(poly);v=c-self.sim.pos;L=norm(v);u=unit(self.sim.channel*37.0) if L<1e-6 else
↪ v/L;up=np.array([-u[1],u[0]]);sgn=self.side.get(ch,1.0)
        if L>700:f,lat=0.55*L,self.params.lateral_frac*L
        elif L>150:f,lat=0.45*L,0.5*L

```

```

        else:f,lat=0.15*L,max(60.0,0.9*L)
        return self.sim.pos+f*u+sgn*lat*up
def _jobs(self):
    jobs=[]
    for ch in self.tracked:
        if ch in self.cleared:continue
        poly=self.tracked[ch];D,_=diameter_rotating_calipers_pair(poly);c,r=mec(poly)
        if r<=19.5:
            p=np.asarray(c,float)
        elif self.params.use_sweep and D<=self.params.sweep_max_diam:
            # use nearest polygon vertex/centroid proxy for first sweep leg; avoids mutating state
            P=np.asarray(poly,float); p=P[int(np.linalg.norm(P-self.sim.pos,axis=1).argmin())]
        else:
            p=np.asarray(self._preview_next_measure_point(ch,poly),float)
        jobs.append(('clear',ch,p,self._dist(p)+0.15*D+30.0))
    if self._need_scan():
        for P in self.pending:jobs.append(('scan',None,np.asarray(P,float),self._dist(P)+6.0*len(self)
            ↪ .unknown)))
    return jobs

def run(problem,n,seed0):
    plan={'level1':STRICT3 if problem==3 else STRICT4,'escalation':[]}; rows=[];t0=time.time()
    for i in range(n):
        srcs=generate_case(np.random.default_rng(seed0+i),problem);sim=Simulator(srcs);rb=PreviewTargetRH
        ↪ 0(sim,plan,Params2(name='preview',theta_miss=0.0,use_belief=False),problem);res=rb.run();rows
        ↪ .append(res)
    st=[r['stats'] for r in rows]
    out={'problem':problem,'n':n,'full':sum(s['ratio']>=1-1e-9 for s in st),'agg':sum(s['total_time_s']
        ↪ for s in st)/sum(s['n_sources'] for s in st),'meanavg':float(np.mean([s['avg_time_s'] for s in
        ↪ st])), 'move':float(np.mean([s['move_m'] for s in st])), 'req':float(np.mean([s['n_requests'] for s
        ↪ in st])), 'miss':sum(s['n_sources']-s['n_cleared'] for s in st),'gaveup':sum(len(r['gave_up']) for r
        ↪ in rows),'wall':time.time()-t0}
    print(json.dumps(out,indent=2))
if __name__=='__main__':
    ap=argparse.ArgumentParser();ap.add_argument('--problem',type=int,required=True);ap.add_argument('--n
        ↪ ',type=int,default=80);ap.add_argument('--seed0',type=int);a=ap.parse_args();run(a.problem,a.n,a.
        ↪ seed0 or (90000 if a.problem==3 else 700000))

```

文件: vendor/portable\_sandbox/ablation\_structural.py

```

import sys, time, json, argparse
sys.path.insert(0,'/mnt/data/extbench/lib')
import numpy as np
from sim import generate_case, Simulator
from strategy2 import Params2, RobotV2
from strategy import norm

STRICT3=[
[0.0,0.0],[1080.8618318011463,0.0],[673.9063293463646,845.0518083002004],[-240.51438429012381,1053.762369
↪ 034913],[-973.8228609568137,468.96837305133465],[-973.8228609568138,-468.9683730513344],[-240.5143842
↪ 9012407,-1053.762369034913],[673.9063293463643,-845.0518083002006],]
STRICT4=[[0.0,0.0],[-500.0,-866.0254037844386],[-1000.0,-1732.0508075688772],[-1500.0,-866.0254037844386]
↪ ,[-2000.0,0.0],[-1500.0,866.0254037844386],[-1000.0,0.0],[-500.0,866.0254037844386],[-1000.0,1732.050
↪ 8075688772],[0.0,1732.0508075688772],[500.0,866.0254037844386],[1000.0,0.0],[500.0,-866.0254037844386]
↪ ],[0.0,-1732.0508075688772],[1000.0,-1732.0508075688772],[1500.0,-866.0254037844386],[2000.0,0.0],[15
↪ 00.0,866.0254037844386],[1000.0,1732.0508075688772]]

```

```

class NearestOnly(RobotV2):
    def _plan_next(self,jobs):
        # Only immediate Euclidean distance, still common scan+source job pool.
        d=[norm(j[2]-self.sim.pos) for j in jobs]
        md=min(d)
        cand=[i for i,x in enumerate(d) if abs(x-md)<1e-9]
        cand.sort(key=lambda i: 0 if jobs[i][0]=='clear' else 1)
        return jobs[cand[0]]

class ScanFirst(RobotV2):
    def _jobs(self):
        jobs=super()._jobs()
        scans=[j for j in jobs if j[0]=='scan']
        return scans if scans else [j for j in jobs if j[0]=='clear']

class SourceFirst(RobotV2):
    def _jobs(self):
        jobs=super()._jobs()
        clears=[j for j in jobs if j[0]=='clear']
        return clears if clears else [j for j in jobs if j[0]=='scan']

class NoPiggyback(RobotV2):
    def _scan_unknown(self, at=None):
        # Same as parent Robot._scan_unknown except do NOT remeasure already-tracked channels at scan
        ↪ points.
        chans=sorted(self.unknown)
        for k,ch in enumerate(chans):
            p=at if (k==0 and at is not None) else self.sim.pos
            res,svd=self.sim.measure(p,ch); self.n_measure+=1
            if res=='direction':
                if ch in self.tracked: self._update_region(ch,self.sim.pos,svd)
                else: self._init_region(ch,svd)
                self.ray[ch]=(self.sim.pos.copy(),svd); self.nosig[ch]=0
            elif res=='no_signal' and ch in self.tracked:
                self._record_nosig(ch,self.sim.pos)
                if self.problem==4:
                    np2=self._shadow_filter(ch,self.tracked[ch],prob4=True)
                    if np2 is not None:self.tracked[ch]=np2
            elif res=='near':
                r,_=self.sim.clear(self.sim.pos,ch); self.n_clear+=1
                if r=='success':self.cleared.add(ch);self.unknown.discard(ch)

VARIANTS={
    'baseline':RobotV2,
    'nearest':NearestOnly,
    'scanfirst':ScanFirst,
    'sourcefirst':SourceFirst,
    'nopiggy':NoPiggyback,
}

def run_variant(problem, variant, n, seed0, use_belief=False, use_sweep=True):
    cls=VARIANTS[variant]
    plan={'level1':STRICT3 if problem==3 else STRICT4,'escalation':[]}
    rows=[]; t0=time.time()
    for i in range(n):
        srcs=generate_case(np.random.default_rng(seed0+i),problem)
        sim=Simulator(srcs)
        p=Params2(name=variant,theta_miss=0.0,use_belief=use_belief,use_sweep=use_sweep)

```

```

        rbt=cls(sim,plan,p,problem=problem)
        r=rbt.run(); r['gave_up']=list(rbt.gave_up); r['scan_points']=len(rbt.scanned)
        rows.append(r)
    st=[r['stats'] for r in rows]
    out={
        'problem':problem, 'variant':variant, 'n':n, 'seed0':seed0, 'belief':use_belief, 'sweep':use_sweep,
        'full':int(sum(s['ratio']>=1-1e-9 for s in st)),
        'mean_ratio':float(np.mean([s['ratio'] for s in st])),
        'mean_avg_s_per_source':float(np.mean([s['avg_time_s'] for s in st])),
        'aggregate_s_per_source':float(sum(s['total_time_s'] for s in st)/sum(s['n_sources'] for s in st)),
        'mean_total_time':float(np.mean([s['total_time_s'] for s in st])),
        'mean_move_m':float(np.mean([s['move_m'] for s in st])),
        'mean_requests':float(np.mean([s['n_requests'] for s in st])),
        'mean_scan_points':float(np.mean([r['scan_points'] for r in rows])),
        'total_missed':int(sum(s['n_sources']-s['n_cleared'] for s in st)),
        'total_gave_up':int(sum(len(r['gave_up']) for r in rows)),
        't_move':float(np.mean([s['t_move'] for s in st])),
        't_measure':float(np.mean([s['t_measure'] for s in st])),
        't_switch':float(np.mean([s['t_switch'] for s in st])),
        't_clear':float(np.mean([s['t_clear'] for s in st])),
        'wall_s':time.time()-t0,
    }
    return out

if __name__=='__main__':
    ap=argparse.ArgumentParser(); ap.add_argument('--problem',type=int,required=True);
    ↪ ap.add_argument('--n',type=int,default=60); ap.add_argument('--seed0',type=int);
    ↪ ap.add_argument('--variants',default='baseline,nearest,scanfirst,sourcefirst,nopiggy');
    ↪ ap.add_argument('--belief',action='store_true'); ap.add_argument('--no-sweep',action='store_true')
    a=ap.parse_args(); seed0=a.seed0 if a.seed0 is not None else (90000 if a.problem==3 else 700000)
    outs=[]
    for v in a.variants.split(','):
        o=run_variant(a.problem,v,a.n,seed0,a.belief,not a.no_sweep); outs.append(o);
        ↪ print(json.dumps(o),flush=True)
    print(json.dumps(outs,indent=2))

```

## 文件: vendor/portable\_sandbox/certify\_q4\_21.py

```

import math,itertools,json,time

def cross(a,b,c):return (b[0]-a[0])*(c[1]-a[1])-(b[1]-a[1])*(c[0]-a[0])
def hull(pts):
    pts=sorted(set(pts)); lo=[];up=[]
    for p in pts:
        while len(lo)>=2 and cross(lo[-2],lo[-1],p)<=0:lo.pop()
        lo.append(p)
    for p in reversed(pts):
        while len(up)>=2 and cross(up[-2],up[-1],p)<=0:up.pop()
        up.append(p)
    return lo[:-1]+up[:-1]
def clip(poly,nx,ny,off):
    if not poly:return []
    L=math.hypot(nx,ny)
    if L==0:return poly if off>=0 else []
    nx/=L;ny/=L;off=off/L+1e-8;out=[];a=poly[-1];fa=nx*a[0]+ny*a[1]-off
    for b in poly:
        fb=nx*b[0]+ny*b[1]-off

```

```

    if (fa<=0)!=(fb<=0):
        t=fa/(fa-fb);out.append((a[0]+t*(b[0]-a[0]),a[1]+t*(b[1]-a[1])))
    if fb<=0:out.append(b)
    a,fa=b,fb
    return out
def contains(poly,p):return bool(poly) and all(cross(a,b,p)>=-1e-6*max(1.,math.dist(a,b)) for a,b in
↪ zip(poly,poly[1:]+poly[:1]))
def extrema(poly,site,R):
    cand=[p for p in poly if math.hypot(*p)<=R+1e-7]
    for a,b in zip(poly,poly[1:]+poly[:1]):
        dx,dy=b[0]-a[0],b[1]-a[1];l2=dx*dx+dy*dy
        if l2<1e-16:continue
        tm=-(a[0]*dx+a[1]*dy)/l2;foot=(a[0]+tm*dx,a[1]+tm*dy);rad2=R*R-foot[0]*foot[0]-foot[1]*foot[1]
        if rad2<-1e-6:continue
        ht=math.sqrt(max(0.,rad2)/l2)
        for t in (tm-ht,tm+ht):
            if -1e-9<=t<=1+1e-9:cand.append((a[0]+t*dx,a[1]+t*dy))
    L=math.hypot(*site);far=(-R*site[0]/L,-R*site[1]/L) if L else (R,0.)
    if contains(poly,far):cand.append(far)
    return cand
def segment_radius(sites,normal,off,R=1800.):
    nx,ny=normal;L=math.hypot(nx,ny)
    if L*R<off-1e-7:return 0.,None
    if not sites:return math.inf,(R*nx/L,R*ny/L)
    seg=clip([(-R,-R),(R,-R),(R,R),(-R,R)],-nx,-ny,-off);worst=0.;wit=None
    for site in sites:
        cell=seg
        for o in sites:
            if o==site:continue
            dx,dy=o[0]-site[0],o[1]-site[1];bound=(dx*(o[0]+site[0])+dy*(o[1]+site[1]))/2
            cell=clip(cell,dx,dy,bound)
            if not cell:break
        cs=extrema(cell,site,R)
        if not cs:continue
        p=max(cs,key=lambda p:math.dist(p,site));v=math.dist(p,site)
        if v>worst:worst,v=p
    return worst,wit
def certify(points,R=1800.,rec=1000.,margin=.001):
    h=hull(points);inr=min((a[0]*b[1]-a[1]*b[0])/math.dist(a,b) for a,b in zip(h,h[1:]+h[:1]))
    if inr<R+margin:return {'certified':False,'reason':'hull','inradius':inr}
    worst=0.;wp=None;pair=None;checked=0;t=time.time()
    for i,j in itertools.permutations(range(len(points)),2):
        a,b=points[i],points[j];dx,dy=b[0]-a[0],b[1]-a[1];L=math.hypot(dx,dy)
        if L==0:continue
        n=(-dy/L,dx/L);off=n[0]*a[0]+n[1]*a[1]
        if off>=R:continue
        sites=[p for p in points if n[0]*p[0]+n[1]*p[1]>off+1e-7]
        v,w=segment_radius(sites,n,off,R);checked+=1
        if v>worst:worst,wp,pair=v,w,(i,j)
    return {'certified':worst<rec-margin,'inradius':inr,'max_radius':worst,'witness':wp,'pair':pair,'checks'
↪ :checked,'runtime':time.time()-t}
outer=1802/math.cos(math.pi/12)
pts=[(0.,0.)]+[(outer*math.cos(2*math.pi*k/12),outer*math.sin(2*math.pi*k/12)) for k in
↪ range(12)]+[(997.32*math.cos(2*math.pi*k/8),997.32*math.sin(2*math.pi*k/8)) for k in range(8)]
print(json.dumps(certify(pts),indent=2))

```

文件: vendor/portable\_sandbox/lib/geom.py

```

""" 几何与定位区域算法库 (CUMCM2026 B 题)

约定:
    角度单位为度, 逆时针为正, x 轴正向为 0 度.
    半平面统一写成  $n \cdot p \leq c$  ( $n$  为单位外法向).
    凸多边形用顶点列表表示 (逆时针, 首尾不重复).
"""

from __future__ import annotations

import math
import numpy as np

EPS = 1e-12

def unit(deg: float) -> np.ndarray:
    r = math.radians(deg)
    return np.array([math.cos(r), math.sin(r)])

def cross(a: np.ndarray, b: np.ndarray) -> float:
    return float(a[0] * b[1] - a[1] * b[0])

def dot(a: np.ndarray, b: np.ndarray) -> float:
    return float(a[0] * b[0] + a[1] * b[1])

def norm(a: np.ndarray) -> float:
    return float(math.hypot(a[0], a[1]))

def ang_deg(v: np.ndarray) -> float:
    return math.degrees(math.atan2(v[1], v[0])) % 360.0

def ang_diff(a: float, b: float) -> float:
    d = (a - b) % 360.0
    return d - 360.0 if d > 180.0 else d

def wedge_halfplanes(S, theta_deg: float, eps_deg: float):
    hps = []
    for sgn in (+1.0, -1.0):
        d = unit(theta_deg + sgn * eps_deg)
        n = np.array([-d[1], d[0]])
        if sgn > 0:
            hps.append((n, dot(n, S)))
        else:
            hps.append((-n, -dot(n, S)))
    return hps

def hp_feasible_point(hps):
    n = len(hps)
    A = np.array([h[0] for h in hps])
    b = np.array([h[1] for h in hps])
    x = np.zeros(2)
    for _ in range(200):
        v = A @ x - b
        act = v > 0
        if not act.any():
            return x
        Aa = A[act]
        g = 2.0 * Aa.T @ (Aa @ x - b[act])
        step = 1.0 / (2.0 * len(Aa) + 1e-9)
        x = x - step * g

```

```

v = A @ x - b
return x if (v <= 1e-7).all() else None

def is_bounded(hps) -> bool:
    ang = sorted(math.degrees(math.atan2(h[0][1], h[0][0])) % 360.0 for h in hps)
    if len(ang) < 2:
        return False
    gaps = [ang[i + 1] - ang[i] for i in range(len(ang) - 1)]
    gaps.append(ang[0] + 360.0 - ang[-1])
    return max(gaps) < 180.0 - 1e-9

def clip_polygon(poly, hp):
    if not len(poly):
        return poly
    n, c = hp
    out = []
    m = len(poly)
    for i in range(m):
        a = poly[i]
        b = poly[(i + 1) % m]
        va = dot(n, a) - c
        vb = dot(n, b) - c
        if va <= 1e-12:
            out.append(a)
        if (va < -1e-12 < vb) or (vb < -1e-12 < va):
            t = va / (va - vb)
            out.append(a + t * (b - a))
    return dedup(out)

def dedup(poly, tol=1e-9):
    if len(poly) < 2:
        return poly
    out = []
    for p in poly:
        if not out or norm(p - out[-1]) > tol:
            out.append(p)
    if len(out) > 1 and norm(out[0] - out[-1]) <= tol:
        out.pop()
    return out

def polygon_area(poly) -> float:
    m = len(poly)
    if m < 3:
        return 0.0
    s = 0.0
    for i in range(m):
        a, b = poly[i], poly[(i + 1) % m]
        s += a[0] * b[1] - b[0] * a[1]
    return abs(s) / 2.0

def polygon_centroid(poly) -> np.ndarray:
    poly = [np.asarray(p, dtype=float) for p in poly]
    m = len(poly)
    if m == 0:
        return np.zeros(2)
    if m == 1:
        return poly[0].copy()
    if m == 2:
        return (poly[0] + poly[1]) / 2.0

```

```

a = 0.0
cx = cy = 0.0
for i in range(m):
    p, q = poly[i], poly[(i + 1) % m]
    cr = p[0] * q[1] - q[0] * p[1]
    a += cr
    cx += (p[0] + q[0]) * cr
    cy += (p[1] + q[1]) * cr
if abs(a) < 1e-12:
    return np.mean(np.array(poly), axis=0)
return np.array([cx / (3.0 * a), cy / (3.0 * a)])

def offset_polygon(poly, d):
    P = [np.asarray(p, dtype=float) for p in poly]
    m = len(P)
    if m < 3:
        return P
    lines = []
    for i in range(m):
        a, b = P[i], P[(i + 1) % m]
        e = b - a
        L = norm(e)
        if L < 1e-12:
            continue
        n = np.array([e[1], -e[0]]) / L
        lines.append((a + d * n, b + d * n))
    out = []
    for i in range(len(lines)):
        p1, p2 = lines[i]
        q1, q2 = lines[(i - 1) % len(lines)]
        r, s = p2 - p1, q2 - q1
        den = cross(r, s)
        if abs(den) < 1e-12:
            out.append(p1)
            continue
        t = cross(q1 - p1, s) / den
        out.append(p1 + t * r)
    return dedup(out)

def polygon_perimeter(poly) -> float:
    m = len(poly)
    if m < 2:
        return 0.0
    return float(sum(norm(poly[(i + 1) % m] - poly[i]) for i in range(m)))

def convex_hull(pts):
    pts = sorted({(round(float(p[0]), 9), round(float(p[1]), 9)) for p in pts})
    if len(pts) <= 2:
        return [np.array(p) for p in pts]
    def half(points):
        st = []
        for p in points:
            while len(st) >= 2 and cross(st[-1] - st[-2], np.array(p) - st[-1]) <= 0:
                st.pop()
            st.append(np.array(p))
        return st
    lo = half(pts)
    up = half(pts[::-1])
    return dedup(lo[:-1] + up[:-1])

```

```

_CIRC_CACHE = {}

def cached_circ_polygon(r: float, n: int = 360):
    key = (round(float(r), 6), int(n))
    if key not in _CIRC_CACHE:
        _CIRC_CACHE[key] = circ_polygon(key[0], key[1])
    return [p.copy() for p in _CIRC_CACHE[key]]

def circ_polygon(r: float, n: int = 360, center=(0.0, 0.0), circumscribe=True):
    c = np.array(center, dtype=float)
    rr = r / math.cos(math.pi / n) if circumscribe else r
    return [c + rr * unit(360.0 * i / n) for i in range(n)]

def localization_polygon(obs, arena=None, box=4.0e6):
    hps = []
    for S, th, eps in obs:
        hps.extend(wedge_halfplanes(np.asarray(S, dtype=float), th, eps))
    bounded_geom = is_bounded(hps)
    if isinstance(arena, (int, float)):
        seed = cached_circ_polygon(float(arena), 360)
    elif arena is None:
        b = box
        seed = [np.array([-b, -b]), np.array([b, -b]), np.array([b, b]), np.array([-b, b])]
    else:
        seed = [np.asarray(p, dtype=float) for p in arena]
    poly = [np.asarray(p, dtype=float) for p in seed]
    for hp in hps:
        poly = clip_polygon(poly, hp)
        if len(poly) == 0:
            return [], bounded_geom
    if len(poly) >= 3 and polygon_area(poly) < 1e-9:
        return [], bounded_geom
    if len(poly) > 12:
        poly = simplify_polygon(poly, tol=0.02)
    return poly, bounded_geom

def diameter_bruteforce(poly) -> float:
    m = len(poly)
    best = 0.0
    for i in range(m):
        for j in range(i + 1, m):
            best = max(best, norm(poly[i] - poly[j]))
    return best

def diameter_pair_bruteforce(poly):
    m = len(poly)
    best, bi, bj = 0.0, 0, 0
    for i in range(m):
        for j in range(i + 1, m):
            d = norm(poly[i] - poly[j])
            if d > best:
                best, bi, bj = d, i, j
    return best, (poly[bi] if m else None), (poly[bj] if m else None)

def diameter_rotating_calipers_pair(poly):
    pts = convex_hull(poly)
    m = len(pts)
    if m < 2:

```

```

        return 0.0, (None, None)
    if m == 2:
        return norm(pts[1] - pts[0]), (pts[0], pts[1])
    def area2(a, b, c):
        return abs(cross(b - a, c - a))
    j = 1
    best, bp = 0.0, (pts[0], pts[1])
    for i in range(m):
        ni = (i + 1) % m
        while True:
            nj = (j + 1) % m
            if area2(pts[i], pts[ni], pts[nj]) > area2(pts[i], pts[ni], pts[j]) + 1e-15:
                j = nj
            else:
                break
        for cand in (pts[i], pts[ni]):
            d = norm(cand - pts[j])
            if d > best:
                best, bp = d, (cand, pts[j])
    return best, bp

def diameter_rotating_calipers(poly) -> float:
    return diameter_rotating_calipers_pair(poly)[0]

def diameter_support_scan(poly, extra_angles=720) -> float:
    pts = convex_hull(poly)
    if len(pts) < 2:
        return 0.0
    angs = []
    m = len(pts)
    for i in range(m):
        e = pts[(i + 1) % m] - pts[i]
        angs.append(math.atan2(e[1], e[0]) + math.pi / 2.0)
        angs.append(math.atan2(e[1], e[0]) - math.pi / 2.0)
    angs += list(np.linspace(0, 2 * math.pi, extra_angles, endpoint=False))
    P = np.array(pts)
    best = 0.0
    for a in angs:
        n = np.array([math.cos(a), math.sin(a)])
        h1 = float((P @ n).max())
        h2 = float((P @ (-n)).max())
        best = max(best, h1 + h2)
    return best

def mec_bruteforce(pts):
    P = [np.asarray(p, dtype=float) for p in pts]
    n = len(P)
    if n == 0:
        return np.zeros(2), 0.0
    if n == 1:
        return P[0].copy(), 0.0
    def ok(c, r):
        return all(norm(p - c) <= r + 1e-9 for p in P)
    best_c, best_r = None, float("inf")
    for i in range(n):
        for j in range(i + 1, n):
            c = (P[i] + P[j]) / 2.0
            r = norm(P[i] - P[j]) / 2.0
            if r < best_r - 1e-12 and ok(c, r):

```

```

        best_c, best_r = c, r
    for i in range(n):
        for j in range(i + 1, n):
            for k in range(j + 1, n):
                a, b, c3 = P[i], P[j], P[k]
                d = 2.0 * (a[0] * (b[1] - c3[1]) + b[0] * (c3[1] - a[1]) + c3[0] * (a[1] - b[1]))
                if abs(d) < 1e-12:
                    continue
                ux = ((a @ a) * (b[1] - c3[1]) + (b @ b) * (c3[1] - a[1]) + (c3 @ c3) * (a[1] - b[1])) / d
                uy = ((a @ a) * (c3[0] - b[0]) + (b @ b) * (a[0] - c3[0]) + (c3 @ c3) * (b[0] - a[0])) / d
                c = np.array([ux, uy])
                r = norm(a - c)
                if r < best_r - 1e-12 and ok(c, r):
                    best_c, best_r = c, r
    return best_c, best_r

def mec_minidisk(poly, rng=None):
    pts = [np.asarray(p, dtype=float) for p in poly]
    if not pts:
        return np.zeros(2), 0.0
    rng = rng or np.random.default_rng(12345)
    idx = rng.permutation(len(pts))
    P = [pts[i] for i in idx]
    c = P[0].copy()
    r = 0.0
    for i in range(1, len(P)):
        p = P[i]
        if norm(p - c) <= r + 1e-9:
            continue
        c, r = p.copy(), 0.0
        for j in range(i):
            q = P[j]
            if norm(q - c) <= r + 1e-9:
                continue
            c = (p + q) / 2.0
            r = norm(p - q) / 2.0
            for k in range(j):
                s = P[k]
                if norm(s - c) <= r + 1e-9:
                    continue
            c, r = circumcircle(p, q, s)
    for _ in range(4):
        bad = [p for p in pts if norm(p - c) > r + 1e-7]
        if not bad:
            break
        r = max(norm(p - c) for p in pts)
    return c, r

def circumcircle(a, b, c):
    d = 2.0 * (a[0] * (b[1] - c[1]) + b[0] * (c[1] - a[1]) + c[0] * (a[1] - b[1]))
    if abs(d) < 1e-14:
        return mec_bruteforce([a, b, c])
    ux = ((a @ a) * (b[1] - c[1]) + (b @ b) * (c[1] - a[1]) + (c @ c) * (a[1] - b[1])) / d
    uy = ((a @ a) * (c[0] - b[0]) + (b @ b) * (a[0] - c[0]) + (c @ c) * (b[0] - a[0])) / d
    ct = np.array([ux, uy])
    return ct, norm(a - ct)

def mec(poly, sample_cap=64, rng=None):
    return mec_minidisk(poly, rng=rng)

```

```

def simplify_polygon(poly, tol=0.02):
    m = len(poly)
    if m <= 4:
        return [np.asarray(p, dtype=float) for p in poly]
    keep = [0]
    for i in range(1, m - 1):
        a = poly[keep[-1]]
        b = poly[i]
        c = poly[i + 1]
        e = c - a
        L = norm(e)
        dist = norm(b - a) if L < 1e-12 else abs(cross(e, b - a)) / L
        if dist >= tol:
            keep.append(i)
    keep.append(m - 1)
    out = [np.asarray(poly[i], dtype=float) for i in keep]
    return dedup(out)

def diameter_disk_covers(poly, pair) -> bool:
    a, b = pair
    if a is None or b is None:
        return True
    scale = max(norm(a - b) ** 2, 1.0)
    return all(dot(p - a, p - b) <= 1e-9 * scale for p in poly)

def sample_in_polygon(poly, n, rng):
    if len(poly) < 3:
        return np.zeros((0, 2))
    P = np.array(poly, dtype=float)
    tris, areas = [], []
    for i in range(1, len(poly) - 1):
        tris.append((P[0], P[i], P[i + 1]))
        areas.append(abs(cross(P[i] - P[0], P[i + 1] - P[0])) / 2.0)
    areas = np.array(areas)
    tot = areas.sum()
    if tot <= 0:
        return np.zeros((0, 2))
    idx = rng.choice(len(tris), size=n, p=areas / tot)
    out = np.zeros((n, 2))
    for k in range(n):
        a, b, c = tris[idx[k]]
        u, v = rng.random(), rng.random()
        if u + v > 1:
            u, v = 1 - u, 1 - v
        out[k] = a + u * (b - a) + v * (c - a)
    return out

def cover_lattice_points(poly, spacing, start=None, shrink=1.0):
    if len(poly) < 3:
        return []
    P = np.array(poly, dtype=float)
    lo, hi = P.min(axis=0), P.max(axis=0)
    dy = spacing * math.sqrt(3) / 2.0
    rows = int((hi[1] - lo[1]) / dy) + 2
    pts = []
    from matplotlib.path import Path
    path = Path(P)
    for r in range(rows):

```

```

        y = lo[1] + r * dy
        x0 = lo[0] + (spacing / 2.0 if r % 2 else 0.0)
        cols = int((hi[0] - x0) / spacing) + 2
        for c2 in range(cols):
            x = x0 + c2 * spacing
            if path.contains_point((x, y)):
                pts.append(np.array([x, y]))
    if not pts:
        pts = [polygon_centroid(poly)]
    cur = np.asarray(start, dtype=float) if start is not None else polygon_centroid(poly)
    order, left = [], list(pts)
    while left:
        k = min(range(len(left)), key=lambda i: norm(left[i] - cur))
        order.append(left.pop(k))
        cur = order[-1]
    return order

def point_in_polygon(p, poly) -> bool:
    ok = True
    m = len(poly)
    for i in range(m):
        a, b = poly[i], poly[(i + 1) % m]
        if cross(b - a, np.asarray(p, float) - a) < -1e-9:
            ok = False
            break
    return ok

```

## 文件：vendor/portable\_sandbox/lib/sim.py

""" 无线电干扰源环境模拟器（按赛题附录 2 / 附件 2 规则实现，用于本地演练与统计检验）。

与官方模拟器的一致性：

- \* 目标区域：半径 1800m 圆域，原点为圆心；机器狗初始位置 (0,0)，初始频道 1
- \* 干扰源：频道互异 (1..20)，位置与有效接收半径 (1000..1500m) 未知
- \* 全向源：距离 $\leq R$  即可接收；定向源：距离 $\leq R$  且位于定向方向  $\pm 90^\circ$  内
- \* 示向度误差：同一地点固定（确定性伪随机场），全局落在  $\pm 1^\circ$
- \* 计时：移动  $|\Delta p|/5$  s；换频道 1 s；检测 5 s；清除 未发现 3 s / 成功 5 s
- \* /clear 不换频道、不受定向覆盖角限制；5m 内检测返回 near；20m 内清除成功

"""

```
from __future__ import annotations
```

```
import math
```

```
from dataclasses import dataclass, field
```

```
import numpy as np
```

```
from geom import ang_deg, norm
```

```
R_ARENA = 1800.0
```

```
SPEED = 5.0
```

```
T_MEASURE = 5.0
```

```
T_SWITCH = 1.0
```

```
T_CLEAR_SUCCESS = 5.0
```

```
T_CLEAR_FAIL = 3.0
```

```
R_NEAR = 5.0
```

```
R_CLEAR = 20.0
```

```
R_MIN = 1000.0
```

```

R_MAX = 1500.0
V_LIMIT = 360000.0

def _mix(x: int) -> int:
    x = (x + 0x9E3779B97F4A7C15) & 0xFFFFFFFFFFFFFFFF
    x = (x ^ (x >> 30)) * 0xBF58476D1CE4E5B9 & 0xFFFFFFFFFFFFFFFF
    x = (x ^ (x >> 27)) * 0x94D049BB133111EB & 0xFFFFFFFFFFFFFFFF
    return x ^ (x >> 31)

def env_error(p, channel: int) -> float:
    """ 固定电磁环境下的示向度误差 (度), 同一地点同一频道恒定, 落在 [-1,1]。"""
    qx = int(round(p[0] * 0.2))          # 5m 网格
    qy = int(round(p[1] * 0.2))
    h = _mix((qx & 0xFFFF) | ((qy & 0xFFFF) << 20) | (channel << 44))
    return (h / 2 ** 63) - 1.0

@dataclass
class Source:
    channel: int
    pos: np.ndarray
    radius: float
    kind: str = "omni"                  # omni | dir
    direction_deg: float | None = None
    cleared: bool = False

    def covers(self, p) -> bool:
        d = norm(np.asarray(p, float) - self.pos)
        if d > self.radius:
            return False
        if self.kind == "dir":
            u = np.array([math.cos(math.radians(self.direction_deg)),
                          math.sin(math.radians(self.direction_deg))])
            if float((np.asarray(p, float) - self.pos) @ u) < 0.0:
                return False
        return True

    def dist(self, p) -> float:
        return norm(np.asarray(p, float) - self.pos)

@dataclass
class Simulator:
    sources: list
    t: float = 0.0
    pos: np.ndarray = field(default_factory=lambda: np.zeros(2))
    channel: int = 1
    n_requests: int = 0
    log: list = field(default_factory=list)
    cleared: list = field(default_factory=list)
    total_move_m: float = 0.0
    t_move: float = 0.0
    t_switch: float = 0.0
    t_measure: float = 0.0
    t_clear: float = 0.0

    def _move(self, p):

```

```

    p = np.asarray(p, float)
    d = norm(p - self.pos)
    dt = d / SPEED
    self.t += dt
    self.total_move_m += d
    self.t_move += dt
    self.pos = p

def measure(self, pos, channel: int):
    self.n_requests += 1
    self._move(pos)
    dt = T_MEASURE
    if channel != self.channel:
        dt += T_SWITCH
        self.t_switch += T_SWITCH
        self.channel = channel
    self.t += dt
    self.t_measure += T_MEASURE

    src = None
    for s in self.sources:
        if s.channel == channel and not s.cleared:
            src = s
            break
    if src is None:
        return "no_signal", None
    if not src.covers(self.pos):
        return "no_signal", None
    d = src.dist(self.pos)
    if d <= R_NEAR:
        return "near", None
    svd = (ang_deg(src.pos - self.pos) + env_error(self.pos, channel)) % 360.0
    return "direction", svd

def clear(self, pos, channel: int):
    self.n_requests += 1
    self._move(pos)
    src = None
    for s in self.sources:
        if s.channel == channel and not s.cleared:
            src = s
            break
    if src is not None and src.dist(self.pos) <= R_CLEAR + 1e-9:
        self.t += T_CLEAR_SUCCESS
        self.t_clear += T_CLEAR_SUCCESS
        src.cleared = True
        self.cleared.append(channel)
        return "success", self.t
    self.t += T_CLEAR_FAIL
    self.t_clear += T_CLEAR_FAIL
    return "no_target_in_range", self.t

def stats(self) -> dict:
    n = len(self.sources)
    nc = len(self.cleared)
    return {
        "n_sources": n, "n_cleared": nc,
        "ratio": nc / n if n else 0.0,
        "total_time_s": self.t,
    }

```

```

        "avg_time_s": self.t / nc if nc else None,
        "n_requests": self.n_requests,
        "move_m": self.total_move_m,
        "t_move": self.t_move, "t_switch": self.t_switch,
        "t_measure": self.t_measure, "t_clear": self.t_clear,
    }

def generate_case(rng, problem=3, n_sources=None):
    if n_sources is None:
        n_sources = int(rng.integers(10, 17))
    channels = rng.choice(np.arange(1, 21), size=n_sources, replace=False)
    srcs = []
    for ch in channels:
        r = R_ARENA * math.sqrt(rng.random())
        a = rng.random() * 2 * math.pi
        pos = np.array([r * math.cos(a), r * math.sin(a)])
        radius = R_MIN + (R_MAX - R_MIN) * rng.random()
        if problem == 3:
            srcs.append(Source(int(ch), pos, radius, "omni", None))
        else:
            kind = "dir" if rng.random() < 0.5 else "omni"
            direction = None if kind == "omni" else float(rng.random() * 360.0)
            srcs.append(Source(int(ch), pos, radius, kind, direction))
    return srcs

```

文件: vendor/portable\_sandbox/lib/strategy.py

```

from __future__ import annotations
import math
from dataclasses import dataclass, field
import numpy as np
from geom import (
    clip_polygon, convex_hull, diameter_rotating_calipers_pair, mec, norm, offset_polygon,
    polygon_area, polygon_centroid, point_in_polygon, unit, wedge_halfplanes,
)
from sim import R_ARENA, R_MIN, SPEED, Simulator
R_CLEAR = 20.0
MARGIN = 0.5

def disk_polygon(center, radius, n=120):
    c=np.asarray(center,float); rr=radius/math.cos(math.pi/n)
    return [c+rr*unit(360.0*i/n) for i in range(n)]

@dataclass
class Params:
    name: str = "adaptive"
    interleave: bool = True
    use_tracking: bool = True
    use_sweep: bool = True
    sweep_max_diam: float = 95.0
    sweep_spacing: float = 33.0
    source_weight: float = 0.85
    lateral_frac: float = 0.30
    max_iter_per_source: int = 40

class Robot:

```

```

def __init__(self, sim: Simulator, cover_pts, params: Params, problem=3):
    self.sim=sim; self.params=params; self.problem=problem
    self.cover=[np.asarray(p,float) for p in cover_pts]; self.pending=list(self.cover)
    self.unknown=set(range(1,21)); self.tracked={}; self.cleared=set(); self.proven_empty=set();
    ↪ self.side={}
    self.n_measure=0; self.n_clear=0; self.n_home_steps=0; self.anomaly=0; self.gave_up=[];
    ↪ self.nosig={}; self.ray={}; self.mlog={}; self.n_filter=0
def _dist(self,p): return norm(np.asarray(p,float)-self.sim.pos)
def _scan_unknown(self, at=None):
    extra=[]
    for ch,poly in self.tracked.items():
        if ch in self.cleared: continue
        D,_=diameter_rotating_calipers_pair(poly)
        if D>80.0: extra.append(ch)
    chans=sorted(self.unknown)+extra
    for k,ch in enumerate(chans):
        p=at if (k==0 and at is not None) else self.sim.pos
        res,svd=self.sim.measure(p,ch); self.n_measure+=1
        if res=="direction":
            if ch in self.tracked: self._update_region(ch,self.sim.pos,svd)
            else: self._init_region(ch,svd)
            self.ray[ch]=(self.sim.pos.copy(),svd); self.nosig[ch]=0
        elif res=="no_signal" and ch in self.tracked:
            self._record_nosig(ch,self.sim.pos)
            if self.problem==4:
                np2=self._shadow_filter(ch,self.tracked[ch],prob4=True)
                if np2 is not None: self.tracked[ch]=np2
        elif res=="near":
            r,_=self.sim.clear(self.sim.pos,ch); self.n_clear+=1
            if r=="success": self.cleared.add(ch); self.unknown.discard(ch)
def _init_region(self,ch,svd):
    self.mlog.setdefault(ch,[]).append((self.sim.pos.copy(),"direction",float(svd)))
    pos=self.sim.pos.copy(); hps=wedge_halfplanes(pos,svd,1.0); poly=disk_polygon((0,0),R_ARENA,360)
    for hp in hps:
        poly=clip_polygon(poly,hp)
        if len(poly)==0: break
    if len(poly)>0:
        disk=disk_polygon(pos,1500.0,120)
        for hp in _poly_halfplanes(disk):
            poly=clip_polygon(poly,hp)
            if len(poly)==0: break
    if len(poly)>=3: self.tracked[ch]=_simplify(poly); self.unknown.discard(ch)
    else: self.anomaly+=1
def _update_region(self,ch,pos,svd):
    self.mlog.setdefault(ch,[]).append((np.asarray(pos,float).copy(),"direction",float(svd)))
    poly=self.tracked.get(ch)
    if poly is None: self._init_region(ch,svd); return
    poly=[p.copy() for p in poly]
    for hp in wedge_halfplanes(np.asarray(pos,float),svd,1.0):
        poly=clip_polygon(poly,hp)
        if len(poly)==0: break
    if len(poly)>=3 and polygon_area(poly)>1e-6: self.tracked[ch]=_simplify(poly)
    else: self.anomaly+=1
def _shadow_filter(self,ch,poly,spacing=15.0,nu=720,prob4=True):
    P=np.array(poly, dtype=float); lo,hi=P.min(axis=0)-spacing,P.max(axis=0)+spacing
    nx=max(2,int((hi[0]-lo[0])/spacing)+1); ny=max(2,int((hi[1]-lo[1])/spacing)+1)
    if nx*ny>40000: return None
    xs=lo[0]+spacing*np.arange(nx); ys=lo[1]+spacing*np.arange(ny); GX,GY=np.meshgrid(xs,ys)

```

```

grid=np.stack([GX.ravel(),GY.ravel()],axis=1); inside=np.array([point_in_polygon(g,poly) for g in
↪ grid]); pts=grid[inside]
if len(pts)==0: return None
U=np.stack([np.cos(2*np.pi*np.arange(nu)/nu),np.sin(2*np.pi*np.arange(nu)/nu)],axis=1)
ok_dir=np.ones(len(pts),dtype=bool); ok_u=np.ones((len(pts),nu),dtype=bool); tol=spacing*1.5;
↪ use_u=bool(prob4)
for q,kind,svd in self.mlog.get(ch,[]):
    v=pts-q; d=np.linalg.norm(v,axis=1)
    if kind=="direction":
        ok_dir&=(d<=1500.0+tol); ang=np.degrees(np.arctan2(v[:,1],v[:,0]));
        ↪ diff=np.abs((ang-svd+180.0)%360.0-180.0)
        tol_ang=np.degrees(np.arctan2(tol,np.maximum(d,1.0)))+1.0; ok_dir&=(diff<=tol_ang)
        if use_u:
            vh=v/np.maximum(d,1e-9)[: ,None]; ok_u&=(vh@U.T<=0.02)
    else:
        if use_u:
            sel=d<=1000.0-tol
            if sel.any():
                vh=v[sel]/np.maximum(d[sel],1e-9)[: ,None]; ok_u[sel]=ok_u[sel]&(vh@U.T>-0.02)
            else: ok_dir&=(d>1000.0-tol)
feas=ok_dir&ok_u.any(axis=1)
if feas.sum()<3:return None
hull=convex_hull(pts[feas])
if len(hull)<3:return None
poly2=offset_polygon(hull,spacing*1.5)
for hp in _poly_halfplanes(poly):
    poly2=clip_polygon(poly2,hp)
    if len(poly2)==0:return None
self.n_filter+=1
return poly2 if len(poly2)>=3 else None
def _record_nosig(self,ch,q):
↪ self.mlog.setdefault(ch,[]).append((np.asarray(q,float).copy(),"no_signal",None))
def _est_pos(self,ch): return polygon_centroid(self.tracked[ch])
def _near_edge(self,ch):
    P=np.array(self.tracked[ch]); d=np.linalg.norm(P-self.sim.pos,axis=1); return P[int(d.argmin())]
def _next_measure_point(self,ch,poly):
    if self.problem==4 and ch in self.ray:
        apex,th=self.ray[ch]; u=unit(th); n=np.array([-u[1],u[0]])
        tmax=max(float((np.asarray(p,float)-apex)@u) for p in poly); tmax=max(tmax,50.0);
        ↪ t_now=float((self.sim.pos-apex)@u)
        t=min(max(t_now+200.0,200.0),tmax); sgn=self.side.get(ch,1.0); self.side[ch]=-sgn
        lat=25.0*sgn if self.nosig.get(ch,0)==0 else 12.0*sgn
        return apex+t*u+lat*n
D,(a,b)=diameter_rotating_calipers_pair(poly); area=polygon_area(poly); width=area/D if D>1e-9
↪ else 0.0
if width<=45.0 and D>80.0:
    u=(b-a)/D; s=float(np.clip((self.sim.pos-a)@u,0.0,D)); t=s+250.0
    if t>D-40.0: t=max(D*0.5,s-250.0) if s>D*0.5 else D
    lat=0.0 if self.params.lateral_frac<=0 else 12.0; sgn=self.side.get(ch,1.0);
    ↪ self.side[ch]=-sgn; n=np.array([-u[1],u[0]])
    return a+float(np.clip(t,0.0,D))*u+sgn*lat*n
c=polygon_centroid(poly); v=c-self.sim.pos; L=norm(v); u=unit(self.sim.channel*37.0) if L<1e-6
↪ else v/L; up=np.array([-u[1],u[0]]); sgn=self.side.get(ch,1.0); self.side[ch]=-sgn
if L>700: f,lat=0.55*L,self.params.lateral_frac*L
elif L>150: f,lat=0.45*L,0.5*L
else: f,lat=0.15*L,max(60.0,0.9*L)
return self.sim.pos+f*u+sgn*lat*up
def _clear_sweep(self,ch,poly):

```

```

from geom import cover_lattice_points
pts=cover_lattice_points(poly,self.params.sweep_spacing,start=self.sim.pos)
for p in pts:
    r,_=self.sim.clear(p,ch); self.n_clear+=1
    if r=="success": return True
return False
def _ray_sweep(self,ch):
    if ch not in self.ray:return False
    apex,th=self.ray[ch]; u=unit(th); n=np.array([-u[1],u[0]]); poly=self.tracked[ch]
    ts=[float((np.asarray(p,float)-apex)@u) for p in poly]; t_lo=max(0.0,min(ts)-20.0);
    ↪ t_hi=max(ts)+30.0; t_now=float((self.sim.pos-apex)@u)
    if abs(t_now-t_lo)>abs(t_now-t_hi): t,t_end,step=t_hi,t_lo,-15.0
    else: t,t_end,step=t_lo,t_hi,15.0
    sgn=1.0
    while (step>0 and t<=t_end) or (step<0 and t>=t_end):
        q=apex+t*u+sgn*13.0*n; r,_=self.sim.clear(q,ch); self.n_clear+=1
        if r=="success":return True
        t+=step; sgn=-sgn
    return False
def home_and_clear(self,ch)->bool:
    for _ in range(self.params.max_iter_per_source):
        poly=self.tracked.get(ch)
        if poly is None or len(poly)<3:return False
        c,r=mec(poly); D,_=diameter_rotating_calipers_pair(poly)
        if not self.params.use_tracking:
            if D>1e-9 and self._clear_sweep(ch,poly):return True
            q=self._next_measure_point(ch,poly); res,svd=self.sim.measure(q,ch); self.n_measure+=1;
            ↪ self.n_home_steps+=1
            if res=="direction":self._update_region(ch,q,svd)
            elif res=="near":
                r2,_=self.sim.clear(q,ch); self.n_clear+=1
                if r2=="success":return True
            continue
        if r<=R_CLEAR-MARGIN:
            res,_=self.sim.clear(c,ch); self.n_clear+=1
            if res=="success":return True
            self.anomaly+=1; return False
        if self.params.use_sweep and D<=self.params.sweep_max_diam and D>1e-9:
            if self._clear_sweep(ch,poly):return True
            self.anomaly+=1
        q=self._next_measure_point(ch,poly); res,svd=self.sim.measure(q,ch); self.n_measure+=1;
        ↪ self.n_home_steps+=1
        if res=="direction":
            self.nosig[ch]=0; self.ray[ch]=(q.copy(),svd); self._update_region(ch,q,svd)
        elif res=="near":
            r2,_=self.sim.clear(q,ch); self.n_clear+=1
            if r2=="success":return True
        else:
            self.nosig[ch]=self.nosig.get(ch,0)+1; self._record_nosig(ch,self.sim.pos)
            if self.problem==4:
                np2=self._shadow_filter(ch,self.tracked[ch],prob4=True)
                if np2 is not None: self.tracked[ch]=np2; poly=self.tracked[ch]
            r3,_=self.sim.clear(self.sim.pos,ch); self.n_clear+=1
            if r3=="success":return True
            if self.problem==4:
                D2,(a2,b2)=diameter_rotating_calipers_pair(poly); width2=polygon_area(poly)/D2 if
                ↪ D2>1e-9 else 0.0
                if self.nosig[ch]>=3 and ch in self.ray:

```

```

        if self._ray_sweep(ch):return True
        elif self.nosig[ch]>=5 and width2>45.0:
            if self._clear_sweep(ch,poly):return True
    return False
def run(self)->dict:
    self._scan_unknown(); self.pending=[q for q in self.pending if self._dist(q)>1e-9]
    while True:
        todo_src=[ch for ch in self.tracked if ch not in self.cleared]; need_scan=bool(self.unknown)
        ↪ and bool(self.pending)
        if not todo_src and not need_scan:break
        options=[]
        if todo_src:
            for ch in todo_src:
                p=self._est_pos(ch) if self.params.interleave else self._near_edge(ch);
                ↪ cost=self._dist(p)*self.params.source_weight
                poly=self.tracked[ch]; D,_=diameter_rotating_calipers_pair(poly); cost+=0.12*D+30.0;
                ↪ options.append((cost,"home",ch,p))
        if need_scan:
            P=min(self.pending,key=lambda q:self._dist(q)); cost=self._dist(P)+6.0*len(self.unknown);
            ↪ cost*=1.0 if self.params.interleave else 0.0; options.append((cost,"scan",None,P))
        if not options:break
        options.sort(key=lambda o:o[0]); _,kind,ch,target=options[0]
        if kind=="scan": self.pending=[q for q in self.pending if norm(q-target)>1e-9];
        ↪ self._scan_unknown(at=target)
        else:
            ok=self.home_and_clear(ch); self.cleared.add(ch)
            if not ok:self.gave_up.append(ch)
    return self.finish()
def finish(self)->dict:
    sim=self.sim; proven=set(self.unknown) if not self.pending else set()
    return {"cleared":sorted(self.cleared),"tracked_channels":sorted(self.tracked),"gave_up":sorted(s
    ↪ elf.gave_up),"proven_empty":sorted(proven),"unresolved":sorted(self.unknown-proven),"n_measur
    ↪ e":self.n_measure,"n_clear":self.n_clear,"n_home_steps":self.n_home_steps,"anomaly":self.anom
    ↪ aly,"stats":sim.stats()}

def _poly_halfplanes(poly):
    hps=[]; m=len(poly)
    for i in range(m):
        a,b=poly[i],poly[(i+1)%m]; e=b-a; n=np.array([e[1],-e[0]]); nn=norm(n)
        if nn<1e-12:continue
        n=n/nn; hps.append((n,float(n@a)))
    return hps

def _simplify(poly,tol=0.05):
    from geom import simplify_polygon
    return simplify_polygon(poly,tol)

def run_trial(sources,cover_pts,params:Params,problem=3):
    sim=Simulator(sources); robot=Robot(sim,cover_pts,params,problem=problem); res=robot.run()
    res["sources"]=[{"ch":s.channel,"pos":s.pos.tolist(),"R":s.radius,"kind":s.kind,"dir":s.direction_deg
    ↪ ,"cleared":s.cleared} for s in sources]
    return res

```

文件: vendor/portable\_sandbox/lib/strategy2.py

```

from __future__ import annotations
import math

```

```

from dataclasses import dataclass, field
import numpy as np
from geom import (
    clip_polygon, convex_hull, cover_lattice_points, diameter_rotating_calipers_pair, mec,
    norm, offset_polygon, point_in_polygon, polygon_area, polygon_centroid, unit,
    wedge_halfplanes,
)
from sim import R_ARENA, R_MAX, R_MIN, SPEED
from strategy import MARGIN, R_CLEAR, Params, Robot, _poly_halfplanes, _simplify

@dataclass
class Params2(Params):
    name: str = "v2"
    theta_miss: float = 0.0
    rho: bool = True
    use_belief: bool = True
    r_learn: bool = True
    front_len: float = 350.0
    nosig_sweep: int = 4
    beta_gain: float = 8000.0

class MissModel:
    def __init__(self, problem=3, grid_step=25.0, nu=90):
        self.problem=problem; self.nu=nu
        xs=np.arange(-R_ARENA,R_ARENA+grid_step,grid_step); X,Y=np.meshgrid(xs,xs);
        ↪ m=(X**2+Y**2)<=R_ARENA**2
        self.grid=np.stack([X[m],Y[m]],axis=1)
        self.U=np.stack([np.cos(2*np.pi*np.arange(nu)/nu),np.sin(2*np.pi*np.arange(nu)/nu)],axis=1)
        self.b_grid=np.arange(1300.0,1500.1,10.0); self.b_post=None; self.detected_dists=[];
        ↪ self._cache={}; self._ver=0
    def observe_detected(self,dist): self.detected_dists.append(float(dist)); self._update_b()
    def _update_b(self):
        if not self.detected_dists or not np.isfinite(self.detected_dists).all(): self.b_post=None; return
        dmax=max(self.detected_dists); w=np.ones_like(self.b_grid)
        for d in self.detected_dists: w=w*np.clip((self.b_grid-d)/(self.b_grid-R_MIN),1e-6,1.0)
        w=np.where(self.b_grid>=max(dmax,R_MIN),w,0.0); w=np.where(self.b_grid<=R_MAX,w,0.0)
        self.b_post=w/w.sum() if w.sum()>0 else None; self._ver+=1; self._cache.clear()
    def mu(self,pts)->float:
        pts=np.asarray(pts,float)
        if len(pts)==0: return 1.0
        key=(self._ver,tuple(sorted((round(float(p[0]),1),round(float(p[1]),1)) for p in pts)))
        if key in self._cache: return self._cache[key]
        if self.b_post is None: bs,ws=np.array([R_MAX]),np.array([1.0])
        else: bs,ws=self.b_grid,self.b_post
        if self.problem==3:
            d=np.sqrt(((self.grid[:,None,:]-pts[None,:,:])**2).sum(-1)).min(axis=1)
            frac=np.clip((d[:,None]-R_MIN)/np.maximum(bs[None,:]-R_MIN,1e-9),0.0,1.0);
            ↪ val=float((frac*ws[None,:]).sum(axis=1).mean()); self._cache[key]=val; return val
        v=self.grid[:,None,:]-pts[None,:,:]; dist=np.sqrt((v**2).sum(-1));
        ↪ front=np.einsum("gmc,uc->gmu",v,self.U)>=0.0
        dd=np.where(front,dist[:, :, None],np.inf).min(axis=1);
        ↪ frac=np.clip((dd[:, :, None]-R_MIN)/np.maximum(bs[None, None, :]-R_MIN,1e-9),0.0,1.0)
        val=float((frac*ws[None, None, :]).sum(axis=2).mean()); self._cache[key]=val; return val
    def expected_missed(self,pts,n_unresolved:int,n_detected:int,n_total_ch=20,n_max_prior=16,n_min_prior=
        ↪ =10)->float:
        mu=self.mu(pts); ns=np.arange(n_min_prior,n_max_prior+1); logp=[]
        for n in ns:
            k=n-n_detected

```

```

        if k<0 or k>n: logp.append(-1e9); continue
        lp=k*math.log(max(mu,1e-12))+n_detected*math.log(max(1-mu,1e-12));
        ↪ lp+=math.lgamma(n+1)-math.lgamma(k+1)-math.lgamma(n_detected+1); logp.append(lp)
    logp=np.array(logp); logp-=logp.max(); p=np.exp(logp); p/=p.sum(); return
    ↪ float((p*(ns-n_detected)).sum())

class DirBelief:
    def __init__(self,nu=72): self.log=[]; self.nu=nu; self.last=None
    def add(self,q,kind,svd=None): self.log.append((np.asarray(q,float).copy(),kind,None if svd is None
    ↪ else float(svd)))
    def evaluate(self,poly,spacing=None):
        if len(poly)<3 or not self.log:return None
        P=np.asarray(poly,float); lo,hi=P.min(axis=0),P.max(axis=0)
        if spacing is None: spacing=max(14.0,math.sqrt(max((hi-lo).prod(),1.0)/1200.0))
        nx=max(2,int((hi[0]-lo[0])/spacing)+1); ny=max(2,int((hi[1]-lo[1])/spacing)+1)
        if nx*ny>2500:return None
        xs=lo[0]+spacing*np.arange(nx); ys=lo[1]+spacing*np.arange(ny); GX,GY=np.meshgrid(xs,ys);
        ↪ grid=np.stack([GX.ravel(),GY.ravel()],axis=1)
        inside=np.array([point_in_polygon(g,poly) for g in grid]); pts=grid[inside]
        if len(pts)<3:return None
        U=np.stack([np.cos(2*np.pi*np.arange(self.nu)/self.nu),np.sin(2*np.pi*np.arange(self.nu)/self.nu)]
        ↪ ,axis=1)
        ok_dir=np.ones(len(pts),dtype=bool); ok_u=np.ones((len(pts),self.nu),dtype=bool); tol=spacing*1.5
        for q,kind,svd in self.log:
            v=pts-q; d=np.linalg.norm(v,axis=1)
            if kind=="direction":
                ok_dir&=(d<=R_MAX+tol); ang=np.degrees(np.arctan2(v[:,1],v[:,0]));
                ↪ diff=np.abs((ang-svd+180.0)%360.0-180.0)
                tol_ang=np.degrees(np.arctan2(tol,np.maximum(d,1.0)))+1.0; ok_dir&=(diff<=tol_ang);
                ↪ vh=v/np.maximum(d,1e-9)[: ,None]; ok_u&=(vh@U.T<=0.02)
            else:
                sel=d<=R_MIN-tol
                if sel.any():
                    vh=v[sel]/np.maximum(d[sel],1e-9)[: ,None]; ok_u[sel]=ok_u[sel]&(vh@U.T>-0.02)
        feas=ok_dir&ok_u.any(axis=1)
        if feas.sum()<3:return None
        w_u=ok_u[feas].sum(axis=0).astype(float)
        if w_u.sum()<=0:return None
        w_u/=w_u.sum(); c=float((w_u*np.cos(2*np.pi*np.arange(self.nu)/self.nu)).sum());
        ↪ s=float((w_u*np.sin(2*np.pi*np.arange(self.nu)/self.nu)).sum())
        mean_u=math.degrees(math.atan2(s,c))%360.0; conc=math.hypot(c,s); hull=convex_hull(pts[feas])
        if len(hull)<3:return None
        region=offset_polygon(hull,spacing*1.5)
        for hp in _poly_halfplanes(poly):
            region=clip_polygon(region,hp)
            if len(region)==0:return None
        self.last={"region":region if len(region)>=3 else None,"mean_u":mean_u,"conc":conc,"u_post":w_u.c
        ↪ opy(),"p_mean":pts[feas].mean(axis=0),"spacing":spacing,"n_feasible":int(feas.sum())}
        return self.last

class RobotV2(Robot):
    def __init__(self,sim,plan,params:Params2,problem=3):
        self.plan1=[np.asarray(p,float) for p in plan["level1"]]; self.plan_esc=[np.asarray(p,float) for p
        ↪ in plan.get("escalation",[])]
        super().__init__(sim,self.plan1+self.plan_esc,params,problem=problem); self.params=params;
        ↪ self.miss_model=MissModel(problem=problem); self.beliefs={}; self.escalated=False;
        ↪ self.scanned=[]; self.n_esc_used=0; self.scan_set=[]
    def _init_region(self,ch,svd):

```

```

        super()._init_region(ch,svd)
        if self.problem==4:self.beliefs.setdefault(ch,DirBelief()).add(self.sim.pos,"direction",svd)
def _update_region(self,ch,pos,svd):
    super()._update_region(ch,pos,svd)
    if self.problem==4:self.beliefs.setdefault(ch,DirBelief()).add(pos,"direction",svd)
def _record_nosig(self,ch,q):
    super()._record_nosig(ch,q)
    if self.problem==4:self.beliefs.setdefault(ch,DirBelief()).add(q,"no_signal")
def _shadow_filter(self,ch,poly,spacing=15.0,nu=720,prob4=True): return None
def _need_scan(self)->bool:
    if not self.unknown or not self.pending:return False
    if self.params.theta_miss<=0.0:return True
    em=self.miss_model.expected_missed(self.scan_set,len(self.unknown),len(self.cleared)); return
    ↪ bool(em>self.params.theta_miss)
def _maybe_escalate(self):
    if not self.unknown:return
    if any(all(norm(np.asarray(q,float)-s)>1e-9 for s in self.scanned) for q in self.pending):return
    em0=self.miss_model.expected_missed(self.scan_set,len(self.unknown),len(self.cleared))
    if em0<=self.params.theta_miss:return
    cand=[q for q in self.plan_esc if all(norm(q-s)>1e-9 for s in self.scanned)]
    if not cand:return
    best,best_gain=None,-1.0
    for q in cand:
        em1=self.miss_model.expected_missed(list(self.scan_set)+[q],len(self.unknown),len(self.cleared))
        ↪ d)); gain=(em0-em1)/max(self._dist(q),50.0)
        if gain>best_gain:best,best_gain=q,gain
    self.pending=list(self.pending)+[np.asarray(best,float)]; self.escalated=True; self.n_esc_used+=1
def _jobs(self):
    jobs=[]
    for ch in self.tracked:
        if ch in self.cleared:continue
        poly=self.tracked[ch]; D,_=diameter_rotating_calipers_pair(poly); c=polygon_centroid(poly);
        ↪ jobs.append(("clear",ch,np.asarray(c,float),self._dist(c)+0.15*D+30.0))
    if self._need_scan():
        for P in self.pending:jobs.append(("scan",None,np.asarray(P,float),self._dist(P)+6.0*len(self)
        ↪ .unknown)))
    return jobs
def _plan_next(self,jobs):
    start=self.sim.pos.copy(); nodes=[j[2] for j in jobs]
    if len(nodes)==1:return jobs[0]
    left=list(range(len(nodes))); order=[]; cur=start
    while left:
        k=min(left,key=lambda i:norm(nodes[i]-cur)); order.append(k); left.remove(k); cur=nodes[k]
def path_len(seq):
    s=0.0; c=start
    for i in seq:s+=norm(nodes[i]-c); c=nodes[i]
    return s
improved=True
while improved:
    improved=False
    for i in range(len(order)-1):
        for j in range(i+1,len(order)):
            new=order[:i]+order[i:j+1][::-1]+order[j+1:]
            if path_len(new)<path_len(order)-1e-9:order,improved=new,True
    first=order[0]; cand=[i for i in range(len(jobs)) if norm(nodes[i]-nodes[first])<1e-6];
    ↪ cand.sort(key=lambda i:0 if jobs[i][0]=="clear" else 1)
    return jobs[cand[0]]
def _do_scan(self,P):

```

```

self.pending=[q for q in self.pending if norm(q-P)>1e-9];
↪ self.scanned.append(np.asarray(P,float)); self._scan_unknown(at=P)
if all(norm(np.asarray(s,float)-P)>1e-9 for s in
↪ self.scan_set):self.scan_set.append(np.asarray(P,float))
self.miss_model.detected_dists=self._detected_dists(); self.miss_model._update_b()
def _detected_dists(self):
    out=[]
    for ch in self.tracked:
        if ch in self.cleared:continue
        poly=self.tracked[ch]; best=0.0
        for s in self.scanned:
            dmin=min(norm(np.asarray(p,float)-np.asarray(s,float)) for p in poly); best=max(best,dmin)
        out.append(float(best))
    return out if out else [R_MIN]
def _do_clear(self,ch):
    ok=self.home_and_clear(ch)
    if not ok:
        poly=self.tracked.get(ch)
        if poly is not None and len(poly)>=3:
            D,_=diameter_rotating_calipers_pair(poly);
            ↪ n_est=polygon_area(poly)/(0.433*self.params.sweep_spacing**2)
            if n_est<=900:ok=self._clear_sweep(ch,poly)
        self.cleared.add(ch)
        if not ok:self.gave_up.append(ch)
def _next_measure_point(self,ch,poly):
    if self.problem==4 and self.params.use_belief and ch in self.beliefs and self.nosig.get(ch,0)>0:
        b=self.beliefs[ch].evaluate(poly)
        if b is not None:
            if b["region"] is not None and polygon_area(b["region"])<polygon_area(poly):
                ↪ self.tracked[ch]=_simplify(b["region"],0.05); poly=self.tracked[ch]
            if b["conc"]>0.15:
                u=unit(b["mean_u"]); goal=np.asarray(b["p_mean"],float)+self.params.front_len*u;
                ↪ v=goal-self.sim.pos; L=norm(v)
                if L>1e-6:
                    uu=v/L; n=np.array([-uu[1],uu[0]]); sgn=self.side.get(ch,1.0); self.side[ch]=-sgn;
                    ↪ step=min(L,max(120.0,0.7*L)); return self.sim.pos+step*uu+sgn*0.18*step*n
    return super()._next_measure_point(ch,poly)
def run(self):
    self._scan_unknown(); self.scanned.append(self.sim.pos.copy());
    ↪ self.scan_set.append(self.sim.pos.copy()); self.pending=[q for q in self.pending if
    ↪ self._dist(q)>1e-9]; guard=0
    while guard<4000:
        guard+=1; self._maybe_escalate(); jobs=self._jobs()
        if not jobs:break
        kind,ch,target,_=self._plan_next(jobs)
        if kind=="scan":self._do_scan(target)
        else:self._do_clear(ch)
    return self.finish()

def run_trial_v2(sources,plan,params:Params2,problem=3):
    from sim import Simulator
    sim=Simulator(sources); robot=RobotV2(sim,plan,params,problem=problem); res=robot.run()
    res["sources"]=[{"ch":s.channel,"pos":s.pos.tolist(),"R":s.radius,"kind":s.kind,"dir":s.direction_deg}
    ↪ ,{"cleared":s.cleared} for s in sources]
    res["escalated"]=robot.escalated; res["n_esc_points"]=robot.n_esc_used;
    ↪ res["scan_points"]=len(robot.scanned); return res

```

文件: vendor/portable\_sandbox/run\_bench.py

```

import sys, time, json
sys.path.insert(0, '/mnt/data/extbench/lib')
import numpy as np
from sim import generate_case
from strategy2 import Params2, run_trial_v2

strict4 = [
[0.0,0.0],[−500.0,−866.0254037844386],[−1000.0,−1732.0508075688772],[−1500.0,−866.0254037844386],[−2000.0
↪ ,0.0],[−1500.0,866.0254037844386],[−1000.0,0.0],[−500.0,866.0254037844386],[−1000.0,1732.050807568877
↪ 2],[0.0,1732.0508075688772],[500.0,866.0254037844386],[1000.0,0.0],[500.0,−866.0254037844386],[0.0,−1
↪ 732.0508075688772],[1000.0,−1732.0508075688772],[1500.0,−866.0254037844386],[2000.0,0.0],[1500.0,866.
↪ 0254037844386],[1000.0,1732.0508075688772]
]
plan={'level1':strict4,'escalation':[]}

def block(n,use_belief,tag):
    rows=[]; t0=time.time()
    for i in range(n):
        rng=np.random.default_rng(700000+i)
        srcs=generate_case(rng,4)
        r=run_trial_v2(srcs,plan,Params2(name=tag,theta_miss=0.0,use_belief=use_belief),4)
        rows.append(r)
        s=r['stats']
        print(tag,i+1,'/',n,'n=',s['n_sources'],'clear=',s['n_cleared'],'avg=',round(s['avg_time_s'] or
↪ 0,2),'gave_up=',len(r['gave_up']),flush=True)
    st=[r['stats'] for r in rows]
    out={
        'tag':tag,'n':n,
        'mean_ratio':float(np.mean([s['ratio'] for s in st])),
        'frac_full_clear':float(np.mean([s['ratio']>=1−1e−9 for s in st])),
        'mean_avg_time_s':float(np.mean([s['avg_time_s'] for s in st])),
        'aggregate_s_per_source':float(sum(s['total_time_s'] for s in st)/sum(s['n_sources'] for s in st)),
        'mean_total_time_s':float(np.mean([s['total_time_s'] for s in st])),
        'mean_move_m':float(np.mean([s['move_m'] for s in st])),
        'mean_requests':float(np.mean([s['n_requests'] for s in st])),
        'total_missed':int(sum(s['n_sources']−s['n_cleared'] for s in st)),
        'total_gave_up':int(sum(len(r['gave_up']) for r in rows)),
        'wall_s':time.time()−t0,
    }
    print('SUMMARY',json.dumps(out),flush=True)
    return out

if __name__=='__main__':
    n=int(sys.argv[1]) if len(sys.argv)>1 else 3
    out=[]
    out.append(block(n,False,'p4_v2_rho_nb'))
    out.append(block(n,True,'p4_v2_belief'))
    print(json.dumps(out,indent=2))

```

文件: vendor/portable\_sandbox/run\_best.py

```

import sys,time,json
sys.path.insert(0, '/mnt/data/extbench/lib')
import numpy as np
from sim import generate_case
from strategy2 import Params2, run_trial_v2

```

```

strict4=[[0.0,0.0],[-500.0,-866.0254037844386],[-1000.0,-1732.0508075688772],[-1500.0,-866.0254037844386]
↪ ,[-2000.0,0.0],[-1500.0,866.0254037844386],[-1000.0,0.0],[-500.0,866.0254037844386],[-1000.0,1732.050
↪ 8075688772],[0.0,1732.0508075688772],[500.0,866.0254037844386],[1000.0,0.0],[500.0,-866.0254037844386]
↪ ],[0.0,-1732.0508075688772],[1000.0,-1732.0508075688772],[1500.0,-866.0254037844386],[2000.0,0.0],[15
↪ 00.0,866.0254037844386],[1000.0,1732.0508075688772]]
plan={'level1':strict4,'escalation':[]}
n=int(sys.argv[1]) if len(sys.argv)>1 else 150
rows=[]; t0=time.time()
for i in range(n):
    rng=np.random.default_rng(700000+i); srcs=generate_case(rng,4)
    r=run_trial_v2(srcs,plan,Params2(name='v2_belief',theta_miss=0.0,use_belief=True),4); rows.append(r)
st=[r['stats'] for r in rows]
full=[s['ratio']>=1-1e-9 for s in st]
out={'n_trials':n,'mean_ratio':float(np.mean([s['ratio'] for s in st])), 'frac_full_clear':float(np.mean(f
↪ ull)), 'full_count':int(sum(full)), 'mean_avg_time_s':float(np.mean([s['avg_time_s'] for s in
↪ st])), 'median_avg_time_s':float(np.median([s['avg_time_s'] for s in
↪ st])), 'p10_avg_time_s':float(np.percentile([s['avg_time_s'] for s in
↪ st],10)), 'p90_avg_time_s':float(np.percentile([s['avg_time_s'] for s in
↪ st],90)), 'aggregate_s_per_source':float(sum(s['total_time_s'] for s in st)/sum(s['n_sources'] for s in
↪ st)), 'mean_total_time_s':float(np.mean([s['total_time_s'] for s in
↪ st])), 'mean_move_m':float(np.mean([s['move_m'] for s in
↪ st])), 'mean_requests':float(np.mean([s['n_requests'] for s in
↪ st])), 'total_sources':int(sum(s['n_sources'] for s in st)), 'total_cleared':int(sum(s['n_cleared'] for s
↪ in st)), 'total_missed':int(sum(s['n_sources']-s['n_cleared'] for s in
↪ st)), 'total_gave_up':int(sum(len(r['gave_up']) for r in rows)), 'time_breakdown':{k:float(np.mean([s[k]
↪ for s in st])) for k in ('t_move','t_switch','t_measure','t_clear')}, 'wall_s':time.time()-t0}
open('/mnt/data/extbench/v2_150_summary.json','w').write(json.dumps(out,indent=2))
print(json.dumps(out,indent=2))

```

## 文件: vendor/portable\_sandbox/run\_mincert25\_external.py

```

import sys,time,json,math,argparse
sys.path.insert(0,'/mnt/data/extbench/lib')
import numpy as np
from sim import generate_case
from strategy2 import Params2, run_trial_v2

def mincert25():
    pts=[[0.0,0.0]]
    for k in range(12):
        a=math.radians(15+30*k); pts.append([950*math.cos(a),950*math.sin(a)])
    for k in range(12):
        a=math.radians(30*k); pts.append([1875*math.cos(a),1875*math.sin(a)])
    return pts
PLAN={'level1':mincert25(),'escalation':[]}

def run(seed0,n):
    rows=[];t0=time.time()
    for i in range(n):
        srcs=generate_case(np.random.default_rng(seed0+i),4)
        r=run_trial_v2(srcs,PLAN,Params2(name='ext_rho_mincert25',theta_miss=0.0,use_belief=False),4);row
↪ s.append(r)
    st=[r['stats'] for r in rows]
    out={
        'seed0':seed0,'n':n,'full':int(sum(s['ratio']>=1-1e-9 for s in st)),
        'mean_ratio':float(np.mean([s['ratio'] for s in st])),
        'mean_avg_s_per_source':float(np.mean([s['avg_time_s'] for s in st])),
    }

```

```

        'aggregate_s_per_source':float(sum(s['total_time_s'] for s in st)/sum(s['n_sources'] for s in st)),
        'mean_total_time':float(np.mean([s['total_time_s'] for s in st])),
        'mean_move_m':float(np.mean([s['move_m'] for s in st])),
        'mean_requests':float(np.mean([s['n_requests'] for s in st])),
        'mean_scan_points':float(np.mean([r['scan_points'] for r in rows])),
        'total_missed':int(sum(s['n_sources']-s['n_cleared'] for s in st)),
        'total_gave_up':int(sum(len(r['gave_up']) for r in rows)),
        'time_breakdown':{k:float(np.mean([s[k] for s in st])) for k in
        ↪ ('t_move', 't_switch', 't_measure', 't_clear')},
        'wall_s':time.time()-t0,
    }
    print(json.dumps(out,indent=2))
if __name__=='__main__':
    ap=argparse.ArgumentParser();ap.add_argument('--seed0',type=int,default=700000);ap.add_argument('--n'
    ↪ ,type=int,default=20);a=ap.parse_args();run(a.seed0,a.n)

```

文件: vendor/portable\_sandbox/run\_q3.py

```

import sys,time,json
sys.path.insert(0,'/mnt/data/extbench/lib')
import numpy as np
from sim import generate_case
from strategy2 import Params2, run_trial_v2
strict3=[
[0.0,0.0],
[1080.8618318011463,0.0],
[673.9063293463646,845.0518083002004],
[-240.51438429012381,1053.762369034913],
[-973.8228609568137,468.96837305133465],
[-973.8228609568138,-468.9683730513344],
[-240.51438429012407,-1053.762369034913],
[673.9063293463643,-845.0518083002006],
]
plan={'level1':strict3,'escalation':[]}
start=int(sys.argv[1]) if len(sys.argv)>1 else 90000
n=int(sys.argv[2]) if len(sys.argv)>2 else 150
rows=[]; t0=time.time()
for i in range(n):
    seed=start+i
    rng=np.random.default_rng(seed); srcs=generate_case(rng,3)
    r=run_trial_v2(srcs,plan,Params2(name='p3_v2_rho',theta_miss=0.0,use_belief=False),3); rows.append(r)
st=[r['stats'] for r in rows]
full=[s['ratio']>=1-1e-9 for s in st]
out={
    'seed_start':start,'n_trials':n,
    'full_count':int(sum(full)), 'frac_full_clear':float(np.mean(full)),
    'mean_ratio':float(np.mean([s['ratio'] for s in st])),
    'mean_avg_time_s':float(np.mean([s['avg_time_s'] for s in st])),
    'median_avg_time_s':float(np.median([s['avg_time_s'] for s in st])),
    'p10_avg_time_s':float(np.percentile([s['avg_time_s'] for s in st],10)),
    'p90_avg_time_s':float(np.percentile([s['avg_time_s'] for s in st],90)),
    'aggregate_s_per_source':float(sum(s['total_time_s'] for s in st)/sum(s['n_sources'] for s in st)),
    'mean_total_time_s':float(np.mean([s['total_time_s'] for s in st])),
    'mean_move_m':float(np.mean([s['move_m'] for s in st])),
    'mean_requests':float(np.mean([s['n_requests'] for s in st])),
    'total_sources':int(sum(s['n_sources'] for s in st)),
    'total_cleared':int(sum(s['n_cleared'] for s in st)),

```

```

'total_missed':int(sum(s['n_sources']-s['n_cleared'] for s in st)),
'total_gave_up':int(sum(len(r['gave_up']) for r in rows)),
'time_breakdown':{k:float(np.mean([s[k] for s in st])) for k in
↪ ('t_move','t_switch','t_measure','t_clear')},
'wall_s':time.time()-t0,
}
print(json.dumps(out,ensure_ascii=False,indent=2))

```

## 文件: vendor/portable\_sandbox/run\_q3\_7pt.py

```

import sys,math,json,time
sys.path.insert(0,'/mnt/data/extbench')
sys.path.insert(0,'/mnt/data/extbench/lib')
import numpy as np
from strict_q3_test import StrictQ3
from sim import generate_case,Simulator
from strategy2 import Params2

r=1150.0
P=[[0.0,0.0]]+[[r*math.cos(k*math.pi/3),r*math.sin(k*math.pi/3)] for k in range(6)]
PLAN={'level1':P,'escalation':[]}

def run(seed0,n):
    rows=[];hard=[];t0=time.time()
    for i in range(n):
        seed=seed0+i; srcs=generate_case(np.random.default_rng(seed),3); sim=Simulator(srcs)
        rb=StrictQ3(sim,PLAN,Params2(name='q3_7pt',theta_miss=0.0,use_belief=False),3)
        try:r0=rb.run()
        except Exception as e:hard.append((seed,str(e)));r0=rb.finish()
        rows.append(r0)
    st=[r['stats'] for r in rows]
    out={'seed0':seed0,'n':n,'full':sum(s['ratio']>=1-1e-9 for s in st),'hard':hard,
        'agg':sum(s['total_time_s'] for s in st)/sum(s['n_sources'] for s in st),
        'meanavg':float(np.mean([s['avg_time_s'] for s in st])),
        'mean_total':float(np.mean([s['total_time_s'] for s in st])),
        'move':float(np.mean([s['move_m'] for s in st])), 'req':float(np.mean([s['n_requests'] for s in
↪ st])),
        'miss':sum(s['n_sources']-s['n_cleared'] for s in st),
        'tm':float(np.mean([s['t_move'] for s in st])), 'tmeasure':float(np.mean([s['t_measure'] for s in
↪ st])), 'tswitch':float(np.mean([s['t_switch'] for s in
↪ st])), 'tclear':float(np.mean([s['t_clear'] for s in st])),
        'wall':time.time()-t0}
    print(json.dumps(out,indent=2))

if __name__=='__main__':
    import argparse
    ap=argparse.ArgumentParser();ap.add_argument('--seed0',type=int,default=90000);ap.add_argument('--n',
↪ type=int,default=200);a=ap.parse_args();run(a.seed0,a.n)

```

## 文件: vendor/portable\_sandbox/run\_q3\_failures.py

```

import sys, json
sys.path.insert(0,'/mnt/data/extbench/lib')
import numpy as np
from sim import generate_case

```

```

from strategy2 import Params2, run_trial_v2
strict3=[[0.0,0.0],[1080.8618318011463,0.0],[673.9063293463646,845.0518083002004],[-240.51438429012381,10]
↪ 53.762369034913],[-973.8228609568137,468.96837305133465],[-973.8228609568138,-468.9683730513344],[-24]
↪ 0.51438429012407,-1053.762369034913],[673.9063293463643,-845.0518083002006]]
plan={'level1':strict3,'escalation':[]}
for seed in range(90150,90600):
    rng=np.random.default_rng(seed); srcs=generate_case(rng,3)
    r=run_trial_v2(srcs,plan,Params2(name='p3_v2_rho',theta_miss=0.0,use_belief=False),3)
    s=r['stats']
    if s['n_cleared']<s['n_sources']:
        missed=[x for x in r['sources'] if not x['cleared']]
        print(json.dumps({'seed':seed,'stats':s,'gave_up':r['gave_up'],'unresolved':r['unresolved'],'miss'
↪ ed':missed},ensure_ascii=False))

```

文件: vendor/portable\_sandbox/run\_q3\_nextgen.py

```

import sys,math,json,time,argparse
from pathlib import Path
_HERE=Path(__file__).resolve().parent
sys.path.insert(0,str(_HERE));sys.path.insert(0,str(_HERE/'lib'))
import numpy as np
import strategy
strategy.MARGIN=2.0
from strict_filtered_test import StrictFiltered
from sim import generate_case,Simulator
from strategy2 import Params2
from geom import norm, clip_polygon, diameter_rotating_calipers_pair, polygon_centroid

BASE_R=1130.0
BASE_RING=[np.array([BASE_R*math.cos(k*math.pi/3),BASE_R*math.sin(k*math.pi/3)],float) for k in range(6)]

def ring_points(n,r,phase=0.0):
    return [np.array([r*math.cos(phase+2*math.pi*k/n),r*math.sin(phase+2*math.pi*k/n)],float) for k in
↪ range(n)]

def path_len(start,nodes,order):
    cur=np.asarray(start,float);s=0.0
    for i in order:
        s+=norm(nodes[i]-cur);cur=nodes[i]
    return s

def route_order(start,nodes):
    if not nodes:return []
    left=list(range(len(nodes)));order=[];cur=np.asarray(start,float)
    while left:
        i=min(left,key=lambda j:norm(nodes[j]-cur));order.append(i);left.remove(i);cur=nodes[i]
    improved=True
    while improved:
        improved=False;base=path_len(start,nodes,order)
        for i in range(len(order)-1):
            for j in range(i+1,len(order)):
                cand=order[:i]+order[i:j+1][::-1]+order[j+1:]
                if path_len(start,nodes,cand)<base-1e-8:
                    order=cand;improved=True;break
        if improved:break
    return order

```

```

class Q3NegDynamic(StrictFiltered):
    def __init__(self,*a,neg=True,conditional=False,threshold=0,outer_n=9,outer_r=1700.0,rotate=False,**k
    ↪ w):
        super().__init__(*a,**kw)
        self.neg_enabled=neg;self.conditional=conditional;self.threshold=threshold;self.outer_n=outer_n;s
        ↪ elf.outer_r=outer_r;self.rotate=rotate
        self.neg_hist={ch:[] for ch in
        ↪ range(1,21)};self.neg_cuts=0;self.conditional_used=False;self.phase_used=0.0

    def _apply_neg(self,ch):
        if not self.neg_enabled or ch not in self.tracked:return
        positives=[np.asarray(e[0],float) for e in self.mlog.get(ch,[]) if e[1]=='direction']
        negatives=self.neg_hist.get(ch,[])
        if not positives or not negatives:return
        poly=self.tracked[ch]
        before=len(poly)
        for p in positives:
            for q in negatives:
                q=np.asarray(q,float)
                n=2.0*(q-p);c=float(q@q-p@p)
                new=clip_polygon(poly,(n,c+1e-7))
                if len(new)>=3:
                    # count only genuine geometric change by vertex/area surrogate
                    if len(new)!=len(poly) or any(norm(np.asarray(a)-np.asarray(b))>1e-7 for a,b in
                    ↪ zip(new,poly[:len(new)])):
                        self.neg_cuts+=1
                poly=new
        if len(poly)>=3:self.tracked[ch]=poly

    def _record_nosig(self,ch,q):
        super()._record_nosig(ch,q)
        q=np.asarray(q,float).copy()
        if not any(norm(q-p)<1e-8 for p in self.neg_hist[ch]):self.neg_hist[ch].append(q)
        self._apply_neg(ch)

    def _init_region(self,ch,svd):
        super()._init_region(ch,svd);self._apply_neg(ch)

    def _update_region(self,ch,pos,svd):
        super()._update_region(ch,pos,svd);self._apply_neg(ch)

    def _scan_unknown(self,at=None):
        extra=[]
        for ch,poly in self.tracked.items():
            if ch in self.cleared:continue
            D,_=diameter_rotating_calipers_pair(poly)
            if D>80.0:extra.append(ch)
        chans=sorted(self.unknown)+extra
        for k,ch in enumerate(chans):
            p=at if (k==0 and at is not None) else self.sim.pos
            res,svd=self.sim.measure(p,ch);self.n_measure+=1
            if res=='direction':
                if ch in self.tracked:self._update_region(ch,self.sim.pos,svd)
                else:self._init_region(ch,svd)
                self.ray[ch]=(self.sim.pos.copy(),svd);self.nosig[ch]=0
            elif res=='no_signal':
                q=self.sim.pos.copy()
                if not any(norm(q-x)<1e-8 for x in self.neg_hist[ch]):self.neg_hist[ch].append(q)

```

```

        if ch in self.tracked:
            super()._record_nosig(ch,q) # only log mlog, avoid duplicate _record_nosig recursion
            self._apply_neg(ch)
        elif res=='near':
            rr,_=self.sim.clear(self.sim.pos,ch);self.n_clear+=1
            if rr=='success':self.cleared.add(ch);self.unknown.discard(ch)

def _choose_outer_ring(self):
    if not self.rotate:return ring_points(self.outer_n,self.outer_r,0.0)
    known=[np.asarray(polygon_centroid(poly),float) for ch,poly in self.tracked.items() if ch not in
    ↪ self.cleared]
    best=None
    # phase only unique within one sector
    for m in range(24):
        ph=(2*math.pi/self.outer_n)*m/24.0
        ring=ring_points(self.outer_n,self.outer_r,ph)
        nodes=ring+known
        order=route_order(self.sim.pos,nodes)
        val=path_len(self.sim.pos,nodes,order)
        if best is None or val<best[0]:best=(val,ph,ring)
    self.phase_used=best[1];return best[2]

def run(self):
    # initial origin scan; retain all negative histories for future discoveries
    self._scan_unknown(); self.scanned.append(self.sim.pos.copy());
    ↪ self.scan_set.append(self.sim.pos.copy())
    known_count=len(set(self.tracked)|self.cleared)
    if self.conditional and known_count<=self.threshold:
        self.pending=self._choose_outer_ring();self.conditional_used=True
    else:
        self.pending=[p.copy() for p in BASE_RING]
    guard=0
    while guard<4000:
        guard+=1;self._maybe_escalate();jobs=self._jobs()
        if not jobs:break
        kind,ch,target,_=self._plan_next(jobs)
        if kind=='scan':self._do_scan(target)
        else:self._do_clear(ch)
    return self.finish()

PLAN={'level1':[[0.,0.]]+[[BASE_R*math.cos(k*math.pi/3),BASE_R*math.sin(k*math.pi/3)] for k in
↪ range(6)],'escalation':[]}

def run(n=200,seed0=90000,variant='base',threshold=0,outer_n=9,outer_r=1700.0):
    cfg={
        'base':dict(neg=False,conditional=False,rotate=False),
        'neg':dict(neg=True,conditional=False,rotate=False),
        'cond':dict(neg=False,conditional=True,rotate=False),
        'condneg':dict(neg=True,conditional=True,rotate=False),
        'condnegrot':dict(neg=True,conditional=True,rotate=True),
    }[variant]
    rows=[];hard=[];conds=[];cuts=[];t0=time.time()
    for i in range(n):
        srcs=generate_case(np.random.default_rng(seed0+i),3);sim=Simulator(srcs)
        rb=Q3NegDynamic(sim,PLAN,Params2(name=variant,theta_miss=0.0,use_belief=False),3,threshold=thresh
        ↪ old,outer_n=outer_n,outer_r=outer_r,**cfg)
        try:r=rb.run()
        except Exception as e:hard.append((seed0+i,str(e)));r=rb.finish()

```

```

        rows.append(r);conds.append(rb.conditional_used);cuts.append(rb.neg_cuts)
    st=[r['stats'] for r in rows]
    out={'variant':variant,'threshold':threshold,'outer_n':outer_n,'outer_r':outer_r,'n':n,'seed0':seed0,
        'full':sum(s['ratio']>=1-1e-9 for s in st),'agg':sum(s['total_time_s'] for s in
        ↪ st)/sum(s['n_sources'] for s in st),
        'meanavg':float(np.mean([s['avg_time_s'] for s in
        ↪ st])), 'mean_total':float(np.mean([s['total_time_s'] for s in st])),
        'move':float(np.mean([s['move_m'] for s in st])), 'req':float(np.mean([s['n_requests'] for s in
        ↪ st])),
        'clear':float(np.mean([s['t_clear'] for s in st])), 'measure':float(np.mean([s['t_measure'] for s
        ↪ in st])), 'switch':float(np.mean([s['t_switch'] for s in st])),
        'conditional_rate':float(np.mean(conds)), 'neg_cuts':float(np.mean(cuts)), 'miss':sum(s['n_sources']
        ↪ -s['n_cleared'] for s in st), 'hard':hard, 'wall':time.time()-t0}
    print(json.dumps(out,indent=2));return out

if __name__=='__main__':
    ap=argparse.ArgumentParser();ap.add_argument('--n',type=int,default=200);ap.add_argument('--seed0',ty
    ↪ pe=int,default=90000);ap.add_argument('--variant',choices=['base','neg','cond','condneg','condneg
    ↪ rot'],default='condnegrot');ap.add_argument('--threshold',type=int,default=0);ap.add_argument('--
    ↪ outer-n',type=int,default=9);ap.add_argument('--outer-r',type=float,default=1700.0);a=ap.parse_ar
    ↪ gs();run(a.n,a.seed0,a.variant,a.threshold,a.outer_n,a.outer_r)

```

文件: vendor/portable\_sandbox/run\_q3\_shared.py

```

import sys,json,time,math,argparse
from pathlib import Path
_HERE=Path(__file__).resolve().parent
sys.path.insert(0,str(_HERE));sys.path.insert(0,str(_HERE/'lib'))
import numpy as np
import strategy
strategy.MARGIN=2.0
from strict_filtered_test import StrictFiltered
from sim import generate_case,Simulator
from strategy2 import Params2
from geom import mec, norm, clip_polygon

class Q3SharedStrict(StrictFiltered):
    def __init__(self,*a,shared_step=100.0,min_channels=3,use_negative_cut=False,**kw):
        super().__init__(*a,**kw);self.shared_step=shared_step;self.min_channels=min_channels;self.use_negative
        ↪ _cut=use_negative_cut;self._did_initial_shared=False;self.shared_reads=0;self.shared_steps=0
    def _scan_unknown(self,at=None):
        initial = not self._did_initial_shared and len(self.scanned)==0 and at is None
        super()._scan_unknown(at=at)
        if initial:
            self._did_initial_shared=True
            self._initial_shared()
    def _initial_shared(self):
        channels=[]
        for ch in self.tracked:
            if ch in self.cleared:continue
            _,r=mec(self.tracked[ch])
            if r>100:channels.append(ch)
        if len(channels)<self.min_channels:return
        start=self.sim.pos.copy();best=None
        for k in range(24):
            ang=2*math.pi*k/24;q=start+self.shared_step*np.array([math.cos(ang),math.sin(ang)])
            gain=0.0

```

```

for ch in channels:
    center,r=mec(self.tracked[ch]);center=np.asarray(center,float)
    # Latest positive anchor. mlog initially has the origin positive.
    positives=[e for e in self.mlog.get(ch,[]) if e[1]=='direction']
    anchor=np.asarray(positives[-1][0],float) if positives else start
    a=anchor-center;b=q-center;den=norm(a)*norm(b)
    sine=abs(float(a[0]*b[1]-a[1]*b[0]))/den if den>1e-9 else 1.0
    est=2*.01754*norm(q-center)/max(.01,sine)
    gain += min(r,max(0.,r-est))
    cand=(gain,-k,q)
    if best is None or cand[:2]>best[:2]:best=cand
if best is None:return
q=best[2]
# one move, many bearings
for ch in channels:
    if ch in self.cleared or ch not in self.tracked:continue
    prev_pos=None
    positives=[e for e in self.mlog.get(ch,[]) if e[1]=='direction']
    if positives:prev_pos=np.asarray(positives[-1][0],float)
    res,svd=self.sim.measure(q,ch);self.n_measure+=1;self.shared_reads+=1
    if res=='direction':
        self.nosig[ch]=0;self.ray[ch]=(self.sim.pos.copy(),svd);self._update_region(ch,self.sim.pos,svd)
    elif res=='near':
        rr,_=self.sim.clear(self.sim.pos,ch);self.n_clear+=1
        if rr=='success':self.cleared.add(ch);self.unknown.discard(ch)
    else:
        self.nosig[ch]=self.nosig.get(ch,0)+1;self._record_nosig(ch,self.sim.pos)
        if self.use_negative_cut and prev_pos is not None and ch in self.tracked:
            # Safe Q3 affine consequence of  $|P-g| \leq R < |Q-g|$ :
            #  $2(Q-P).g \leq |Q|^2-|P|^2$  (strict truth, relaxed closed halfplane).
            n=2.0*(self.sim.pos-prev_pos);c=float(self.sim.pos@self.sim.pos-prev_pos@prev_pos)
            p2=clip_polygon(self.tracked[ch],(n,c))
            if len(p2)>=3:self.tracked[ch]=p2

self.shared_steps+=1

def make_plan(r=1150.0):
    pts=[[0.,0.]]+[[r*math.cos(k*math.pi/3),r*math.sin(k*math.pi/3)] for k in range(6)]
    return {'level1':pts,'escalation':[]}

def run(n=200,seed0=90000,r=1150.,step=100.,min_channels=3,neg=False):
    plan=make_plan(r);rows=[];hard=[];ss=[];sr=[];t0=time.time()
    for i in range(n):
        srcs=generate_case(np.random.default_rng(seed0+i),3);sim=Simulator(srcs);rb=Q3SharedStrict(sim,plan,Par
        ↪ ams2(name='q3shared',theta_miss=0.0,use_belief=False),3,shared_step=step,min_channels=min_channels,
        ↪ use_negative_cut=neg)
        try:rr=rb.run()
        except Exception as e:hard.append((seed0+i,str(e)));rr=rb.finish()
        rows.append(rr);ss.append(rb.shared_steps);sr.append(rb.shared_reads)
    st=[x['stats'] for x in rows]

```

```

out={'n':n,'seed0':seed0,'ring':r,'step':step,'min_channels':min_channels,'neg':neg,'full':sum(s['ratio']
↪ ]>=1-1e-9 for s in st),'agg':sum(s['total_time_s'] for s in st)/sum(s['n_sources'] for s in
↪ st),'meanavg':float(np.mean([s['avg_time_s'] for s in
↪ st])), 'mean_total':float(np.mean([s['total_time_s'] for s in st])), 'move':float(np.mean([s['move_m']
↪ for s in st])), 'req':float(np.mean([s['n_requests'] for s in
↪ st])), 'clear':float(np.mean([s['t_clear'] for s in st])), 'measure':float(np.mean([s['t_measure'] for s
↪ in st])), 'switch':float(np.mean([s['t_switch'] for s in st])), 'shared_steps':float(np.mean(ss)), 'sha
↪ red_reads':float(np.mean(sr)), 'miss':sum(s['n_sources']-s['n_cleared'] for s in
↪ st), 'hard':hard, 'wall':time.time()-t0}
print(json.dumps(out,indent=2));return out
if __name__=='__main__':
    ap=argparse.ArgumentParser();ap.add_argument('--n',type=int,default=200);ap.add_argument('--seed0',type=
↪ int,default=900000);ap.add_argument('--ring',type=float,default=1150);ap.add_argument('--step',type=f
↪ loat,default=100);ap.add_argument('--min_channels',type=int,default=3);ap.add_argument('--neg',actio
↪ n='store_true');a=ap.parse_args();run(a.n,a.seed0,a.ring,a.step,a.min_channels,a.neg)

```

## 文件: vendor/portable\_sandbox/run\_q4\_21pt.py

```

import sys,json,time,math
from pathlib import Path
_HERE=Path(__file__).resolve().parent
sys.path.insert(0,str(_HERE));sys.path.insert(0,str(_HERE/'lib'))
import numpy as np
import strategy
strategy.MARGIN=2.0
from strict_filtered_test import StrictFiltered
from sim import generate_case,Simulator
from strategy2 import Params2

outer=1802/math.cos(math.pi/12)
PTS=[(0.,0.)]
PTS += [(outer*math.cos(2*math.pi*k/12),outer*math.sin(2*math.pi*k/12)) for k in range(12)]
PTS += [(997.32*math.cos(2*math.pi*k/8),997.32*math.sin(2*math.pi*k/8)) for k in range(8)]
PLAN={'level1':[list(p) for p in PTS],'escalation':[]}

def run(n=20,seed0=700000,outfile=None):
    rows=[];hard=[];t0=time.time()
    for i in range(n):
        sracs=generate_case(np.random.default_rng(seed0+i),4);sim=Simulator(sracs);rb=StrictFiltered(sim,PLAN,Par
↪ ms2(name='q4_21',theta_miss=0.0,use_belief=False),4)
        try:r=rb.run()
        except Exception as e:hard.append((seed0+i,str(e)));r=rb.finish()
        rows.append(r)
    st=[r['stats'] for r in rows]
    out={'n':n,'seed0':seed0,'full':sum(s['ratio']>=1-1e-9 for s in st),'agg':sum(s['total_time_s'] for s in
↪ st)/sum(s['n_sources'] for s in st),'meanavg':float(np.mean([s['avg_time_s'] for s in
↪ st])), 'mean_total':float(np.mean([s['total_time_s'] for s in st])), 'move':float(np.mean([s['move_m']
↪ for s in st])), 'req':float(np.mean([s['n_requests'] for s in
↪ st])), 'clear':float(np.mean([s['t_clear'] for s in st])), 'measure':float(np.mean([s['t_measure'] for s
↪ in st])), 'switch':float(np.mean([s['t_switch'] for s in
↪ st])), 'miss':sum(s['n_sources']-s['n_cleared'] for s in st), 'hard':hard, 'wall':time.time()-t0}
    print(json.dumps(out,indent=2))
    if outfile:
        open(outfile,'w').write(json.dumps(out,indent=2))
    return out
if __name__=='__main__':
    import argparse

```

```
ap=argparse.ArgumentParser();ap.add_argument('--n',type=int,default=20);ap.add_argument('--seed0',type=i
↪ nt,default=700000);ap.add_argument('--outfile');a=ap.parse_args();run(a.n,a.seed0,a.outfile)
```

## 文件: vendor/portable\_sandbox/run\_q4\_23pt.py

```
import sys,json,time
from pathlib import Path
_HERE=Path(__file__).resolve().parent
sys.path.insert(0,str(_HERE));sys.path.insert(0,str(_HERE/'lib'))
import numpy as np
import strategy
strategy.MARGIN=2.0
from strict_filtered_test import StrictFiltered
from sim import generate_case,Simulator
from strategy2 import Params2

PTS=[
(0.000000,0.000000),(1864.500000,0.000000),(1614.704365,932.250000),(932.250000,1614.704365),(0.000000,18
↪ 64.500000),(-932.250000,1614.704365),(-1614.704365,932.250000),(-1864.500000,0.000000),(-1614.704365,
↪ -932.250000),(-932.250000,-1614.704365),(0.000000,-1864.500000),(932.250000,-1614.704365),(1614.704365
↪ ,-932.250000),(1076.467178,0.001385),(673.445446,707.456053),(35.090167,1039.476106),(-652.783202,727
↪ .398374),(-1076.469577,0.000000),(-804.167008,-363.249827),(-538.234788,-932.250000),(93.843216,-877.
↪ 978803),(538.234788,-932.250000),(955.618224,-253.126730)
]
PLAN={'level1':[list(p) for p in PTS],'escalation':[]}]

def run(n=100,seed0=700000):
rows=[];hard=[];t0=time.time()
for i in range(n):
srcs=generate_case(np.random.default_rng(seed0+i),4);sim=Simulator(srcs);rb=StrictFiltered(sim,PLAN,Par
↪ ams2(name='q4_23',theta_miss=0.0,use_belief=False),4)
try:r=rb.run()
except Exception as e:hard.append((seed0+i,str(e)));r=rb.finish()
rows.append(r)
st=[r['stats'] for r in rows]
out={'n':n,'seed0':seed0,'full':sum(s['ratio']>=1-1e-9 for s in st),'agg':sum(s['total_time_s'] for s in
↪ st)/sum(s['n_sources'] for s in st),'meanavg':float(np.mean([s['avg_time_s'] for s in
↪ st])),('mean_total':float(np.mean([s['total_time_s'] for s in st])),('move':float(np.mean([s['move_m']
↪ for s in st])),('req':float(np.mean([s['n_requests'] for s in
↪ st])),('clear':float(np.mean([s['t_clear'] for s in st])),('measure':float(np.mean([s['t_measure'] for s
↪ in st])),('switch':float(np.mean([s['t_switch'] for s in
↪ st])),('miss':sum(s['n_sources']-s['n_cleared'] for s in st),'hard':hard,'wall':time.time()-t0}
print(json.dumps(out,indent=2))
if __name__=='__main__':
import argparse
ap=argparse.ArgumentParser();ap.add_argument('--n',type=int,default=100);ap.add_argument('--seed0',type=
↪ int,default=700000);a=ap.parse_args();run(a.n,a.seed0)
```

## 文件: vendor/portable\_sandbox/run\_q4\_nextgen.py

```
import sys,math,json,time,argparse
from pathlib import Path
_HERE=Path(__file__).resolve().parent
sys.path.insert(0,str(_HERE));sys.path.insert(0,str(_HERE/'lib'))
import numpy as np
```

```

import strategy
strategy.MARGIN=2.0
from run_q4_shared_pair import SharedPairStrict
from run_q4_shared import PTS21
from sim import generate_case, Simulator
from strategy2 import Params2
from geom import mec, norm, polygon_centroid

# ----- safe geometry helpers -----
def circle_intersections(a,b,r):
    a=np.asarray(a,float);b=np.asarray(b,float);d=norm(b-a)
    if d<1e-9 or d>2*r:return []
    mid=(a+b)/2; h=math.sqrt(max(0.0,r*r-d*d/4)); u=(b-a)/d; n=np.array([-u[1],u[0]])
    return [mid+h*n, mid-h*n]

def closest_guaranteed_clear(poly, origin, radius=18.0, nxt=None):
    P=[np.asarray(p,float) for p in poly]; origin=np.asarray(origin,float)
    def valid(q): return all(norm(q-v)<=radius+1e-8 for v in P)
    choices=[]
    if valid(origin): choices.append(origin.copy())
    c,r=mec(P); c=np.asarray(c,float)
    if r>radius+1e-8:return None
    choices.append(c)
    targets=[origin] + ([np.asarray(nxt,float)] if nxt is not None else [])
    for target in targets:
        for v in P:
            d=norm(target-v)
            if d>1e-12:
                q=v+radius*(target-v)/d
                if valid(q): choices.append(q)
    for i,a in enumerate(P):
        for b in P[:i]:
            for q in circle_intersections(a,b,radius):
                if valid(q):choices.append(q)
    # point on origin->next segment minimizing detour while inside intersection
    if nxt is not None:
        nxt=np.asarray(nxt,float); dvec=nxt-origin; A=float(dvec@dvec)
        if A>1e-12:
            lo,hi=0.0,1.0
            for v in P:
                sv=origin-v; B=2*float(dvec@sv); C=float(sv@sv-radius*radius); disc=B*B-4*A*C
                if disc < -1e-9: lo,hi=1.0,0.0;break
                disc=max(0.0,disc); rt=math.sqrt(disc); t0=(-B-rt)/(2*A);t1=(-B+rt)/(2*A)
                lo=max(lo,t0);hi=min(hi,t1)
            if lo<=hi+1e-12:
                for t in (max(0,min(1,lo)),max(0,min(1,hi))):
                    q=origin+t*dvec
                    if valid(q):choices.append(q)
    if not choices:return None
    if nxt is None:return min(choices,key=lambda q:norm(q-origin))
    return min(choices,key=lambda q:norm(q-origin)+norm(np.asarray(nxt)-q))

class ClearRegionPair(SharedPairStrict):
    def __init__(self,*a,clear_radius=18.0,clear_lookahead=True,**kw):
        super().__init__(*a,**kw);self.clear_radius=clear_radius;self.clear_lookahead=clear_lookahead;sel
        ↪ f.clear_region_uses=0;self.clear_region_saved=0.0
    def _next_global_goal(self, exclude_ch=None):
        jobs=self._jobs()

```

```

    if not jobs: return None
    # filter current source if possible to get lookahead beyond it
    other=[j for j in jobs if not (j[0]=='clear' and j[1]==exclude_ch)]
    if not other: return None
    try: return np.asarray(self._plan_next(other)[2],float)
    except Exception: return None
def home_and_clear(self,ch):
    # same base loop, but only replace certified MEC clear target with closest point in
    ↪ guaranteed-clear region
    for _ in range(self.params.max_iter_per_source):
        poly=self.tracked.get(ch)
        if poly is None or len(poly)<3: return False
        c,r=mec(poly)
        from geom import diameter_rotating_calipers_pair, polygon_area
        D,_=diameter_rotating_calipers_pair(poly)
        if not self.params.use_tracking:
            if D>1e-9 and self._clear_sweep(ch,poly): return True
            q=self._next_measure_point(ch,poly); res,svd=self.sim.measure(q,ch); self.n_measure+=1; self.n_h
            ↪ ome_steps+=1
            if res=='direction': self._update_region(ch,q,svd)
            elif res=='near':
                r2,_=self.sim.clear(q,ch); self.n_clear+=1
                if r2=='success': return True
            continue
        if r<=self.clear_radius+1e-9:
            nxt=self._next_global_goal(ch) if self.clear_lookahead else None
            q=closest_guaranteed_clear(poly,self.sim.pos,self.clear_radius,nxt)
            if q is None: q=np.asarray(c,float)
            olddist=norm(np.asarray(c,float)-self.sim.pos); newdist=norm(q-self.sim.pos)
            self.clear_region_saved += max(0.0,olddist-newdist); self.clear_region_uses+=1
            res,_=self.sim.clear(q,ch); self.n_clear+=1
            if res=='success': return True
            self.anomaly+=1; return False
        if self.params.use_sweep and D<=self.params.sweep_max_diam and D>1e-9:
            if self._clear_sweep(ch,poly): return True
            self.anomaly+=1
            q=self._next_measure_point(ch,poly); res,svd=self.sim.measure(q,ch); self.n_measure+=1; self.n_h
            ↪ ome_steps+=1
            if res=='direction':
                self.nosig[ch]=0; self.ray[ch]=(q.copy(),svd); self._update_region(ch,q,svd)
            elif res=='near':
                r2,_=self.sim.clear(q,ch); self.n_clear+=1
                if r2=='success': return True
            else:
                self.nosig[ch]=self.nosig.get(ch,0)+1; self._record_nosig(ch,self.sim.pos)
                r3,_=self.sim.clear(self.sim.pos,ch); self.n_clear+=1
                if r3=='success': return True
                if self.problem==4:
                    D2,_=diameter_rotating_calipers_pair(poly); width2=polygon_area(poly)/D2 if D2>1e-9
                    ↪ else 0.0
                    if self.nosig[ch]>=3 and ch in self.ray:
                        if self._ray_sweep(ch): return True
                    elif self.nosig[ch]>=5 and width2>45.0:
                        if self._clear_sweep(ch,poly): return True
    return False

class EnrouteClearRegionPair(ClearRegionPair):
    def __init__(self,*a,enroute=True,enroute_min_unc=120.0,enroute_min_gain=70.0,enroute_max=5,enroute_m
    ↪ in_leg=250.0,**kw):

```

```

super().__init__(*a,**kw);self.enroute=enroute;self.enroute_min_unc=enroute_min_unc;self.enroute_
↪ min_gain=enroute_min_gain;self.enroute_max=enroute_max;self.enroute_min_leg=enroute_min_leg;s
↪ elf.enroute_reads=0;self.enroute_positive=0;self.enroute_nosig=0;self._enroute_hist={}
def _predict(self,poly,anchor,q):
    center,r=mec(poly);center=np.asarray(center,float);anchor=np.asarray(anchor,float);q=np.asarray(q
↪ ,float)
    a=anchor-center;b=q-center;den=norm(a)*norm(b)
    sine=abs(float(a[0]*b[1]-a[1]*b[0]))/den if den>1e-9 else 0.0
    if sine<0.03:return r,0.0
    est=min(r, math.tan(math.radians(1.0))*(norm(a)+norm(b))/sine)
    prob=max(0.0,min(1.0,(1500.0-norm(b))/500.0))
    return est,prob
def _collect_enroute(self,destination,active_ch=None):
    if not self.enroute:return
    start=self.sim.pos.copy();dest=np.asarray(destination,float);L=norm(dest-start)
    if L<self.enroute_min_leg:return
    dvec=dest-start
    entries=[]
    for ch,poly in list(self.tracked.items()):
        if ch in self.cleared or ch==active_ch:continue
        _,unc=mec(poly)
        if unc<self.enroute_min_unc:continue
        positives=[e for e in self.mlog.get(ch,[]) if e[1]=='direction']
        if not positives:continue
        anchor=np.asarray(positives[-1][0],float)
        attempts=self._enroute_hist.setdefault(ch,[])
        end_est,end_prob=self._predict(poly,anchor,dest)
        center=np.asarray(mec(poly)[0],float)
        ts=[.15,.3,.45,.6,.75,.9]
        tproj=float((center-start@dvec)/(L*L))
        if .08<tproj<.92:ts.append(tproj)
        best=None
        for t in ts:
            q=start+t*dvec
            if any(norm(q-p)<90 for p in attempts):continue
            if any(norm(q-np.asarray(e[0],float))<75 for e in positives):continue
            est,prob=self._predict(poly,anchor,q)
            gain=(unc-est)*prob - .75*(unc-end_est)*end_prob
            if gain<self.enroute_min_gain:continue
            score=gain/(1.0+norm(q-center)/1500.0)
            cand=(score,t,q,ch)
            if best is None or cand[0]>best[0]:best=cand
        if best:entries.append(best)
    # Pick top distinct channels and execute in path order. Points stay on same segment => zero
    ↪ movement detour.
    entries=sorted(sorted(entries,key=lambda e:e[0],reverse=True)[:self.enroute_max],key=lambda e:e[1])
    for _,t,q,ch in entries:
        if ch in self.cleared or ch not in self.tracked:continue
        self._enroute_hist[ch].append(q.copy())
        res,svd=self.sim.measure(q,ch);self.n_measure+=1;self.enroute_reads+=1
        if res=='direction':
            self.enroute_positive+=1;self.nosig[ch]=0;self.ray[ch]=(self.sim.pos.copy(),svd);self._up
↪ date_region(ch,self.sim.pos,svd)
        elif res=='near':
            rr,_=self.sim.clear(self.sim.pos,ch);self.n_clear+=1
            if rr=='success':self.cleared.add(ch);self.unknown.discard(ch)
        else:
            self.enroute_nosig+=1;self.nosig[ch]=self.nosig.get(ch,0)+1;self._record_nosig(ch,self.si
↪ m.pos)

```

```

        # generic Q4 no_signal does not contract position
def _do_scan(self,P):
    self._collect_enroute(P,None)
    return super()._do_scan(P)

def run(n=20,seed0=700000,variant='base'):
    plan={'level1':[list(p) for p in PTS21],'escalation':[]};rows=[];hard=[];extras=[];t=time.time()
    cls={'base':SharedPairStrict,'clear':ClearRegionPair,'enroute':EnrouteClearRegionPair}[variant]
    for i in range(n):
        srcs=generate_case(np.random.default_rng(seed0+i),4);sim=Simulator(srcs);rb=cls(sim,plan,Params2(
            ↪ name=variant,theta_miss=0,use_belief=False),4,shared_radius=50,fresh_only=True)
        try:r=rb.run()
        except Exception as e:hard.append((seed0+i,str(e)));r=rb.finish()
        rows.append(r)
        extras.append({
            'clear_region_uses':getattr(rb,'clear_region_uses',0),'clear_region_saved_m':getattr(rb,'clea
            ↪ r_region_saved',0.0),
            'enroute_reads':getattr(rb,'enroute_reads',0),'enroute_positive':getattr(rb,'enroute_positive'
            ↪ ',0),'enroute_nosig':getattr(rb,'enroute_nosig',0),
            'pair_actions':getattr(rb,'pair_actions',0),'pair_clips':getattr(rb,'pair_clips',0)
        })
    st=[r['stats'] for r in rows]
    out={'variant':variant,'n':n,'seed0':seed0,'full':sum(s['ratio']>=1-1e-9 for s in
    ↪ st),'agg':sum(s['total_time_s'] for s in st)/sum(s['n_sources'] for s in
    ↪ st),'meanavg':float(np.mean([s['avg_time_s'] for s in
    ↪ st])),('mean_total':float(np.mean([s['total_time_s'] for s in
    ↪ st])),('move':float(np.mean([s['move_m'] for s in st])),('req':float(np.mean([s['n_requests'] for s
    ↪ in st])),('clear':float(np.mean([s['t_clear'] for s in
    ↪ st])),('measure':float(np.mean([s['t_measure'] for s in st])),('switch':float(np.mean([s['t_switch']
    ↪ for s in st])),('miss':sum(s['n_sources']-s['n_cleared'] for s in
    ↪ st),'hard':hard,'wall':time.time()-t}
    for k in extras[0]:out[k]=float(np.mean([e[k] for e in extras]))
    print(json.dumps(out,indent=2));return out

if __name__=='__main__':
    ap=argparse.ArgumentParser();ap.add_argument('--n',type=int,default=20);ap.add_argument('--seed0',typ
    ↪ e=int,default=700000);ap.add_argument('--variant',choices=['base','clear','enroute'],default='enr
    ↪ oute');a=ap.parse_args();run(a.n,a.seed0,a.variant)

```

文件: vendor/portable\_sandbox/run\_q4\_pairlocal.py

```

import sys,math,json,time,argparse
from pathlib import Path
_HERE=Path(__file__).resolve().parent
sys.path.insert(0,str(_HERE));sys.path.insert(0,str(_HERE/'lib'))
import numpy as np
import strategy
strategy.MARGIN=2.0
from run_q4_shared import Shared50Strict,PTS21
from sim import generate_case,Simulator
from strategy2 import Params2
from geom import norm,mec,clip_polygon,unit,polygon_centroid

class PairLocalStrict(Shared50Strict):
    def __init__(self,*a,rounds=5,quantile=.2,width_frac=.04,explore=55.0,**kw):
        super().__init__(*a,**kw)

```

```

self.rounds=rounds;self.quantile=quantile;self.width_frac=width_frac;self.explore=explore
self.pair_actions=0;self.pair_clips=0;self.explore_clears=0;self.opt_fallbacks=0

def _pair(self,ch,poly):
    if ch not in self.ray:return None
    apex,th=self.ray[ch];apex=np.asarray(apex,float);u=unit(th);side=np.array([-u[1],u[0]])
    projs=[float((np.asarray(p,float)-apex)@u) for p in poly];lo,hi=min(projs),max(projs)
    if hi-lo<1e-6:return None
    step=lo+self.quantile*(hi-lo)
    width=max(15.,min(140.,self.width_frac*step));width=max(width,step*math.tan(math.radians(1.0))+.02)
    P=np.asarray(poly,float)
    pair=None
    for _ in range(24):
        mid=apex+step*u;pair=[mid-width*side,mid+width*side]
        if max(norm(q-v) for q in pair for v in P)<999.9:break
        step=.5*step+.25*(lo+hi);width=max(width,step*math.tan(math.radians(1.0))+.02)
    all_in=max(norm(q-v) for q in pair for v in P)<999.9
    pair.sort(key=lambda q:norm(q-self.sim.pos))
    return pair,apex,u,step,all_in

def _do_clear(self,ch):
    attempted_explore=False;attempted=set()
    for _ in range(self.rounds):
        poly=self.tracked.get(ch)
        if poly is None or len(poly)<3:raise RuntimeError('missing feasible region')
        c,r=mec(poly);c=np.asarray(c,float)
        if r<=18.0:
            rr,_=self.sim.clear(c,ch);self.n_clear+=1
            if rr=='success':self.cleared.add(ch);return
            # numerical mismatch: do not certify; fall back later
        elif r<=self.explore and not attempted_explore:
            attempted_explore=True;self.explore_clears+=1
            q=np.asarray(polygon_centroid(poly),float);rr,_=self.sim.clear(q,ch);self.n_clear+=1
            if rr=='success':self.cleared.add(ch);return
        made=self._pair(ch,poly)
        if made is None:break
        pair,apex,u,step,all_in=made
        negative=[]
        for q in pair:
            key=(round(float(q[0]),6),round(float(q[1]),6))
            if key in attempted:continue
            attempted.add(key)
            res,svd=self.sim.measure(q,ch);self.n_measure+=1;self.n_home_steps+=1;self.pair_actions+=1
            if res=='direction':
                self.nosig[ch]=0;self.ray[ch]=(self.sim.pos.copy(),svd);self._update_region(ch,self.s
                ↪ im.pos,svd)
                break
            elif res=='near':
                rr,_=self.sim.clear(self.sim.pos,ch);self.n_clear+=1
                if rr=='success':self.cleared.add(ch);return
            else:
                self.nosig[ch]=self.nosig.get(ch,0)+1;self._record_nosig(ch,self.sim.pos);negative.ap
                ↪ pend(np.asarray(self.sim.pos,float).copy())
        if len(negative)==2 and all_in and step>0 and ch in self.tracked:
            constant=float(u@apex+step)
            new=clip_polygon(self.tracked[ch],(u,constant))
            if len(new)>=3:
                self.tracked[ch]=new;self.pair_clips+=1;self.nosig[ch]=0

```

```

        # loop and re-evaluate geometry
        # guaranteed optical fallback
        poly=self.tracked.get(ch)
        if poly is None or len(poly)<3:raise RuntimeError('no region before fallback')
        self.opt_fallbacks+=1
        for q in self._safe_lattice(poly,self.params.sweep_spacing):
            rr,_=self.sim.clear(q,ch);self.n_clear+=1
            if rr=='success':self.cleared.add(ch);return
        raise RuntimeError(f'safe lattice exhausted ch={ch}')

def run(n=20,seed0=700000,rounds=5,q=.2,w=.04,explore=55.0):
    plan={'level1':[list(p) for p in PTS21],'escalation':[]};rows=[];hard=[];extra=[];t=time.time()
    for i in range(n):
        srcs=generate_case(np.random.default_rng(seed0+i),4);sim=Simulator(srcs)
        rb=PairLocalStrict(sim,plan,Params2(name='pairlocal',theta_miss=0,use_belief=False),4,shared_radi
        ↪ us=50,fresh_only=True,rounds=rounds,quantile=q,width_frac=w,explore=explore)
        try:r=rb.run()
        except Exception as e:hard.append((seed0+i,str(e)));r=rb.finish()
        rows.append(r);extra.append((rb.pair_actions,rb.pair_clips,rb.explore_clears,rb.opt_fallbacks))
    st=[r['stats'] for r in rows]
    out={'n':n,'seed0':seed0,'rounds':rounds,'q':q,'w':w,'explore':explore,'full':sum(s['ratio']>=1-1e-9
    ↪ for s in st),
        'agg':sum(s['total_time_s'] for s in st)/sum(s['n_sources'] for s in
        ↪ st),'meanavg':float(np.mean([s['avg_time_s'] for s in
        ↪ st])), 'mean_total':float(np.mean([s['total_time_s'] for s in st])),
        'move':float(np.mean([s['move_m'] for s in st])), 'req':float(np.mean([s['n_requests'] for s in
        ↪ st])), 'clear':float(np.mean([s['t_clear'] for s in
        ↪ st])), 'measure':float(np.mean([s['t_measure'] for s in
        ↪ st])), 'switch':float(np.mean([s['t_switch'] for s in st])),
        'pair_actions':float(np.mean([x[0] for x in extra])), 'pair_clips':float(np.mean([x[1] for x in
        ↪ extra])), 'explore_clears':float(np.mean([x[2] for x in
        ↪ extra])), 'opt_fallbacks':float(np.mean([x[3] for x in extra])),
        'miss':sum(s['n_sources']-s['n_cleared'] for s in st),'hard':hard,'wall':time.time()-t}
    print(json.dumps(out,indent=2));return out

if __name__=='__main__':
    ap=argparse.ArgumentParser();ap.add_argument('--n',type=int,default=20);ap.add_argument('--seed0',typ
    ↪ e=int,default=700000);ap.add_argument('--rounds',type=int,default=5);ap.add_argument('--q',type=f
    ↪ loat,default=.2);ap.add_argument('--w',type=float,default=.04);ap.add_argument('--explore',type=f
    ↪ loat,default=55.0);a=ap.parse_args();run(a.n,a.seed0,a.rounds,a.q,a.w,a.explore)

```

文件: vendor/portable\_sandbox/run\_q4\_routing\_ablate.py

```

import sys,math,json,time,argparse
from pathlib import Path
_HERE=Path(__file__).resolve().parent
sys.path.insert(0,str(_HERE));sys.path.insert(0,str(_HERE/'lib'))
import numpy as np
import strategy
strategy.MARGIN=2.0
from run_q4_shared import Shared50Strict,PTS21
from run_q4_shared_pair import SharedPairStrict
from run_q4_pairlocal import PairLocalStrict
from run_q4_routingplus_fast import nearest_seed,insertion_seed,two_opt,or_opt,route_length
from sim import generate_case,Simulator
from strategy2 import Params2

```

```

from geom import norm

class PlannerMixin:
    def __init__(self,*a,**kw):
        super().__init__(*a,**kw);self._route_keys2=[];self.rp_improve=0.;self.rp_calls=0
    def _key(self,j):
        kind,ch,p,_=j
        return ('clear',int(ch)) if kind=='clear' else ('scan',round(float(p[0]),5),round(float(p[1]),5))
    def _plan_next(self,jobs):
        if len(jobs)==1:return jobs[0]
        points=[np.asarray(j[2],float) for j in jobs]+[self.sim.pos.copy()]
        matrix=[[norm(a-b) for b in points] for a in points]
        seeds=[nearest_seed(matrix),insertion_seed(matrix)]
        ordered=sorted(range(len(jobs)),key=lambda i:math.atan2(points[i][1],points[i][0]))
        if ordered:
            cut=min(range(len(ordered)),key=lambda k:matrix[-1][ordered[k]])
            angular=ordered[cut:]+ordered[:cut];seeds.append(angular);seeds.append([angular[0]]+list(reversed(angular[1:])))
        if self._route_keys2:
            mapping={self._key(g):i for i,g in enumerate(jobs)}
            warm=[mapping[k] for k in self._route_keys2 if k in mapping]
            for point in range(len(jobs)):
                if point in warm:continue
                best=min(range(len(warm)+1),key=lambda i:matrix[-1 if i==0 else
                    ↪ warm[i-1]][point]+(matrix[point][warm[i]]-matrix[-1 if i==0 else warm[i-1]][warm[i]]
                    ↪ if i<len(warm) else 0))
                warm.insert(best,point)
            if len(warm)==len(jobs):seeds.append(warm)
        base=route_length(seeds[0],matrix)
        routes=[two_opt(seed,matrix) for seed in seeds]
        route=min(routes,key=lambda
            ↪ r:route_length(r,matrix));route=or_opt(route,matrix,max_moves=12,max_chain=3)
        self.rp_improve+=max(0.,base-route_length(route,matrix));self.rp_calls+=1
        self._route_keys2=[self._key(jobs[i]) for i in route]
        return jobs[route[0]]

class RoutingShared(PlannerMixin,Shared50Strict): pass
class RoutingPair(PlannerMixin,SharedPairStrict): pass
class RoutingPairLocal(PlannerMixin,PairLocalStrict): pass

CLASSES={'shared':RoutingShared,'pair':RoutingPair,'pairlocal':RoutingPairLocal}

def run(n=20,seed0=700000,variant='pair'):
    plan={'level1':[list(p) for p in
        ↪ PTS21],'escalation':[]};rows=[];hard=[];extras=[];t=time.time();cls=CLASSES[variant]
    for i in range(n):
        srcs=generate_case(np.random.default_rng(seed0+i),4);sim=Simulator(srcs)
        kw=dict(shared_radius=50,fresh_only=True)
        if variant=='pairlocal':kw.update(rounds=5,quantile=.2,width_frac=.04,explore=55.0)
        rb=cls(sim,plan,Params2(name=variant,theta_miss=0,use_belief=False),4,**kw)
        try:r=rb.run()
        except Exception as e:hard.append((seed0+i,str(e)));r=rb.finish()
        rows.append(r);extras.append((getattr(rb,'rp_improve',0.),getattr(rb,'pair_actions',0.),getattr(r,
            ↪ b,'pair_clips',0.)))
    st=[r['stats'] for r in rows]

```

```

out={'variant':variant,'n':n,'seed0':seed0,'full':sum(s['ratio']>=1-1e-9 for s in
→ st),'agg':sum(s['total_time_s'] for s in st)/sum(s['n_sources'] for s in
→ st),'meanavg':float(np.mean([s['avg_time_s'] for s in
→ st])), 'mean_total':float(np.mean([s['total_time_s'] for s in
→ st])), 'move':float(np.mean([s['move_m'] for s in st])), 'req':float(np.mean([s['n_requests'] for s
→ in st])), 'clear':float(np.mean([s['t_clear'] for s in
→ st])), 'measure':float(np.mean([s['t_measure'] for s in st])), 'switch':float(np.mean([s['t_switch']
→ for s in st])), 'planner_improve_m':float(np.mean([x[0] for x in
→ extras])), 'pair_actions':float(np.mean([x[1] for x in extras])), 'pair_clips':float(np.mean([x[2]
→ for x in extras])), 'miss':sum(s['n_sources']-s['n_cleared'] for s in
→ st),'hard':hard,'wall':time.time()-t}
print(json.dumps(out,indent=2));return out
if __name__=='__main__':
    ap=argparse.ArgumentParser();ap.add_argument('--variant',choices=list(CLASSES),default='pair');ap.add
→ _argument('--n',type=int,default=20);ap.add_argument('--seed0',type=int,default=700000);a=ap.pars
→ e_args();run(a.n,a.seed0,a.variant)

```

## 文件: vendor/portable\_sandbox/run\_q4\_routingplus.py

```

import sys,math,json,time,argparse
from pathlib import Path
_HERE=Path(__file__).resolve().parent
sys.path.insert(0,str(_HERE));sys.path.insert(0,str(_HERE/'lib'))
import numpy as np
import strategy
strategy.MARGIN=2.0
from run_q4_shared_pair import SharedPairStrict
from run_q4_shared import PTS21
from sim import generate_case,Simulator
from strategy2 import Params2
from geom import norm

class RoutingPlusPair(SharedPairStrict):
    def __init__(self,*a,oropt=True,warm=True,starts=4,**kw):
        super().__init__(*a,**kw);self.rp_oropt=oropt;self.rp_warm=warm;self.rp_starts=starts;self._rp_pr
        ↪ ev=[];self.rp_improve=0.0

    def _key(self,j):
        kind,ch,p,_=j
        if kind=='clear': return ('clear',int(ch))
        return ('scan',round(float(p[0]),5),round(float(p[1]),5))

    @staticmethod
    def _path(start,nodes,route):
        cur=start;s=0.0
        for i in route:s+=norm(nodes[i]-cur);cur=nodes[i]
        return s

    def _nearest(self,start,nodes,first=None):
        rem=set(range(len(nodes)));route=[];cur=start
        if first is not None:route=[first];rem.remove(first);cur=nodes[first]
        while rem:
            i=min(rem,key=lambda k:(norm(nodes[k]-cur),k));route.append(i);rem.remove(i);cur=nodes[i]
        return route

    def _insert(self,start,nodes):
        n=len(nodes);rem=set(range(n))

```

```

if not rem: return []
first=max(rem,key=lambda i: norm(nodes[i]-start)); route=[first]; rem.remove(first)
while rem:
    best=None
    for p in rem:
        for idx in range(len(route)+1):
            a=start if idx==0 else nodes[route[idx-1]]
            b=None if idx==len(route) else nodes[route[idx]]
            extra=norm(nodes[p]-a)+(0 if b is None else norm(nodes[p]-b)-norm(a-b))
            cand=(extra,p,idx)
            if best is None or cand<best: best=cand
        _,p,idx=best; route.insert(idx,p); rem.remove(p)
    return route

def _twoopt(self,start,nodes,route,max_moves=80):
    route=list(route)
    for _ in range(max_moves):
        base=self._path(start,nodes,route); best=base; chg=None
        for i in range(len(route)-1):
            for j in range(i+1,len(route)):
                cand=route[:i]+route[i:j+1][::-1]+route[j+1:]
                L=self._path(start,nodes,cand)
                if L<best-1e-8: best=L; chg=(i,j)
        if chg is None: break
        i,j=chg; route[i:j+1]=reversed(route[i:j+1])
    return route

def _oropt(self,start,nodes,route,max_moves=12,max_chain=3):
    route=list(route); n=len(route)
    for _ in range(max_moves):
        base=self._path(start,nodes,route); best=base; best_route=None
        for k in range(1,min(max_chain,n-1)+1):
            for i in range(n-k+1):
                chain=route[i:i+k]; rest=route[:i]+route[i+k:]
                for j in range(len(rest)+1):
                    for rev in ([False,True] if k>1 else [False]):
                        c=chain[::-1] if rev else chain
                        cand=rest[:j]+c+rest[j:]
                        L=self._path(start,nodes,cand)
                        if L<best-1e-8: best=L; best_route=cand
        if best_route is None: break
        route=self._twoopt(start,nodes,best_route,max_moves=20)
    return route

def _plan_next(self,jobs):
    if len(jobs)<=1: return jobs[0]
    start=self.sim.pos.copy(); nodes=[np.asarray(j[2],float) for j in jobs]
    seeds=[self._nearest(start,nodes)]
    if self.rp_starts>1: seeds.append(self._insert(start,nodes))
    if self.rp_starts>2:
        angular=sorted(range(len(nodes)),key=lambda i: math.atan2(nodes[i][1],nodes[i][0]))
        cut=min(range(len(angular)),key=lambda
            ↪ k: norm(nodes[angular[k]]-start)); angular=angular[cut:]+angular[:cut]
        seeds.append(angular)
        if self.rp_starts>3 and len(angular)>1: seeds.append([angular[0]]+angular[:0:-1])
    if self.rp_warm and self._rp_prev:
        mapping={self._key(j):i for i,j in enumerate(jobs)}
        warm=[mapping[k] for k in self._rp_prev if k in mapping]

```

```

        for i in range(len(jobs)):
            if i in warm:continue
            bestidx=min(range(len(warm)+1),key=lambda idx: norm(nodes[i]-(start if idx==0 else
↪ nodes[warm[idx-1]])) + (0 if idx==len(warm) else
↪ norm(nodes[i]-nodes[warm[idx]])-norm((start if idx==0 else
↪ nodes[warm[idx-1]])-nodes[warm[idx]])))
            warm.insert(bestidx,i)
            if len(warm)==len(jobs):seeds.append(warm)
        routes=[]
        for s in seeds:
            r=self._twoopt(start,nodes,s)
            routes.append(r)
        route=min(routes,key=lambda r:self._path(start,nodes,r))
        before=self._path(start,nodes,seeds[0])
        if self.rp_oropt:route=self._oropt(start,nodes,route)
        after=self._path(start,nodes,route);self.rp_improve+=max(0,before-after)
        self._rp_prev=[self._key(jobs[i]) for i in route]
        return jobs[route[0]]

def run(n=20,seed0=700000,oropt=True,warm=True,starts=4):
    plan={'level1':[list(p) for p in PTS21],'escalation':[]};rows=[];hard=[];imp=[];t=time.time()
    for i in range(n):
        srcs=generate_case(np.random.default_rng(seed0+i),4);sim=Simulator(srcs);rb=RoutingPlusPair(sim,p
↪ lan,Params2(name='rp',theta_miss=0,use_belief=False),4,shared_radius=50,fresh_only=True,oropt
↪ =oropt,warm=warm,starts=starts)
        try:r=rb.run()
        except Exception as e:hard.append((seed0+i,str(e)));r=rb.finish()
        rows.append(r);imp.append(rb.rp_improve)
    st=[r['stats'] for r in rows]
    out={'n':n,'seed0':seed0,'oropt':oropt,'warm':warm,'starts':starts,'full':sum(s['ratio']>=1-1e-9 for s
↪ in st),'agg':sum(s['total_time_s'] for s in st)/sum(s['n_sources'] for s in
↪ st),'meanavg':float(np.mean([s['avg_time_s'] for s in
↪ st])), 'mean_total':float(np.mean([s['total_time_s'] for s in
↪ st])), 'move':float(np.mean([s['move_m'] for s in st])), 'req':float(np.mean([s['n_requests'] for s
↪ in st])), 'clear':float(np.mean([s['t_clear'] for s in
↪ st])), 'measure':float(np.mean([s['t_measure'] for s in st])), 'switch':float(np.mean([s['t_switch']
↪ for s in st])), 'planner_improve_m':float(np.mean(imp)), 'miss':sum(s['n_sources']-s['n_cleared']
↪ for s in st), 'hard':hard, 'wall':time.time()-t}
    print(json.dumps(out,indent=2));return out
if __name__=='__main__':
    ap=argparse.ArgumentParser();ap.add_argument('--n',type=int,default=20);ap.add_argument('--seed0',typ
↪ e=int,default=700000);ap.add_argument('--no-oropt',action='store_true');ap.add_argument('--no-war
↪ m',action='store_true');ap.add_argument('--starts',type=int,default=4);a=ap.parse_args();run(a.n,
↪ a.seed0,not a.no_oropt,not a.no_warm,a.starts)

```

文件: vendor/portable\_sandbox/run\_q4\_routingplus\_bound.py

```

import sys,math,json,time,argparse
from pathlib import Path
_HERE=Path(__file__).resolve().parent
sys.path.insert(0,str(_HERE));sys.path.insert(0,str(_HERE/'lib'))
import numpy as np
import strategy
strategy.MARGIN=2.0
from run_q4_shared_pair_bound import SharedPairBoundStrict
from run_q4_shared import PTS21
from sim import generate_case,Simulator

```

```

from strategy2 import Params2
from geom import norm

class RoutingPlusPair(SharedPairBoundStrict):
    def __init__(self,*a,oropt=True,warm=True,starts=4,**kw):
        super().__init__(*a,**kw);self.rp_oropt=oropt;self.rp_warm=warm;self.rp_starts=starts;self.rp_pr
        ↪ ev=[];self.rp_improve=0.0

    def _key(self,j):
        kind,ch,p,_=j
        if kind=='clear': return ('clear',int(ch))
        return ('scan',round(float(p[0]),5),round(float(p[1]),5))

    @staticmethod
    def _path(start,nodes,route):
        cur=start;s=0.0
        for i in route:s+=norm(nodes[i]-cur);cur=nodes[i]
        return s

    def _nearest(self,start,nodes,first=None):
        rem=set(range(len(nodes)));route=[];cur=start
        if first is not None:route=[first];rem.remove(first);cur=nodes[first]
        while rem:
            i=min(rem,key=lambda k:(norm(nodes[k]-cur),k));route.append(i);rem.remove(i);cur=nodes[i]
        return route

    def _insert(self,start,nodes):
        n=len(nodes);rem=set(range(n))
        if not rem:return []
        first=max(rem,key=lambda i:norm(nodes[i]-start));route=[first];rem.remove(first)
        while rem:
            best=None
            for p in rem:
                for idx in range(len(route)+1):
                    a=start if idx==0 else nodes[route[idx-1]]
                    b=None if idx==len(route) else nodes[route[idx]]
                    extra=norm(nodes[p]-a)+(0 if b is None else norm(nodes[p]-b)-norm(a-b))
                    cand=(extra,p,idx)
                    if best is None or cand<best:best=cand
            _,p,idx=best;route.insert(idx,p);rem.remove(p)
        return route

    def _twoopt(self,start,nodes,route,max_moves=80):
        route=list(route)
        for _ in range(max_moves):
            base=self._path(start,nodes,route);best=base;chg=None
            for i in range(len(route)-1):
                for j in range(i+1,len(route)):
                    cand=route[:i]+route[i:j+1][::-1]+route[j+1:]
                    L=self._path(start,nodes,cand)
                    if L<best-1e-8:best=L;chg=(i,j)
            if chg is None:break
            i,j=chg;route[i:j+1]=reversed(route[i:j+1])
        return route

    def _oropt(self,start,nodes,route,max_moves=12,max_chain=3):
        route=list(route);n=len(route)
        for _ in range(max_moves):

```

```

        base=self._path(start,nodes,route);best=base;best_route=None
        for k in range(1,min(max_chain,n-1)+1):
            for i in range(n-k+1):
                chain=route[i:i+k];rest=route[:i]+route[i+k:]
                for j in range(len(rest)+1):
                    for rev in ([False,True] if k>1 else [False]):
                        c=chain[::-1] if rev else chain
                        cand=rest[:j]+c+rest[j:]
                        L=self._path(start,nodes,cand)
                        if L<best-1e-8:best=L;best_route=cand
            if best_route is None:break
        route=self._twoopt(start,nodes,best_route,max_moves=20)
    return route

def _plan_next(self,jobs):
    if len(jobs)<=1:return jobs[0]
    start=self.sim.pos.copy();nodes=[np.asarray(j[2],float) for j in jobs]
    seeds=[self._nearest(start,nodes)]
    if self.rp_starts>1:seeds.append(self._insert(start,nodes))
    if self.rp_starts>2:
        angular=sorted(range(len(nodes)),key=lambda i:math.atan2(nodes[i][1],nodes[i][0]))
        cut=min(range(len(angular)),key=lambda
        ↪ k:norm(nodes[angular[k]]-start));angular=angular[cut:]+angular[:cut]
        seeds.append(angular)
        if self.rp_starts>3 and len(angular)>1:seeds.append([angular[0]]+angular[:0:-1])
    if self.rp_warm and self._rp_prev:
        mapping={self._key(j):i for i,j in enumerate(jobs)}
        warm=[mapping[k] for k in self._rp_prev if k in mapping]
        for i in range(len(jobs)):
            if i in warm:continue
            bestidx=min(range(len(warm)+1),key=lambda idx: norm(nodes[i]-(start if idx==0 else
            ↪ nodes[warm[idx-1]])) + (0 if idx==len(warm) else
            ↪ norm(nodes[i]-nodes[warm[idx]])-norm((start if idx==0 else
            ↪ nodes[warm[idx-1]])-nodes[warm[idx]])))
            warm.insert(bestidx,i)
        if len(warm)==len(jobs):seeds.append(warm)
    routes=[]
    for s in seeds:
        r=self._twoopt(start,nodes,s)
        routes.append(r)
    route=min(routes,key=lambda r:self._path(start,nodes,r))
    before=self._path(start,nodes,seeds[0])
    if self.rp_oropt:route=self._oropt(start,nodes,route)
    after=self._path(start,nodes,route);self.rp_improve+=max(0,before-after)
    self._rp_prev=[self._key(jobs[i]) for i in route]
    return jobs[route[0]]

def run(n=20,seed0=700000,oropt=True,warm=True,starts=4):
    plan={'level1':[list(p) for p in PTS21],'escalation':[]};rows=[];hard=[];imp=[];t=time.time()
    for i in range(n):
        sracs=generate_case(np.random.default_rng(seed0+i),4);sim=Simulator(sracs);rb=RoutingPlusPair(sim,p
        ↪ lan,Params2(name='rp',theta_miss=0,use_belief=False),4,shared_radius=50,fresh_only=True,oropt
        ↪ =oropt,warm=warm,starts=starts)
        try:r=rb.run()
        except Exception as e:hard.append((seed0+i,str(e)));r=rb.finish()
        rows.append(r);imp.append(rb.rp_improve)
    st=[r['stats'] for r in rows]

```

```

out={'n':n,'seed0':seed0,'oropt':oropt,'warm':warm,'starts':starts,'full':sum(s['ratio']>=1-1e-9 for s
→ in st),'agg':sum(s['total_time_s'] for s in st)/sum(s['n_sources'] for s in
→ st),'meanavg':float(np.mean([s['avg_time_s'] for s in
→ st])), 'mean_total':float(np.mean([s['total_time_s'] for s in
→ st])), 'move':float(np.mean([s['move_m'] for s in st])), 'req':float(np.mean([s['n_requests'] for s
→ in st])), 'clear':float(np.mean([s['t_clear'] for s in
→ st])), 'measure':float(np.mean([s['t_measure'] for s in st])), 'switch':float(np.mean([s['t_switch']
→ for s in st])), 'planner_improve_m':float(np.mean(imp)), 'miss':sum(s['n_sources']-s['n_cleared']
→ for s in st), 'hard':hard, 'wall':time.time()-t}
print(json.dumps(out,indent=2));return out
if __name__=='__main__':
    ap=argparse.ArgumentParser();ap.add_argument('--n',type=int,default=20);ap.add_argument('--seed0',typ
→ e=int,default=700000);ap.add_argument('--no-oropt',action='store_true');ap.add_argument('--no-war
→ m',action='store_true');ap.add_argument('--starts',type=int,default=4);a=ap.parse_args();run(a.n,
→ a.seed0,not a.no_oropt,not a.no_warm,a.starts)

```

文件: vendor/portable\_sandbox/run\_q4\_routingplus\_enroute.py

```

import sys,math,json,time,argparse
from pathlib import Path
_HERE=Path(__file__).resolve().parent
sys.path.insert(0,str(_HERE));sys.path.insert(0,str(_HERE/'lib'))
import numpy as np
import strategy
strategy.MARGIN=2.0
from run_q4_nextgen import EnrouteClearRegionPair
from run_q4_shared import PTS21
from run_q4_routingplus_fast import nearest_seed,insertion_seed,two_opt,or_opt,route_length
from sim import generate_case,Simulator
from strategy2 import Params2
from geom import norm

class RoutingPlusEnroute(EnrouteClearRegionPair):
    def __init__(self,*a,oropt_on=True,warm=True,starts=4,**kw):
        super().__init__(*a,**kw);self.rp_oropt=oropt_on;self.rp_warm=warm;self.rp_starts=starts;self._ro
        ute_keys2=[];self.rp_improve=0.;self.rp_calls=0
    def _key(self,j):
        kind,ch,p,_=j
        return ('clear',int(ch)) if kind=='clear' else ('scan',round(float(p[0]),5),round(float(p[1]),5))
    def _plan_next(self,jobs):
        if len(jobs)==1:return jobs[0]
        points=[np.asarray(j[2],float) for j in jobs]+[self.sim.pos.copy()]
        matrix=[[norm(a-b) for b in points] for a in points]
        seeds=[nearest_seed(matrix)]
        if self.rp_starts>1:seeds.append(insertion_seed(matrix))
        if self.rp_starts>2:
            ordered=sorted(range(len(jobs)),key=lambda i:math.atan2(points[i][1],points[i][0]))
            if ordered:
                cut=min(range(len(ordered)),key=lambda k:matrix[-1][ordered[k]])
                angular=ordered[cut:]+ordered[:cut];seeds.append(angular)
                if self.rp_starts>3:seeds.append([angular[0]]+list(reversed(angular[1:])))
        if self.rp_warm and self._route_keys2:
            mapping={self._key(g):i for i,g in enumerate(jobs)}
            warm=[mapping[k] for k in self._route_keys2 if k in mapping]
            for point in range(len(jobs)):
                if point in warm:continue

```

```

        best=min(range(len(warm)+1),key=lambda i:matrix[-1 if i==0 else
↪ warm[i-1]][point]+(matrix[point][warm[i]]-matrix[-1 if i==0 else warm[i-1]][warm[i]]
↪ if i<len(warm) else 0))
        warm.insert(best,point)
        if len(warm)==len(jobs):seeds.append(warm)
        base=route_length(seeds[0],matrix);routes=[two_opt(seed,matrix) for seed in
↪ seeds];route=min(routes,key=lambda r:route_length(r,matrix))
        if self.rp_oropt:route=or_opt(route,matrix,max_moves=12,max_chain=3)
        final=route_length(route,matrix);self.rp_improve+=max(0.,base-final);self.rp_calls+=1
        self._route_keys2=[self._key(jobs[i]) for i in route]
        return jobs[route[0]]

def run(n=20,seed0=700000,enroute_gain=70.0,enroute_max=5):
    plan={'level1':[list(p) for p in PTS21],'escalation':[]};rows=[];hard=[];ext=[];t=time.time()
    for i in range(n):
        srcs=generate_case(np.random.default_rng(seed0+i),4);sim=Simulator(srcs)
        rb=RoutingPlusEnroute(sim,plan,Params2(name='rpe',theta_miss=0,use_belief=False),4,shared_radius=
↪ 50,fresh_only=True,enroute_min_gain=enroute_gain,enroute_max=enroute_max)
        try:r=rb.run()
        except Exception as e:hard.append((seed0+i,str(e)));r=rb.finish()
        rows.append(r);ext.append((rb.rp_improve,rb.enroute_reads,rb.enroute_positive,rb.enroute_nosig,rb
↪ .clear_region_saved))
    st=[r['stats'] for r in rows]
    out={'n':n,'seed0':seed0,'enroute_gain':enroute_gain,'enroute_max':enroute_max,'full':sum(s['ratio']>
↪ =1-1e-9 for s in st),'agg':sum(s['total_time_s'] for s in st)/sum(s['n_sources'] for s in
↪ st),'meanavg':float(np.mean([s['avg_time_s'] for s in
↪ st])),('mean_total':float(np.mean([s['total_time_s'] for s in
↪ st])),('move':float(np.mean([s['move_m'] for s in st])),('req':float(np.mean([s['n_requests'] for s
↪ in st])),('clear':float(np.mean([s['t_clear'] for s in
↪ st])),('measure':float(np.mean([s['t_measure'] for s in st])),('switch':float(np.mean([s['t_switch']
↪ for s in st])),('planner_improve_m':float(np.mean([x[0] for x in
↪ ext])),('enroute_reads':float(np.mean([x[1] for x in ext])),('enroute_positive':float(np.mean([x[2]
↪ for x in ext])),('enroute_nosig':float(np.mean([x[3] for x in
↪ ext])),('clear_region_saved_m':float(np.mean([x[4] for x in
↪ ext])),('miss':sum(s['n_sources']-s['n_cleared'] for s in st),'hard':hard,'wall':time.time()-t}
    print(json.dumps(out,indent=2));return out
if __name__=='__main__':
    ap=argparse.ArgumentParser();ap.add_argument('--n',type=int,default=20);ap.add_argument('--seed0',typ
↪ e=int,default=700000);ap.add_argument('--gain',type=float,default=70.0);ap.add_argument('--max',t
↪ ype=int,default=5);a=ap.parse_args();run(a.n,a.seed0,a.gain,a.max)

```

文件: vendor/portable\_sandbox/run\_q4\_routingplus\_fast.py

```

import sys,math,json,time,argparse
from pathlib import Path
_HERE=Path(__file__).resolve().parent
sys.path.insert(0,str(_HERE));sys.path.insert(0,str(_HERE/'lib'))
import numpy as np
import strategy
strategy.MARGIN=2.0
from run_q4_shared_pair import SharedPairStrict
from run_q4_shared import PTS21
from sim import generate_case,Simulator
from strategy2 import Params2
from geom import norm

```

```

def route_length(route,matrix):
    start=len(matrix)-1
    return sum(matrix[start if i==0 else route[i-1]][j] for i,j in enumerate(route))

def two_opt(route,matrix,moves=80):
    route=list(route);start=len(matrix)-1
    for _ in range(moves):
        best=-.01;change=None
        for i in range(len(route)-1):
            a=start if i==0 else route[i-1];b=route[i]
            for j in range(i+1,len(route)):
                c=route[j];d=route[j+1] if j+1<len(route) else None
                delta=matrix[a][c]-matrix[a][b]
                if d is not None:delta+=matrix[b][d]-matrix[c][d]
                if delta<best:best=delta;change=(i,j)
        if change is None:break
        i,j=change;route[i:j+1]=reversed(route[i:j+1])
    return route

def or_opt(route,matrix,max_moves=12,max_chain=3):
    route=list(route);start=len(matrix)-1;n=len(route)
    for _ in range(max_moves):
        best=-.01;change=None
        for k in range(1,min(max_chain,n-1)+1):
            for i in range(n-k+1):
                chain=route[i:i+k];rest=route[:i]+route[i+k:]
                a=start if i==0 else route[i-1];b=route[i+k] if i+k<n else None
                removal=-matrix[a][chain[0]]
                if b is not None:removal+=matrix[a][b]-matrix[chain[-1]][b]
                for j in range(len(rest)+1):
                    if j==i:continue
                    c=start if j==0 else rest[j-1];d=rest[j] if j<len(rest) else None
                    for reverse in ((False,True) if k>1 else (False,)):
                        first,last=(chain[-1],chain[0]) if reverse else (chain[0],chain[-1])
                        delta=removal+matrix[c][first]
                        if d is not None:delta+=matrix[last][d]-matrix[c][d]
                        if delta<best:best=delta;change=(i,k,j,reverse)
                if change is None:break
            i,k,j,reverse=change;chain=route[i:i+k];rest=route[:i]+route[i+k:]
            if reverse:chain.reverse()
            route=rest[:j]+chain+rest[j:]
            route=two_opt(route,matrix,20)
    return route

def nearest_seed(matrix,first=None):
    n=len(matrix)-1;remaining=set(range(n));route=[];last=n
    if first is not None:route=[first];remaining.remove(first);last=first
    while remaining:
        nxt=min(remaining,key=lambda
            ↪ i:(matrix[last][i],i));route.append(nxt);remaining.remove(nxt);last=nxt
    return route

def insertion_seed(matrix):
    n=len(matrix)-1;start=n;remaining=set(range(n))
    if not remaining:return []
    first=max(remaining,key=lambda i:matrix[start][i]);route=[first];remaining.remove(first)
    while remaining:
        best=None

```

```

        for point in remaining:
            for index in range(len(route)+1):
                a=start if index==0 else route[index-1];b=route[index] if index<len(route) else None
                extra=matrix[a][point]+(matrix[point][b]-matrix[a][b] if b is not None else 0)
                candidate=(extra,point,index)
                if best is None or candidate<best:best=candidate
            _,point,index=best;route.insert(index,point);remaining.remove(point)
    return route

class RoutingPlusFast(SharedPairStrict):
    def __init__(self,*a,oropt_on=True,warm=True,starts=4,**kw):
        super().__init__(*a,**kw);self.rp_oropt=oropt_on;self.rp_warm=warm;self.rp_starts=starts;self._route_keys2=[];self.rp_improve=0.;self.rp_calls=0
    def _key(self,j):
        kind,ch,p,_=j
        return ('clear',int(ch)) if kind=='clear' else ('scan',round(float(p[0]),5),round(float(p[1]),5))
    def _plan_next(self,jobs):
        if len(jobs)==1:return jobs[0]
        points=[np.asarray(j[2],float) for j in jobs]+[self.sim.pos.copy()]
        matrix=[[norm(a-b) for b in points] for a in points]
        seeds=[nearest_seed(matrix)]
        if self.rp_starts>1:seeds.append(insertion_seed(matrix))
        if self.rp_starts>2:
            ordered=sorted(range(len(jobs)),key=lambda i:math.atan2(points[i][1],points[i][0]))
            if ordered:
                cut=min(range(len(ordered)),key=lambda k:matrix[-1][ordered[k]])
                angular=ordered[cut:]+ordered[:cut];seeds.append(angular)
                if self.rp_starts>3:seeds.append([angular[0]]+list(reversed(angular[1:])))
        if self.rp_warm and self._route_keys2:
            mapping={self._key(g):i for i,g in enumerate(jobs)}
            warm=[mapping[k] for k in self._route_keys2 if k in mapping]
            for point in range(len(jobs)):
                if point in warm:continue
                best=min(range(len(warm)+1),key=lambda i:matrix[-1 if i==0 else
                    ↪ warm[i-1]][point]+(matrix[point][warm[i]]-matrix[-1 if i==0 else warm[i-1]][warm[i]]
                    ↪ if i<len(warm) else 0))
                warm.insert(best,point)
            if len(warm)==len(jobs):seeds.append(warm)
        base=route_length(seeds[0],matrix)
        candidates=[]
        for seed in seeds:
            route=two_opt(seed,matrix)
            candidates.append(route)
        route=min(candidates,key=lambda r:route_length(r,matrix))
        if self.rp_oropt:route=or_opt(route,matrix,max_moves=12,max_chain=3)
        final=route_length(route,matrix);self.rp_improve+=max(0.,base-final);self.rp_calls+=1
        self._route_keys2=[self._key(jobs[i]) for i in route]
        return jobs[route[0]]

def run(n=20,seed0=700000,oropt_on=True,warm=True,starts=4):
    plan={'level1':[list(p) for p in PTS21],'escalation':[]};rows=[];hard=[];imp=[];calls=[];t=time.time()
    for i in range(n):
        sracs=generate_case(np.random.default_rng(seed0+i),4);sim=Simulator(sracs);rb=RoutingPlusFast(sim,p
        ↪ lan,Params2(name='rpf',theta_miss=0,use_belief=False),4,shared_radius=50,fresh_only=True,orop
        ↪ t_on=oropt_on,warm=warm,starts=starts)
        try:r=rb.run()
        except Exception as e:hard.append((seed0+i,str(e)));r=rb.finish()
        rows.append(r);imp.append(rb.rp_improve);calls.append(rb.rp_calls)

```

```

st=[r['stats'] for r in rows]
out={'n':n,'seed0':seed0,'oropt':oropt_on,'warm':warm,'starts':starts,'full':sum(s['ratio']>=1-1e-9
↪ for s in st),'agg':sum(s['total_time_s'] for s in st)/sum(s['n_sources'] for s in
↪ st),'meanavg':float(np.mean([s['avg_time_s'] for s in
↪ st])), 'mean_total':float(np.mean([s['total_time_s'] for s in
↪ st])), 'move':float(np.mean([s['move_m'] for s in st])), 'req':float(np.mean([s['n_requests'] for s
↪ in st])), 'clear':float(np.mean([s['t_clear'] for s in
↪ st])), 'measure':float(np.mean([s['t_measure'] for s in st])), 'switch':float(np.mean([s['t_switch']
↪ for s in st])), 'planner_improve_m':float(np.mean(imp)), 'planner_calls':float(np.mean(calls)), 'mis
↪ s':sum(s['n_sources']-s['n_cleared'] for s in st), 'hard':hard, 'wall':time.time()-t}
print(json.dumps(out,indent=2));return out
if __name__=='__main__':
    ap=argparse.ArgumentParser();ap.add_argument('--n',type=int,default=20);ap.add_argument('--seed0',typ
↪ e=int,default=700000);ap.add_argument('--no-oropt',action='store_true');ap.add_argument('--no-war
↪ m',action='store_true');ap.add_argument('--starts',type=int,default=4);a=ap.parse_args();run(a.n,
↪ a.seed0,not a.no_oropt,not a.no_warm,a.starts)

```

## 文件: vendor/portable\_sandbox/run\_q4\_serviceanchor.py

```

import sys,math,json,time,argparse
from pathlib import Path
_HERE=Path(__file__).resolve().parent
sys.path.insert(0,str(_HERE));sys.path.insert(0,str(_HERE/'lib'))
import numpy as np
import strategy
strategy.MARGIN=2.0
from run_q4_routing_ablate import PlannerMixin
from run_q4_pairlocal import PairLocalStrict
from run_q4_shared import PTS21
from sim import generate_case,Simulator
from strategy2 import Params2
from geom import mec,polygon_centroid,diameter_rotating_calipers_pair

class ServiceAnchorPairLocal(PlannerMixin,PairLocalStrict):
    def _preview_service(self,ch):
        poly=self.tracked[ch];c,r=mec(poly)
        if r<=18.0:return np.asarray(c,float)
        if r<=self.explore:return np.asarray(polygon_centroid(poly),float)
        made=self._pair(ch,poly)
        if made is not None:
            pair,apex,u,step,all_in=made
            return np.asarray(pair[0],float)
        return np.asarray(polygon_centroid(poly),float)
    def _jobs(self):
        jobs=[]
        for ch in self.tracked:
            if ch in self.cleared:continue
            poly=self.tracked[ch];D,_=diameter_rotating_calipers_pair(poly);p=self._preview_service(ch)
            jobs.append(('clear',ch,p,self._dist(p)+0.12*D+20.0))
        if self._need_scan():
            for P in self.pending:jobs.append(('scan',None,np.asarray(P,float),self._dist(P)+6.0*len(self)
↪ .unknown)))
        return jobs

def run(n=20,seed0=700000):
    plan={'level1':[list(p) for p in PTS21],'escalation':[]};rows=[];hard=[];t=time.time();imps=[]
    for i in range(n):

```

```

srcs=generate_case(np.random.default_rng(seed0+i),4);sim=Simulator(srcs)
rb=ServiceAnchorPairLocal(sim,plan,Params2(name='svcanchor',theta_miss=0,use_belief=False),4,shar
↪ ed_radius=50,fresh_only=True,rounds=5,quantile=.2,width_frac=.04,explore=55.0)
try:r=rb.run()
except Exception as e:hard.append((seed0+i,str(e)));r=rb.finish()
rows.append(r);imps.append(rb.rp_improve)
st=[r['stats'] for r in rows]
out={'n':n,'seed0':seed0,'full':sum(s['ratio']>=1-1e-9 for s in st),'agg':sum(s['total_time_s'] for s
↪ in st)/sum(s['n_sources'] for s in st),'meanavg':float(np.mean([s['avg_time_s'] for s in
↪ st])), 'mean_total':float(np.mean([s['total_time_s'] for s in
↪ st])), 'move':float(np.mean([s['move_m'] for s in st])), 'req':float(np.mean([s['n_requests'] for s
↪ in st])), 'clear':float(np.mean([s['t_clear'] for s in
↪ st])), 'measure':float(np.mean([s['t_measure'] for s in st])), 'switch':float(np.mean([s['t_switch']
↪ for s in st])), 'planner_improve_m':float(np.mean(imps)), 'miss':sum(s['n_sources']-s['n_cleared']
↪ for s in st), 'hard':hard, 'wall':time.time()-t}
print(json.dumps(out,indent=2));return out
if __name__=='__main__':
    ap=argparse.ArgumentParser();ap.add_argument('--n',type=int,default=20);ap.add_argument('--seed0',typ
↪ e=int,default=700000);a=ap.parse_args();run(a.n,a.seed0)

```

文件: vendor/portable\_sandbox/run\_q4\_shared.py

```

import sys,json,time,math,argparse
from pathlib import Path
_HERE=Path(__file__).resolve().parent
sys.path.insert(0,str(_HERE));sys.path.insert(0,str(_HERE/'lib'))
import numpy as np
import strategy
strategy.MARGIN=2.0
from strict_filtered_test import StrictFiltered
from sim import generate_case,Simulator
from strategy2 import Params2
from geom import mec, polygon_centroid, norm

PTS23=[
(0.000000,0.000000),(1864.500000,0.000000),(1614.704365,932.250000),(932.250000,1614.704365),(0.000000,18
↪ 64.500000),(-932.250000,1614.704365),(-1614.704365,932.250000),(-1864.500000,0.000000),(-1614.704365,
↪ -932.250000),(-932.250000,-1614.704365),(0.000000,-1864.500000),(932.250000,-1614.704365),(1614.704365
↪ ,-932.250000),(1076.467178,0.001385),(673.445446,707.456053),(35.090167,1039.476106),(-652.783202,727
↪ .398374),(-1076.469577,0.000000),(-804.167008,-363.249827),(-538.234788,-932.250000),(93.843216,-877.
↪ 978803),(538.234788,-932.250000),(955.618224,-253.126730)
]
outer=1802/math.cos(math.pi/12)
PTS21=[(0.,0.)]+[(outer*math.cos(2*math.pi*k/12),outer*math.sin(2*math.pi*k/12)) for k in
↪ range(12)]+[(997.32*math.cos(2*math.pi*k/8),997.32*math.sin(2*math.pi*k/8)) for k in range(8)]

class Shared50Strict(StrictFiltered):
    def __init__(self,*a,shared_radius=50.0,fresh_only=True,min_unc=100.0,min_gain=25.0,detour_weight=1.0,**
↪ kw):
        super().__init__(*a,**kw);self.shared_radius=shared_radius;self.fresh_only=fresh_only;self.min_unc=min_
↪ unc;self.min_gain=min_gain;self.detour_weight=detour_weight;self.shared_count=0;self.shared_reads=0
    def _do_scan(self,P):
        old=set(self.tracked)
        super()._do_scan(P)
        fresh=[ch for ch in self.tracked if ch not in old and ch not in self.cleared]
        self._shared_step(fresh if self.fresh_only else [ch for ch in self.tracked if ch not in self.cleared])
    def _shared_step(self,candidates_ch):

```

```

channels=[]
for ch in candidates_ch:
    if ch in self.cleared or ch not in self.tracked:continue
    _,unc=mec(self.tracked[ch])
    if unc>self.min_unc:channels.append(ch)
if not channels:return False
origin=self.sim.pos.copy()
jobs=self._jobs()
next_goal=origin
if jobs:
    try:next_goal=np.asarray(self._plan_next(jobs)[2],float)
    except Exception:pass
opts=[]
for idx in range(24):
    a=idx*2*math.pi/24;q=origin+self.shared_radius*np.array([math.cos(a),math.sin(a)])
    elig=[]
    for ch in channels:
        center,unc=mec(self.tracked[ch]);center=np.asarray(center,float)
        anchor=np.asarray(self.mlog[ch][-1][0],float) if self.mlog.get(ch) else origin
        va=anchor-center;vb=q-center;den=norm(va)*norm(vb)
        sine=abs(float(va[0]*vb[1]-va[1]*vb[0]))/den if den>1e-9 else 0.0
        est=min(unc,2*.01754*norm(q-center)/max(.01,sine))
        # Heuristic receive probability affects action selection only, never safe state.
        prob=max(0.,min(1.,(1500-norm(q-center))/500))
        gain=min(unc,max(0.,unc-est))*prob
        if gain>=self.min_gain:elig.append((gain,ch))
    if not elig:continue
    det=self.shared_radius+norm(q-next_goal)-norm(origin-next_goal)
    score=sum(g for g,_ in elig)-self.detour_weight*det
    opts.append((score,idx,q,[ch for _,ch in elig]))
if not opts:return False
score,idx,q,chs=max(opts,key=lambda x:(x[0],-x[1]))
if score<=0:return False
# Keep current channel first to reduce switch if possible.
if self.sim.channel in chs:
    chs.remove(self.sim.channel);chs.insert(0,self.sim.channel)
for ch in chs:
    if ch in self.cleared or ch not in self.tracked:continue
    res,svd=self.sim.measure(q,ch);self.n_measure+=1
    if res=='direction':
        self.nosig[ch]=0;self.ray[ch]=(self.sim.pos.copy(),svd);self._update_region(ch,self.sim.pos,svd)
    elif res=='near':
        rr,_=self.sim.clear(self.sim.pos,ch);self.n_clear+=1
        if rr=='success':self.cleared.add(ch);self.unknown.discard(ch)
    else:
        self.nosig[ch]=self.nosig.get(ch,0)+1;self._record_nosig(ch,self.sim.pos)
        # Strict Q4: no source-position contraction on no_signal.
    self.shared_reads+=1
self.shared_count+=1
return True

def run(n=20,seed0=700000,layout=21,radius=50.0,fresh=True):
    pts=PTS21 if layout==21 else PTS23;plan={'level1':[list(p) for p in pts],'escalation':[]}
    rows=[];hard=[];counts=[];reads=[];t0=time.time()
    for i in range(n):
        sracs=generate_case(np.random.default_rng(seed0+i),4);sim=Simulator(sracs);rb=Shared50Strict(sim,plan,Par
        ↪ ams2(name='q4_shared',theta_miss=0.0,use_belief=False),4,shared_radius=radius,fresh_only=fresh)
        try:r=rb.run()

```

```

except Exception as e:hard.append((seed0+i,str(e)));r=rb.finish()
rows.append(r);counts.append(rb.shared_count);reads.append(rb.shared_reads)
st=[r['stats'] for r in rows]
out={'layout':layout,'radius':radius,'fresh':fresh,'n':n,'seed0':seed0,'full':sum(s['ratio']>=1-1e-9 for s
↪ in st),'agg':sum(s['total_time_s'] for s in st)/sum(s['n_sources'] for s in
↪ st),'meanavg':float(np.mean([s['avg_time_s'] for s in
↪ st])), 'mean_total':float(np.mean([s['total_time_s'] for s in st])), 'move':float(np.mean([s['move_m']
↪ for s in st])), 'req':float(np.mean([s['n_requests'] for s in
↪ st])), 'clear':float(np.mean([s['t_clear'] for s in st])), 'measure':float(np.mean([s['t_measure'] for s
↪ in st])), 'switch':float(np.mean([s['t_switch'] for s in st])), 'shared_steps':float(np.mean(counts)),
↪ 'shared_reads':float(np.mean(reads)), 'miss':sum(s['n_sources']-s['n_cleared'] for s in
↪ st),'hard':hard,'wall':time.time()-t0}
print(json.dumps(out,indent=2));return out
if __name__=='__main__':
ap=argparse.ArgumentParser();ap.add_argument('--n',type=int,default=20);ap.add_argument('--seed0',type=i
↪ nt,default=700000);ap.add_argument('--layout',type=int,choices=[21,23],default=21);ap.add_argument('
↪ --radius',type=float,default=50);ap.add_argument('--all',action='store_true');a=ap.parse_args();run(a
↪ .n,a.seed0,a.layout,a.radius,not a.all)

```

## 文件: vendor/portable\_sandbox/run\_q4\_shared\_pair.py

```

import sys,math,json,time,argparse
from pathlib import Path
_HERE=Path(__file__).resolve().parent
sys.path.insert(0,str(_HERE));sys.path.insert(0,str(_HERE/'lib'))
import numpy as np
import strategy
strategy.MARGIN=2.0
from run_q4_shared import Shared50Strict, PTS21
from sim import generate_case,Simulator
from strategy2 import Params2
from geom import norm, unit, clip_polygon, diameter_rotating_calipers_pair

class SharedPairStrict(Shared50Strict):
def __init__(self,*a,**kw):
super().__init__(*a,**kw);self._pair_pending={};self._pair_ctx={};self.pair_actions=0;self.pair_clips=0
def _make_pair(self,ch,poly):
if ch not in self.ray or self.nosig.get(ch,0)!=0:return None
apex,th=self.ray[ch];apex=np.asarray(apex,float);u=unit(th);side=np.array([-u[1],u[0]])
projs=[float((np.asarray(p,float)-apex)@u) for p in poly];lo,hi=min(projs),max(projs)
if hi-lo<1e-6:return None
step=lo+0.2*(hi-lo);width=max(15.,min(140.,0.04*step));width=max(width,step*math.tan(math.radians(1.0)))
↪ +.05)
P=np.asarray(poly,float)
pair=None
for _ in range(24):
mid=apex+step*u;pair=[mid-width*side,mid+width*side]
if max(norm(q-v) for q in pair for v in P)<999.9:break
step=.5*step+.25*(lo+hi);width=max(width,step*math.tan(math.radians(1.0))+.05)
if pair is None or max(norm(q-v) for q in pair for v in P)>=999.9:return None
pair.sort(key=lambda q: norm(q-self.sim.pos))
return pair,apex,u,step
def _next_measure_point(self,ch,poly):
pend=self._pair_pending.get(ch)
if pend:
return pend.pop(0)
made=self._make_pair(ch,poly)

```

```

if made is not None:
    pair,apex,u,step=made
    self._pair_pending[ch]=[pair[1]]
    self._pair_ctx[ch]={'points':[pair[0],pair[1]],'negative':[],'apex':apex,'u':u,'step':step}
    return pair[0]
return super()._next_measure_point(ch,poly)
def _update_region(self,ch,pos,svd):
    # A direction result ends the current pair attempt; the normal positive update is safe.
    if ch in self._pair_ctx:
        self._pair_pending.pop(ch,None);self._pair_ctx.pop(ch,None)
    return super()._update_region(ch,pos,svd)
def _record_nosig(self,ch,q):
    super()._record_nosig(ch,q)
    ctx=self._pair_ctx.get(ch)
    if ctx is None:return
    q=np.asarray(q,float);ctx['negative'].append(q.copy());self.pair_actions+=1
    if len(ctx['negative'])<2:return
    # Both designed probes are certified <1000m from every point in the pre-pair polygon.
    # Thus for Q4 both no_signal readings are directional shadows, not range failures.
    # The source cannot lie beyond the segment joining the two transverse probes in
    # the latest positive bearing cone: keep forward projection <= apex + step.
    u=ctx['u'];apex=ctx['apex'];constant=float(u@apex+ctx['step'])
    old=self.tracked.get(ch)
    if old is not None:
        new=clip_polygon(old,(u,constant))
        if len(new)>=3:
            self.tracked[ch]=new;self.pair_clips+=1
    self._pair_pending.pop(ch,None);self._pair_ctx.pop(ch,None)

def run(n=20,seed0=700000):
    plan={'level1':[list(p) for p in PTS21],'escalation':[]};rows=[];hard=[];pa=[];pc=[];t=time.time()
    for i in range(n):
        sracs=generate_case(np.random.default_rng(seed0+i),4);sim=Simulator(sracs);rb=SharedPairStrict(sim,plan,P
        ↪ arams2(name='pair',theta_miss=0,use_belief=False),4,shared_radius=50,fresh_only=True)
        try:r=rb.run()
        except Exception as e:hard.append((seed0+i,str(e)));r=rb.finish()
        rows.append(r);pa.append(rb.pair_actions);pc.append(rb.pair_clips)
    st=[r['stats'] for r in rows]
    out={'n':n,'seed0':seed0,'full':sum(s['ratio']>=1-1e-9 for s in st),'agg':sum(s['total_time_s'] for s in
    ↪ st)/sum(s['n_sources'] for s in st),'meanavg':float(np.mean([s['avg_time_s'] for s in
    ↪ st])),('mean_total':float(np.mean([s['total_time_s'] for s in st])),('move':float(np.mean([s['move_m']
    ↪ for s in st])),('req':float(np.mean([s['n_requests'] for s in
    ↪ st])),('clear':float(np.mean([s['t_clear'] for s in st])),('measure':float(np.mean([s['t_measure'] for s
    ↪ in st])),('switch':float(np.mean([s['t_switch'] for s in st])),('pair_actions':float(np.mean(pa)),('pai
    ↪ r_clips':float(np.mean(pc)),('miss':sum(s['n_sources']-s['n_cleared'] for s in
    ↪ st),'hard':hard,'wall':time.time()-t}
    print(json.dumps(out,indent=2));return out
if __name__=='__main__':
    ap=argparse.ArgumentParser();ap.add_argument('--n',type=int,default=20);ap.add_argument('--seed0',type=i
    ↪ nt,default=700000);a=ap.parse_args();run(a.n,a.seed0)

```

文件: vendor/portable\_sandbox/run\_q4\_shared\_pair\_bound.py

""""Safety-hardened paired-negative prototype for the preserved Q4 sandbox.

This file fixes a specific atomic/resumable-context hazard in the original run\_q4\_shared\_pair.py: only a no\_signal observed at the \*intended designed pair

probe coordinate\* may count as paired-negative evidence. Unrelated same-channel no\_signal observations are logged but ignored by the pair context.

Important: this hardens evidence binding only. The directional-shadow cut itself must still be re-derived against the real project/API convention before porting.

```

"""
from __future__ import annotations
import sys, math, json, time, argparse
from pathlib import Path
_HERE=Path(__file__).resolve().parent
sys.path.insert(0,str(_HERE));sys.path.insert(0,str(_HERE/'lib'))
import numpy as np
import strategy
strategy.MARGIN=2.0
from run_q4_shared import Shared50Strict, PTS21
from sim import generate_case, Simulator
from strategy2 import Params2
from geom import norm, unit, clip_polygon

PAIR_MATCH_TOL=1e-6

class SharedPairBoundStrict(Shared50Strict):
    def __init__(self,*a,**kw):
        super().__init__(*a,**kw)
        self._pair_pending={};self._pair_ctx={}
        self.pair_actions=0;self.pair_clips=0;self.pair_ignored_unrelated=0

    def _make_pair(self,ch,poly):
        if ch not in self.ray or self.nosig.get(ch,0)!=0:return None
        apex,th=self.ray[ch];apex=np.asarray(apex,float);u=unit(th);side=np.array([-u[1],u[0]])
        projs=[float((np.asarray(p,float)-apex)@u) for p in poly];lo,hi=min(projs),max(projs)
        if hi-lo<1e-6:return None
        step=lo+0.2*(hi-lo)
        width=max(15.,min(140.,0.04*step));width=max(width,step*math.tan(math.radians(1.0))+.05)
        P=np.asarray(poly,float);pair=None
        for _ in range(24):
            mid=apex+step*u;pair=[mid-width*side,mid+width*side]
            if max(norm(q-v) for q in pair for v in P)<999.9:break
            step=.5*step+.25*(lo+hi);width=max(width,step*math.tan(math.radians(1.0))+.05)
        if pair is None or max(norm(q-v) for q in pair for v in P)>=999.9:return None
        pair.sort(key=lambda q: norm(q-self.sim.pos))
        return pair,apex,u,step,P.copy()

    def _next_measure_point(self,ch,poly):
        pend=self._pair_pending.get(ch)
        if pend:
            q=pend.pop(0)
            ctx=self._pair_ctx.get(ch)
            if ctx is not None:ctx['expected']=np.asarray(q,float).copy()
            return q
        made=self._make_pair(ch,poly)
        if made is not None:
            pair,apex,u,step,pre_poly=made
            self._pair_pending[ch]=[pair[1]]
            self._pair_ctx[ch]={
                'points': [np.asarray(pair[0],float).copy(),np.asarray(pair[1],float).copy()],
                'negative': [], 'matched': set(), 'expected': np.asarray(pair[0],float).copy(),
                'apex': apex.copy(), 'u': u.copy(), 'step': float(step), 'pre_poly': pre_poly,
            }

```

```

        return pair[0]
    return super()._next_measure_point(ch,poly)

def _update_region(self,ch,pos,svd):
    # Any actual direction result ends the current pair attempt.
    if ch in self._pair_ctx:
        self._pair_pending.pop(ch,None);self._pair_ctx.pop(ch,None)
    return Shared50Strict._update_region(self,ch,pos,svd)

def _record_nosig(self,ch,q):
    # Always preserve ordinary strict Q4 logging semantics.
    Shared50Strict._record_nosig(self,ch,q)
    ctx=self._pair_ctx.get(ch)
    if ctx is None:return
    q=np.asarray(q,float)
    expected=ctx.get('expected')
    if expected is None or norm(q-expected)>PAIR_MATCH_TOL:
        self.pair_ignored_unrelated+=1
        return
    # Bind to a concrete designed probe identity and prevent duplicate use.
    idx=min(range(2),key=lambda i:norm(q-ctx['points'][i]))
    if norm(q-ctx['points'][idx])>PAIR_MATCH_TOL or idx in ctx['matched']:
        self.pair_ignored_unrelated+=1
        return
    ctx['matched'].add(idx);ctx['negative'].append(q.copy());self.pair_actions+=1
    if len(ctx['negative'])<2:return
    # Re-certify the deterministic in-range premise against the pre-pair safe polygon.
    if any(max(norm(point-v) for v in ctx['pre_poly'])>=999.9 for point in ctx['points']):
        self._pair_pending.pop(ch,None);self._pair_ctx.pop(ch,None)
        return
    u=ctx['u'];apex=ctx['apex'];constant=float(u@apex+ctx['step'])
    old=self.tracked.get(ch)
    if old is not None:
        new=clip_polygon(old,(u,constant))
        if len(new)>=3:
            self.tracked[ch]=new;self.pair_clips+=1
    self._pair_pending.pop(ch,None);self._pair_ctx.pop(ch,None)

def run(n=20,seed0=700000):
    plan={'level1':[list(p) for p in
    ↪ PTS21],'escalation':[]};rows=[];hard=[];pa=[];pc=[];ig=[];t=time.time()
    for i in range(n):
        srcs=generate_case(np.random.default_rng(seed0+i),4);sim=Simulator(srcs)
        rb=SharedPairBoundStrict(sim,plan,Params2(name='pair_bound',theta_miss=0,use_belief=False),4,shar
        ↪ ed_radius=50,fresh_only=True)
        try:r=rb.run()
        except Exception as e:hard.append((seed0+i,str(e)));r=rb.finish()
        rows.append(r);pa.append(rb.pair_actions);pc.append(rb.pair_clips);ig.append(rb.pair_ignored_unre
        ↪ lated)
    st=[r['stats'] for r in rows]
    out={'n':n,'seed0':seed0,'full':sum(s['ratio']>=1-1e-9 for s in st),
        'agg':sum(s['total_time_s'] for s in st)/sum(s['n_sources'] for s in st),
        'meanavg':float(np.mean([s['avg_time_s'] for s in st])),
        'mean_total':float(np.mean([s['total_time_s'] for s in st])),
        'move':float(np.mean([s['move_m'] for s in st])), 'req':float(np.mean([s['n_requests'] for s in
        ↪ st])),
        'clear':float(np.mean([s['t_clear'] for s in st])), 'measure':float(np.mean([s['t_measure'] for s
        ↪ in st])),

```

```

        'switch':float(np.mean([s['t_switch'] for s in st])), 'pair_actions':float(np.mean(pa)),
        'pair_clips':float(np.mean(pc)), 'ignored_unrelated':int(sum(ig)),
        'miss':sum(s['n_sources']-s['n_cleared'] for s in st), 'hard':hard, 'wall':time.time()-t}
    print(json.dumps(out,indent=2));return out

if __name__=='__main__':
    ap=argparse.ArgumentParser();ap.add_argument('--n',type=int,default=20);ap.add_argument('--seed0',typ
    ↪ e=int,default=700000);a=ap.parse_args();run(a.n,a.seed0)

```

## 文件: vendor/portable\_sandbox/run\_q4\_shared\_richroute.py

```

import sys,math,json,time,argparse
from pathlib import Path
_HERE=Path(__file__).resolve().parent
sys.path.insert(0,str(_HERE));sys.path.insert(0,str(_HERE/'lib'))
import numpy as np
from geom import norm
from run_q4_shared import Shared50Strict, PTS21, PTS23
from sim import generate_case, Simulator
from strategy2 import Params2

def pathlen(order,pts,start):
    cur=start;s=0.0
    for i in order:s+=norm(pts[i]-cur);cur=pts[i]
    return s

def nearest(pts,start,first=None):
    left=list(range(len(pts)));out=[];cur=start
    if first is not None:out=[first];left.remove(first);cur=pts[first]
    while left:
        k=min(left,key=lambda i:(norm(pts[i]-cur),i));out.append(k);left.remove(k);cur=pts[k]
    return out

def insertion(pts,start):
    n=len(pts)
    if not n:return []
    left=set(range(n));first=max(left,key=lambda i:(norm(pts[i]-start),-i));out=[first];left.remove(first)
    while left:
        best=None
        for x in sorted(left):
            for j in range(len(out)+1):
                a=start if j==0 else pts[out[j-1]];extra=norm(pts[x]-a)
                if j<len(out):extra+=norm(pts[x]-pts[out[j]])-norm(a-pts[out[j]])
                cand=(extra,x,j)
                if best is None or cand<best:best=cand
        _,x,j=best;out.insert(j,x);left.remove(x)
    return out

def twoopt(order,pts,start,maxmoves=80):
    order=list(order)
    for _ in range(maxmoves):
        base=pathlen(order,pts,start);best=base;bo=None
        for i in range(len(order)-1):
            for j in range(i+1,len(order)):
                cand=order[:i]+order[i:j+1][::-1]+order[j+1:];v=pathlen(cand,pts,start)
                if v<best-1e-9:best=v;bo=cand
    if bo is None:break

```

```

    order=bo
    return order

def oropt(order,pts,start,maxmoves=16,maxchain=3):
    order=list(order)
    for _ in range(maxmoves):
        base=pathlen(order,pts,start);best=base;bo=None;n=len(order)
        for k in range(1,min(maxchain,n-1)+1):
            for i in range(n-k+1):
                chain=order[i:i+k];rest=order[:i]+order[i+k:]
                for j in range(len(rest)+1):
                    for rev in ([False,True] if k>1 else [False]):
                        c=chain[::-1] if rev else chain
                        cand=rest[:j]+c+rest[j:];v=pathlen(cand,pts,start)
                        if v<best-1e-9:best=v;bo=cand
            if bo is None:break
        order=twoopt(bo,pts,start,20)
    return order

class RichRouteShared(Shared50Strict):
    def _plan_next(self,jobs):
        if len(jobs)<=1:return jobs[0]
        start=self.sim.pos.copy();pts=[np.asarray(j[2],float) for j in jobs];n=len(pts)
        seeds=[nearest(pts,start),insertion(pts,start)]
        ordered=sorted(range(n),key=lambda i:(math.atan2(pts[i][1],pts[i][0]),i))
        if ordered:
            cut=min(range(n),key=lambda k:norm(pts[ordered[k]]-start));ang=ordered[cut:]+ordered[:cut]
            seeds += [ang,[ang[0]]+list(reversed(ang[1:]))]
        cand=[]
        for s in seeds:
            r=twoopt(s,pts,start);cand.append(r)
        r=min(cand,key=lambda o:pathlen(o,pts,start));r=ropt(r,pts,start)
        first=r[0]
        # if identical locations, prefer source clear
        same=[i for i in range(n) if norm(pts[i]-pts[first])<1e-6]
        same.sort(key=lambda i:(0 if jobs[i][0]=='clear' else 1,i))
        return jobs[same[0]]

def run(n=20,seed0=700000,layout=21):
    pts=PTS21 if layout==21 else PTS23;plan={'level1':[list(p) for p in
    ↪ pts],'escalation':[]};rows=[];hard=[];t=time.time();steps=[];reads=[]
    for i in range(n):
        sracs=generate_case(np.random.default_rng(seed0+i),4);sim=Simulator(sracs);rb=RichRouteShared(sim,plan,Pa
    ↪ rams2(name='rich',theta_miss=0,use_belief=False),4,shared_radius=50,fresh_only=True)
        try:r=rb.run()
        except Exception as e:hard.append((seed0+i,str(e)));r=rb.finish()
        rows.append(r);steps.append(rb.shared_count);reads.append(rb.shared_reads)
    st=[r['stats'] for r in rows]
    out={'n':n,'seed0':seed0,'layout':layout,'full':sum(s['ratio']>=1-1e-9 for s in
    ↪ st),'agg':sum(s['total_time_s'] for s in st)/sum(s['n_sources'] for s in
    ↪ st),'meanavg':float(np.mean([s['avg_time_s'] for s in
    ↪ st])), 'mean_total':float(np.mean([s['total_time_s'] for s in st])), 'move':float(np.mean([s['move_m']
    ↪ for s in st])), 'req':float(np.mean([s['n_requests'] for s in
    ↪ st])), 'clear':float(np.mean([s['t_clear'] for s in st])), 'measure':float(np.mean([s['t_measure'] for s
    ↪ in st])), 'switch':float(np.mean([s['t_switch'] for s in st])), 'shared_steps':float(np.mean(steps)), '
    ↪ shared_reads':float(np.mean(reads)), 'miss':sum(s['n_sources']-s['n_cleared'] for s in
    ↪ st), 'hard':hard, 'wall':time.time()-t}
    print(json.dumps(out,indent=2));return out

```

```

if __name__=='__main__':
    ap=argparse.ArgumentParser();ap.add_argument('--n',type=int,default=20);ap.add_argument('--seed0',type=i
↪ nt,default=700000);ap.add_argument('--layout',type=int,default=21);a=ap.parse_args();run(a.n,a.seed0
↪ ,a.layout)

```

文件: vendor/portable\_sandbox/run\_q4\_staged.py

```

import sys,math,json,time,argparse
from pathlib import Path
_HERE=Path(__file__).resolve().parent
sys.path.insert(0,str(_HERE));sys.path.insert(0,str(_HERE/'lib'))
import numpy as np
import strategy
strategy.MARGIN=2.0
from run_q4_shared_pair import SharedPairStrict
from run_q4_shared import PTS21
from run_q4_nextgen import closest_guaranteed_clear
from sim import generate_case,Simulator
from strategy2 import Params2
from geom import norm,mec,diameter_rotating_calipers_pair,polygon_area,polygon_centroid,unit

class StagedPairStrict(SharedPairStrict):
    def __init__(self,*a,max_radio_steps=8,use_preview=True,**kw):
        super().__init__(*a,**kw)
        self.max_radio_steps=max_radio_steps;self.use_preview=use_preview
        self.stage_count={};self.stage_clear_attempts={};self.stage_mode={}
        self.staged_actions=0;self.staged_measures=0;self.staged_clear_steps=0

    def _preview_base_measure(self,ch,poly):
        if self.problem==4 and ch in self.ray:
            apex,th=self.ray[ch];apex=np.asarray(apex,float);u=unit(th);n=np.array([-u[1],u[0]])
            tmax=max(float((np.asarray(p,float)-apex)@u) for p in poly);tmax=max(tmax,50.0)
            t_now=float((self.sim.pos-apex)@u);t=min(max(t_now+200.0,200.0),tmax)
            sgn=self.side.get(ch,1.0);lat=25.0*sgn if self.nosig.get(ch,0)==0 else 12.0*sgn
            return apex+t*u+lat*n
        D,(a,b)=diameter_rotating_calipers_pair(poly);area=polygon_area(poly);width=area/D if D>1e-9 else
↪ 0.0
        if width<=45.0 and D>80.0:
            u=(b-a)/D;s=float(np.clip((self.sim.pos-a)@u,0.0,D));t=s+250.0
            if t>D-40.0:t=max(D*0.5,s-250.0) if s>D*0.5 else D
            lat=0.0 if self.params.lateral_frac<=0 else
↪ 12.0;sgn=self.side.get(ch,1.0);n=np.array([-u[1],u[0]])
            return a+float(np.clip(t,0.0,D))*u+sgn*lat*n
        c=polygon_centroid(poly);v=c-self.sim.pos;L=norm(v);u=unit(self.sim.channel*37.0) if L<1e-6 else
↪ v/L;up=np.array([-u[1],u[0]]);sgn=self.side.get(ch,1.0)
        if L>700:f,lat=0.55*L,self.params.lateral_frac*L
        elif L>150:f,lat=0.45*L,0.5*L
        else:f,lat=0.15*L,max(60.0,0.9*L)
        return self.sim.pos+f*u+sgn*lat*up

    def _unattempted_lattice(self,ch,poly):
        pts=self._safe_lattice(poly,self.params.sweep_spacing)
        seen=self.stage_clear_attempts.setdefault(ch,set())
        return [p for p in pts if (round(float(p[0]),6),round(float(p[1]),6)) not in seen]

    def _preview_source_anchor(self,ch):
        poly=self.tracked[ch];c,r=mec(poly);D,_=diameter_rotating_calipers_pair(poly)

```

```

if r<=18.0+1e-9:
    q=closest_guaranteed_clear(poly,self.sim.pos,18.0,None)
    return np.asarray(q if q is not None else c,float)
if D<=self.params.sweep_max_diam or self.stage_count.get(ch,0)>=self.max_radio_steps:
    pts=self._unattempted_lattice(ch,poly)
    if pts:return np.asarray(pts[0],float)
pend=self._pair_pending.get(ch)
if pend:return np.asarray(pend[0],float)
if self.nosig.get(ch,0)==0:
    made=self._make_pair(ch,poly)
    if made is not None:
        pair,*_=made
        return np.asarray(pair[0],float)
return np.asarray(self._preview_base_measure(ch,poly),float)

def _jobs(self):
    jobs=[]
    for ch in self.tracked:
        if ch in self.cleared:continue
        poly=self.tracked[ch];D,_=diameter_rotating_calipers_pair(poly)
        anchor=self._preview_source_anchor(ch) if self.use_preview else
        ↪ np.asarray(polygon_centroid(poly),float)
        jobs.append(('clear',ch,anchor,self._dist(anchor)+0.12*D+20.0))
    if self._need_scan():
        for P in self.pending:jobs.append(('scan',None,np.asarray(P,float),self._dist(P)+6.0*len(self
        ↪ .unknown)))
    return jobs

def _mark_clear_attempt(self,ch,q):
    self.stage_clear_attempts.setdefault(ch,set()).add((round(float(q[0]),6),round(float(q[1]),6)))

def _do_clear(self,ch):
    # One bounded action per global scheduling decision.
    if ch in self.cleared or ch not in self.tracked:return
    self.staged_actions+=1
    poly=self.tracked[ch];c,r=mec(poly);D,_=diameter_rotating_calipers_pair(poly)

    # Guaranteed direct clear region.
    if r<=18.0+1e-9:
        q=closest_guaranteed_clear(poly,self.sim.pos,18.0,None)
        if q is None:q=np.asarray(c,float)
        rr,_=self.sim.clear(q,ch);self.n_clear+=1;self.staged_clear_steps+=1;self._mark_clear_attempt
        ↪ (ch,q)
        if rr=='success':self.cleared.add(ch);return
        # unexpected numerical/model mismatch: fall through to exhaustive cover on later stages
        self.stage_count[ch]=self.max_radio_steps
        return

    # Once region is small or radio budget used, execute ONE safe optical lattice center.
    if D<=self.params.sweep_max_diam or self.stage_count.get(ch,0)>=self.max_radio_steps:
        pts=self._unattempted_lattice(ch,poly)
        if not pts:raise RuntimeError(f'safe lattice exhausted ch={ch}')
        q=pts[0];rr,_=self.sim.clear(q,ch);self.n_clear+=1;self.staged_clear_steps+=1;self._mark_clea
        ↪ r_attempt(ch,q)
        if rr=='success':self.cleared.add(ch)
        return

    # One radio localization action. SharedPairStrict may make/use a certified pair.

```

```

q=self._next_measure_point(ch,poly)
res,svd=self.sim.measure(q,ch);self.n_measure+=1;self.n_home_steps+=1;self.staged_measures+=1
self.stage_count[ch]=self.stage_count.get(ch,0)+1
if res=='direction':
    self.nosig[ch]=0;self.ray[ch]=(self.sim.pos.copy(),svd);self._update_region(ch,self.sim.pos,s
    ↪ vd)
elif res=='near':
    rr,_=self.sim.clear(self.sim.pos,ch);self.n_clear+=1;self.staged_clear_steps+=1
    if rr=='success':self.cleared.add(ch)
else:
    self.nosig[ch]=self.nosig.get(ch,0)+1;self._record_nosig(ch,self.sim.pos)
    # Cheap exact-position clear attempt; failure gives no geometric cut.
    rr,_=self.sim.clear(self.sim.pos,ch);self.n_clear+=1;self.staged_clear_steps+=1;self._mark_cl
    ↪ ear_attempt(ch,self.sim.pos)
    if rr=='success':self.cleared.add(ch)

def run(n=20,seed0=700000,preview=True,maxsteps=8):
    plan={'level1':[list(p) for p in PTS21],'escalation':[]};rows=[];hard=[];ext=[];t=time.time()
    for i in range(n):
        srcs=generate_case(np.random.default_rng(seed0+i),4);sim=Simulator(srcs)
        rb=StagedPairStrict(sim,plan,Params2(name='staged',theta_miss=0,use_belief=False),4,shared_radius
        ↪ =50,fresh_only=True,max_radio_steps=maxsteps,use_preview=preview)
        try:r=rb.run()
        except Exception as e:hard.append((seed0+i,str(e)));r=rb.finish()
        rows.append(r);ext.append((rb.staged_actions,rb.staged_measures,rb.staged_clear_steps,rb.pair_act
        ↪ ions,rb.pair_clips))
    st=[r['stats'] for r in rows]
    out={'n':n,'seed0':seed0,'preview':preview,'maxsteps':maxsteps,'full':sum(s['ratio']>=1-1e-9 for s in
    ↪ st),
        'agg':sum(s['total_time_s'] for s in st)/sum(s['n_sources'] for s in
        ↪ st),'meanavg':float(np.mean([s['avg_time_s'] for s in st])),
        'mean_total':float(np.mean([s['total_time_s'] for s in st])),
        'move':float(np.mean([s['move_m']
        ↪ for s in st])),
        'req':float(np.mean([s['n_requests'] for s in st])),
        'clear':float(np.mean([s['t_clear'] for s in st])),
        'measure':float(np.mean([s['t_measure'] for s
        ↪ in st])),
        'switch':float(np.mean([s['t_switch'] for s in st])),
        'staged_actions':float(np.mean([e[0] for e in ext])),
        'staged_measures':float(np.mean([e[1] for e
        ↪ in ext])),
        'staged_clear_steps':float(np.mean([e[2] for e in ext])),
        'pair_actions':float(np.mean([e[3] for e in ext])),
        'pair_clips':float(np.mean([e[4] for e in
        ↪ ext])),
        'miss':sum(s['n_sources']-s['n_cleared'] for s in st),
        'hard':hard,'wall':time.time()-t}
    print(json.dumps(out,indent=2));return out

if __name__=='__main__':
    ap=argparse.ArgumentParser();ap.add_argument('--n',type=int,default=20);ap.add_argument('--seed0',typ
    ↪ e=int,default=700000);ap.add_argument('--no-preview',action='store_true');ap.add_argument('--maxs
    ↪ teps',type=int,default=8);a=ap.parse_args();run(a.n,a.seed0,not a.no_preview,a.maxsteps)

```

文件: vendor/portable\_sandbox/run\_q4\_staged\_routingplus.py

```

import sys,json,time,argparse
from pathlib import Path
_HERE=Path(__file__).resolve().parent
sys.path.insert(0,str(_HERE));sys.path.insert(0,str(_HERE/'lib'))
import numpy as np
import strategy
strategy.MARGIN=2.0

```

```

from run_q4_routing_ablate import PlannerMixin
from run_q4_staged import StagedPairStrict
from run_q4_shared import PTS21
from sim import generate_case, Simulator
from strategy2 import Params2

class StagedRoutingPlus(PlannerMixin, StagedPairStrict): pass

def run(n=20, seed0=700000, maxsteps=8):
    plan={'level1':[list(p) for p in PTS21], 'escalation':[]}; rows=[]; hard=[]; ext=[]; t=time.time()
    for i in range(n):
        srcs=generate_case(np.random.default_rng(seed0+i), 4); sim=Simulator(srcs)
        rb=StagedRoutingPlus(sim, plan, Params2(name='stagedrp', theta_miss=0, use_belief=False), 4, shared_rad
        ↪ ius=50, fresh_only=True, max_radio_steps=maxsteps, use_preview=True)
        try:r=rb.run()
        except Exception as e:hard.append((seed0+i, str(e))); r=rb.finish()
        rows.append(r); ext.append((rb.rp_improve, rb.staged_actions, rb.staged_measures, rb.staged_clear_ste
        ↪ ps, rb.pair_actions, rb.pair_clips))
    st=[r['stats'] for r in rows]
    out={'n':n, 'seed0':seed0, 'maxsteps':maxsteps, 'full':sum(s['ratio']>=1-1e-9 for s in
    ↪ st), 'agg':sum(s['total_time_s'] for s in st)/sum(s['n_sources'] for s in
    ↪ st), 'meanavg':float(np.mean([s['avg_time_s'] for s in
    ↪ st])), 'mean_total':float(np.mean([s['total_time_s'] for s in
    ↪ st])), 'move':float(np.mean([s['move_m'] for s in st])), 'req':float(np.mean([s['n_requests'] for s
    ↪ in st])), 'clear':float(np.mean([s['t_clear'] for s in
    ↪ st])), 'measure':float(np.mean([s['t_measure'] for s in st])), 'switch':float(np.mean([s['t_switch']
    ↪ for s in st])), 'planner_improve_m':float(np.mean([x[0] for x in
    ↪ ext])), 'staged_actions':float(np.mean([x[1] for x in ext])), 'staged_measures':float(np.mean([x[2]
    ↪ for x in ext])), 'staged_clear_steps':float(np.mean([x[3] for x in
    ↪ ext])), 'pair_actions':float(np.mean([x[4] for x in ext])), 'pair_clips':float(np.mean([x[5] for x
    ↪ in ext])), 'miss':sum(s['n_sources']-s['n_cleared'] for s in st), 'hard':hard, 'wall':time.time()-t}
    print(json.dumps(out, indent=2)); return out
if __name__=='__main__':
    ap=argparse.ArgumentParser(); ap.add_argument('--n', type=int, default=20); ap.add_argument('--seed0', typ
    ↪ e=int, default=700000); ap.add_argument('--maxsteps', type=int, default=8); a=ap.parse_args(); run(a.n,
    ↪ a.seed0, a.maxsteps)

```

文件: vendor/portable\_sandbox/run\_rho30.py

```

import sys, time, json
sys.path.insert(0, '/mnt/data/extbench/lib')
import numpy as np
from sim import generate_case
from strategy2 import Params2, run_trial_v2
strict4=[[0.0, 0.0], [-500.0, -866.0254037844386], [-1000.0, -1732.0508075688772], [-1500.0, -866.0254037844386]
    ↪ , [-2000.0, 0.0], [-1500.0, 866.0254037844386], [-1000.0, 0.0], [-500.0, 866.0254037844386], [-1000.0, 1732.050
    ↪ 8075688772], [0.0, 1732.0508075688772], [500.0, 866.0254037844386], [1000.0, 0.0], [500.0, -866.0254037844386]
    ↪ ], [0.0, -1732.0508075688772], [1000.0, -1732.0508075688772], [1500.0, -866.0254037844386], [2000.0, 0.0], [15
    ↪ 00.0, 866.0254037844386], [1000.0, 1732.0508075688772]]
plan={'level1':strict4, 'escalation':[]}
rows=[]
for i in range(30):
    rng=np.random.default_rng(700000+i); srcs=generate_case(rng, 4)
    rows.append(run_trial_v2(srcs, plan, Params2(name='v2_rho', theta_miss=0.0, use_belief=False), 4))
st=[r['stats'] for r in rows]

```

```

print(json.dumps({'full':sum(s['ratio']>=1-1e-9 for s in
↪ st),'mean_avg_time_s':float(np.mean([s['avg_time_s'] for s in
↪ st])), 'aggregate_s_per_source':float(sum(s['total_time_s'] for s in st)/sum(s['n_sources'] for s in
↪ st)), 'mean_move_m':float(np.mean([s['move_m'] for s in
↪ st])), 'mean_requests':float(np.mean([s['n_requests'] for s in
↪ st])), 'missed':sum(s['n_sources']-s['n_cleared'] for s in st), 'gave_up':sum(len(r['gave_up']) for r in
↪ rows)},indent=2))

```

## 文件: vendor/portable\_sandbox/run\_safe7\_filtered.py

```

import sys,math,json,time
from pathlib import Path
_HERE=Path(__file__).resolve().parent
sys.path.insert(0,str(_HERE));sys.path.insert(0,str(_HERE/'lib'))
import numpy as np
import strategy
strategy.MARGIN=2.0
from strict_filtered_test import StrictFiltered
from sim import generate_case,Simulator
from strategy2 import Params2

r=1150.0
PTS=[[0.,0.]]+[[r*math.cos(k*math.pi/3),r*math.sin(k*math.pi/3)] for k in range(6)]
PLAN={'level1':PTS,'escalation':[]}

def run(n=200,seed0=90000):
    rows=[];hard=[];t0=time.time()
    for i in range(n):
        srscs=generate_case(np.random.default_rng(seed0+i),3);sim=Simulator(srscs);rb=StrictFiltered(sim,PLAN,Par
↪   ams2(name='safe7',theta_miss=0.0,use_belief=False),3)
        try:r0=rb.run()
        except Exception as e:hard.append((seed0+i,str(e)));r0=rb.finish()
        rows.append(r0)
    st=[r['stats'] for r in rows]
    out={'n':n,'seed0':seed0,'full':sum(s['ratio']>=1-1e-9 for s in st),'agg':sum(s['total_time_s'] for s in
↪   st)/sum(s['n_sources'] for s in st),'meanavg':float(np.mean([s['avg_time_s'] for s in
↪   st])), 'move':float(np.mean([s['move_m'] for s in st])), 'req':float(np.mean([s['n_requests'] for s in
↪   st])), 'clear':float(np.mean([s['t_clear'] for s in st])), 'measure':float(np.mean([s['t_measure'] for s
↪   in st])), 'switch':float(np.mean([s['t_switch'] for s in
↪   st])), 'miss':sum(s['n_sources']-s['n_cleared'] for s in st), 'hard':hard, 'wall':time.time()-t0}
    print(json.dumps(out,indent=2))
if __name__=='__main__':
    import argparse
    ap=argparse.ArgumentParser();ap.add_argument('--n',type=int,default=200);ap.add_argument('--seed0',type=
↪   int,default=90000);a=ap.parse_args();run(a.n,a.seed0)

```

## 文件: vendor/portable\_sandbox/run\_safe7\_minr.py

```

import sys,math,json,time
from pathlib import Path
_HERE=Path(__file__).resolve().parent
sys.path.insert(0,str(_HERE));sys.path.insert(0,str(_HERE/'lib'))
import numpy as np
import strategy
strategy.MARGIN=2.0
from strict_filtered_test import StrictFiltered

```

```

from sim import generate_case, Simulator
from strategy2 import Params2

r=1122.95583245792
PTS=[[0.,0.]]+[[r*math.cos(k*math.pi/3),r*math.sin(k*math.pi/3)] for k in range(6)]
PLAN={'level1':PTS,'escalation':[]}

def run(n=200,seed0=90000):
    rows=[];hard=[];t0=time.time()
    for i in range(n):
        srcs=generate_case(np.random.default_rng(seed0+i),3);sim=Simulator(srcs);rb=StrictFiltered(sim,PLAN,Par
        ↪ ams2(name='safe7',theta_miss=0.0,use_belief=False),3)
        try:r0=rb.run()
        except Exception as e:hard.append((seed0+i,str(e)));r0=rb.finish()
        rows.append(r0)
    st=[r['stats'] for r in rows]
    out={'n':n,'seed0':seed0,'full':sum(s['ratio']>=1-1e-9 for s in st),'agg':sum(s['total_time_s'] for s in
    ↪ st)/sum(s['n_sources'] for s in st),'meanavg':float(np.mean([s['avg_time_s'] for s in
    ↪ st])), 'move':float(np.mean([s['move_m'] for s in st])), 'req':float(np.mean([s['n_requests'] for s in
    ↪ st])), 'clear':float(np.mean([s['t_clear'] for s in st])), 'measure':float(np.mean([s['t_measure'] for s
    ↪ in st])), 'switch':float(np.mean([s['t_switch'] for s in
    ↪ st])), 'miss':sum(s['n_sources']-s['n_cleared'] for s in st), 'hard':hard, 'wall':time.time()-t0}
    print(json.dumps(out,indent=2))
if __name__=='__main__':
    import argparse
    ap=argparse.ArgumentParser();ap.add_argument('--n',type=int,default=200);ap.add_argument('--seed0',type=
    ↪ int,default=90000);a=ap.parse_args();run(a.n,a.seed0)

```

文件: vendor/portable\_sandbox/run\_safe7\_r1130.py

```

import sys,math,json,time
from pathlib import Path
_HERE=Path(__file__).resolve().parent
sys.path.insert(0,str(_HERE));sys.path.insert(0,str(_HERE/'lib'))
import numpy as np
import strategy
strategy.MARGIN=2.0
from strict_filtered_test import StrictFiltered
from sim import generate_case, Simulator
from strategy2 import Params2

r=1130.0
PTS=[[0.,0.]]+[[r*math.cos(k*math.pi/3),r*math.sin(k*math.pi/3)] for k in range(6)]
PLAN={'level1':PTS,'escalation':[]}

def run(n=200,seed0=90000):
    rows=[];hard=[];t0=time.time()
    for i in range(n):
        srcs=generate_case(np.random.default_rng(seed0+i),3);sim=Simulator(srcs);rb=StrictFiltered(sim,PLAN,Par
        ↪ ams2(name='safe7',theta_miss=0.0,use_belief=False),3)
        try:r0=rb.run()
        except Exception as e:hard.append((seed0+i,str(e)));r0=rb.finish()
        rows.append(r0)
    st=[r['stats'] for r in rows]

```

```

out={'n':n,'seed0':seed0,'full':sum(s['ratio']>=1-1e-9 for s in st),'agg':sum(s['total_time_s'] for s in
↪ st)/sum(s['n_sources'] for s in st),'meanavg':float(np.mean([s['avg_time_s'] for s in
↪ st])), 'move':float(np.mean([s['move_m'] for s in st])), 'req':float(np.mean([s['n_requests'] for s in
↪ st])), 'clear':float(np.mean([s['t_clear'] for s in st])), 'measure':float(np.mean([s['t_measure'] for s
↪ in st])), 'switch':float(np.mean([s['t_switch'] for s in
↪ st])), 'miss':sum(s['n_sources']-s['n_cleared'] for s in st), 'hard':hard, 'wall':time.time()-t0}
print(json.dumps(out,indent=2))
if __name__=='__main__':
    import argparse
    ap=argparse.ArgumentParser();ap.add_argument('--n',type=int,default=200);ap.add_argument('--seed0',type=
↪ int,default=90000);a=ap.parse_args();run(a.n,a.seed0)

```

## 文件: vendor/portable\_sandbox/strict\_filtered\_test.py

```

import sys,time,json,math,argparse
sys.path.insert(0,'/mnt/data/extbench/lib')
import numpy as np
import strategy,strategy2
from sim import generate_case,Simulator
from strategy2 import Params2,RobotV2
from geom import norm,cross

def identity_simplify(poly,tol=0.05): return [np.asarray(p,float).copy() for p in poly]
strategy._simplify=identity_simplify; strategy2._simplify=identity_simplify

STRICT3=[[0.0,0.0],[1080.8618318011463,0.0],[673.9063293463646,845.0518083002004],[-240.51438429012381,10
↪ 53.762369034913],[-973.8228609568137,468.96837305133465],[-973.8228609568138,-468.9683730513344],[-24
↪ 0.51438429012407,-1053.762369034913],[673.9063293463643,-845.0518083002006]]
def mincert25():
    p=[[0.0,0.0]]
    for k in range(12):
        a=math.radians(15+30*k);p.append([950*math.cos(a),950*math.sin(a)])
    for k in range(12):
        a=math.radians(30*k);p.append([1875*math.cos(a),1875*math.sin(a)])
    return p
STRICT4=mincert25()

def inside_convex(q,P):
    # P expected CCW from clipping; tolerate either orientation by sign consistency.
    m=len(P); signs=[]
    for i in range(m):
        a=P[i];b=P[(i+1)%m];v=b-a;w=q-a;z=v[0]*w[1]-v[1]*w[0]
        if abs(z)>1e-9:signs.append(1 if z>0 else -1)
    return not signs or all(s==signs[0] for s in signs)

def segdist(q,a,b):
    v=b-a;d=float(v@v)
    if d<=1e-18:return norm(q-a)
    t=max(0.0,min(1.0,float((q-a)@v/d)));return norm(q-(a+t*v))

def polydist(q,P):
    if inside_convex(q,P): return 0.0
    return min(segdist(q,P[i],P[(i+1)%len(P)]) for i in range(len(P)))

class StrictFiltered(RobotV2):
    def __init__(self,*a,**kw):super().__init__(*a,**kw);self.hard=[]
    def _safe_lattice(self,poly,spacing=33.0):

```

```

P=np.asarray(poly,float); rcov=spacing/math.sqrt(3.0); pad=rcov+1e-6
lo=P.min(axis=0)-pad;hi=P.max(axis=0)+pad;dy=spacing*math.sqrt(3)/2
r0=math.floor(lo[1]/dy)-1;r1=math.ceil(hi[1]/dy)+1;pts=[]
for r in range(r0,r1+1):
    y=r*dy;xoff=spacing/2 if (r&1) else
    ↪ 0.0;c0=math.floor((lo[0]-xoff)/spacing)-1;c1=math.ceil((hi[0]-xoff)/spacing)+1
    for c in range(c0,c1+1):
        x=xoff+c*spacing;q=np.array([x,y],float)
        if polydist(q,P)<=rcov+1e-7:pts.append(q)
cur=self.sim.pos.copy();out=[];left=pts
while left:
    k=min(range(len(left)),key=lambda i:norm(left[i]-cur));cur=left[k];out.append(left.pop(k))
return out
def _clear_sweep(self,ch,poly):
    for p in self._safe_lattice(poly,self.params.sweep_spacing):
        r,_=self.sim.clear(p,ch);self.n_clear+=1
        if r=='success':return True
    return False
def _do_clear(self,ch):
    ok=self.home_and_clear(ch)
    if not ok:
        poly=self.tracked.get(ch)
        if poly is not None and len(poly)>=3:ok=self._clear_sweep(ch,poly)
    if ok:self.cleared.add(ch)
    else:self.hard.append(ch);raise RuntimeError('strict fallback failed')

def run(problem,seed0,n,mec18=False):
    if mec18:strategy.MARGIN=2.0
    else:strategy.MARGIN=0.5
    plan={'level1':STRICT3 if problem==3 else STRICT4,'escalation':[]}; rows=[];hard=[];t0=time.time()
    for i in range(n):
        srCs=generate_case(np.random.default_rng(seed0+i),problem);sim=Simulator(srCs);rb=StrictFiltered(sim,pl
        ↪ an,Params2(name='sf',theta_miss=0.0,use_belief=False),problem)
        try:r=rb.run()
        except Exception as e:hard.append((seed0+i,str(e)));r=rb.finish()
        rows.append(r)
    st=[r['stats'] for r in rows]
    out={'problem':problem,'n':n,'full':sum(s['ratio']>=1-1e-9 for s in st),'agg':sum(s['total_time_s'] for s
    ↪ in st)/sum(s['n_sources'] for s in st),'meanavg':float(np.mean([s['avg_time_s'] for s in
    ↪ st])), 'move':float(np.mean([s['move_m'] for s in st])), 'req':float(np.mean([s['n_requests'] for s in
    ↪ st])), 'clear':float(np.mean([s['t_clear'] for s in st])), 'measure':float(np.mean([s['t_measure'] for s
    ↪ in st])), 'switch':float(np.mean([s['t_switch'] for s in
    ↪ st])), 'miss':sum(s['n_sources']-s['n_cleared'] for s in
    ↪ st), 'hard':hard, 'wall':time.time()-t0};print(json.dumps(out,indent=2))
if __name__=='__main__':
    ap=argparse.ArgumentParser();ap.add_argument('--problem',type=int,required=True);ap.add_argument('--seed
    ↪ 0',type=int);ap.add_argument('--n',type=int,default=100);ap.add_argument('--mec18',action='store_tru
    ↪ e');a=ap.parse_args();run(a.problem,a.seed0 or (90000 if a.problem==3 else 700000),a.n,a.mec18)

```

文件: vendor/portable\_sandbox/strict\_q3\_mec18.py

```

import sys,json,time,argparse
from pathlib import Path
_HERE=Path(__file__).resolve().parent
sys.path.insert(0,str(_HERE));sys.path.insert(0,str(_HERE/'lib'))
import strict_q3_test as sq
import strategy

```

```

strategy.MARGIN=2.0
import numpy as np
from sim import generate_case, Simulator
from strategy2 import Params2

def run(seed0,n):
    rows=[];hard=[];t0=time.time()
    for i in range(n):
        srcs=generate_case(np.random.default_rng(seed0+i),3);sim=Simulator(srcs);rb=sq.StrictQ3(sim,sq.PLAN,Par
        ↪ ams2(name='mec18',theta_miss=0.0,use_belief=False),3)
        try:r=rb.run()
        except Exception as e:hard.append((seed0+i,str(e)));r=rb.finish()
        rows.append(r)
    st=[r['stats'] for r in rows]
    print(json.dumps({'n':n,'full':sum(s['ratio']>=1-1e-9 for s in st),'agg':sum(s['total_time_s'] for s in
    ↪ st)/sum(s['n_sources'] for s in st),'meanavg':float(np.mean([s['avg_time_s'] for s in
    ↪ st])), 'move':float(np.mean([s['move_m'] for s in st])), 'req':float(np.mean([s['n_requests'] for s in
    ↪ st])), 'clear':float(np.mean([s['t_clear'] for s in st])), 'miss':sum(s['n_sources']-s['n_cleared'] for
    ↪ s in st), 'hard':hard, 'wall':time.time()-t0},indent=2))
if __name__=='__main__':
    ap=argparse.ArgumentParser();ap.add_argument('--seed0',type=int,default=90000);ap.add_argument('--n',typ
    ↪ e=int,default=200);a=ap.parse_args();run(a.seed0,a.n)

```

文件: vendor/portable\_sandbox/strict\_q3\_test.py

```

import sys,time,json,math
sys.path.insert(0,'/mnt/data/extbench/lib')
import numpy as np
import strategy, strategy2
from sim import generate_case, Simulator
from strategy2 import Params2, RobotV2
from geom import norm, polygon_centroid

STRICT3=[[0.0,0.0],[1080.8618318011463,0.0],[673.9063293463646,845.0518083002004],[-240.51438429012381,10
    ↪ 53.762369034913],[-973.8228609568137,468.96837305133465],[-973.8228609568138,-468.9683730513344],[-24
    ↪ 0.51438429012407,-1053.762369034913],[673.9063293463643,-845.0518083002006]]
PLAN={'level1':STRICT3,'escalation':[]}]

# Disable under-approximating polygon simplification globally.
def identity_simplify(poly,tol=0.05):
    return [np.asarray(p,float).copy() for p in poly]
strategy._simplify=identity_simplify
strategy2._simplify=identity_simplify

class StrictQ3(RobotV2):
    def __init__(self,*a,**kw):
        super().__init__(*a,**kw); self.hard_fail=[]
    def _safe_lattice(self, poly, spacing=33.0):
        # Full-plane triangular lattice over expanded bbox. Keeping centers outside the polygon
        # preserves the spacing/sqrt(3) covering guarantee at polygon boundaries.
        P=np.asarray(poly,float)
        lo=P.min(axis=0)-spacing; hi=P.max(axis=0)+spacing
        dy=spacing*math.sqrt(3)/2.0
        r0=math.floor(lo[1]/dy)-1; r1=math.ceil(hi[1]/dy)+1
        pts=[]
        for r in range(r0,r1+1):
            y=r*dy; xoff=spacing/2.0 if (r&1) else 0.0

```

```

        c0=math.floor((lo[0]-xoff)/spacing)-1; c1=math.ceil((hi[0]-xoff)/spacing)+1
        for c in range(c0,c1+1):
            x=xoff+c*spacing
            if lo[0]<=x<=hi[0] and lo[1]<=y<=hi[1]: pts.append(np.array([x,y],float))
        # nearest-neighbor ordering from current position only affects time, not coverage.
        cur=self.sim.pos.copy(); out=[]; left=pts
        while left:
            k=min(range(len(left)),key=lambda i:norm(left[i]-cur)); cur=left[k]; out.append(left.pop(k))
        return out
def _clear_sweep(self,ch,poly):
    for p in self._safe_lattice(poly,self.params.sweep_spacing):
        r,_=self.sim.clear(p,ch); self.n_clear+=1
        if r=='success': return True
    return False
def _do_clear(self,ch):
    ok=self.home_and_clear(ch)
    if not ok:
        poly=self.tracked.get(ch)
        if poly is not None and len(poly)>=3:
            # No 900-point cutoff in ultimate fallback: correctness first.
            ok=self._clear_sweep(ch,poly)
    if ok:
        self.cleared.add(ch)
    else:
        self.hard_fail.append(ch)
        # keep it unresolved, but avoid infinite loop by stopping this run explicitly
        raise RuntimeError(f'strict fallback failed ch={ch}')

def run(seed0,n):
    rows=[]; hard=[]; t0=time.time()
    for i in range(n):
        seed=seed0+i; sracs=generate_case(np.random.default_rng(seed),3); sim=Simulator(sracs)
        rb=StrictQ3(sim,PLAN,Params2(name='strictq3',theta_miss=0.0,use_belief=False),3)
        try: res=rb.run()
        except Exception as e:
            hard.append((seed,str(e))); res=rb.finish()
        rows.append((seed,res,rb))
    st=[r[1]['stats'] for r in rows]
    out={
        'seed0':seed0,'n':n,'full':sum(s['ratio']>=1-1e-9 for s in st),'hard_fail':hard,
        'mean_ratio':float(np.mean([s['ratio'] for s in st])),
        'mean_avg_s_per_source':float(np.mean([s['avg_time_s'] for s in st if s['avg_time_s'] is not None])),
        'aggregate_s_per_source':float(sum(s['total_time_s'] for s in st)/sum(s['n_sources'] for s in st)),
        'mean_total_time':float(np.mean([s['total_time_s'] for s in st])),
        'mean_move_m':float(np.mean([s['move_m'] for s in st])),
        'mean_requests':float(np.mean([s['n_requests'] for s in st])),
        'total_missed':int(sum(s['n_sources']-s['n_cleared'] for s in st)),
        't_move':float(np.mean([s['t_move'] for s in st])), 't_measure':float(np.mean([s['t_measure'] for s
        ↪ in st])),
        't_switch':float(np.mean([s['t_switch'] for s in st])), 't_clear':float(np.mean([s['t_clear'] for s
        ↪ in st])),
        'wall_s':time.time()-t0,
    }
    return out
if __name__=='__main__':
    import argparse
    ap=argparse.ArgumentParser(); ap.add_argument('--seed0',type=int,default=90000);
    ↪ ap.add_argument('--n',type=int,default=1000); a=ap.parse_args()

```

```
print(json.dumps(run(a.seed0,a.n),indent=2))
```

## 文件: vendor/portable\_sandbox/strict\_q4\_mec18.py

```
import sys,json,time,argparse
from pathlib import Path
_HERE=Path(__file__).resolve().parent
sys.path.insert(0,str(_HERE));sys.path.insert(0,str(_HERE/'lib'))
import strict_q4_mincert25 as sq
import strategy
# stricter deterministic threshold: MEC radius <= 18m
strategy.MARGIN=2.0
from sim import generate_case,Simulator
from strategy2 import Params2
import numpy as np

def run(seed0,n):
    rows=[];t0=time.time();hard=[]
    for i in range(n):
        srcs=generate_case(np.random.default_rng(seed0+i),4);sim=Simulator(srcs);rb=sq.StrictQ4(sim,sq.PLAN,Par
        ↪ ams2(name='mec18',theta_miss=0.0,use_belief=False),4)
        try:r=rb.run()
        except Exception as e:hard.append((seed0+i,str(e)));r=rb.finish()
        rows.append(r)
    st=[r['stats'] for r in rows]
    print(json.dumps({'n':n,'full':sum(s['ratio']>=1-1e-9 for s in st),'agg':sum(s['total_time_s'] for s in
    ↪ st)/sum(s['n_sources'] for s in st),'meanavg':float(np.mean([s['avg_time_s'] for s in
    ↪ st])), 'move':float(np.mean([s['move_m'] for s in st])), 'req':float(np.mean([s['n_requests'] for s in
    ↪ st])), 'clear':float(np.mean([s['t_clear'] for s in st])), 'miss':sum(s['n_sources']-s['n_cleared'] for
    ↪ s in st),'hard':hard,'wall':time.time()-t0},indent=2))
if __name__=='__main__':
    ap=argparse.ArgumentParser();ap.add_argument('--seed0',type=int,default=700000);ap.add_argument('--n',ty
    ↪ pe=int,default=20);a=ap.parse_args();run(a.seed0,a.n)
```

## 文件: vendor/portable\_sandbox/strict\_q4\_mincert25.py

```
import sys,time,json,math,argparse
sys.path.insert(0,'/mnt/data/extbench/lib')
import numpy as np
import strategy,strategy2
from sim import generate_case,Simulator
from strategy2 import Params2,RobotV2
from geom import norm

def identity_simplify(poly,tol=0.05): return [np.asarray(p,float).copy() for p in poly]
strategy._simplify=identity_simplify; strategy2._simplify=identity_simplify

def mincert25():
    pts=[[0.0,0.0]]
    for k in range(12):
        a=math.radians(15+30*k);pts.append([950*math.cos(a),950*math.sin(a)])
    for k in range(12):
        a=math.radians(30*k);pts.append([1875*math.cos(a),1875*math.sin(a)])
    return pts
PLAN={'level1':mincert25(),'escalation':[]}
```

```

class StrictQ4(RobotV2):
    def __init__(self,*a,**kw): super().__init__(*a,**kw);self.hard_fail=[]
    def _safe_lattice(self,poly,spacing=33.0):
        P=np.asarray(poly,float);lo=P.min(axis=0)-spacing;hi=P.max(axis=0)+spacing;dy=spacing*math.sqrt(3)
        ↪ )/2
        r0=math.floor(lo[1]/dy)-1;r1=math.ceil(hi[1]/dy)+1;pts=[]
        for r in range(r0,r1+1):
            y=r*dy;xoff=spacing/2 if (r&1) else
            ↪ 0.0;c0=math.floor((lo[0]-xoff)/spacing)-1;c1=math.ceil((hi[0]-xoff)/spacing)+1
            for c in range(c0,c1+1):
                x=xoff+c*spacing
                if lo[0]<=x<=hi[0] and lo[1]<=y<=hi[1]:pts.append(np.array([x,y],float))
        cur=self.sim.pos.copy();out=[];left=pts
        while left:
            k=min(range(len(left)),key=lambda i:norm(left[i]-cur));cur=left[k];out.append(left.pop(k))
        return out
    def _clear_sweep(self,ch,poly):
        for p in self._safe_lattice(poly,self.params.sweep_spacing):
            r,_=self.sim.clear(p,ch);self.n_clear+=1
            if r=='success':return True
        return False
    def _do_clear(self,ch):
        ok=self.home_and_clear(ch)
        if not ok:
            poly=self.tracked.get(ch)
            if poly is not None and len(poly)>=3: ok=self._clear_sweep(ch,poly)
        if ok:self.cleared.add(ch)
        else:
            self.hard_fail.append(ch);raise RuntimeError(f'strict fallback failed ch={ch}')

def run(seed0,n):
    rows=[];hard=[];t0=time.time()
    for i in range(n):
        seed=seed0+i;srcs=generate_case(np.random.default_rng(seed),4);sim=Simulator(srcs);rb=StrictQ4(si
        ↪ m,PLAN,Params2(name='strictq4',theta_miss=0.0,use_belief=False),4)
        try:res=rb.run()
        except Exception as e:hard.append((seed,str(e)));res=rb.finish()
        rows.append((seed,res,rb))
    st=[r[1]['stats'] for r in rows]
    out={'seed0':seed0,'n':n,'full':sum(s['ratio']>=1-1e-9 for s in st),'hard_fail':hard,
        'mean_ratio':float(np.mean([s['ratio'] for s in st])),
        'mean_avg_s_per_source':float(np.mean([s['avg_time_s'] for s in st if s['avg_time_s'] is not
        ↪ None])),
        'aggregate_s_per_source':float(sum(s['total_time_s'] for s in st)/sum(s['n_sources'] for s in
        ↪ st)),
        'mean_total_time':float(np.mean([s['total_time_s'] for s in st])),
        'mean_move_m':float(np.mean([s['move_m'] for s in st])),
        ↪ 'mean_requests':float(np.mean([s['n_requests'] for s in st])),
        'total_missed':int(sum(s['n_sources']-s['n_cleared'] for s in st)),
        'time_breakdown':{k:float(np.mean([s[k] for s in st])) for k in
        ↪ ('t_move','t_switch','t_measure','t_clear')},
        'wall_s':time.time()-t0}
    return out
if __name__=='__main__':
    ap=argparse.ArgumentParser();ap.add_argument('--seed0',type=int,default=700000);ap.add_argument('--n'
    ↪ ,type=int,default=20);a=ap.parse_args();print(json.dumps(run(a.seed0,a.n),indent=2))

```

文件: vendor/portable\_sandbox/strict\_q4\_sched\_ablate.py

```
import sys,json,time,argparse
sys.path.insert(0,'/mnt/data/extbench')
sys.path.insert(0,'/mnt/data/extbench/lib')
import numpy as np
from strict_q4_mincert25 import StrictQ4, PLAN
from sim import generate_case, Simulator
from strategy2 import Params2
from geom import norm

class Nearest(StrictQ4):
    def _plan_next(self,jobs):
        d=[norm(j[2]-self.sim.pos) for j in jobs];m=min(d);cand=[i for i,x in enumerate(d) if
        ↪ abs(x-m)<1e-9];cand.sort(key=lambda i:0 if jobs[i][0]=='clear' else 1);return jobs[cand[0]]
class ScanFirst(StrictQ4):
    def _jobs(self):
        jobs=super()._jobs();s=[j for j in jobs if j[0]=='scan'];return s if s else [j for j in jobs if
        ↪ j[0]=='clear']
class SourceFirst(StrictQ4):
    def _jobs(self):
        jobs=super()._jobs();c=[j for j in jobs if j[0]=='clear'];return c if c else [j for j in jobs if
        ↪ j[0]=='scan']
CL={'baseline':StrictQ4,'nearest':Nearest,'scanfirst':ScanFirst,'sourcefirst':SourceFirst}

def run(v,n,seed0):
    rows=[];t0=time.time(); hard=[]
    for i in range(n):
        srcs=generate_case(np.random.default_rng(seed0+i),4);sim=Simulator(srcs);rb=CL[v](sim,PLAN,Params2
        ↪ 2(name=v,theta_miss=0.0,use_belief=False),4)
        try:r=rb.run()
        except Exception as e:hard.append((seed0+i,str(e)));r=rb.finish()
        rows.append(r)
    st=[r['stats'] for r in rows]
    o={'v':v,'n':n,'full':sum(s['ratio']>=1-1e-9 for s in st),'agg':sum(s['total_time_s'] for s in
    ↪ st)/sum(s['n_sources'] for s in st),'meanavg':float(np.mean([s['avg_time_s'] for s in
    ↪ st])), 'move':float(np.mean([s['move_m'] for s in st])), 'req':float(np.mean([s['n_requests'] for s
    ↪ in st])), 'tclear':float(np.mean([s['t_clear'] for s in
    ↪ st])), 'miss':sum(s['n_sources']-s['n_cleared'] for s in
    ↪ st), 'hard':hard, 'wall':time.time()-t0};print(json.dumps(o))
if __name__=='__main__':
    ap=argparse.ArgumentParser();ap.add_argument('v');ap.add_argument('--n',type=int,default=10);ap.add_a
    ↪ rgument('--seed0',type=int,default=700000);a=ap.parse_args();run(a.v,a.n,a.seed0)
```

文件: vendor/q3\_breakthrough\_sandbox/lib/geom.py

""" 几何与定位区域算法库 (CUMCM2026 B 题)

约定:

角度单位为度, 逆时针为正, x 轴正向为 0 度.

半平面统一写成  $n \cdot p \leq c$  ( $n$  为单位外法向).

凸多边形用顶点列表表示 (逆时针, 首尾不重复).

"""

from \_\_future\_\_ import annotations

import math

import numpy as np

```

EPS = 1e-12

def unit(deg: float) -> np.ndarray:
    r = math.radians(deg)
    return np.array([math.cos(r), math.sin(r)])

def cross(a: np.ndarray, b: np.ndarray) -> float:
    return float(a[0] * b[1] - a[1] * b[0])

def dot(a: np.ndarray, b: np.ndarray) -> float:
    return float(a[0] * b[0] + a[1] * b[1])

def norm(a: np.ndarray) -> float:
    return float(math.hypot(a[0], a[1]))

def ang_deg(v: np.ndarray) -> float:
    return math.degrees(math.atan2(v[1], v[0])) % 360.0

def ang_diff(a: float, b: float) -> float:
    d = (a - b) % 360.0
    return d - 360.0 if d > 180.0 else d

def wedge_halfplanes(S, theta_deg: float, eps_deg: float):
    hps = []
    for sgn in (+1.0, -1.0):
        d = unit(theta_deg + sgn * eps_deg)
        n = np.array([-d[1], d[0]])
        if sgn > 0:
            hps.append((n, dot(n, S)))
        else:
            hps.append((-n, -dot(n, S)))
    return hps

def hp_feasible_point(hps):
    n = len(hps)
    A = np.array([h[0] for h in hps])
    b = np.array([h[1] for h in hps])
    x = np.zeros(2)
    for _ in range(200):
        v = A @ x - b
        act = v > 0
        if not act.any():
            return x
        Aa = A[act]
        g = 2.0 * Aa.T @ (Aa @ x - b[act])
        step = 1.0 / (2.0 * len(Aa) + 1e-9)
        x = x - step * g
    v = A @ x - b
    return x if (v <= 1e-7).all() else None

def is_bounded(hps) -> bool:
    ang = sorted(math.degrees(math.atan2(h[0][1], h[0][0])) % 360.0 for h in hps)
    if len(ang) < 2:
        return False
    gaps = [ang[i + 1] - ang[i] for i in range(len(ang) - 1)]
    gaps.append(ang[0] + 360.0 - ang[-1])
    return max(gaps) < 180.0 - 1e-9

```

```

def clip_polygon(poly, hp):
    if not len(poly):
        return poly
    n, c = hp
    out = []
    m = len(poly)
    for i in range(m):
        a = poly[i]
        b = poly[(i + 1) % m]
        va = dot(n, a) - c
        vb = dot(n, b) - c
        if va <= 1e-12:
            out.append(a)
        if (va < -1e-12 < vb) or (vb < -1e-12 < va):
            t = va / (va - vb)
            out.append(a + t * (b - a))
    return dedup(out)

def dedup(poly, tol=1e-9):
    if len(poly) < 2:
        return poly
    out = []
    for p in poly:
        if not out or norm(p - out[-1]) > tol:
            out.append(p)
    if len(out) > 1 and norm(out[0] - out[-1]) <= tol:
        out.pop()
    return out

def polygon_area(poly) -> float:
    m = len(poly)
    if m < 3:
        return 0.0
    s = 0.0
    for i in range(m):
        a, b = poly[i], poly[(i + 1) % m]
        s += a[0] * b[1] - b[0] * a[1]
    return abs(s) / 2.0

def polygon_centroid(poly) -> np.ndarray:
    poly = [np.asarray(p, dtype=float) for p in poly]
    m = len(poly)
    if m == 0:
        return np.zeros(2)
    if m == 1:
        return poly[0].copy()
    if m == 2:
        return (poly[0] + poly[1]) / 2.0
    a = 0.0
    cx = cy = 0.0
    for i in range(m):
        p, q = poly[i], poly[(i + 1) % m]
        cr = p[0] * q[1] - q[0] * p[1]
        a += cr
        cx += (p[0] + q[0]) * cr
        cy += (p[1] + q[1]) * cr
    if abs(a) < 1e-12:
        return np.mean(np.array(poly), axis=0)
    return np.array([cx / (3.0 * a), cy / (3.0 * a)])

```

```

def offset_polygon(poly, d):
    P = [np.asarray(p, dtype=float) for p in poly]
    m = len(P)
    if m < 3:
        return P
    lines = []
    for i in range(m):
        a, b = P[i], P[(i + 1) % m]
        e = b - a
        L = norm(e)
        if L < 1e-12:
            continue
        n = np.array([e[1], -e[0]]) / L
        lines.append((a + d * n, b + d * n))
    out = []
    for i in range(len(lines)):
        p1, p2 = lines[i]
        q1, q2 = lines[(i - 1) % len(lines)]
        r, s = p2 - p1, q2 - q1
        den = cross(r, s)
        if abs(den) < 1e-12:
            out.append(p1)
            continue
        t = cross(q1 - p1, s) / den
        out.append(p1 + t * r)
    return dedup(out)

def polygon_perimeter(poly) -> float:
    m = len(poly)
    if m < 2:
        return 0.0
    return float(sum(norm(poly[(i + 1) % m] - poly[i]) for i in range(m)))

def convex_hull(pts):
    pts = sorted([(round(float(p[0]), 9), round(float(p[1]), 9)) for p in pts])
    if len(pts) <= 2:
        return [np.array(p) for p in pts]
    def half(points):
        st = []
        for p in points:
            while len(st) >= 2 and cross(st[-1] - st[-2], np.array(p) - st[-1]) <= 0:
                st.pop()
            st.append(np.array(p))
        return st
    lo = half(pts)
    up = half(pts[::-1])
    return dedup(lo[:-1] + up[:-1])

_CIRC_CACHE = {}

def cached_circ_polygon(r: float, n: int = 360):
    key = (round(float(r), 6), int(n))
    if key not in _CIRC_CACHE:
        _CIRC_CACHE[key] = circ_polygon(key[0], key[1])
    return [p.copy() for p in _CIRC_CACHE[key]]

def circ_polygon(r: float, n: int = 360, center=(0.0, 0.0), circumscribe=True):
    c = np.array(center, dtype=float)

```

```

rr = r / math.cos(math.pi / n) if circumscribe else r
return [c + rr * unit(360.0 * i / n) for i in range(n)]

def localization_polygon(obs, arena=None, box=4.0e6):
    hps = []
    for S, th, eps in obs:
        hps.extend(wedge_halfplanes(np.asarray(S, dtype=float), th, eps))
    bounded_geom = is_bounded(hps)
    if isinstance(arena, (int, float)):
        seed = cached_circ_polygon(float(arena), 360)
    elif arena is None:
        b = box
        seed = [np.array([-b, -b]), np.array([b, -b]), np.array([b, b]), np.array([-b, b])]
    else:
        seed = [np.asarray(p, dtype=float) for p in arena]
    poly = [np.asarray(p, dtype=float) for p in seed]
    for hp in hps:
        poly = clip_polygon(poly, hp)
        if len(poly) == 0:
            return [], bounded_geom
    if len(poly) >= 3 and polygon_area(poly) < 1e-9:
        return [], bounded_geom
    if len(poly) > 12:
        poly = simplify_polygon(poly, tol=0.02)
    return poly, bounded_geom

def diameter_bruteforce(poly) -> float:
    m = len(poly)
    best = 0.0
    for i in range(m):
        for j in range(i + 1, m):
            best = max(best, norm(poly[i] - poly[j]))
    return best

def diameter_pair_bruteforce(poly):
    m = len(poly)
    best, bi, bj = 0.0, 0, 0
    for i in range(m):
        for j in range(i + 1, m):
            d = norm(poly[i] - poly[j])
            if d > best:
                best, bi, bj = d, i, j
    return best, (poly[bi] if m else None), (poly[bj] if m else None)

def diameter_rotating_calipers_pair(poly):
    pts = convex_hull(poly)
    m = len(pts)
    if m < 2:
        return 0.0, (None, None)
    if m == 2:
        return norm(pts[1] - pts[0]), (pts[0], pts[1])
    def area2(a, b, c):
        return abs(cross(b - a, c - a))
    j = 1
    best, bp = 0.0, (pts[0], pts[1])
    for i in range(m):
        ni = (i + 1) % m
        while True:
            nj = (j + 1) % m

```

```

        if area2(pts[i], pts[ni], pts[nj]) > area2(pts[i], pts[ni], pts[j]) + 1e-15:
            j = nj
        else:
            break
    for cand in (pts[i], pts[ni]):
        d = norm(cand - pts[j])
        if d > best:
            best, bp = d, (cand, pts[j])
    return best, bp

def diameter_rotating_calipers(poly) -> float:
    return diameter_rotating_calipers_pair(poly)[0]

def diameter_support_scan(poly, extra_angles=720) -> float:
    pts = convex_hull(poly)
    if len(pts) < 2:
        return 0.0
    angs = []
    m = len(pts)
    for i in range(m):
        e = pts[(i + 1) % m] - pts[i]
        angs.append(math.atan2(e[1], e[0]) + math.pi / 2.0)
        angs.append(math.atan2(e[1], e[0]) - math.pi / 2.0)
    angs += list(np.linspace(0, 2 * math.pi, extra_angles, endpoint=False))
    P = np.array(pts)
    best = 0.0
    for a in angs:
        n = np.array([math.cos(a), math.sin(a)])
        h1 = float((P @ n).max())
        h2 = float((P @ (-n)).max())
        best = max(best, h1 + h2)
    return best

def mec_bruteforce(pts):
    P = [np.asarray(p, dtype=float) for p in pts]
    n = len(P)
    if n == 0:
        return np.zeros(2), 0.0
    if n == 1:
        return P[0].copy(), 0.0
    def ok(c, r):
        return all(norm(p - c) <= r + 1e-9 for p in P)
    best_c, best_r = None, float("inf")
    for i in range(n):
        for j in range(i + 1, n):
            c = (P[i] + P[j]) / 2.0
            r = norm(P[i] - P[j]) / 2.0
            if r < best_r - 1e-12 and ok(c, r):
                best_c, best_r = c, r
    for i in range(n):
        for j in range(i + 1, n):
            for k in range(j + 1, n):
                a, b, c3 = P[i], P[j], P[k]
                d = 2.0 * (a[0] * (b[1] - c3[1]) + b[0] * (c3[1] - a[1]) + c3[0] * (a[1] - b[1]))
                if abs(d) < 1e-12:
                    continue
                ux = ((a @ a) * (b[1] - c3[1]) + (b @ b) * (c3[1] - a[1]) + (c3 @ c3) * (a[1] - b[1])) / d
                uy = ((a @ a) * (c3[0] - b[0]) + (b @ b) * (a[0] - c3[0]) + (c3 @ c3) * (b[0] - a[0])) / d
                c = np.array([ux, uy])

```

```

        r = norm(a - c)
        if r < best_r - 1e-12 and ok(c, r):
            best_c, best_r = c, r
    return best_c, best_r

def mec_minidisk(poly, rng=None):
    pts = [np.asarray(p, dtype=float) for p in poly]
    if not pts:
        return np.zeros(2), 0.0
    rng = rng or np.random.default_rng(12345)
    idx = rng.permutation(len(pts))
    P = [pts[i] for i in idx]
    c = P[0].copy()
    r = 0.0
    for i in range(1, len(P)):
        p = P[i]
        if norm(p - c) <= r + 1e-9:
            continue
        c, r = p.copy(), 0.0
        for j in range(i):
            q = P[j]
            if norm(q - c) <= r + 1e-9:
                continue
            c = (p + q) / 2.0
            r = norm(p - q) / 2.0
            for k in range(j):
                s = P[k]
                if norm(s - c) <= r + 1e-9:
                    continue
            c, r = circumcircle(p, q, s)
    for _ in range(4):
        bad = [p for p in pts if norm(p - c) > r + 1e-7]
        if not bad:
            break
        r = max(norm(p - c) for p in pts)
    return c, r

def circumcircle(a, b, c):
    d = 2.0 * (a[0] * (b[1] - c[1]) + b[0] * (c[1] - a[1]) + c[0] * (a[1] - b[1]))
    if abs(d) < 1e-14:
        return mec_bruteforce([a, b, c])
    ux = ((a @ a) * (b[1] - c[1]) + (b @ b) * (c[1] - a[1]) + (c @ c) * (a[1] - b[1])) / d
    uy = ((a @ a) * (c[0] - b[0]) + (b @ b) * (a[0] - c[0]) + (c @ c) * (b[0] - a[0])) / d
    ct = np.array([ux, uy])
    return ct, norm(a - ct)

def mec(poly, sample_cap=64, rng=None):
    return mec_minidisk(poly, rng=rng)

def simplify_polygon(poly, tol=0.02):
    m = len(poly)
    if m <= 4:
        return [np.asarray(p, dtype=float) for p in poly]
    keep = [0]
    for i in range(1, m - 1):
        a = poly[keep[-1]]
        b = poly[i]
        c = poly[i + 1]
        e = c - a

```

```

        L = norm(e)
        dist = norm(b - a) if L < 1e-12 else abs(cross(e, b - a)) / L
        if dist >= tol:
            keep.append(i)
    keep.append(m - 1)
    out = [np.asarray(poly[i], dtype=float) for i in keep]
    return dedup(out)

def diameter_disk_covers(poly, pair) -> bool:
    a, b = pair
    if a is None or b is None:
        return True
    scale = max(norm(a - b) ** 2, 1.0)
    return all(dot(p - a, p - b) <= 1e-9 * scale for p in poly)

def sample_in_polygon(poly, n, rng):
    if len(poly) < 3:
        return np.zeros((0, 2))
    P = np.array(poly, dtype=float)
    tris, areas = [], []
    for i in range(1, len(poly) - 1):
        tris.append((P[0], P[i], P[i + 1]))
        areas.append(abs(cross(P[i] - P[0], P[i + 1] - P[0])) / 2.0)
    areas = np.array(areas)
    tot = areas.sum()
    if tot <= 0:
        return np.zeros((0, 2))
    idx = rng.choice(len(tris), size=n, p=areas / tot)
    out = np.zeros((n, 2))
    for k in range(n):
        a, b, c = tris[idx[k]]
        u, v = rng.random(), rng.random()
        if u + v > 1:
            u, v = 1 - u, 1 - v
        out[k] = a + u * (b - a) + v * (c - a)
    return out

def cover_lattice_points(poly, spacing, start=None, shrink=1.0):
    if len(poly) < 3:
        return []
    P = np.array(poly, dtype=float)
    lo, hi = P.min(axis=0), P.max(axis=0)
    dy = spacing * math.sqrt(3) / 2.0
    rows = int((hi[1] - lo[1]) / dy) + 2
    pts = []
    from matplotlib.path import Path
    path = Path(P)
    for r in range(rows):
        y = lo[1] + r * dy
        x0 = lo[0] + (spacing / 2.0 if r % 2 else 0.0)
        cols = int((hi[0] - x0) / spacing) + 2
        for c2 in range(cols):
            x = x0 + c2 * spacing
            if path.contains_point((x, y)):
                pts.append(np.array([x, y]))
    if not pts:
        pts = [polygon_centroid(poly)]
    cur = np.asarray(start, dtype=float) if start is not None else polygon_centroid(poly)
    order, left = [], list(pts)

```

```

while left:
    k = min(range(len(left)), key=lambda i: norm(left[i] - cur))
    order.append(left.pop(k))
    cur = order[-1]
return order

def point_in_polygon(p, poly) -> bool:
    ok = True
    m = len(poly)
    for i in range(m):
        a, b = poly[i], poly[(i + 1) % m]
        if cross(b - a, np.asarray(p, float) - a) < -1e-9:
            ok = False
            break
    return ok

```

## 文件：vendor/q3\_breakthrough\_sandbox/lib/sim.py

""" 无线电干扰源环境模拟器（按赛题附录 2 / 附件 2 规则实现，用于本地演练与统计检验）。

与官方模拟器的一致性：

- \* 目标区域：半径 1800m 圆域，原点为圆心；机器狗初始位置 (0,0)，初始频道 1
- \* 干扰源：频道互异 (1..20)，位置与有效接收半径 (1000..1500m) 未知
- \* 全向源：距离 $\leq R$  即可接收；定向源：距离 $\leq R$  且位于定向方向  $\pm 90^\circ$  内
- \* 示向度误差：同一地点固定（确定性伪随机场），全局落在  $\pm 1^\circ$
- \* 计时：移动  $|\Delta p|/5$  s；换频道 1 s；检测 5 s；清除 未发现 3 s / 成功 5 s
- \* /clear 不换频道、不受定向覆盖角限制；5m 内检测返回 near；20m 内清除成功

```
from __future__ import annotations
```

```
import math
from dataclasses import dataclass, field
```

```
import numpy as np
```

```
from geom import ang_deg, norm
```

```

R_ARENA = 1800.0
SPEED = 5.0
T_MEASURE = 5.0
T_SWITCH = 1.0
T_CLEAR_SUCCESS = 5.0
T_CLEAR_FAIL = 3.0
R_NEAR = 5.0
R_CLEAR = 20.0
R_MIN = 1000.0
R_MAX = 1500.0
V_LIMIT = 360000.0

```

```

def _mix(x: int) -> int:
    x = (x + 0x9E3779B97F4A7C15) & 0xFFFFFFFFFFFFFFFF
    x = (x ^ (x >> 30)) * 0xBF58476D1CE4E5B9 & 0xFFFFFFFFFFFFFFFF
    x = (x ^ (x >> 27)) * 0x94D049BB133111EB & 0xFFFFFFFFFFFFFFFF
    return x ^ (x >> 31)

```

```

def env_error(p, channel: int) -> float:
    """ 固定电磁环境下的示向度误差 (度), 同一地点同一频道恒定, 落在 [-1,1]。"""
    qx = int(round(p[0] * 0.2))          # 5m 网格
    qy = int(round(p[1] * 0.2))
    h = _mix((qx & 0xFFFF) | ((qy & 0xFFFF) << 20) | (channel << 44))
    return (h / 2 ** 63) - 1.0

@dataclass
class Source:
    channel: int
    pos: np.ndarray
    radius: float
    kind: str = "omni"                  # omni | dir
    direction_deg: float | None = None
    cleared: bool = False

    def covers(self, p) -> bool:
        d = norm(np.asarray(p, float) - self.pos)
        if d > self.radius:
            return False
        if self.kind == "dir":
            u = np.array([math.cos(math.radians(self.direction_deg)),
                          math.sin(math.radians(self.direction_deg))])
            if float((np.asarray(p, float) - self.pos) @ u) < 0.0:
                return False
        return True

    def dist(self, p) -> float:
        return norm(np.asarray(p, float) - self.pos)

@dataclass
class Simulator:
    sources: list
    t: float = 0.0
    pos: np.ndarray = field(default_factory=lambda: np.zeros(2))
    channel: int = 1
    n_requests: int = 0
    log: list = field(default_factory=list)
    cleared: list = field(default_factory=list)
    total_move_m: float = 0.0
    t_move: float = 0.0
    t_switch: float = 0.0
    t_measure: float = 0.0
    t_clear: float = 0.0

    def _move(self, p):
        p = np.asarray(p, float)
        d = norm(p - self.pos)
        dt = d / SPEED
        self.t += dt
        self.total_move_m += d
        self.t_move += dt
        self.pos = p

    def measure(self, pos, channel: int):
        self.n_requests += 1
        self._move(pos)

```

```

dt = T_MEASURE
if channel != self.channel:
    dt += T_SWITCH
    self.t_switch += T_SWITCH
    self.channel = channel
self.t += dt
self.t_measure += T_MEASURE

src = None
for s in self.sources:
    if s.channel == channel and not s.cleared:
        src = s
        break
if src is None:
    return "no_signal", None
if not src.covers(self.pos):
    return "no_signal", None
d = src.dist(self.pos)
if d <= R_NEAR:
    return "near", None
svd = (ang_deg(src.pos - self.pos) + env_error(self.pos, channel)) % 360.0
return "direction", svd

def clear(self, pos, channel: int):
    self.n_requests += 1
    self._move(pos)
    src = None
    for s in self.sources:
        if s.channel == channel and not s.cleared:
            src = s
            break
    if src is not None and src.dist(self.pos) <= R_CLEAR + 1e-9:
        self.t += T_CLEAR_SUCCESS
        self.t_clear += T_CLEAR_SUCCESS
        src.cleared = True
        self.cleared.append(channel)
        return "success", self.t
    self.t += T_CLEAR_FAIL
    self.t_clear += T_CLEAR_FAIL
    return "no_target_in_range", self.t

def stats(self) -> dict:
    n = len(self.sources)
    nc = len(self.cleared)
    return {
        "n_sources": n, "n_cleared": nc,
        "ratio": nc / n if n else 0.0,
        "total_time_s": self.t,
        "avg_time_s": self.t / nc if nc else None,
        "n_requests": self.n_requests,
        "move_m": self.total_move_m,
        "t_move": self.t_move, "t_switch": self.t_switch,
        "t_measure": self.t_measure, "t_clear": self.t_clear,
    }

def generate_case(rng, problem=3, n_sources=None):
    if n_sources is None:
        n_sources = int(rng.integers(10, 17))

```

```

channels = rng.choice(np.arange(1, 21), size=n_sources, replace=False)
srcs = []
for ch in channels:
    r = R_ARENA * math.sqrt(rng.random())
    a = rng.random() * 2 * math.pi
    pos = np.array([r * math.cos(a), r * math.sin(a)])
    radius = R_MIN + (R_MAX - R_MIN) * rng.random()
    if problem == 3:
        srcs.append(Source(int(ch), pos, radius, "omni", None))
    else:
        kind = "dir" if rng.random() < 0.5 else "omni"
        direction = None if kind == "omni" else float(rng.random() * 360.0)
        srcs.append(Source(int(ch), pos, radius, kind, direction))
return srcs

```

文件: vendor/q3\_breakthrough\_sandbox/lib/strategy.py

```

from __future__ import annotations
import math
from dataclasses import dataclass, field
import numpy as np
from geom import (
    clip_polygon, convex_hull, diameter_rotating_calipers_pair, mec, norm, offset_polygon,
    polygon_area, polygon_centroid, point_in_polygon, unit, wedge_halfplanes,
)
from sim import R_ARENA, R_MIN, SPEED, Simulator
R_CLEAR = 20.0
MARGIN = 0.5

def disk_polygon(center, radius, n=120):
    c=np.asarray(center,float); rr=radius/math.cos(math.pi/n)
    return [c+rr*unit(360.0*i/n) for i in range(n)]

@dataclass
class Params:
    name: str = "adaptive"
    interleave: bool = True
    use_tracking: bool = True
    use_sweep: bool = True
    sweep_max_diam: float = 95.0
    sweep_spacing: float = 33.0
    source_weight: float = 0.85
    lateral_frac: float = 0.30
    max_iter_per_source: int = 40

class Robot:
    def __init__(self, sim: Simulator, cover_pts, params: Params, problem=3):
        self.sim=sim; self.params=params; self.problem=problem
        self.cover=[np.asarray(p,float) for p in cover_pts]; self.pending=list(self.cover)
        self.unknown=set(range(1,21)); self.tracked={}; self.cleared=set(); self.proven_empty=set();
        ↪ self.side={}
        self.n_measure=0; self.n_clear=0; self.n_home_steps=0; self.anomaly=0; self.gave_up=[];
        ↪ self.nosig={}; self.ray={}; self.mlog={}; self.n_filter=0
    def _dist(self,p): return norm(np.asarray(p,float)-self.sim.pos)
    def _scan_unknown(self, at=None):
        extra=[]
        for ch,poly in self.tracked.items():

```

```

        if ch in self.cleared: continue
        D,_=diameter_rotating_calipers_pair(poly)
        if D>80.0: extra.append(ch)
chans=sorted(self.unknown)+extra
for k,ch in enumerate(chans):
    p=at if (k==0 and at is not None) else self.sim.pos
    res,svd=self.sim.measure(p,ch); self.n_measure+=1
    if res=="direction":
        if ch in self.tracked: self._update_region(ch,self.sim.pos,svd)
        else: self._init_region(ch,svd)
        self.ray[ch]=(self.sim.pos.copy(),svd); self.nosig[ch]=0
    elif res=="no_signal" and ch in self.tracked:
        self._record_nosig(ch,self.sim.pos)
        if self.problem==4:
            np2=self._shadow_filter(ch,self.tracked[ch],prob4=True)
            if np2 is not None: self.tracked[ch]=np2
    elif res=="near":
        r,_=self.sim.clear(self.sim.pos,ch); self.n_clear+=1
        if r=="success": self.cleared.add(ch); self.unknown.discard(ch)
def _init_region(self,ch,svd):
    self.mlog.setdefault(ch,[]).append((self.sim.pos.copy(),"direction",float(svd)))
    pos=self.sim.pos.copy(); hps=wedge_halfplanes(pos,svd,1.0); poly=disk_polygon((0,0),R_arena,360)
    for hp in hps:
        poly=clip_polygon(poly,hp)
        if len(poly)==0: break
    if len(poly)>0:
        disk=disk_polygon(pos,1500.0,120)
        for hp in _poly_halfplanes(disk):
            poly=clip_polygon(poly,hp)
            if len(poly)==0: break
    if len(poly)>=3: self.tracked[ch]=_simplify(poly); self.unknown.discard(ch)
    else: self.anomaly+=1
def _update_region(self,ch,pos,svd):
    self.mlog.setdefault(ch,[]).append((np.asarray(pos,float).copy(),"direction",float(svd)))
    poly=self.tracked.get(ch)
    if poly is None: self._init_region(ch,svd); return
    poly=[p.copy() for p in poly]
    for hp in wedge_halfplanes(np.asarray(pos,float),svd,1.0):
        poly=clip_polygon(poly,hp)
        if len(poly)==0: break
    if len(poly)>=3 and polygon_area(poly)>1e-6: self.tracked[ch]=_simplify(poly)
    else: self.anomaly+=1
def _shadow_filter(self,ch,poly,spacing=15.0,nu=720,prob4=True):
    P=np.array(poly,dtype=float); lo,hi=P.min(axis=0)-spacing,P.max(axis=0)+spacing
    nx=max(2,int((hi[0]-lo[0])/spacing)+1); ny=max(2,int((hi[1]-lo[1])/spacing)+1)
    if nx*ny>40000: return None
    xs=lo[0]+spacing*np.arange(nx); ys=lo[1]+spacing*np.arange(ny); GX,GY=np.meshgrid(xs,ys)
    grid=np.stack([GX.ravel(),GY.ravel()],axis=1); inside=np.array([point_in_polygon(g,poly) for g in
↪ grid]); pts=grid[inside]
    if len(pts)==0: return None
    U=np.stack([np.cos(2*np.pi*np.arange(nu)/nu),np.sin(2*np.pi*np.arange(nu)/nu)],axis=1)
    ok_dir=np.ones(len(pts),dtype=bool); ok_u=np.ones((len(pts),nu),dtype=bool); tol=spacing*1.5;
↪ use_u=bool(prob4)
    for q,kind,svd in self.mlog.get(ch,[]):
        v=pts-q; d=np.linalg.norm(v,axis=1)
        if kind=="direction":
            ok_dir&=(d<=1500.0+tol); ang=np.degrees(np.arctan2(v[:,1],v[:,0]));
↪ diff=np.abs((ang-svd+180.0)%360.0-180.0)

```

```

        tol_ang=np.degrees(np.arctan2(tol,np.maximum(d,1.0)))+1.0; ok_dir&=(diff<=tol_ang)
        if use_u:
            vh=v/np.maximum(d,1e-9)[: ,None]; ok_u&=(vh@U.T<=0.02)
        else:
            if use_u:
                sel=d<=1000.0-tol
                if sel.any():
                    vh=v[sel]/np.maximum(d[sel],1e-9)[: ,None]; ok_u[sel]=ok_u[sel]&(vh@U.T>-0.02)
                else: ok_dir&=(d>1000.0-tol)
            feas=ok_dir&ok_u.any(axis=1)
            if feas.sum()<3:return None
            hull=convex_hull(pts[feas])
            if len(hull)<3:return None
            poly2=offset_polygon(hull,spacing*1.5)
            for hp in _poly_halfplanes(poly):
                poly2=clip_polygon(poly2,hp)
                if len(poly2)==0:return None
            self.n_filter+=1
            return poly2 if len(poly2)>=3 else None
def _record_nosig(self,ch,q):
    ↪ self.mlog.setdefault(ch,[]).append((np.asarray(q,float).copy(),"no_signal",None))
def _est_pos(self,ch): return polygon_centroid(self.tracked[ch])
def _near_edge(self,ch):
    P=np.array(self.tracked[ch]); d=np.linalg.norm(P-self.sim.pos,axis=1); return P[int(d.argmin())]
def _next_measure_point(self,ch,poly):
    if self.problem==4 and ch in self.ray:
        apex,th=self.ray[ch]; u=unit(th); n=np.array([-u[1],u[0]])
        tmax=max(float((np.asarray(p,float)-apex)@u) for p in poly); tmax=max(tmax,50.0);
        ↪ t_now=float((self.sim.pos-apex)@u)
        t=min(max(t_now+200.0,200.0),tmax); sgn=self.side.get(ch,1.0); self.side[ch]=-sgn
        lat=25.0*sgn if self.nosig.get(ch,0)==0 else 12.0*sgn
        return apex+t*u+lat*n
D,(a,b)=diameter_rotating_calipers_pair(poly); area=polygon_area(poly); width=area/D if D>1e-9
    ↪ else 0.0
if width<=45.0 and D>80.0:
    u=(b-a)/D; s=float(np.clip((self.sim.pos-a)@u,0.0,D)); t=s+250.0
    if t>D-40.0: t=max(D*0.5,s-250.0) if s>D*0.5 else D
    lat=0.0 if self.params.lateral_frac<=0 else 12.0; sgn=self.side.get(ch,1.0);
    ↪ self.side[ch]=-sgn; n=np.array([-u[1],u[0]])
    return a+float(np.clip(t,0.0,D))*u+sgn*lat*n
c=polygon_centroid(poly); v=c-self.sim.pos; L=norm(v); u=unit(self.sim.channel*37.0) if L<1e-6
    ↪ else v/L; up=np.array([-u[1],u[0]]); sgn=self.side.get(ch,1.0); self.side[ch]=-sgn
if L>700: f,lat=0.55*L,self.params.lateral_frac*L
elif L>150: f,lat=0.45*L,0.5*L
else: f,lat=0.15*L,max(60.0,0.9*L)
return self.sim.pos+f*u+sgn*lat*up
def _clear_sweep(self,ch,poly):
    from geom import cover_lattice_points
    pts=cover_lattice_points(poly,self.params.sweep_spacing,start=self.sim.pos)
    for p in pts:
        r,_=self.sim.clear(p,ch); self.n_clear+=1
        if r=="success": return True
    return False
def _ray_sweep(self,ch):
    if ch not in self.ray:return False
    apex,th=self.ray[ch]; u=unit(th); n=np.array([-u[1],u[0]]); poly=self.tracked[ch]
    ts=[float((np.asarray(p,float)-apex)@u) for p in poly]; t_lo=max(0.0,min(ts)-20.0);
    ↪ t_hi=max(ts)+30.0; t_now=float((self.sim.pos-apex)@u)

```

```

if abs(t_now-t_lo)>abs(t_now-t_hi): t,t_end,step=t_hi,t_lo,-15.0
else: t,t_end,step=t_lo,t_hi,15.0
sgn=1.0
while (step>0 and t<=t_end) or (step<0 and t>=t_end):
    q=apex+t*u+sgn*13.0*n; r,_=self.sim.clear(q,ch); self.n_clear+=1
    if r=="success":return True
    t+=step; sgn=-sgn
return False
def home_and_clear(self,ch)->bool:
    for _ in range(self.params.max_iter_per_source):
        poly=self.tracked.get(ch)
        if poly is None or len(poly)<3:return False
        c,r=mec(poly); D,_=diameter_rotating_calipers_pair(poly)
        if not self.params.use_tracking:
            if D>1e-9 and self._clear_sweep(ch,poly):return True
            q=self._next_measure_point(ch,poly); res,svd=self.sim.measure(q,ch); self.n_measure+=1;
            ↪ self.n_home_steps+=1
            if res=="direction":self._update_region(ch,q,svd)
            elif res=="near":
                r2,_=self.sim.clear(q,ch); self.n_clear+=1
                if r2=="success":return True
            continue
        if r<=R_CLEAR-MARGIN:
            res,_=self.sim.clear(c,ch); self.n_clear+=1
            if res=="success":return True
            self.anomaly+=1; return False
        if self.params.use_sweep and D<=self.params.sweep_max_diam and D>1e-9:
            if self._clear_sweep(ch,poly):return True
            self.anomaly+=1
        q=self._next_measure_point(ch,poly); res,svd=self.sim.measure(q,ch); self.n_measure+=1;
        ↪ self.n_home_steps+=1
        if res=="direction":
            self.nosig[ch]=0; self.ray[ch]=(q.copy(),svd); self._update_region(ch,q,svd)
        elif res=="near":
            r2,_=self.sim.clear(q,ch); self.n_clear+=1
            if r2=="success":return True
        else:
            self.nosig[ch]=self.nosig.get(ch,0)+1; self._record_nosig(ch,self.sim.pos)
            if self.problem==4:
                np2=self._shadow_filter(ch,self.tracked[ch],prob4=True)
                if np2 is not None: self.tracked[ch]=np2; poly=self.tracked[ch]
            r3,_=self.sim.clear(self.sim.pos,ch); self.n_clear+=1
            if r3=="success":return True
            if self.problem==4:
                D2,(a2,b2)=diameter_rotating_calipers_pair(poly); width2=polygon_area(poly)/D2 if
                ↪ D2>1e-9 else 0.0
                if self.nosig[ch]>=3 and ch in self.ray:
                    if self._ray_sweep(ch):return True
                elif self.nosig[ch]>=5 and width2>45.0:
                    if self._clear_sweep(ch,poly):return True
            return False
def run(self)->dict:
    self._scan_unknown(); self.pending=[q for q in self.pending if self._dist(q)>1e-9]
    while True:
        todo_src=[ch for ch in self.tracked if ch not in self.cleared]; need_scan=bool(self.unknown)
        ↪ and bool(self.pending)
        if not todo_src and not need_scan:break
        options=[]

```

```

        if todo_src:
            for ch in todo_src:
                p=self._est_pos(ch) if self.params.interleave else self._near_edge(ch);
                ↪ cost=self._dist(p)*self.params.source_weight
                poly=self.tracked[ch]; D,_=diameter_rotating_calipers_pair(poly); cost+=0.12*D+30.0;
                ↪ options.append((cost,"home",ch,p))
            if need_scan:
                P=min(self.pending,key=lambda q:self._dist(q)); cost=self._dist(P)+6.0*len(self.unknown);
                ↪ cost*=1.0 if self.params.interleave else 0.0; options.append((cost,"scan",None,P))
            if not options:break
            options.sort(key=lambda o:o[0]); _,kind,ch,target=options[0]
            if kind=="scan": self.pending=[q for q in self.pending if norm(q-target)>1e-9];
            ↪ self._scan_unknown(at=target)
        else:
            ok=self.home_and_clear(ch); self.cleared.add(ch)
            if not ok:self.gave_up.append(ch)
        return self.finish()
def finish(self)->dict:
    sim=self.sim; proven=set(self.unknown) if not self.pending else set()
    return {"cleared":sorted(self.cleared),"tracked_channels":sorted(self.tracked),"gave_up":sorted(s
    ↪ elf.gave_up),"proven_empty":sorted(proven),"unresolved":sorted(self.unknown-proven),"n_measur
    ↪ e":self.n_measure,"n_clear":self.n_clear,"n_home_steps":self.n_home_steps,"anomaly":self.anom
    ↪ aly,"stats":sim.stats()}

def _poly_halfplanes(poly):
    hps=[]; m=len(poly)
    for i in range(m):
        a,b=poly[i],poly[(i+1)%m]; e=b-a; n=np.array([e[1],-e[0]]); nn=norm(n)
        if nn<1e-12:continue
        n=n/nn; hps.append((n,float(n@a)))
    return hps

def _simplify(poly,tol=0.05):
    from geom import simplify_polygon
    return simplify_polygon(poly,tol)

def run_trial(sources,cover_pts,params:Params,problem=3):
    sim=Simulator(sources); robot=Robot(sim,cover_pts,params,problem=problem); res=robot.run()
    res["sources"]=[{"ch":s.channel,"pos":s.pos.tolist(),"R":s.radius,"kind":s.kind,"dir":s.direction_deg
    ↪ ,"cleared":s.cleared} for s in sources]
    return res

```

文件: vendor/q3\_breakthrough\_sandbox/lib/strategy2.py

```

from __future__ import annotations
import math
from dataclasses import dataclass, field
import numpy as np
from geom import (
    clip_polygon, convex_hull, cover_lattice_points, diameter_rotating_calipers_pair, mec,
    norm, offset_polygon, point_in_polygon, polygon_area, polygon_centroid, unit,
    wedge_halfplanes,
)
from sim import R_ARENA, R_MAX, R_MIN, SPEED
from strategy import MARGIN, R_CLEAR, Params, Robot, _poly_halfplanes, _simplify

@dataclass

```

```

class Params2(Params):
    name: str = "v2"
    theta_miss: float = 0.0
    rho: bool = True
    use_belief: bool = True
    r_learn: bool = True
    front_len: float = 350.0
    nosig_sweep: int = 4
    beta_gain: float = 8000.0

class MissModel:
    def __init__(self, problem=3, grid_step=25.0, nu=90):
        self.problem=problem; self.nu=nu
        xs=np.arange(-R_ARENA,R_ARENA+grid_step,grid_step); X,Y=np.meshgrid(xs,xs);
        ↪ m=(X**2+Y**2)<=R_ARENA**2
        self.grid=np.stack([X[m],Y[m]],axis=1)
        self.U=np.stack([np.cos(2*np.pi*np.arange(nu)/nu),np.sin(2*np.pi*np.arange(nu)/nu)],axis=1)
        self.b_grid=np.arange(1300.0,1500.1,10.0); self.b_post=None; self.detected_dists=[];
        ↪ self._cache={}; self._ver=0
    def observe_detected(self,dist): self.detected_dists.append(float(dist)); self._update_b()
    def _update_b(self):
        if not self.detected_dists or not np.isfinite(self.detected_dists).all(): self.b_post=None; return
        dmax=max(self.detected_dists); w=np.ones_like(self.b_grid)
        for d in self.detected_dists: w=w*np.clip((self.b_grid-d)/(self.b_grid-R_MIN),1e-6,1.0)
        w=np.where(self.b_grid>=max(dmax,R_MIN),w,0.0); w=np.where(self.b_grid<=R_MAX,w,0.0)
        self.b_post=w/w.sum() if w.sum()>0 else None; self._ver+=1; self._cache.clear()
    def mu(self,pts)->float:
        pts=np.asarray(pts,float)
        if len(pts)==0:return 1.0
        key=(self._ver,tuple(sorted((round(float(p[0]),1),round(float(p[1]),1)) for p in pts)))
        if key in self._cache:return self._cache[key]
        if self.b_post is None: bs,ws=np.array([R_MAX]),np.array([1.0])
        else: bs,ws=self.b_grid,self.b_post
        if self.problem==3:
            d=np.sqrt(((self.grid[:,None,:]-pts[None,:,:])**2).sum(-1)).min(axis=1)
            frac=np.clip((d[:,None]-R_MIN)/np.maximum(bs[None,:]-R_MIN,1e-9),0.0,1.0);
            ↪ val=float((frac*ws[None,:]).sum(axis=1).mean()); self._cache[key]=val; return val
        v=self.grid[:,None,:]-pts[None,:,:]; dist=np.sqrt((v**2).sum(-1));
        ↪ front=np.einsum("gmc,uc->gmu",v,self.U)>=0.0
        dd=np.where(front,dist[:, :, None],np.inf).min(axis=1);
        ↪ frac=np.clip((dd[:, :, None]-R_MIN)/np.maximum(bs[None, None, :]-R_MIN,1e-9),0.0,1.0)
        val=float((frac*ws[None, None, :]).sum(axis=2).mean()); self._cache[key]=val; return val
    def expected_missed(self,pts,n_unresolved:int,n_detected:int,n_total_ch=20,n_max_prior=16,n_min_prior
    ↪ =10)->float:
        mu=self.mu(pts); ns=np.arange(n_min_prior,n_max_prior+1); logp=[]
        for n in ns:
            k=n-n_detected
            if k<0 or k>n: logp.append(-1e9); continue
            lp=k*math.log(max(mu,1e-12))+n_detected*math.log(max(1-mu,1e-12));
            ↪ lp+=math.lgamma(n+1)-math.lgamma(k+1)-math.lgamma(n_detected+1); logp.append(lp)
        logp=np.array(logp); logp=-logp.max(); p=np.exp(logp); p/=p.sum(); return
        ↪ float((p*(ns-n_detected)).sum())

class DirBelief:
    def __init__(self,nu=72): self.log=[]; self.nu=nu; self.last=None
    def add(self,q,kind,svd=None): self.log.append((np.asarray(q,float).copy(),kind,None if svd is None
    ↪ else float(svd)))
    def evaluate(self,poly,spacing=None):

```

```

if len(poly)<3 or not self.log:return None
P=np.asarray(poly,float); lo,hi=P.min(axis=0),P.max(axis=0)
if spacing is None: spacing=max(14.0,math.sqrt(max((hi-lo).prod(),1.0)/1200.0))
nx=max(2,int((hi[0]-lo[0])/spacing)+1); ny=max(2,int((hi[1]-lo[1])/spacing)+1)
if nx*ny>2500:return None
xs=lo[0]+spacing*np.arange(nx); ys=lo[1]+spacing*np.arange(ny); GX,GY=np.meshgrid(xs,ys);
↪ grid=np.stack([GX.ravel(),GY.ravel()],axis=1)
inside=np.array([point_in_polygon(g,poly) for g in grid]); pts=grid[inside]
if len(pts)<3:return None
U=np.stack([np.cos(2*np.pi*np.arange(self.nu)/self.nu),np.sin(2*np.pi*np.arange(self.nu)/self.nu)]
↪ ,axis=1)
ok_dir=np.ones(len(pts),dtype=bool); ok_u=np.ones((len(pts),self.nu),dtype=bool); tol=spacing*1.5
for q,kind,svd in self.log:
    v=pts-q; d=np.linalg.norm(v,axis=1)
    if kind=="direction":
        ok_dir&=(d<=R_MAX+tol); ang=np.degrees(np.arctan2(v[:,1],v[:,0]));
        ↪ diff=np.abs((ang-svd+180.0)%360.0-180.0)
        tol_ang=np.degrees(np.arctan2(tol,np.maximum(d,1.0)))+1.0; ok_dir&=(diff<=tol_ang);
        ↪ vh=v/np.maximum(d,1e-9)[: ,None]; ok_u&=(vh@U.T<=0.02)
    else:
        sel=d<=R_MIN-tol
        if sel.any():
            vh=v[sel]/np.maximum(d[sel],1e-9)[: ,None]; ok_u[sel]=ok_u[sel]&(vh@U.T>-0.02)
feas=ok_dir&ok_u.any(axis=1)
if feas.sum()<3:return None
w_u=ok_u[feas].sum(axis=0).astype(float)
if w_u.sum()<=0:return None
w_u/=w_u.sum(); c=float((w_u*np.cos(2*np.pi*np.arange(self.nu)/self.nu)).sum());
↪ s=float((w_u*np.sin(2*np.pi*np.arange(self.nu)/self.nu)).sum())
mean_u=math.degrees(math.atan2(s,c))%360.0; conc=math.hypot(c,s); hull=convex_hull(pts[feas])
if len(hull)<3:return None
region=offset_polygon(hull,spacing*1.5)
for hp in _poly_halfplanes(poly):
    region=clip_polygon(region,hp)
    if len(region)==0:return None
self.last={"region":region if len(region)>=3 else None,"mean_u":mean_u,"conc":conc,"u_post":w_u.c}
↪ copy(),"p_mean":pts[feas].mean(axis=0),"spacing":spacing,"n_feasible":int(feas.sum())}
return self.last

```

```

class RobotV2(Robot):
    def __init__(self,sim,plan,params:Params2,problem=3):
        self.plan1=[np.asarray(p,float) for p in plan["level1"]]; self.plan_esc=[np.asarray(p,float) for p
        ↪ in plan.get("escalation",[])]
        super().__init__(sim,self.plan1+self.plan_esc,params,problem=problem); self.params=params;
        ↪ self.miss_model=MissModel(problem=problem); self.beliefs={}; self.escalated=False;
        ↪ self.scanned=[]; self.n_esc_used=0; self.scan_set=[]
    def _init_region(self,ch,svd):
        super()._init_region(ch,svd)
        if self.problem==4:self.beliefs.setdefault(ch,DirBelief()).add(self.sim.pos,"direction",svd)
    def _update_region(self,ch,pos,svd):
        super()._update_region(ch,pos,svd)
        if self.problem==4:self.beliefs.setdefault(ch,DirBelief()).add(pos,"direction",svd)
    def _record_nosig(self,ch,q):
        super()._record_nosig(ch,q)
        if self.problem==4:self.beliefs.setdefault(ch,DirBelief()).add(q,"no_signal")
    def _shadow_filter(self,ch,poly,spacing=15.0,nu=720,prob4=True): return None
    def _need_scan(self)->bool:
        if not self.unknown or not self.pending:return False

```

```

        if self.params.theta_miss<=0.0:return True
        em=self.miss_model.expected_missed(self.scan_set,len(self.unknown),len(self.cleared)); return
        ↪ bool(em>self.params.theta_miss)
def _maybe_escalate(self):
    if not self.unknown:return
    if any(all(norm(np.asarray(q,float)-s)>1e-9 for s in self.scanned) for q in self.pending):return
    em0=self.miss_model.expected_missed(self.scan_set,len(self.unknown),len(self.cleared))
    if em0<=self.params.theta_miss:return
    cand=[q for q in self.plan_esc if all(norm(q-s)>1e-9 for s in self.scanned)]
    if not cand:return
    best,best_gain=None,-1.0
    for q in cand:
        em1=self.miss_model.expected_missed(list(self.scan_set)+[q],len(self.unknown),len(self.cleared))
        ↪ d)); gain=(em0-em1)/max(self._dist(q),50.0)
        if gain>best_gain:best,best_gain=q,gain
    self.pending=list(self.pending)+[np.asarray(best,float)]; self.escalated=True; self.n_esc_used+=1
def _jobs(self):
    jobs=[]
    for ch in self.tracked:
        if ch in self.cleared:continue
        poly=self.tracked[ch]; D,_=diameter_rotating_calipers_pair(poly); c=polygon_centroid(poly);
        ↪ jobs.append(("clear",ch,np.asarray(c,float),self._dist(c)+0.15*D+30.0))
    if self._need_scan():
        for P in self.pending:jobs.append(("scan",None,np.asarray(P,float),self._dist(P)+6.0*len(self)
        ↪ .unknown)))
    return jobs
def _plan_next(self,jobs):
    start=self.sim.pos.copy(); nodes=[j[2] for j in jobs]
    if len(nodes)==1:return jobs[0]
    left=list(range(len(nodes))); order=[]; cur=start
    while left:
        k=min(left,key=lambda i:norm(nodes[i]-cur)); order.append(k); left.remove(k); cur=nodes[k]
    def path_len(seq):
        s=0.0; c=start
        for i in seq:s+=norm(nodes[i]-c); c=nodes[i]
        return s
    improved=True
    while improved:
        improved=False
        for i in range(len(order)-1):
            for j in range(i+1,len(order)):
                new=order[:i]+order[i:j+1][::-1]+order[j+1:]
                if path_len(new)<path_len(order)-1e-9:order,improved=new,True
    first=order[0]; cand=[i for i in range(len(jobs)) if norm(nodes[i]-nodes[first])<1e-6];
    ↪ cand.sort(key=lambda i:0 if jobs[i][0]=="clear" else 1)
    return jobs[cand[0]]
def _do_scan(self,P):
    self.pending=[q for q in self.pending if norm(q-P)>1e-9];
    ↪ self.scanned.append(np.asarray(P,float)); self._scan_unknown(at=P)
    if all(norm(np.asarray(s,float)-P)>1e-9 for s in
    ↪ self.scan_set):self.scan_set.append(np.asarray(P,float))
    self.miss_model.detected_dists=self._detected_dists(); self.miss_model._update_b()
def _detected_dists(self):
    out=[]
    for ch in self.tracked:
        if ch in self.cleared:continue
        poly=self.tracked[ch]; best=0.0
        for s in self.scanned:

```

```

        dmin=min(norm(np.asarray(p,float)-np.asarray(s,float)) for p in poly); best=max(best,dmin)
    out.append(float(best))
    return out if out else [R_MIN]
def _do_clear(self,ch):
    ok=self.home_and_clear(ch)
    if not ok:
        poly=self.tracked.get(ch)
        if poly is not None and len(poly)>=3:
            D,_=diameter_rotating_calipers_pair(poly);
            ↪ n_est=polygon_area(poly)/(0.433*self.params.sweep_spacing**2)
            if n_est<=900:ok=self._clear_sweep(ch,poly)
        self.cleared.add(ch)
    if not ok:self.gave_up.append(ch)
def _next_measure_point(self,ch,poly):
    if self.problem==4 and self.params.use_belief and ch in self.beliefs and self.nosig.get(ch,0)>0:
        b=self.beliefs[ch].evaluate(poly)
        if b is not None:
            if b["region"] is not None and polygon_area(b["region"])<polygon_area(poly):
                ↪ self.tracked[ch]=simplify(b["region"],0.05); poly=self.tracked[ch]
            if b["conc"]>0.15:
                u=unit(b["mean_u"]); goal=np.asarray(b["p_mean"],float)+self.params.front_len*u;
                ↪ v=goal-self.sim.pos; L=norm(v)
                if L>1e-6:
                    uu=v/L; n=np.array([-uu[1],uu[0]]); sgn=self.side.get(ch,1.0); self.side[ch]=-sgn;
                    ↪ step=min(L,max(120.0,0.7*L)); return self.sim.pos+step*uu+sgn*0.18*step*n
    return super()._next_measure_point(ch,poly)
def run(self):
    self._scan_unknown(); self.scanned.append(self.sim.pos.copy());
    ↪ self.scan_set.append(self.sim.pos.copy()); self.pending=[q for q in self.pending if
    ↪ self._dist(q)>1e-9]; guard=0
    while guard<4000:
        guard+=1; self._maybe_escalate(); jobs=self._jobs()
        if not jobs:break
        kind,ch,target,_=self._plan_next(jobs)
        if kind=="scan":self._do_scan(target)
        else:self._do_clear(ch)
    return self.finish()

def run_trial_v2(sources,plan,params:Params2,problem=3):
    from sim import Simulator
    sim=Simulator(sources); robot=RobotV2(sim,plan,params,problem=problem); res=robot.run()
    res["sources"]=[{"ch":s.channel,"pos":s.pos.tolist(),"R":s.radius,"kind":s.kind,"dir":s.direction_deg}
    ↪ ,{"cleared":s.cleared} for s in sources]
    res["escalated"]=robot.escalated; res["n_esc_points"]=robot.n_esc_used;
    ↪ res["scan_points"]=len(robot.scanned); return res

```

文件: vendor/q3\_breakthrough\_sandbox/q3\_breakthrough.py

```

import sys,math,json,time,argparse
from pathlib import Path
ROOT=Path(__file__).resolve().parent
sys.path.insert(0,str(ROOT));sys.path.insert(0,str(ROOT/'lib'))
import numpy as np
import strategy
strategy.MARGIN=2.0
from run_q3_nextgen import Q3NegDynamic, PLAN
from sim import generate_case,Simulator

```

```

from strategy2 import Params2
from geom import norm, polygon_centroid, diameter_rotating_calipers_pair, clip_polygon, mec

def area(poly):
    return abs(sum(float(a[0]*b[1]-a[1]*b[0]) for a,b in zip(poly,poly[1:]+poly[:1]))) / 2 if len(poly) >= 3
    ↪ else 0.

def quadrature(poly, positives, negatives, count=3):
    D,(a,b)=diameter_rotating_calipers_pair(poly)
    if D<1e-9: return [(1.,np.asarray(polygon_centroid(poly),float),1000.,1500.)]
    u=(b-a)/D; vals=[float(np.asarray(p)@u) for p in poly]; lo,hi=min(vals),max(vals); rows=[]
    for i in range(count):
        l=lo+(hi-lo)*i/count; h=lo+(hi-lo)*(i+1)/count
        slab=clip_polygon(poly,(-u,-l)); slab=clip_polygon(slab,(u,h)) if len(slab)>=3 else []
        if len(slab)<3: continue
        c=np.asarray(polygon_centroid(slab),float); w=area(slab)
        rlo=max([1000.]+[norm(c-p) for p in positives])
        rhi=min([1500.]+[norm(c-q) for q in negatives])
        if w>1e-9 and rlo<=rhi+1e-6: rows.append((w,c,rlo,max(rlo,rhi)))
    s=sum(r[0] for r in rows)
    return [(w/s,c,lo,hi) for w,c,lo,hi in rows] if s else []

def negative_posterior(poly,q,positives):
    pp=list(poly); q=np.asarray(q,float)
    for p in positives:
        p=np.asarray(p,float); n=2*(q-p); c=float(q@q-p@p)
        new=clip_polygon(pp,(n,c+1e-7))
        if len(new)>=3: pp=new
    return pp

def positive_posterior(poly,q,source,negatives):
    ang=math.degrees(math.atan2(source[1]-q[1],source[0]-q[0]))%360
    outs=[]
    # Use sandbox bearing wedge update helper by clipping with two halfplanes manually.
    for noise in (-1.,0.,1.):
        theta=math.radians((ang+noise)%360); eps=math.radians(1.0)
        # direction vector range [theta-eps,theta+eps]; source relative vector must lie between rays.
        d1=np.array([math.cos(theta-eps),math.sin(theta-eps)]); d2=np.array([math.cos(theta+eps),math.sin(theta+eps)])
        ↪ theta+eps)])
        # cross(d1, x-q)>=0 -> [d1_y,-d1_x] dot (x-q)<=0
        n1=np.array([d1[1],-d1[0]]); c1=float(n1@q)
        # cross(d2, x-q)<=0 -> [-d2_y,d2_x] dot (x-q)<=0
        n2=np.array([-d2[1],d2[0]]); c2=float(n2@q)
        pp=clip_polygon(poly,(n1,c1)); pp=clip_polygon(pp,(n2,c2)) if len(pp)>=3 else []
        if len(pp)<3: continue
        # positive at q plus old negatives gives Q3 affine cuts
        for nq in negatives:
            nq=np.asarray(nq,float); n=2*(nq-q); c=float(nq@nq-q@q)
            new=clip_polygon(pp,(n,c+1e-7))
            if len(new)>=3: pp=new
        outs.append(pp)
    return outs

def finish_cost(point,poly,source):
    if len(poly)<3: return 3000.
    c,r=mec(poly); c=np.asarray(c,float); point=np.asarray(point,float); source=np.asarray(source,float)
    if r<=18:
        d=norm(point-c); off=min(d,max(0.,18-r)); target=c+(point-c)*(off/d) if d>1e-9 else c

```

```

        return norm(point-target)+5.
# cost proxy: approach centroid + one further action + residual correction
target=np.asarray(polygon_centroid(poly),float)
residual=max(0.,norm(target-source)-18.)
return norm(point-target)+40.+residual

def band_cover(poly,radius=19.5,maximum=10):
    D,(a,b)=diameter_rotating_calipers_pair(poly)
    if D<1e-9:return [np.asarray(a,float)]
    u=(b-a)/D;v=np.array([-u[1],u[0]])
    xs=[float(np.asarray(p)@u) for p in poly];ys=[float(np.asarray(p)@v) for p in poly]
    lo,hi=min(xs),max(xs);bot,top=min(ys),max(ys);half=(top-bot)/2
    if half>=radius:return []
    step=2*math.sqrt(max(0.,radius*radius-half*half));cnt=max(1,math.ceil((hi-lo)/step))
    if cnt>maximum:return []
    mid=(bot+top)/2
    return [((lo+(i+.5)*(hi-lo)/cnt)*u + mid*v) for i in range(cnt)]

class Q3Round1(Q3NegDynamic):
    def __init__(self,*a,**kw):super().__init__(*a,**kw);self.lookahead_choices=0
    def _next_measure_point(self,ch,poly):
        original=super()._next_measure_point(ch,poly);c,r=mec(poly)
        if r<60:return original
        positives=[np.asarray(e[0],float) for e in self.mlog.get(ch,[]) if e[1]=='direction']
        negatives=[np.asarray(q,float) for q in self.neg_hist.get(ch,[])]
        if not positives:return original
        nodes=quadrature(poly,positives,negatives,3)
        if not nodes:return original
        anchor=positives[-1];svd=[e[2] for e in self.mlog.get(ch,[]) if e[1]=='direction'][-1]
        th=math.radians(float(svd));u=np.array([math.cos(th),math.sin(th)]);side=np.array([-u[1],u[0]])
        projs=[float((np.asarray(p)-anchor)@u) for p in poly];lo,hi=min(projs),max(projs)
        cand=[np.asarray(original,float),np.asarray(polygon_centroid(poly),float)]
        for qtl in (.15,.45,.75,1.0):
            step=lo+qtl*(hi-lo)
            for w in (25.,100.):
                for sign in (-1.,1.):cand.append(anchor+step*u+sign*w*side)
        best=None
        for q in cand:
            if min(norm(q-p) for p in positives)<2:continue
            if negatives and min(norm(q-n) for n in negatives)<2:continue
            sc=norm(q-self.sim.pos)+5.
            negp=negative_posterior(poly,q,positives)
            for wt,g,rlo,rhi in nodes:
                d=norm(q-g);prob=float(d<=rlo) if rhi-rlo<1e-6 else max(0.,min(1.,(rhi-d)/(rhi-rlo)))
                posc=0.
                if prob:
                    if d<=5:posc=5.
                else:
                    poss=positive_posterior(poly,q,g,negatives)
                    posc=sum(finish_cost(q,pp,g) for pp in poss)/len(poss) if poss else 3000.
                negc=finish_cost(q,negp,g)
                sc += wt*(prob*posc+(1-prob)*negc)
            if best is None or sc<best[0]:best=(sc,q)
        self.lookahead_choices+=1
        return best[1] if best else original

class Q3Round2(Q3Round1):
    def __init__(self,*a,**kw):super().__init__(*a,**kw);self.band_sweeps=0;self.band_attempts=0
    def home_and_clear(self,ch):

```

```

for _ in range(self.params.max_iter_per_source):
    poly=self.tracked.get(ch)
    if poly is None or len(poly)<3:return False
    c,r=mec(poly);D,_=diameter_rotating_calipers_pair(poly)
    if r<=18.0:
        rr,_=self.sim.clear(c,ch);self.n_clear+=1
        if rr=='success':return True
        return False
    plan=band_cover(poly,19.5,10)
    if len(plan)>1:
        # Compare deterministic complete band vs immediate radio action + rough residual.
        seqs=[plan,list(reversed(plan))]
        def seqcost(seq):
            cur=self.sim.pos.copy();cost=0.
            for p in seq:cost+=norm(p-cur)+3.;cur=p
            return cost
        seq=min(seqs,key=seqcost);band=seqcost(seq)
        q=self._next_measure_point(ch,poly);radio=norm(q-self.sim.pos)+5.+max(40.,.15*D)
        if band-25. < radio:
            self.band_sweeps+=1
            for p in seq:
                rr,_=self.sim.clear(p,ch);self.n_clear+=1;self.band_attempts+=1
                if rr=='success':return True
            raise RuntimeError('certified band exhausted')
    if self.params.use_sweep and D<=self.params.sweep_max_diam and D>1e-9:
        if self._clear_sweep(ch,poly):return True
    q=self._next_measure_point(ch,poly);res,svd=self.sim.measure(q,ch);self.n_measure+=1;self.n_h
↪ ome_steps+=1
    if res=='direction':self.nosig[ch]=0;self.ray[ch]=(q.copy(),svd);self._update_region(ch,q,svd)
    elif res=='near':
        rr,_=self.sim.clear(q,ch);self.n_clear+=1
        if rr=='success':return True
    else:
        self.nosig[ch]=self.nosig.get(ch,0)+1;self._record_nosig(ch,self.sim.pos)
        # Q3 no_signal implies d>R>=1000, so clear here is guaranteed failure. Skip it.
return False

class Q3Round3(Q3Round2):
    def __init__(self,*a,**kw):
        super().__init__(*a,**kw);self.base_phase=0.0;self.early16=False
    def _need_scan(self):
        if len(set(self.tracked)|self.cleared)>=16:
            self.early16=True;return False
        return super()._need_scan()
    def _choose_base_ring(self):
        known=[np.asarray(polygon_centroid(poly),float) for ch,poly in self.tracked.items() if ch not in
↪ self.cleared]
        best=None
        for m in range(24):
            ph=(math.pi/3)*m/24.0
            ring=[np.array([1130*math.cos(ph+k*math.pi/3),1130*math.sin(ph+k*math.pi/3)],float) for k in
↪ range(6)]
            nodes=ring+known
            # cheap NN+2opt-equivalent path proxy from the current origin position
            left=list(range(len(nodes)));cur=self.sim.pos.copy();order=[];L=0.
            while left:
                j=min(left,key=lambda i:norm(nodes[i]-cur));L+=norm(nodes[j]-cur);cur=nodes[j];order.appe
↪ nd(j);left.remove(j)

```

```

        # modest bias toward placing stations near already-known source regions
        near=sum(min(norm(c-r) for r in ring) for c in known) if known else 0.
        score=L+0.15*near
        if best is None or score<best[0]:best=(score,ph,ring)
    self.base_phase=best[1];return best[2]
def run(self):
    self._scan_unknown();self.scanned.append(self.sim.pos.copy());self.scan_set.append(self.sim.pos.c
    ↪ opy())
    known_count=len(set(self.tracked)|self.cleared)
    if self.conditional and known_count<=self.threshold:
        self.pending=self._choose_outer_ring();self.conditional_used=True
    else:self.pending=self._choose_base_ring()
    guard=0
    while guard<4000:
        guard+=1;self._maybe_escalate();jobs=self._jobs()
        if not jobs:break
        kind,ch,target,_=self._plan_next(jobs)
        if kind=='scan':self._do_scan(target)
        else:self._do_clear(ch)
    return self.finish()

CLASSES={'r1':Q3Round1,'r2':Q3Round2,'r3':Q3Round3}

def run(variant='r1',n=20,seed0=90000):
    rows=[];hard=[];extras=[];t=time.time();cls=CLASSES[variant]
    for i in range(n):
        src=generate_case(np.random.default_rng(seed0+i),3);sim=Simulator(src)
        rb=cls(sim,PLAN,Params2(name=variant,theta_miss=0,use_belief=False),3,neg=True,conditional=True,t
        ↪ hreshold=0,outer_n=9,outer_r=1700,rotate=True)
        try:o=rb.run()
        except Exception as e:hard.append((seed0+i,type(e).__name__,str(e)));o=rb.finish()
        rows.append(o);extras.append(rb)
    st=[r['stats'] for r in rows]
    out={'variant':variant,'n':n,'seed0':seed0,'full':sum(s['ratio']>=1-1e-9 for s in
    ↪ st),'sources':sum(s['n_sources'] for s in st),'cleared':sum(s['n_cleared'] for s in
    ↪ st),'agg':sum(s['total_time_s'] for s in st)/sum(s['n_sources'] for s in
    ↪ st),'meanavg':float(np.mean([s['avg_time_s'] for s in
    ↪ st])), 'mean_total':float(np.mean([s['total_time_s'] for s in
    ↪ st])), 'move':float(np.mean([s['move_m'] for s in st])), 'req':float(np.mean([s['n_requests'] for s
    ↪ in st])), 'clear':float(np.mean([s['t_clear'] for s in
    ↪ st])), 'measure':float(np.mean([s['t_measure'] for s in st])), 'switch':float(np.mean([s['t_switch']
    ↪ for s in st])), 'lookahead':float(np.mean([getattr(r,'lookahead_choices',0) for r in
    ↪ extras])), 'band_sweeps':float(np.mean([getattr(r,'band_sweeps',0) for r in
    ↪ extras])), 'band_attempts':float(np.mean([getattr(r,'band_attempts',0) for r in
    ↪ extras])), 'service_offset':float(np.mean([x for r in extras for x in
    ↪ getattr(r,'service_offsets',[])]) if any(getattr(r,'service_offsets',[]) for r in extras) else
    ↪ 0),'hard':hard,'wall':time.time()-t}
    print(json.dumps(out,indent=2));return out

if __name__=='__main__':
    ap=argparse.ArgumentParser();ap.add_argument('--variant',choices=CLASSES,default='r1');ap.add_argumen
    ↪ t('--n',type=int,default=20);ap.add_argument('--seed0',type=int,default=90000);a=ap.parse_args();
    ↪ run(a.variant,a.n,a.seed0)

```

文件: vendor/q3\_breakthrough\_sandbox/reference/run\_q3\_7pt.py

```
import sys,math,json,time
```

```

sys.path.insert(0, '/mnt/data/extbench')
sys.path.insert(0, '/mnt/data/extbench/lib')
import numpy as np
from strict_q3_test import StrictQ3
from sim import generate_case, Simulator
from strategy2 import Params2

r=1150.0
P=[[0.0,0.0]]+[[r*math.cos(k*math.pi/3),r*math.sin(k*math.pi/3)] for k in range(6)]
PLAN={'level1':P,'escalation':[]}

def run(seed0,n):
    rows=[];hard=[];t0=time.time()
    for i in range(n):
        seed=seed0+i; srcs=generate_case(np.random.default_rng(seed),3); sim=Simulator(srcs)
        rb=StrictQ3(sim,PLAN,Params2(name='q3_7pt',theta_miss=0.0,use_belief=False),3)
        try:r0=rb.run()
        except Exception as e:hard.append((seed,str(e)));r0=rb.finish()
        rows.append(r0)
    st=[r['stats'] for r in rows]
    out={'seed0':seed0,'n':n,'full':sum(s['ratio']>=1-1e-9 for s in st),'hard':hard,
        'agg':sum(s['total_time_s'] for s in st)/sum(s['n_sources'] for s in st),
        'meanavg':float(np.mean([s['avg_time_s'] for s in st])),
        'mean_total':float(np.mean([s['total_time_s'] for s in st])),
        'move':float(np.mean([s['move_m'] for s in st])), 'req':float(np.mean([s['n_requests'] for s in
        ↪ st])),
        'miss':sum(s['n_sources']-s['n_cleared'] for s in st),
        'tm':float(np.mean([s['t_move'] for s in st])), 'tmeasure':float(np.mean([s['t_measure'] for s in
        ↪ st])), 'tswitch':float(np.mean([s['t_switch'] for s in
        ↪ st])), 'tclear':float(np.mean([s['t_clear'] for s in st])),
        'wall':time.time()-t0}
    print(json.dumps(out,indent=2))

if __name__=='__main__':
    import argparse
    ap=argparse.ArgumentParser();ap.add_argument('--seed0',type=int,default=90000);ap.add_argument('--n',
    ↪ type=int,default=200);a=ap.parse_args();run(a.seed0,a.n)

```

文件: vendor/q3\_breakthrough\_sandbox/reference/run\_q3\_nextgen.py

```

import sys,math,json,time,argparse
from pathlib import Path
_HERE=Path(__file__).resolve().parent.parent
sys.path.insert(0,str(_HERE));sys.path.insert(0,str(_HERE/'lib'))
import numpy as np
import strategy
strategy.MARGIN=2.0
from strict_filtered_test import StrictFiltered
from sim import generate_case, Simulator
from strategy2 import Params2
from geom import norm, clip_polygon, diameter_rotating_calipers_pair, polygon_centroid

BASE_R=1130.0
BASE_RING=[np.array([BASE_R*math.cos(k*math.pi/3),BASE_R*math.sin(k*math.pi/3)],float) for k in range(6)]

def ring_points(n,r,phase=0.0):
    return [np.array([r*math.cos(phase+2*math.pi*k/n),r*math.sin(phase+2*math.pi*k/n)],float) for k in
    ↪ range(n)]

```

```

def path_len(start,nodes,order):
    cur=np.asarray(start,float);s=0.0
    for i in order:
        s+=norm(nodes[i]-cur);cur=nodes[i]
    return s

def route_order(start,nodes):
    if not nodes:return []
    left=list(range(len(nodes)));order=[];cur=np.asarray(start,float)
    while left:
        i=min(left,key=lambda j: norm(nodes[j]-cur));order.append(i);left.remove(i);cur=nodes[i]
    improved=True
    while improved:
        improved=False;base=path_len(start,nodes,order)
        for i in range(len(order)-1):
            for j in range(i+1,len(order)):
                cand=order[:i]+order[i:j+1][::-1]+order[j+1:]
                if path_len(start,nodes,cand)<base-1e-8:
                    order=cand;improved=True;break
            if improved:break
    return order

class Q3NegDynamic(StrictFiltered):
    def __init__(self,*a,neg=True,conditional=False,threshold=0,outer_n=9,outer_r=1700.0,rotate=False,**k
↪ w):
        super().__init__(*a,**kw)
        self.neg_enabled=neg;self.conditional=conditional;self.threshold=threshold;self.outer_n=outer_n;s
↪ elf.outer_r=outer_r;self.rotate=rotate
        self.neg_hist={ch:[] for ch in
↪ range(1,21)};self.neg_cuts=0;self.conditional_used=False;self.phase_used=0.0

    def _apply_neg(self,ch):
        if not self.neg_enabled or ch not in self.tracked:return
        positives=[np.asarray(e[0],float) for e in self.mlog.get(ch,[]) if e[1]=='direction']
        negatives=self.neg_hist.get(ch,[])
        if not positives or not negatives:return
        poly=self.tracked[ch]
        before=len(poly)
        for p in positives:
            for q in negatives:
                q=np.asarray(q,float)
                n=2.0*(q-p);c=float(q@q-p@p)
                new=clip_polygon(poly,(n,c+1e-7))
                if len(new)>=3:
                    # count only genuine geometric change by vertex/area surrogate
                    if len(new)!=len(poly) or any(norm(np.asarray(a)-np.asarray(b))>1e-7 for a,b in
↪ zip(new,poly[:len(new)])):
                        self.neg_cuts+=1
                    poly=new
        if len(poly)>=3:self.tracked[ch]=poly

    def _record_nosig(self,ch,q):
        super()._record_nosig(ch,q)
        q=np.asarray(q,float).copy()
        if not any(norm(q-p)<1e-8 for p in self.neg_hist[ch]):self.neg_hist[ch].append(q)
        self._apply_neg(ch)

```

```

def _init_region(self, ch, svd):
    super()._init_region(ch, svd); self._apply_neg(ch)

def _update_region(self, ch, pos, svd):
    super()._update_region(ch, pos, svd); self._apply_neg(ch)

def _scan_unknown(self, at=None):
    extra=[]
    for ch, poly in self.tracked.items():
        if ch in self.cleared: continue
        D, _=diameter_rotating_calipers_pair(poly)
        if D>80.0: extra.append(ch)
    chans=sorted(self.unknown)+extra
    for k, ch in enumerate(chans):
        p=at if (k==0 and at is not None) else self.sim.pos
        res, svd=self.sim.measure(p, ch); self.n_measure+=1
        if res=='direction':
            if ch in self.tracked: self._update_region(ch, self.sim.pos, svd)
            else: self._init_region(ch, svd)
            self.ray[ch]=(self.sim.pos.copy(), svd); self.nosig[ch]=0
        elif res=='no_signal':
            q=self.sim.pos.copy()
            if not any(norm(q-x)<1e-8 for x in self.neg_hist[ch]): self.neg_hist[ch].append(q)
            if ch in self.tracked:
                super()._record_nosig(ch, q) # only log mlog, avoid duplicate _record_nosig recursion
                self._apply_neg(ch)
        elif res=='near':
            rr, _=self.sim.clear(self.sim.pos, ch); self.n_clear+=1
            if rr=='success': self.cleared.add(ch); self.unknown.discard(ch)

def _choose_outer_ring(self):
    if not self.rotate: return ring_points(self.outer_n, self.outer_r, 0.0)
    known=[np.asarray(polygon_centroid(poly), float) for ch, poly in self.tracked.items() if ch not in
    ↪ self.cleared]
    best=None
    # phase only unique within one sector
    for m in range(24):
        ph=(2*math.pi/self.outer_n)*m/24.0
        ring=ring_points(self.outer_n, self.outer_r, ph)
        nodes=ring+known
        order=route_order(self.sim.pos, nodes)
        val=path_len(self.sim.pos, nodes, order)
        if best is None or val<best[0]: best=(val, ph, ring)
    self.phase_used=best[1]; return best[2]

def run(self):
    # initial origin scan; retain all negative histories for future discoveries
    self._scan_unknown(); self.scanned.append(self.sim.pos.copy());
    ↪ self.scan_set.append(self.sim.pos.copy())
    known_count=len(set(self.tracked)|self.cleared)
    if self.conditional and known_count<=self.threshold:
        self.pending=self._choose_outer_ring(); self.conditional_used=True
    else:
        self.pending=[p.copy() for p in BASE_RING]
    guard=0
    while guard<4000:
        guard+=1; self._maybe_escalate(); jobs=self._jobs()
        if not jobs: break

```

```

        kind,ch,target,_=self._plan_next(jobs)
        if kind=='scan':self._do_scan(target)
        else:self._do_clear(ch)
    return self.finish()

PLAN={'level1':[[0.,0.]]+[[BASE_R*math.cos(k*math.pi/3),BASE_R*math.sin(k*math.pi/3)] for k in
↪ range(6)],'escalation':[]}

def run(n=200,seed0=90000,variant='base',threshold=0,outer_n=9,outer_r=1700.0):
    cfg={
        'base':dict(neg=False,conditional=False,rotate=False),
        'neg':dict(neg=True,conditional=False,rotate=False),
        'cond':dict(neg=False,conditional=True,rotate=False),
        'condneg':dict(neg=True,conditional=True,rotate=False),
        'condnegrot':dict(neg=True,conditional=True,rotate=True),
    }[variant]
    rows=[];hard=[];conds=[];cuts=[];t0=time.time()
    for i in range(n):
        srcls=generate_case(np.random.default_rng(seed0+i),3);sim=Simulator(srcls)
        rb=Q3NegDynamic(sim,PLAN,Params2(name=variant,theta_miss=0.0,use_belief=False),3,threshold=thresh
↪ old,outer_n=outer_n,outer_r=outer_r,**cfg)
        try:r=rb.run()
        except Exception as e:hard.append((seed0+i,str(e)));r=rb.finish()
        rows.append(r);conds.append(rb.conditional_used);cuts.append(rb.neg_cuts)
    st=[r['stats'] for r in rows]
    out={'variant':variant,'threshold':threshold,'outer_n':outer_n,'outer_r':outer_r,'n':n,'seed0':seed0,
        'full':sum(s['ratio']>=1-1e-9 for s in st),'agg':sum(s['total_time_s'] for s in
↪ st)/sum(s['n_sources'] for s in st),
        'meanavg':float(np.mean([s['avg_time_s'] for s in
↪ st])),
        'mean_total':float(np.mean([s['total_time_s'] for s in st])),
        'move':float(np.mean([s['move_m'] for s in st])),
        'req':float(np.mean([s['n_requests'] for s in
↪ st])),
        'clear':float(np.mean([s['t_clear'] for s in st])),
        'measure':float(np.mean([s['t_measure'] for s
↪ in st])),
        'switch':float(np.mean([s['t_switch'] for s in st])),
        'conditional_rate':float(np.mean(conds)),
        'neg_cuts':float(np.mean(cuts)),
        'miss':sum(s['n_sources']
↪ s['n_cleared'] for s in st),
        'hard':hard,'wall':time.time()-t0}
    print(json.dumps(out,indent=2));return out

if __name__=='__main__':
    ap=argparse.ArgumentParser();ap.add_argument('--n',type=int,default=200);ap.add_argument('--seed0',ty
↪ pe=int,default=90000);ap.add_argument('--variant',choices=['base','neg','cond','condneg','condneg
↪ rot'],default='condnegrot');ap.add_argument('--threshold',type=int,default=0);ap.add_argument('--
↪ outer_n',type=int,default=9);ap.add_argument('--outer_r',type=float,default=1700.0);a=ap.parse_ar
↪ gs();run(a.n,a.seed0,a.variant,a.threshold,a.outer_n,a.outer_r)

```

文件: vendor/q3\_breakthrough\_sandbox/reference/run\_q3\_shared.py

```

import sys,json,time,math,argparse
from pathlib import Path
_HERE=Path(__file__).resolve().parent.parent
sys.path.insert(0,str(_HERE));sys.path.insert(0,str(_HERE/'lib'))
import numpy as np
import strategy
strategy.MARGIN=2.0
from strict_filtered_test import StrictFiltered
from sim import generate_case,Simulator
from strategy2 import Params2

```

```

from geom import mec, norm, clip_polygon

class Q3SharedStrict(StrictFiltered):
    def __init__(self,*a,shared_step=100.0,min_channels=3,use_negative_cut=False,**kw):
        super().__init__(*a,**kw);self.shared_step=shared_step;self.min_channels=min_channels;self.use_negative_
        ↪ _cut=use_negative_cut;self._did_initial_shared=False;self.shared_reads=0;self.shared_steps=0
    def _scan_unknown(self,at=None):
        initial = not self._did_initial_shared and len(self.scanned)==0 and at is None
        super()._scan_unknown(at=at)
        if initial:
            self._did_initial_shared=True
            self._initial_shared()
    def _initial_shared(self):
        channels=[]
        for ch in self.tracked:
            if ch in self.cleared:continue
            _,r=mec(self.tracked[ch])
            if r>100:channels.append(ch)
        if len(channels)<self.min_channels:return
        start=self.sim.pos.copy();best=None
        for k in range(24):
            ang=2*math.pi*k/24;q=start+self.shared_step*np.array([math.cos(ang),math.sin(ang)])
            gain=0.0
            for ch in channels:
                center,r=mec(self.tracked[ch]);center=np.asarray(center,float)
                # Latest positive anchor. mlog initially has the origin positive.
                positives=[e for e in self.mlog.get(ch,[]) if e[1]=='direction']
                anchor=np.asarray(positives[-1][0],float) if positives else start
                a=anchor-center;b=q-center;den=norm(a)*norm(b)
                sine=abs(float(a[0]*b[1]-a[1]*b[0]))/den if den>1e-9 else 1.0
                est=2*.01754*norm(q-center)/max(.01,sine)
                gain += min(r,max(0.,r-est))
            cand=(gain,-k,q)
            if best is None or cand[:2]>best[:2]:best=cand
        if best is None:return
        q=best[2]
        # one move, many bearings
        for ch in channels:
            if ch in self.cleared or ch not in self.tracked:continue
            prev_pos=None
            positives=[e for e in self.mlog.get(ch,[]) if e[1]=='direction']
            if positives:prev_pos=np.asarray(positives[-1][0],float)
            res,svd=self.sim.measure(q,ch);self.n_measure+=1;self.shared_reads+=1
            if res=='direction':
                self.nosig[ch]=0;self.ray[ch]=(self.sim.pos.copy(),svd);self._update_region(ch,self.sim.pos,svd)
            elif res=='near':
                rr,_=self.sim.clear(self.sim.pos,ch);self.n_clear+=1
                if rr=='success':self.cleared.add(ch);self.unknown.discard(ch)
            else:
                self.nosig[ch]=self.nosig.get(ch,0)+1;self._record_nosig(ch,self.sim.pos)
                if self.use_negative_cut and prev_pos is not None and ch in self.tracked:
                    # Safe Q3 affine consequence of  $|P-g| \leq R < |Q-g|$ :
                    #  $2(Q-P).g \leq |Q|^2 - |P|^2$  (strict truth, relaxed closed halfplane).
                    n=2.0*(self.sim.pos-prev_pos);c=float(self.sim.pos@self.sim.pos-prev_pos@prev_pos)
                    p2=clip_polygon(self.tracked[ch],(n,c))
                    if len(p2)>=3:self.tracked[ch]=p2

self.shared_steps+=1

```

```

def make_plan(r=1150.0):
    pts=[[0.,0.]]+[[r*math.cos(k*math.pi/3),r*math.sin(k*math.pi/3)] for k in range(6)]
    return {'level1':pts,'escalation':[]}

def run(n=200,seed0=90000,r=1150.,step=100.,min_channels=3,neg=False):
    plan=make_plan(r);rows=[];hard=[];ss=[];sr=[];t0=time.time()
    for i in range(n):
        srcs=generate_case(np.random.default_rng(seed0+i),3);sim=Simulator(srcs);rb=Q3SharedStrict(sim,plan,Par
        ↪ ams2(name='q3shared',theta_miss=0.0,use_belief=False),3,shared_step=step,min_channels=min_channels,
        ↪ use_negative_cut=neg)
        try:rr=rb.run()
        except Exception as e:hard.append((seed0+i,str(e)));rr=rb.finish()
        rows.append(rr);ss.append(rb.shared_steps);sr.append(rb.shared_reads)
    st=[x['stats'] for x in rows]
    out={'n':n,'seed0':seed0,'ring':r,'step':step,'min_channels':min_channels,'neg':neg,'full':sum(s['ratio'
    ↪ ]>=1-1e-9 for s in st),'agg':sum(s['total_time_s'] for s in st)/sum(s['n_sources'] for s in
    ↪ st),'meanavg':float(np.mean([s['avg_time_s'] for s in
    ↪ st])),,'mean_total':float(np.mean([s['total_time_s'] for s in st])),,'move':float(np.mean([s['move_m']
    ↪ for s in st])),,'req':float(np.mean([s['n_requests'] for s in
    ↪ st])),,'clear':float(np.mean([s['t_clear'] for s in st])),,'measure':float(np.mean([s['t_measure'] for s
    ↪ in st])),,'switch':float(np.mean([s['t_switch'] for s in st])),,'shared_steps':float(np.mean(ss)),,'sha
    ↪ red_reads':float(np.mean(sr)),,'miss':sum(s['n_sources']-s['n_cleared'] for s in
    ↪ st),'hard':hard,'wall':time.time()-t0}
    print(json.dumps(out,indent=2));return out
if __name__=='__main__':
    ap=argparse.ArgumentParser();ap.add_argument('--n',type=int,default=200);ap.add_argument('--seed0',type=
    ↪ int,default=90000);ap.add_argument('--ring',type=float,default=1150);ap.add_argument('--step',type=f
    ↪ loat,default=100);ap.add_argument('--min-channels',type=int,default=3);ap.add_argument('--neg',actio
    ↪ n='store_true');a=ap.parse_args();run(a.n,a.seed0,a.ring,a.step,a.min_channels,a.neg)

```

文件: vendor/q3\_breakthrough\_sandbox/reference/run\_safe7\_r1130.py

```

import sys,math,json,time
from pathlib import Path
_HERE=Path(__file__).resolve().parent.parent
sys.path.insert(0,str(_HERE));sys.path.insert(0,str(_HERE/'lib'))
import numpy as np
import strategy
strategy.MARGIN=2.0
from strict_filtered_test import StrictFiltered
from sim import generate_case,Simulator
from strategy2 import Params2

r=1130.0
PTS=[[0.,0.]]+[[r*math.cos(k*math.pi/3),r*math.sin(k*math.pi/3)] for k in range(6)]
PLAN={'level1':PTS,'escalation':[]}

def run(n=200,seed0=90000):
    rows=[];hard=[];t0=time.time()
    for i in range(n):
        srcs=generate_case(np.random.default_rng(seed0+i),3);sim=Simulator(srcs);rb=StrictFiltered(sim,PLAN,Par
        ↪ ams2(name='safe7',theta_miss=0.0,use_belief=False),3)
        try:r0=rb.run()
        except Exception as e:hard.append((seed0+i,str(e)));r0=rb.finish()
        rows.append(r0)
    st=[r['stats'] for r in rows]

```

```

out={'n':n,'seed0':seed0,'full':sum(s['ratio']>=1-1e-9 for s in st),'agg':sum(s['total_time_s'] for s in
↪ st)/sum(s['n_sources'] for s in st),'meanavg':float(np.mean([s['avg_time_s'] for s in
↪ st])), 'move':float(np.mean([s['move_m'] for s in st])), 'req':float(np.mean([s['n_requests'] for s in
↪ st])), 'clear':float(np.mean([s['t_clear'] for s in st])), 'measure':float(np.mean([s['t_measure'] for s
↪ in st])), 'switch':float(np.mean([s['t_switch'] for s in
↪ st])), 'miss':sum(s['n_sources']-s['n_cleared'] for s in st), 'hard':hard, 'wall':time.time()-t0}
print(json.dumps(out,indent=2))
if __name__=='__main__':
    import argparse
    ap=argparse.ArgumentParser();ap.add_argument('--n',type=int,default=200);ap.add_argument('--seed0',type=
↪ int,default=90000);a=ap.parse_args();run(a.n,a.seed0)

```

文件: vendor/q3\_breakthrough\_sandbox/reference/strict\_q3\_mec18.py

```

import sys,json,time,argparse
from pathlib import Path
_HERE=Path(__file__).resolve().parent.parent
sys.path.insert(0,str(_HERE));sys.path.insert(0,str(_HERE/'lib'))
import strict_q3_test as sq
import strategy
strategy.MARGIN=2.0
import numpy as np
from sim import generate_case,Simulator
from strategy2 import Params2

def run(seed0,n):
    rows=[];hard=[];t0=time.time()
    for i in range(n):
        sracs=generate_case(np.random.default_rng(seed0+i),3);sim=Simulator(sracs);rb=sq.StrictQ3(sim,sq.PLAN,Par
↪ ms2(name='mec18',theta_miss=0.0,use_belief=False),3)
        try:r=rb.run()
        except Exception as e:hard.append((seed0+i,str(e)));r=rb.finish()
        rows.append(r)
    st=[r['stats'] for r in rows]
    print(json.dumps({'n':n,'full':sum(s['ratio']>=1-1e-9 for s in st),'agg':sum(s['total_time_s'] for s in
↪ st)/sum(s['n_sources'] for s in st),'meanavg':float(np.mean([s['avg_time_s'] for s in
↪ st])), 'move':float(np.mean([s['move_m'] for s in st])), 'req':float(np.mean([s['n_requests'] for s in
↪ st])), 'clear':float(np.mean([s['t_clear'] for s in st])), 'miss':sum(s['n_sources']-s['n_cleared'] for
↪ s in st), 'hard':hard, 'wall':time.time()-t0},indent=2))
if __name__=='__main__':
    ap=argparse.ArgumentParser();ap.add_argument('--seed0',type=int,default=90000);ap.add_argument('--n',typ
↪ e=int,default=200);a=ap.parse_args();run(a.seed0,a.n)

```

文件: vendor/q3\_breakthrough\_sandbox/run\_q3\_nextgen.py

```

import sys,math,json,time,argparse
from pathlib import Path
_HERE=Path(__file__).resolve().parent
sys.path.insert(0,str(_HERE));sys.path.insert(0,str(_HERE/'lib'))
import numpy as np
import strategy
strategy.MARGIN=2.0
from strict_filtered_test import StrictFiltered
from sim import generate_case,Simulator
from strategy2 import Params2
from geom import norm, clip_polygon, diameter_rotating_calipers_pair, polygon_centroid

```

```

BASE_R=1130.0
BASE_RING=[np.array([BASE_R*math.cos(k*math.pi/3),BASE_R*math.sin(k*math.pi/3)],float) for k in range(6)]

def ring_points(n,r,phase=0.0):
    return [np.array([r*math.cos(phase+2*math.pi*k/n),r*math.sin(phase+2*math.pi*k/n)],float) for k in
    ↪ range(n)]

def path_len(start,nodes,order):
    cur=np.asarray(start,float);s=0.0
    for i in order:
        s+=norm(nodes[i]-cur);cur=nodes[i]
    return s

def route_order(start,nodes):
    if not nodes:return []
    left=list(range(len(nodes)));order=[];cur=np.asarray(start,float)
    while left:
        i=min(left,key=lambda j: norm(nodes[j]-cur));order.append(i);left.remove(i);cur=nodes[i]
    improved=True
    while improved:
        improved=False;base=path_len(start,nodes,order)
        for i in range(len(order)-1):
            for j in range(i+1,len(order)):
                cand=order[i]+order[i:j+1][::-1]+order[j+1:]
                if path_len(start,nodes,cand)<base-1e-8:
                    order=cand;improved=True;break
            if improved:break
    return order

class Q3NegDynamic(StrictFiltered):
    def __init__(self,*a,neg=True,conditional=False,threshold=0,outer_n=9,outer_r=1700.0,rotate=False,**k
    ↪ w):
        super().__init__(*a,**kw)
        self.neg_enabled=neg;self.conditional=conditional;self.threshold=threshold;self.outer_n=outer_n;s
    ↪ elf.outer_r=outer_r;self.rotate=rotate
        self.neg_hist={ch:[] for ch in
    ↪ range(1,21)};self.neg_cuts=0;self.conditional_used=False;self.phase_used=0.0

    def _apply_neg(self,ch):
        if not self.neg_enabled or ch not in self.tracked:return
        positives=[np.asarray(e[0],float) for e in self.mlog.get(ch,[]) if e[1]=='direction']
        negatives=self.neg_hist.get(ch,[])
        if not positives or not negatives:return
        poly=self.tracked[ch]
        before=len(poly)
        for p in positives:
            for q in negatives:
                q=np.asarray(q,float)
                n=2.0*(q-p);c=float(q@q-p@p)
                new=clip_polygon(poly,(n,c+1e-7))
                if len(new)>=3:
                    # count only genuine geometric change by vertex/area surrogate
                    if len(new)!=len(poly) or any(norm(np.asarray(a)-np.asarray(b))>1e-7 for a,b in
    ↪ zip(new,poly[:len(new)])):
                        self.neg_cuts+=1
                    poly=new
        if len(poly)>=3:self.tracked[ch]=poly

```

```

def _record_nosig(self, ch, q):
    super()._record_nosig(ch, q)
    q = np.asarray(q, float).copy()
    if not any(norm(q-p) < 1e-8 for p in self.neg_hist[ch]): self.neg_hist[ch].append(q)
    self._apply_neg(ch)

def _init_region(self, ch, svd):
    super()._init_region(ch, svd); self._apply_neg(ch)

def _update_region(self, ch, pos, svd):
    super()._update_region(ch, pos, svd); self._apply_neg(ch)

def _scan_unknown(self, at=None):
    extra = []
    for ch, poly in self.tracked.items():
        if ch in self.cleared: continue
        D, _ = diameter_rotating_calipers_pair(poly)
        if D > 80.0: extra.append(ch)
    chans = sorted(self.unknown) + extra
    for k, ch in enumerate(chans):
        p = at if (k == 0 and at is not None) else self.sim.pos
        res, svd = self.sim.measure(p, ch); self.n_measure += 1
        if res == 'direction':
            if ch in self.tracked: self._update_region(ch, self.sim.pos, svd)
            else: self._init_region(ch, svd)
            self.ray[ch] = (self.sim.pos.copy(), svd); self.nosig[ch] = 0
        elif res == 'no_signal':
            q = self.sim.pos.copy()
            if not any(norm(q-x) < 1e-8 for x in self.neg_hist[ch]): self.neg_hist[ch].append(q)
            if ch in self.tracked:
                super()._record_nosig(ch, q) # only log mlog, avoid duplicate _record_nosig recursion
                self._apply_neg(ch)
        elif res == 'near':
            rr, _ = self.sim.clear(self.sim.pos, ch); self.n_clear += 1
            if rr == 'success': self.cleared.add(ch); self.unknown.discard(ch)

def _choose_outer_ring(self):
    if not self.rotate: return ring_points(self.outer_n, self.outer_r, 0.0)
    known = [np.asarray(polygon_centroid(poly), float) for ch, poly in self.tracked.items() if ch not in
    ↪ self.cleared]
    best = None
    # phase only unique within one sector
    for m in range(24):
        ph = (2 * math.pi / self.outer_n) * m / 24.0
        ring = ring_points(self.outer_n, self.outer_r, ph)
        nodes = ring + known
        order = route_order(self.sim.pos, nodes)
        val = path_len(self.sim.pos, nodes, order)
        if best is None or val < best[0]: best = (val, ph, ring)
    self.phase_used = best[1]; return best[2]

def run(self):
    # initial origin scan; retain all negative histories for future discoveries
    self._scan_unknown(); self.scanned.append(self.sim.pos.copy());
    ↪ self.scan_set.append(self.sim.pos.copy())
    known_count = len(set(self.tracked) | self.cleared)
    if self.conditional and known_count <= self.threshold:

```

```

        self.pending=self._choose_outer_ring();self.conditional_used=True
    else:
        self.pending=[p.copy() for p in BASE_RING]
    guard=0
    while guard<4000:
        guard+=1;self._maybe_escalate();jobs=self._jobs()
        if not jobs:break
        kind,ch,target,_=self._plan_next(jobs)
        if kind=='scan':self._do_scan(target)
        else:self._do_clear(ch)
    return self.finish()

PLAN={'level1':[[0.,0.]]+[[BASE_R*math.cos(k*math.pi/3),BASE_R*math.sin(k*math.pi/3)] for k in
↪ range(6)],'escalation':[]}

def run(n=200,seed0=90000,variant='base',threshold=0,outer_n=9,outer_r=1700.0):
    cfg={
        'base':dict(neg=False,conditional=False,rotate=False),
        'neg':dict(neg=True,conditional=False,rotate=False),
        'cond':dict(neg=False,conditional=True,rotate=False),
        'condneg':dict(neg=True,conditional=True,rotate=False),
        'condnegrot':dict(neg=True,conditional=True,rotate=True),
    }[variant]
    rows=[];hard=[];conds=[];cuts=[];t0=time.time()
    for i in range(n):
        srcls=generate_case(np.random.default_rng(seed0+i),3);sim=Simulator(srcls)
        rb=Q3NegDynamic(sim,PLAN,Params2(name=variant,theta_miss=0.0,use_belief=False),3,threshold=thresh
↪ old,outer_n=outer_n,outer_r=outer_r,**cfg)
        try:r=rb.run()
        except Exception as e:hard.append((seed0+i,str(e)));r=rb.finish()
        rows.append(r);conds.append(rb.conditional_used);cuts.append(rb.neg_cuts)
    st=[r['stats'] for r in rows]
    out={'variant':variant,'threshold':threshold,'outer_n':outer_n,'outer_r':outer_r,'n':n,'seed0':seed0,
        'full':sum(s['ratio']>=1-1e-9 for s in st),'agg':sum(s['total_time_s'] for s in
↪ st)/sum(s['n_sources'] for s in st),
        'meanavg':float(np.mean([s['avg_time_s'] for s in
↪ st])),
        'mean_total':float(np.mean([s['total_time_s'] for s in st])),
        'move':float(np.mean([s['move_m'] for s in st])),
        'req':float(np.mean([s['n_requests'] for s in
↪ st])),
        'clear':float(np.mean([s['t_clear'] for s in st])),
        'measure':float(np.mean([s['t_measure'] for s
↪ in st])),
        'switch':float(np.mean([s['t_switch'] for s in st])),
        'conditional_rate':float(np.mean(conds)),
        'neg_cuts':float(np.mean(cuts)),
        'miss':sum(s['n_sources']
↪ s['n_cleared'] for s in st),
        'hard':hard,'wall':time.time()-t0}
    print(json.dumps(out,indent=2));return out

if __name__=='__main__':
    ap=argparse.ArgumentParser();ap.add_argument('--n',type=int,default=200);ap.add_argument('--seed0',ty
↪ pe=int,default=90000);ap.add_argument('--variant',choices=['base','neg','cond','condneg','condneg
↪ rot'],default='condnegrot');ap.add_argument('--threshold',type=int,default=0);ap.add_argument('--
↪ outer_n',type=int,default=9);ap.add_argument('--outer_r',type=float,default=1700.0);a=ap.parse_ar
↪ gs();run(a.n,a.seed0,a.variant,a.threshold,a.outer_n,a.outer_r)

```

文件: vendor/q3\_breakthrough\_sandbox/run\_safe7\_r1130.py

```

import sys,math,json,time
from pathlib import Path
_HERE=Path(__file__).resolve().parent

```

```

sys.path.insert(0,str(_HERE));sys.path.insert(0,str(_HERE/'lib'))
import numpy as np
import strategy
strategy.MARGIN=2.0
from strict_filtered_test import StrictFiltered
from sim import generate_case, Simulator
from strategy2 import Params2

r=1130.0
PTS=[[0.,0.]]+[[r*math.cos(k*math.pi/3),r*math.sin(k*math.pi/3)] for k in range(6)]
PLAN={'level1':PTS,'escalation':[]}

def run(n=200,seed0=90000):
    rows=[];hard=[];t0=time.time()
    for i in range(n):
        sracs=generate_case(np.random.default_rng(seed0+i),3);sim=Simulator(sracs);rb=StrictFiltered(sim,PLAN,Par
        ↪ ams2(name='safe7',theta_miss=0.0,use_belief=False),3)
        try:r0=rb.run()
        except Exception as e:hard.append((seed0+i,str(e)));r0=rb.finish()
        rows.append(r0)
    st=[r['stats'] for r in rows]
    out={'n':n,'seed0':seed0,'full':sum(s['ratio']>=1-1e-9 for s in st),'agg':sum(s['total_time_s'] for s in
    ↪ st)/sum(s['n_sources'] for s in st),'meanavg':float(np.mean([s['avg_time_s'] for s in
    ↪ st])), 'move':float(np.mean([s['move_m'] for s in st])), 'req':float(np.mean([s['n_requests'] for s in
    ↪ st])), 'clear':float(np.mean([s['t_clear'] for s in st])), 'measure':float(np.mean([s['t_measure'] for s
    ↪ in st])), 'switch':float(np.mean([s['t_switch'] for s in
    ↪ st])), 'miss':sum(s['n_sources']-s['n_cleared'] for s in st), 'hard':hard, 'wall':time.time()-t0}
    print(json.dumps(out,indent=2))
if __name__=='__main__':
    import argparse
    ap=argparse.ArgumentParser();ap.add_argument('--n',type=int,default=200);ap.add_argument('--seed0',type=
    ↪ int,default=90000);a=ap.parse_args();run(a.n,a.seed0)

```

文件: vendor/q3\_breakthrough\_sandbox/strict\_filtered\_test.py

```

import sys,time,json,math,argparse
sys.path.insert(0,'/mnt/data/extbench/lib')
import numpy as np
import strategy, strategy2
from sim import generate_case, Simulator
from strategy2 import Params2, RobotV2
from geom import norm, cross

def identity_simplify(poly,tol=0.05): return [np.asarray(p,float).copy() for p in poly]
strategy._simplify=identity_simplify; strategy2._simplify=identity_simplify

STRICT3=[[0.0,0.0],[1080.8618318011463,0.0],[673.9063293463646,845.0518083002004],[-240.51438429012381,10
↪ 53.762369034913],[-973.8228609568137,468.96837305133465],[-973.8228609568138,-468.9683730513344],[-24
↪ 0.51438429012407,-1053.762369034913],[673.9063293463643,-845.0518083002006]]
def mincert25():
    p=[[0.0,0.0]]
    for k in range(12):
        a=math.radians(15+30*k);p.append([950*math.cos(a),950*math.sin(a)])
    for k in range(12):
        a=math.radians(30*k);p.append([1875*math.cos(a),1875*math.sin(a)])
    return p
STRICT4=mincert25()

```

```

def inside_convex(q,P):
    # P expected CCW from clipping; tolerate either orientation by sign consistency.
    m=len(P); signs=[]
    for i in range(m):
        a=P[i];b=P[(i+1)%m];v=b-a;w=q-a;z=v[0]*w[1]-v[1]*w[0]
        if abs(z)>1e-9:signs.append(1 if z>0 else -1)
    return not signs or all(s==signs[0] for s in signs)

def segdist(q,a,b):
    v=b-a;d=float(v@v)
    if d<=1e-18:return norm(q-a)
    t=max(0.0,min(1.0,float((q-a)@v/d)));return norm(q-(a+t*v))

def polydist(q,P):
    if inside_convex(q,P): return 0.0
    return min(segdist(q,P[i],P[(i+1)%len(P)]) for i in range(len(P)))

class StrictFiltered(RobotV2):
    def __init__(self,*a,**kw):super().__init__(*a,**kw);self.hard=[]
    def _safe_lattice(self,poly,spacing=33.0):
        P=np.asarray(poly,float); rcov=spacing/math.sqrt(3.0); pad=rcov+1e-6
        lo=P.min(axis=0)-pad;hi=P.max(axis=0)+pad;dy=spacing*math.sqrt(3)/2
        r0=math.floor(lo[1]/dy)-1;r1=math.ceil(hi[1]/dy)+1;pts=[]
        for r in range(r0,r1+1):
            y=r*dy;xoff=spacing/2 if (r&1) else
            ↪ 0.0;c0=math.floor((lo[0]-xoff)/spacing)-1;c1=math.ceil((hi[0]-xoff)/spacing)+1
            for c in range(c0,c1+1):
                x=xoff+c*spacing;q=np.array([x,y],float)
                if polydist(q,P)<=rcov+1e-7:pts.append(q)
        cur=self.sim.pos.copy();out=[];left=pts
        while left:
            k=min(range(len(left)),key=lambda i:norm(left[i]-cur));cur=left[k];out.append(left.pop(k))
        return out
    def _clear_sweep(self,ch,poly):
        for p in self._safe_lattice(poly,self.params.sweep_spacing):
            r,_=self.sim.clear(p,ch);self.n_clear+=1
            if r=='success':return True
        return False
    def _do_clear(self,ch):
        ok=self.home_and_clear(ch)
        if not ok:
            poly=self.tracked.get(ch)
            if poly is not None and len(poly)>=3:ok=self._clear_sweep(ch,poly)
            if ok:self.cleared.add(ch)
        else:self.hard.append(ch);raise RuntimeError('strict fallback failed')

def run(problem,seed0,n,mec18=False):
    if mec18:strategy.MARGIN=2.0
    else:strategy.MARGIN=0.5
    plan={'level1':STRICT3 if problem==3 else STRICT4,'escalation':[]}; rows=[];hard=[];t0=time.time()
    for i in range(n):
        sracs=generate_case(np.random.default_rng(seed0+i),problem);sim=Simulator(sracs);rb=StrictFiltered(sim,pl
        ↪ an,Params2(name='sf',theta_miss=0.0,use_belief=False),problem)
        try:r=rb.run()
        except Exception as e:hard.append((seed0+i,str(e)));r=rb.finish()
        rows.append(r)
    st=[r['stats'] for r in rows]

```

```

out={'problem':problem,'n':n,'full':sum(s['ratio']>=1-1e-9 for s in st),'agg':sum(s['total_time_s'] for s
↪ in st)/sum(s['n_sources'] for s in st),'meanavg':float(np.mean([s['avg_time_s'] for s in
↪ st])), 'move':float(np.mean([s['move_m'] for s in st])), 'req':float(np.mean([s['n_requests'] for s in
↪ st])), 'clear':float(np.mean([s['t_clear'] for s in st])), 'measure':float(np.mean([s['t_measure'] for s
↪ in st])), 'switch':float(np.mean([s['t_switch'] for s in
↪ st])), 'miss':sum(s['n_sources']-s['n_cleared'] for s in
↪ st), 'hard':hard, 'wall':time.time()-t0};print(json.dumps(out,indent=2))
if __name__=='__main__':
    ap=argparse.ArgumentParser();ap.add_argument('--problem',type=int,required=True);ap.add_argument('--seed'
↪ 0',type=int);ap.add_argument('--n',type=int,default=100);ap.add_argument('--mec18',action='store_true'
↪ e');a=ap.parse_args();run(a.problem,a.seed0 or (90000 if a.problem==3 else 700000),a.n,a.mec18)

```

## 文件: vendor/q3\_breakthrough\_sandbox/strict\_q3\_test.py

```

import sys,time,json,math
sys.path.insert(0,'/mnt/data/extbench/lib')
import numpy as np
import strategy, strategy2
from sim import generate_case, Simulator
from strategy2 import Params2, RobotV2
from geom import norm, polygon_centroid

STRICT3=[[0.0,0.0],[1080.8618318011463,0.0],[673.9063293463646,845.0518083002004],[-240.51438429012381,10
↪ 53.762369034913],[-973.8228609568137,468.96837305133465],[-973.8228609568138,-468.9683730513344],[-24
↪ 0.51438429012407,-1053.762369034913],[673.9063293463643,-845.0518083002006]]
PLAN={'level1':STRICT3,'escalation':[]}]

# Disable under-approximating polygon simplification globally.
def identity_simplify(poly,tol=0.05):
    return [np.asarray(p,float).copy() for p in poly]
strategy._simplify=identity_simplify
strategy2._simplify=identity_simplify

class StrictQ3(RobotV2):
    def __init__(self,*a,**kw):
        super().__init__(*a,**kw); self.hard_fail=[]
    def _safe_lattice(self, poly, spacing=33.0):
        # Full-plane triangular lattice over expanded bbox. Keeping centers outside the polygon
        # preserves the spacing/sqrt(3) covering guarantee at polygon boundaries.
        P=np.asarray(poly,float)
        lo=P.min(axis=0)-spacing; hi=P.max(axis=0)+spacing
        dy=spacing*math.sqrt(3)/2.0
        r0=math.floor(lo[1]/dy)-1; r1=math.ceil(hi[1]/dy)+1
        pts=[]
        for r in range(r0,r1+1):
            y=r*dy; xoff=spacing/2.0 if (r&1) else 0.0
            c0=math.floor((lo[0]-xoff)/spacing)-1; c1=math.ceil((hi[0]-xoff)/spacing)+1
            for c in range(c0,c1+1):
                x=xoff+c*spacing
                if lo[0]<=x<=hi[0] and lo[1]<=y<=hi[1]: pts.append(np.array([x,y],float))
        # nearest-neighbor ordering from current position only affects time, not coverage.
        cur=self.sim.pos.copy(); out=[]; left=pts
        while left:
            k=min(range(len(left)),key=lambda i:norm(left[i]-cur)); cur=left[k]; out.append(left.pop(k))
        return out
    def _clear_sweep(self,ch,poly):
        for p in self._safe_lattice(poly,self.params.sweep_spacing):

```

```

        r,_=self.sim.clear(p,ch); self.n_clear+=1
        if r=='success': return True
    return False
def _do_clear(self,ch):
    ok=self.home_and_clear(ch)
    if not ok:
        poly=self.tracked.get(ch)
        if poly is not None and len(poly)>=3:
            # No 900-point cutoff in ultimate fallback: correctness first.
            ok=self._clear_sweep(ch,poly)
    if ok:
        self.cleared.add(ch)
    else:
        self.hard_fail.append(ch)
        # keep it unresolved, but avoid infinite loop by stopping this run explicitly
        raise RuntimeError(f'strict fallback failed ch={ch}')

def run(seed0,n):
    rows=[]; hard=[]; t0=time.time()
    for i in range(n):
        seed=seed0+i; srcs=generate_case(np.random.default_rng(seed),3); sim=Simulator(srcs)
        rb=StrictQ3(sim,PLAN,Params2(name='strictq3',theta_miss=0.0,use_belief=False),3)
        try: res=rb.run()
        except Exception as e:
            hard.append((seed,str(e))); res=rb.finish()
        rows.append((seed,res,rb))
    st=[r[1]['stats'] for r in rows]
    out={
        'seed0':seed0,'n':n,'full':sum(s['ratio']>=1-1e-9 for s in st),'hard_fail':hard,
        'mean_ratio':float(np.mean([s['ratio'] for s in st])),
        'mean_avg_s_per_source':float(np.mean([s['avg_time_s'] for s in st if s['avg_time_s'] is not None])),
        'aggregate_s_per_source':float(sum(s['total_time_s'] for s in st)/sum(s['n_sources'] for s in st)),
        'mean_total_time':float(np.mean([s['total_time_s'] for s in st])),
        'mean_move_m':float(np.mean([s['move_m'] for s in st])),
        'mean_requests':float(np.mean([s['n_requests'] for s in st])),
        'total_missed':int(sum(s['n_sources']-s['n_cleared'] for s in st)),
        't_move':float(np.mean([s['t_move'] for s in st])), 't_measure':float(np.mean([s['t_measure'] for s
↵ in st])),
        't_switch':float(np.mean([s['t_switch'] for s in st])), 't_clear':float(np.mean([s['t_clear'] for s
↵ in st])),
        'wall_s':time.time()-t0,
    }
    return out
if __name__=='__main__':
    import argparse
    ap=argparse.ArgumentParser(); ap.add_argument('--seed0',type=int,default=90000);
    ↵ ap.add_argument('--n',type=int,default=1000); a=ap.parse_args()
    print(json.dumps(run(a.seed0,a.n),indent=2))

```

文件: q1q2/generate\_fast.py

```

import math, itertools, json
from pathlib import Path
import numpy as np
from scipy.spatial import ConvexHull
from shapely.geometry import Polygon, Point
import matplotlib

```

```

matplotlib.use('Agg')
import matplotlib.pyplot as plt
from matplotlib import font_manager

# use CJK font if available
for fam in ['Noto Sans CJK SC', 'AR PL UMing CN', 'DejaVu Sans']:
    try:
        path=font_manager.findfont(fam, fallback_to_default=False)
        plt.rcParams['font.family']=fam
        break
    except Exception:
        pass
plt.rcParams['axes.unicode_minus']=False

ROOT=Path(__file__).resolve().parent; FIG=ROOT/'figures'; QA=ROOT/'qa'
FIG.mkdir(parents=True,exist_ok=True); QA.mkdir(parents=True,exist_ok=True)

def unit(t): return np.array([math.cos(t),math.sin(t)])
def wedge_polygon(S,theta,delta,L=5000):
    S=np.array(S,float); a=S+L*unit(theta-delta); b=S+L*unit(theta+delta)
    return Polygon([tuple(S),tuple(a),tuple(b)])
def sector_polygon(rmin=5,rmax=1500,delta=math.radians(1),n=240):
    outer=[(rmax*math.cos(a),rmax*math.sin(a)) for a in np.linspace(-delta,delta,n//2)]
    inner=[(rmin*math.cos(a),rmin*math.sin(a)) for a in np.linspace(delta,-delta,n//2)]
    return Polygon(outer+inner)
def poly_pts(g):
    if g.is_empty: return np.empty((0,2))
    h=g.convex_hull
    if h.geom_type == 'Polygon':
        return np.array(h.exterior.coords[:-1])
    if h.geom_type == 'LineString':
        return np.array(h.coords)
    if h.geom_type == 'Point':
        return np.array([h.coords[0]])
    return np.empty((0,2))
def diameter(P):
    if len(P)<2: return 0.0
    best=0
    for i,j in itertools.combinations(range(len(P)),2):
        best=max(best,float(np.linalg.norm(P[i]-P[j])))
    return best

def circle2(a,b): c=(a+b)/2; return c,float(np.linalg.norm(a-b))/2

def circle3(a,b,c):
    ax,ay=a;bx,by=b;cx,cy=c;d=2*(ax*(by-cy)+bx*(cy-ay)+cx*(ay-by))
    if abs(d)<1e-12: return None
    ux=((ax*ax+ay*ay)*(by-cy)+(bx*bx+by*by)*(cy-ay)+(cx*cx+cy*cy)*(ay-by))/d
    uy=((ax*ax+ay*ay)*(cx-bx)+(bx*bx+by*by)*(ax-cx)+(cx*cx+cy*cy)*(bx-ax))/d
    cc=np.array([ux,uy]); return cc,float(np.linalg.norm(cc-a))
def mec(P):
    P=np.array(P,float); n=len(P); best=(1e99,None)
    def cover(c,r): return np.max(np.linalg.norm(P-c,axis=1))<=r+1e-8
    for i,j in itertools.combinations(range(n),2):
        c,r=circle2(P[i],P[j])
        if r<best[0] and cover(c,r): best=(r,c)
    for i,j,k in itertools.combinations(range(n),3):
        q=circle3(P[i],P[j],P[k])
        if q:

```

```

        c,r=q
        if r<best[0] and cover(c,r):best=(r,c)
    if best[1] is None:
        return np.mean(P,axis=0),0
    return best[1],best[0]

# Regenerate existing figures with CJK font
# Q1 figure
true=np.array([820.,360.]);
↪ stations=[np.array([-500.,-200.]),np.array([1100.,-600.]),np.array([300.,1200.])]
errs=np.radians([0.7,-0.4,0.9]); delta=math.radians(1)
# intersect polygons using large box
box=Polygon([(-4000,-4000),(4000,-4000),(4000,4000),(-4000,4000)])
region=box
fig,ax=plt.subplots(figsize=(7.2,6.2))
for S,e,col in zip(stations,errs,['tab:blue','tab:orange','tab:green']):
    theta=math.atan2(true[1]-S[1],true[0]-S[0])+e
    W=wedge_polygon(S,theta,delta,5000)
    region=region.intersection(W)
    x,y=W.exterior.xy;ax.fill(x,y,alpha=.08,color=col);ax.plot(*S,'o',color=col)
P=poly_pts(region);D=diameter(P)
# farthest pair
bi=bj=0;bd=-1
for i,j in itertools.combinations(range(len(P)),2):
    d=np.linalg.norm(P[i]-P[j])
    if d>bd:bd=d;bi=i;bj=j
A,B=P[bi],P[bj]
pr=np.vstack([P,P[0]]);ax.fill(pr[:,0],pr[:,1],alpha=.35,color='crimson',label='可行定位区域
↪ $P$');ax.plot([A[0],B[0]],[A[1],B[1]],'k-',lw=2,label=f'直径 $D={D:.1f}$
↪ m');ax.plot(*true,'k*',ms=10,label='真实源 (示意)')
ax.set_aspect('equal');ax.grid(alpha=.2);ax.legend(fontsize=9);ax.set_title('Q1:
↪ 有界测向楔形的集合交会定位');ax.set_xlabel('x / m');ax.set_ylabel('y /
↪ m');fig.tight_layout();fig.savefig(FIG/'q1_wedge_intersection.png',dpi=200);plt.close(fig)

tri=np.array([[0.,0.],[1000.,0.],[500.,math.sqrt(3)/2*1000]])
Dtri=1000.;mid=(tri[0]+tri[1])/2;cm,rm=mec(tri)
fig,ax=plt.subplots(figsize=(7.2,5.6));tr=np.vstack([tri,tri[0]]);ax.fill(tr[:,0],tr[:,1],alpha=.12);ax.p
↪ lot(tr[:,0],tr[:,1],'k-',lw=2)
t=np.linspace(0,2*math.pi,500);ax.plot(mid[0]+500*np.cos(t),mid[1]+500*np.sin(t),'--',lw=2,label='直径圆
↪ (半径 $D/2$)');ax.plot(cm[0]+rm*np.cos(t),cm[1]+rm*np.sin(t),'-',lw=2,label='最小包围圆
↪ MEC');ax.set_aspect('equal');ax.grid(alpha=.2);ax.legend();ax.set_title('Q1:
↪ 正三角形反例与最小包围圆');ax.set_xlabel('x / m');ax.set_ylabel('y /
↪ m');fig.tight_layout();fig.savefig(FIG/'q1_mec_counterexample.png',dpi=200);plt.close(fig)

# Q2 signal region lens
fig,ax=plt.subplots(figsize=(8,5.6));t=np.linspace(0,2*math.pi,700)
for c,col in [(5,0),'tab:blue'],((1500,0),'tab:blue'),((1000,0),'tab:orange')]:ax.plot(c[0]+1000*np.cos(
↪ t),c[1]+1000*np.sin(t),color=col,alpha=.65)
xx=np.linspace(-250,1800,520);yy=np.linspace(-1000,1000,420);X,Y=np.meshgrid(xx,yy)
mask1=((X-5)**2+Y**2<=1e6)&((X-1500)**2+Y**2<=1e6);mask2=((X-5)**2+Y**2<=1e6)&((X-1000)**2+Y**2<=1e6)
ax.contourf(X,Y,mask2.astype(int),levels=[.5,1.5],colors=['tab:orange'],alpha=.16);ax.contourf(X,Y,mask1.
↪ astype(int),levels=[.5,1.5],colors=['tab:blue'],alpha=.28)
ax.plot([5,1500],[0,0],'k-',lw=3,label='零角宽近似的第一次可行射线段');ax.text(260,810,'$C_{sig}^*$',color=
↪ 'tab:orange',fontsize=12);ax.text(690,210,'$C_{sig}$',color='tab:blue',fontsize=12)
ax.set_aspect('equal');ax.set_xlim(-200,1800);ax.set_ylim(-1000,1000);ax.grid(alpha=.18);ax.set_xlabel('
↪ 沿第一次示向方向 / m');ax.set_ylabel('横向基线 / m');ax.set_title('Q2:
↪ 第一次正观测扩大保证二次有信号的区域');fig.tight_layout();fig.savefig(FIG/'q2_signal_regions.png',dpi=2
↪ 00);plt.close(fig)

```

```

# Fast approximate heatmap
F1=sector_polygon(); delta=math.radians(1)
# only boundary + few interior true source samples
g=[]
for a in [-delta,0,delta]:
    for r in [100,300,600,900,1200,1500]:g.append(np.array([r*math.cos(a),r*math.sin(a)]))
for a in np.linspace(-delta,delta,7):
    for r in [450,1000,1450]:g.append(np.array([r*math.cos(a),r*math.sin(a)]))
evals=np.radians([-1,0,1])
cert=[]
for a in np.linspace(-delta,delta,15):
    for r in np.linspace(5,1500,25):cert.append(np.array([r*math.cos(a),r*math.sin(a)]))
def Japprox(xy):
    x=np.array(xy,float);worst=0
    for p in g:
        if np.linalg.norm(x-p)<=5:continue
        beta=math.atan2(p[1]-x[1],p[0]-x[0])
        for e in evals:
            q=F1.intersection(wedge_polygon(x,beta+e,delta,5000)).difference(Point(*x).buffer(5,resolution=
            ↪ n=12))
            worst=max(worst,diameter(poly_pts(q)))
    return worst
xs=np.arange(200,1001,100);ys=np.arange(0,801,100);J=np.full((len(ys),len(xs)),np.nan)
for iy,y in enumerate(ys):
    for ix,x in enumerate(xs):
        z=np.array([x,y],float)
        if all(np.linalg.norm(z-p)<=max(1000,np.linalg.norm(p))+1e-9 for p in cert):J[iy,ix]=Japprox(z)
best_idx=np.unravel_index(np.nanargmin(J),J.shape);best=(xs[best_idx[1]],ys[best_idx[0]],J[best_idx])
fig,ax=plt.subplots(figsize=(8,5.8));im=ax.imshow(np.ma.masked_invalid(J),origin='lower',extent=[150,1050]
↪ , -50,850],aspect='auto');fig.colorbar(im,ax=ax,label='近似最坏定位直径 $J_D$ /
↪ m');ax.plot(best[0],best[1], 'r*',ms=13,label=f'网格最优示意 ({best[0]},
↪ {best[1]})');ax.set_xlabel('第二测站前向位移 x / m');ax.set_ylabel('横向位移 |y| / m');ax.set_title('Q2:
↪ 保证信号域内的 min-max 目标示意 (非官方数值)
↪ ');ax.legend();fig.tight_layout();fig.savefig(FIG/'q2_minmax_heatmap.png',dpi=200);plt.close(fig)

# linearized tradeoff
phis=np.radians(np.linspace(5,175,400));dlt=delta
fig,ax=plt.subplots(figsize=(7.5,5.2))
for r1,r2 in [(1000,1000),(1200,800),(1500,600)]:
    val=2*dlt/np.abs(np.sin(phis))*np.sqrt(r1*r1+r2*r2+2*r1*r2*np.abs(np.cos(phis)))
    ax.plot(np.degrees(phis),val,label=f'$r_1$={r1}$ m, $r_2$={r2}$ m')
ax.set_yscale('log');ax.grid(alpha=.25);ax.legend(fontsize=9);ax.set_xlabel('两次真实 bearing 的交会角
↪ $\phi$ / deg');ax.set_ylabel('一阶近似直径 $D_{lin}$ / m (对数轴)');ax.set_title('Q2: 交会角与测站距离
↪ 的解析折中');fig.tight_layout();fig.savefig(FIG/'q2_linearized_tradeoff.png',dpi=200);plt.close(fig)

# Validation on random convex polygons
rng=np.random.default_rng(20260912);mismatch=0;jung=0;kvals=[]
for _ in range(200):
    pts=rng.normal(size=(12,2))*rng.uniform(50,1000);P=pts[ConvexHull(pts).vertices];D=diameter(P);cm,rm=
    ↪ mec(P);k=2*rm/D;kvals.append(k)
    if rm < D/2-1e-6 or rm > D/math.sqrt(3)+1e-5:jung+=1
# exact equilateral
summary={'random_convex_cases':200,'jung_violations':jung,'kappa_min':float(min(kvals)),'kappa_max':float
↪ (max(kvals)),'q2_grid_best':{'x_m':int(best[0]),'abs_y_m':int(best[1]),'approx_J_m':float(best[2])}}
(QA/'validation_summary.json').write_text(json.dumps(summary,ensure_ascii=False,indent=2),encoding='utf-8')
with open(QA/'q2_heatmap_values.csv','w',encoding='utf-8') as f:
    f.write('abs_y_m','+','.join(map(str,xs))+'\n')

```

```

        for iy,y in enumerate(ys):f.write(str(y)+' ','+','.join(' ' if not np.isfinite(v) else f'{v:.3f}' for v
        ↪ in J[iy]))+'\n')
print(json.dumps(summary,ensure_ascii=False,indent=2))

```

文件: q1q2/generate\_q1q2\_artifacts.py

```

import math, itertools, json
from pathlib import Path
import numpy as np
from scipy.spatial import ConvexHull
from scipy.optimize import linprog
from shapely.geometry import Polygon, Point
from shapely.ops import unary_union
import matplotlib.pyplot as plt

ROOT = Path(__file__).resolve().parent
FIG = ROOT/'figures'
QA = ROOT/'qa'
FIG.mkdir(exist_ok=True, parents=True); QA.mkdir(exist_ok=True, parents=True)

EPS=1e-9

def unit(theta):
    return np.array([math.cos(theta), math.sin(theta)], dtype=float)

def wedge_halfplanes(S, theta, delta):
    # halfplanes a*x+b*y <= c corresponding to cross(u-, p-S)>=0 and cross(u+, p-S)<=0
    S=np.asarray(S,float)
    um=unit(theta-delta); up=unit(theta+delta)
    # cross(um,p-S)>=0 => um_x(y-Sy)-um_y(x-Sx)>=0 => um_y x - um_x y <= um_y Sx - um_x Sy
    h1=(um[1], -um[0], um[1]*S[0]-um[0]*S[1])
    # cross(up,p-S)<=0 => up_x(y-Sy)-up_y(x-Sx)<=0 => -up_y x + up_x y <= -up_y Sx + up_x Sy
    h2=(-up[1], up[0], -up[1]*S[0]+up[0]*S[1])
    return [h1,h2]

def halfplane_vertices(H, tol=1e-8):
    H=np.asarray(H,float)
    pts=[]
    for i,j in itertools.combinations(range(len(H)),2):
        a1,b1,c1=H[i]; a2,b2,c2=H[j]
        det=a1*b2-a2*b1
        if abs(det)<1e-12: continue
        x=(c1*b2-c2*b1)/det; y=(a1*c2-a2*c1)/det
        p=np.array([x,y])
        if np.all(H[:,2]@p <= H[:,2]+tol): pts.append(p)
    if len(pts)<3: return np.array(pts)
    # unique
    arr=np.array(pts)
    uniq=[]
    for p in arr:
        if not any(np.linalg.norm(p-q)<1e-7 for q in uniq): uniq.append(p)
    arr=np.array(uniq)
    if len(arr)<3: return arr
    hull=ConvexHull(arr)
    return arr[hull.vertices]

def is_unbounded(H):

```

```

H=np.asarray(H,float)
A=H[:, :2]; b=H[:, 2]
# Check feasibility first
res=linprog([0,0], A_ub=A,b_ub=b,bounds=[(None,None),(None,None)],method='highs')
if not res.success: return False
for d in ([1,0],[-1,0],[0,1],[0,-1]):
    res=linprog([-d[0],-d[1]], A_ub=A,b_ub=b,bounds=[(None,None),(None,None)],method='highs')
    if res.status==3: return True
return False

def polygon_diameter_brute(P):
    P=np.asarray(P,float); best=(-1,None,None)
    for i,j in itertools.combinations(range(len(P)),2):
        d2=float(np.dot(P[i]-P[j],P[i]-P[j]))
        if d2>best[0]: best=(d2,i,j)
    return math.sqrt(best[0]), best[1], best[2]

def polygon_diameter_calipers(P):
    # Standard rotating calipers for CCW convex polygon. Falls back for small/degenerate.
    P=np.asarray(P,float); n=len(P)
    if n<2: return 0.0,0,0
    if n==2: return float(np.linalg.norm(P[0]-P[1])),0,1
    def area2(a,b,c): return abs(np.cross(b-a,c-a))
    j=1; best_d2=0; best=(0,1)
    for i in range(n):
        ni=(i+1)%n
        # advance j while area increases
        while area2(P[i],P[ni],P[(j+1)%n]) > area2(P[i],P[ni],P[j]) + 1e-12:
            j=(j+1)%n
        for a,b in [(i,j),(ni,j)]:
            d2=float(np.dot(P[a]-P[b],P[a]-P[b]))
            if d2>best_d2:
                best_d2=d2; best=(a,b)
    return math.sqrt(best_d2), best[0], best[1]

def circle_from_2(a,b):
    c=(a+b)/2; r=float(np.linalg.norm(a-b))/2; return c,r

def circle_from_3(a,b,c):
    ax,ay=a; bx,by=b; cx,cy=c
    d=2*(ax*(by-cy)+bx*(cy-ay)+cx*(ay-by))
    if abs(d)<1e-12: return None
    ux=((ax*ax+ay*ay)*(by-cy)+(bx*bx+by*by)*(cy-ay)+(cx*cx+cy*cy)*(ay-by))/d
    uy=((ax*ax+ay*ay)*(cx-bx)+(bx*bx+by*by)*(ax-cx)+(cx*cx+cy*cy)*(bx-ax))/d
    center=np.array([ux,uy]); r=float(np.linalg.norm(center-a)); return center,r

def mec_deterministic(P,tol=1e-8):
    P=np.asarray(P,float); n=len(P)
    if n==0: return np.array([0.,0.]),0.0,()
    if n==1: return P[0].copy(),0.0,(0,)
    best=(float('inf'),None,None)
    def covers(c,r): return np.max(np.linalg.norm(P-c,axis=1)) <= r+tol
    for i,j in itertools.combinations(range(n),2):
        c,r=circle_from_2(P[i],P[j])
        if r<best[0] and covers(c,r): best=(r,c,(i,j))
    for i,j,k in itertools.combinations(range(n),3):
        cc=circle_from_3(P[i],P[j],P[k])
        if cc is None: continue
        c,r=cc

```

```

        if r<best[0] and covers(c,r): best=(r,c,(i,j,k))
    if best[1] is None:
        # degenerate collinear: farthest pair
        D,i,j=polygon_diameter_brute(P); c,r=circle_from_2(P[i],P[j]); return c,r,(i,j)
    return best[1],best[0],best[2]

def thales_ok(P,A,B,tol=1e-8):
    vals=[float(np.dot(v-A,v-B)) for v in P]
    return max(vals)<=tol, max(vals)

def wedge_polygon(S, theta, delta, L=5000):
    S=np.asarray(S,float)
    a=S + L*unit(theta-delta); b=S + L*unit(theta+delta)
    return Polygon([tuple(S),tuple(a),tuple(b)])

def sector_polygon(rmin=5,rmax=1500,amin=-math.radians(1),amax=math.radians(1),n=800):
    outer=[(rmax*math.cos(a),rmax*math.sin(a)) for a in np.linspace(amin,amax,n//2)]
    inner=[(rmin*math.cos(a),rmin*math.sin(a)) for a in np.linspace(amax,amin,n//2)]
    return Polygon(outer+inner)

def poly_coords(geom):
    if geom.is_empty: return np.empty((0,2))
    g=geom.convex_hull
    return np.array(g.exterior.coords[:-1])

def diameter_shapely(geom):
    P=poly_coords(geom)
    if len(P)<2: return 0.0
    return polygon_diameter_brute(P)[0]

def approximate_J(x, F1, g_samples, e_samples, delta=math.radians(1)):
    x=np.asarray(x,float); worst=0.0
    for g in g_samples:
        if np.linalg.norm(g-x)<=5:
            post=0.0
            worst=max(worst,post); continue
        beta=math.atan2(g[1]-x[1],g[0]-x[0])
        for e in e_samples:
            theta=beta+e
            W=wedge_polygon(x,theta,delta,L=5000)
            F2=F1.intersection(W)
            # normal direction implies >5m; removing disk may make nonconvex. Diameter equals diameter of
            # ↪ convex hull of exact set.
            F2=F2.difference(Point(*x).buffer(5,resolution=32))
            post=diameter_shapely(F2)
            worst=max(worst,post)
    return worst

# ----- Figure 1: Q1 wedge intersection -----
true=np.array([820.,360.])
stations=[np.array([-500.,-200.]), np.array([1100.,-600.]), np.array([300.,1200.])]
errs=np.radians([0.7,-0.4,0.9]); delta=math.radians(1)
H=[]
for S,e in zip(stations,errs):
    theta=math.atan2(true[1]-S[1],true[0]-S[0])+e
    H += wedge_halfplanes(S,theta,delta)
P=halfplane_vertices(H)
D,ia,ib=polygon_diameter_calipers(P); A=P[ia]; B=P[ib]

```

```

cmec,rmec,supp=mec_deterministic(P)
ok,tmax=thales_ok(P,A,B)

fig,ax=plt.subplots(figsize=(7.2,6.3))
colors=['tab:blue','tab:orange','tab:green']
for S,e,col in zip(stations,errs,colors):
    theta=math.atan2(true[1]-S[1],true[0]-S[0])+e
    wp=wedge_polygon(S,theta,delta,L=2500)
    xs,ys=wp.exterior.xy
    ax.fill(xs,ys,alpha=.10,color=col)
    ax.plot(S[0],S[1], 'o',color=col); ax.text(S[0]+25,S[1]+25,'$$$',color=col)
if len(P)>=3:
    pp=np.vstack([P,P[0]])
    ax.fill(pp[:,0],pp[:,1],alpha=.35,label='可行定位区域 $$$',color='crimson')
    ax.plot([A[0],B[0]],[A[1],B[1]], 'k-',lw=2,label=f'直径 $D={D:.1f}$ m')
ax.plot(*true,'k*',ms=10,label='真实源（仅用于示意）')
ax.set_aspect('equal'); ax.grid(alpha=.2); ax.legend(loc='best',fontsize=9)
ax.set_xlabel('x / m'); ax.set_ylabel('y / m'); ax.set_title('Q1: 有界测向楔形的集合交会定位')
fig.tight_layout(); fig.savefig(FIG/'q1_wedge_intersection.png',dpi=220); plt.close(fig)

# Figure 2: diameter circle vs MEC counterexample
tri=np.array([[0.,0.],[1000.,0.],[500.,math.sqrt(3)/2*1000]])
D2,i2,j2=polygon_diameter_brute(tri); A2=tri[i2]; B2=tri[j2]
mid=(A2+B2)/2; c2,r2,_=mec_deterministic(tri)
fig,ax=plt.subplots(figsize=(7.2,5.8))
tr=np.vstack([tri,tri[0]]); ax.plot(tr[:,0],tr[:,1], 'k-',lw=2); ax.fill(tr[:,0],tr[:,1],alpha=.12)
for p,l in zip(tri,['A','B','C']): ax.plot(*p,'ko'); ax.text(p[0]+18,p[1]+18,l)
for c,r,ls,label in [(mid,D2/2,'--','某一直径对对应的直径圆'),(c2,r2,'-','最小包围圆 MEC')]:
    tt=np.linspace(0,2*math.pi,400); ax.plot(c[0]+r*np.cos(tt),c[1]+r*np.sin(tt),ls,lw=2,label=label)
ax.set_aspect('equal'); ax.grid(alpha=.2); ax.legend(fontsize=9); ax.set_title('Q1:
↪ 正三角形说明直径圆并不总能覆盖')
ax.set_xlabel('x / m'); ax.set_ylabel('y / m'); fig.tight_layout();
↪ fig.savefig(FIG/'q1_mec_counterexample.png',dpi=220); plt.close(fig)

# Figure 3: canonical Csig and Csig*
fig,ax=plt.subplots(figsize=(8,5.6))
t=np.linspace(0,2*math.pi,800)
# conservative: centers 5 and 1500, radius1000; enhanced: 5 and1000
for center,style in [(5,0),'tab:blue'],[(1500,0),'tab:blue'],[(1000,0),'tab:orange']]:
    ax.plot(center[0]+1000*np.cos(t),center[1]+1000*np.sin(t),color=style,alpha=.65)
# fill lenses through grid mask
xx=np.linspace(-300,1800,700); yy=np.linspace(-1000,1000,600); X,Y=np.meshgrid(xx,yy)
mask1=((X-5)**2+Y**2<=1000**2)&((X-1500)**2+Y**2<=1000**2)
mask2=((X-5)**2+Y**2<=1000**2)&((X-1000)**2+Y**2<=1000**2)
ax.contourf(X,Y,mask2.astype(int),levels=[.5,1.5],colors=['tab:orange'],alpha=.16)
ax.contourf(X,Y,mask1.astype(int),levels=[.5,1.5],colors=['tab:blue'],alpha=.25)
ax.plot([5,1500],[0,0], 'k-',lw=3,label='零角宽近似的第一次可行射线段')
ax.text(260,800,'$C_{sig}^*$',color='tab:orange',fontsize=12)
ax.text(680,210,'$C_{sig}$',color='tab:blue',fontsize=12)
ax.set_aspect('equal'); ax.set_xlim(-200,1800); ax.set_ylim(-1000,1000); ax.grid(alpha=.18)
ax.set_xlabel('沿第一次示向方向 / m'); ax.set_ylabel('横向基线 / m'); ax.set_title('Q2:
↪ 第一次正观测扩大了保证二次有信号的区域')
fig.tight_layout(); fig.savefig(FIG/'q2_signal_regions.png',dpi=220); plt.close(fig)

# Figure 4: approximate J heatmap for canonical +/-1deg sector
Fl=sector_polygon()
# source samples: boundary rays + radial middle + outer arc
rs=np.concatenate([np.linspace(5,1500,30),np.array([1000])])

```

```

g_samples=[]
for a in [-delta,0,delta]:
    for r in rs: g_samples.append(np.array([r*math.cos(a),r*math.sin(a)]))
for a in np.linspace(-delta,delta,21):
    for r in [300,600,900,1200,1500]: g_samples.append(np.array([r*math.cos(a),r*math.sin(a)]))
e_samples=np.radians(np.linspace(-1,1,5))
xs=np.arange(200,1001,100); ys=np.arange(0,801,100)
J=np.full((len(ys),len(xs)),np.nan)
# guaranteed-signal check approximate on dense g samples: dist <= max(1000,dist to S1)
cert_samples=[]
for a in np.linspace(-delta,delta,61):
    for r in np.linspace(5,1500,80): cert_samples.append(np.array([r*math.cos(a),r*math.sin(a)]))
for iy,y in enumerate(ys):
    for ix,x in enumerate(xs):
        z=np.array([x,y])
        if all(np.linalg.norm(z-g) <= max(1000,np.linalg.norm(g)) + 1e-9 for g in cert_samples):
            J[iy,ix]=approximate_J(z,F1,g_samples,e_samples,delta)
finite=np.isfinite(J)
best_idx=np.unravel_index(np.nanargmin(J),J.shape); best=(xs[best_idx[1]],ys[best_idx[0]],J[best_idx])
fig,ax=plt.subplots(figsize=(8,5.8))
masked=np.ma.masked_invalid(J)
im=ax.imshow(masked,origin='lower',extent=[xs[0]-50,xs[-1]+50,ys[0]-50,ys[-1]+50],aspect='auto')
fig.colorbar(im,ax=ax,label='近似最坏定位直径 $J_{D}$ / m')
ax.plot(best[0],best[1], 'r*',ms=13,label=f'网格最优示意 ({best[0]}, {best[1]})')
ax.set_xlabel('第二测站前向位移 x / m'); ax.set_ylabel('横向位移 |y| / m'); ax.set_title('Q2: 保证信号域内的
↪ min-max 目标示意 (非官方数值)'); ax.legend()
fig.tight_layout(); fig.savefig(FIG/'q2_minmax_heatmap.png',dpi=220); plt.close(fig)

# Figure 5: linearized geometry tradeoff
phis=np.radians(np.linspace(5,175,400)); dlt=math.radians(1)
fig,ax=plt.subplots(figsize=(7.5,5.2))
for r1,r2 in [(1000,1000),(1200,800),(1500,600)]:
    val=2*dlt/np.abs(np.sin(phis))*np.sqrt(r1*r1+r2*r2+2*r1*r2*np.abs(np.cos(phis)))
    ax.plot(np.degrees(phis),val,label=f'$r_1={r1}$m, $r_2={r2}$m')
ax.set_yscale('log'); ax.grid(alpha=.25); ax.legend(fontsize=9); ax.set_xlabel('两次真实 bearing 的交会角
↪ $\phi$ / deg'); ax.set_ylabel('一阶近似直径 $D_{lin}$ / m (log)'); ax.set_title('Q2:
↪ 交会角与测站距离的解析折中')
fig.tight_layout(); fig.savefig(FIG/'q2_linearized_tradeoff.png',dpi=220); plt.close(fig)

# Validation: calipers vs brute; Thales/MEC equivalence on random convex polygons
rng=np.random.default_rng(20260912)
max_d_err=0.0; mismatch=0; jung_viol=0; cases=1000
kappas=[]
for _ in range(cases):
    pts=rng.normal(size=(rng.integers(5,25),2))*rng.uniform(50,1000)
    hull=ConvexHull(pts); P0=pts[hull.vertices]
    Db,ib,jb=polygon_diameter_brute(P0); Dc,ic,jc=polygon_diameter_calipers(P0)
    max_d_err=max(max_d_err,abs(Db-Dc))
    A=P0[ic]; B=P0[jc]; th,_=thales_ok(P0,A,B,tol=1e-7)
    c,r,_=mec_deterministic(P0,tol=1e-7)
    mecok=(r <= Dc/2 + 1e-6)
    if th!=mecok: mismatch+=1
    if r < Dc/2-1e-6 or r > Dc/math.sqrt(3)+1e-5: jung_viol+=1
    kappas.append(2*r/Dc)

# Canonical support / signal region sanity: random samples from exact 1D case
# Csig* exact lens B((5,0),1000) intersect B((1000,0),1000)
def in_lens(z):

```

```

    z=np.asarray(z); return np.linalg.norm(z-np.array([5.,0.]<=1000+1e-9 and
    ⇨ np.linalg.norm(z-np.array([1000.,0.]<=1000+1e-9
viol=0; checked=0
for _ in range(20000):
    z=np.array([rng.uniform(-500,1600),rng.uniform(-1000,1000)])
    if not in_lens(z): continue
    checked+=1
    for r in rng.uniform(5,1500,20):
        g=np.array([r,0.]); rho=max(1000,r)
        if np.linalg.norm(z-g)>rho+1e-8: viol+=1; break

summary={
    'q1_random_convex_cases':cases,
    'max_abs_diameter_error_calipers_vs_brute_m':max_d_err,
    'thales_vs_mec_mismatch_count':mismatch,
    'jung_violation_count':jung_viol,
    'kappa_min':float(np.min(kappas)),
    'kappa_max':float(np.max(kappas)),
    'canonical_csigstar_random_points_checked':checked,
    'canonical_csigstar_guarantee_violations':viol,
    'illustrative_q1_D_m':D,
    'illustrative_q1_Rmec_m':rmec,
    'illustrative_q1_kappa':2*rmec/D,
    'illustrative_q1_thales_cover':ok,
    'q2_heatmap_best_grid_point':{'x':int(best[0]),'abs_y':int(best[1]),'approx_J_m':float(best[2])}
}
(QA/'validation_summary.json').write_text(json.dumps(summary,ensure_ascii=False,indent=2),encoding='utf-8')

# small CSV heatmap
with open(QA/'q2_heatmap_values.csv','w',encoding='utf-8') as f:
    f.write('abs_y_m','+','.join(map(str,xs))+'\n')
    for iy,y in enumerate(ys):
        vals=[(' if not np.isfinite(v) else f'{v:.6f}') for v in J[iy]]
        f.write(str(y)+'+','.join(vals)+'\n')

print(json.dumps(summary,ensure_ascii=False,indent=2))

```